Softwareentwicklung mit Java 2

Klassen · Vererbung und dynamische Bindung · Interfaces · Spezielle Klassen · Generics

Ausführliches Lehrskript mit Erläuterungen, Originalfolien, Fehlerhinweisen und Folienabgleich

Grundlage: Vorlesungsfolien von Herbert Prähofer (2026), FH Hagenberg / JKU Linz.

Bearbeitungsstand: 29. Oktober 2026

Die fünf Foliensätze werden in einem durchgehenden Lehrtext mit Originalabbildungen und fachlichen Korrekturen behandelt. Java-Syntax erscheint einheitlich in blauer Monospace-Schrift. Ergänzungen und Korrekturhinweise sind kein Teil der Originalfolien.

Das organisatorische Dokument wurde zur Orientierung ausgewertet, aber wie gewünscht nicht in den Lernstoff übernommen.

Inhaltsverzeichnis

1 Klassen ... 3
1.1 Klassen, Objekte und Referenzen ... 4
1.2 Aufbau, Felder, Methoden und Konstruktoren ... 10
1.3 Packages, qualifizierte Namen und Imports ... 17
1.4 Sichtbarkeit, Information Hiding und Properties ... 21
1.5 final, unveränderliche Objekte und Wertegleichheit ... 26
1.6 Entwurfsrichtlinien und Fallstudie Telefonbuch ... 32
2 Vererbung und dynamische Bindung ... 36
2.0 Statische und dynamische Betrachtung; UML ... 36
2.1 Vererbung, super, abstract und final ... 38
2.2 Subtypen, dynamische Bindung, Typtests und Exceptions ... 47
2.3 Kovariante Rückgabetypen und geprüfte Exceptions ... 56
2.4 Fallstudie List: Basisklasse und Implementierungen ... 60
2.5 Entwurfsmuster: Wrapper, Decorator, Adapter, Factory, Template Method und Strategy ... 66
3 Interfaces ... 82
3.0 Vorschau: generische Typen und Methoden ... 82
3.1 Motivation: Einfach- und Mehrfachvererbung ... 84
3.2 Interfaces als Verträge und Typen ... 85
3.3 Default-Methoden und die Weiterentwicklung von Interfaces ... 89
3.4 Statische Konstanten, Fabriken und private Hilfsmethoden ... 92
3.5 Bibliotheksinterfaces: Runnable, Iterable, Ordnung und Kopien ... 94
3.6 Fallstudie List und MutableList: lesende versus ändernde Zugriffe ... 100
4 Spezielle Klassen ... 106
4.1 Record-Klassen ... 106
4.2 Enumerationsklassen ... 111
4.3 Geschachtelte, lokale und anonyme Klassen ... 115
4.4 Fallstudie List: Iteratoren, Adapter und differenzierte Interfaces ... 131
5 Generics ... 143
5.1 Motivation: Typsicherheit statt Object und Casts ... 143
5.2 Grundlagen: Typparameter, Parametrisierung und generische Methoden ... 145
5.3 Typconstraints und vergleichbare Elemente ... 150
5.4 Invarianz, Ko- und Kontravarianz und Wildcards ... 155
5.5 Type Erasure, Rohtypen und Laufzeitgrenzen ... 166
5.6 Arrays, Kovarianz und generische Arraytypen ... 170
5.7 Typinferenz und Erzeugung generischer Werte ... 174
5.8 Fallstudie List: die generische Listenarchitektur ... 178
5.9 Zusammenfassung und systematische Einordnung ... 188
Anhang A Fachliche Korrekturen und besonders wichtige Codefallen ... 191
Anhang B Vollständiger Folienabgleich ... 196

Die blau gesetzten Kapitelüberschriften sind anklickbar. Die angegebenen Seitenzahlen beziehen sich auf diese Ausgabe.

1 Klassen

Grundbegriffe für das gesamte Skript

Ein Programm ist eine Folge von Anweisungen im Quelltext. Der Java-`Compiler (javac)` überprüft diesen Text und erzeugt ausführbaren Bytecode, während die Java Virtual `Machine (JVM)` ihn zur Laufzeit ausführt. Eine Klasse ist eine Typdefinition mit `Feldern (Daten)` und `Methoden (Operationen)`. Ein Objekt ist eine zur Laufzeit erzeugte Instanz dieser Klasse; eine Referenz ist ein Wert, über den ein solches Objekt angesprochen wird. Der statische Typ ist die vom Compiler betrachtete Typangabe einer Variable oder eines Ausdrucks, der dynamische Typ bezeichnet bei Referenzwerten die Klasse des tatsächlich referenzierten Objekts.

Eine Methode besitzt Parameter, einen Rückgabetyp und einen ausführbaren Rumpf oder – bei abstrakten Methoden – nur eine Deklaration. Der Aufruf einer Methode kann je nach Objektklasse zur Laufzeit unterschiedliche überschriebene Implementierungen `erreichen (dynamische Bindung)`. Ein Interface definiert eine Schnittstelle, also einen vertraglich festgelegten Satz von Operationen, ohne zwingend eine eigene Objektdatenstruktur vorzugeben. Ein Typparameter, später ausführlich in Thema 5 behandelt, wird als Platzhalter für einen Referenztyp deklariert und ermöglicht statische Typprüfung bei wiederverwendbaren Klassen und Methoden.

Die Beispiele der Vorlesung verwenden eigene Lehr-Klassen mit Namen wie `List`, `ArrayList`, `LinkedList` oder `Map`. Die gleichnamigen Typen der Java-Standardbibliothek haben zum Teil andere `Arten (Klasse oder Interface)`, Methoden oder Eigenschaften. Eine ausdrückliche Präfixangabe wie `java.util.List` bezeichnet immer den Bibliothekstyp. `Auslassungspunkte (...)` in Originalfolien sind schematische Platzhalter und kein grundsätzlich unmittelbar ausführbarer Java-Code.

Hinweis zur Lesart: In den erklärenden Abschnitten bezeichnet eine blau gesetzte Monospace-Schreibweise konkrete Java-Syntax, etwa `new Timer()`, `List<String>` oder `@Override`. Dagegen bezeichnet das normal gesetzte Wort „Klasse“ einen fachlichen Begriff. Schematische Bezeichnungen der Vorlesung sind nicht automatisch mit Klassen der Java-Standardbibliothek identisch.

Für den Ablauf eines Programms sind drei Zeitpunkte auseinanderzuhalten: Das Schreiben des Quelltexts, das Übersetzen mit `javac` und die Ausführung in der JVM. Eine Deklaration kann bereits beim Übersetzen unzulässig sein, etwa `int x = "Text";`. Andere Probleme entstehen erst beim Ausführen, etwa die Dereferenzierung von `null` oder ein Cast auf einen falschen dynamischen Typ. Diese Unterscheidung wird bei Vererbung, Interfaces und Generics wiederverwendet.

1.1 Klassen, Objekte und Referenzen

Eine Klasse ist in Java gleichzeitig eine Beschreibung des Zustands und des Verhaltens von Objekten sowie ein Typ, mit dem Variablen, Parameter und Rückgabewerte deklariert werden. Im Klassenkopf stehen Klassenname und gegebenenfalls Vererbungs- und Interface-Angaben. Im Klassenkörper stehen Felder, Konstruktoren und Methoden. Felder speichern die Daten eines konkreten Objekts; Methoden definieren zulässige Operationen. Das Beispiel `Timer` verwendet drei ganzzahlige Felder für Stunden, Minuten und Sekunden und Operationen zum Zurücksetzen und Weiterzählen. Seine Feldwerte gehören nicht zur Variablen, die auf den `Timer` verweist, sondern zum `Timer`-Objekt selbst.

Die Deklaration `Timer t;` reserviert eine Variable des Referenztyps `Timer`, erzeugt aber ausdrücklich noch keinen `Timer`. Erst `t = new Timer();` erzeugt ein Objekt, initialisiert dessen Felder und speichert in `t` eine Referenz auf das neue Objekt. Java trennt also Objekterzeugung und Referenzvariable. Die Folien zeichnen lokale Variablen vereinfacht im Stack und Objekte im Heap. Das ist ein übliches Laufzeitmodell; auf Spezifikationsebene garantiert Java die semantischen Regeln, nicht für jede Implementierung eine bestimmte physische Anordnung im Speicher. Eine nicht initialisierte lokale Variable darf vor einer Zuweisung nicht gelesen werden. Objektfelder erhalten hingegen zunächst Standardwerte, beispielsweise `0` für `int`, `false` für `boolean` und `null` für Referenztypen, sofern sie nicht ausdrücklich initialisiert werden.

Die Zuweisung `t3 = t2;` kopiert nur die Referenz. Danach zeigen beide Variablen auf dasselbe Objekt. Wird `t3.hrs = 5;` ausgeführt, liefert `t2.hrs` ebenfalls `5`; es handelt sich nicht um eine versteckte Kopie der Uhr. Solche gemeinsam genutzten veränderlichen Objekte können Seiteneffekte erzeugen: Eine Operation über eine Referenz verändert den Zustand, den eine andere Referenz beobachtet. Das ist nicht grundsätzlich falsch, muss aber beim Entwurf kontrolliert werden.

Bei den acht primitiven Datentypen `byte`, `short`, `int`, `long`, `float`, `double`, `char` und `boolean` liegt demgegenüber Wertesemantik vor: `int j = i;` kopiert den aktuellen Zahlenwert. Ein späteres `i++` verändert `j` nicht. Ein Array ist dagegen selbst ein Objekt; auch `Timer[] arr` ist daher eine Referenz, und seine Elemente enthalten weitere Referenzen. `new Timer[2]` erzeugt ein Array mit zwei zunächst `null`-Elementen, aber noch keine zwei `Timer`-Objekte.

Weil manche APIs Objekt- beziehungsweise generische Typen verlangen, besitzt jeder primitive Typ eine passende Wrapper-Klasse: `Boolean`, `Byte`, `Short`, `Integer`, `Long`, `Character`, `Float` und `Double`. Eine Konvertierung von `int` nach `Integer` heißt Boxing, die Gegenrichtung Unboxing. Java kann beide Umwandlungen automatisch durchführen: `Integer n = 7; int x = n;`. Der erste Ausdruck erzeugt beziehungsweise beschafft eine Wrapper-Repräsentation; beim zweiten wird der primitive Wert entnommen. Ein `Integer` kann auch `null` sein; dann führt automatisches Unboxing zu einer `NullPointerException`.

`null` ist der besondere Referenzwert „kein Objekt“. Eine Variable darf `null` speichern, solange ihr Referenztyp das zulässt. Die Typprüfung des Compilers erlaubt dennoch einen Methodenaufruf wie `p.firstName()`, weil `Person` die Methode definiert; wenn `p` zur Laufzeit `null` ist, scheitert die Dereferenzierung mit einer `NullPointerException`. `null` kann in APIs uneinheitlich „unbekannt“, „nicht vorhanden“, „noch nicht gesetzt“ oder „nicht gefunden“ bedeuten. Deshalb muss die Bedeutung pro Parameter und Rückgabewert festgelegt werden.

Ein expliziter Test `p != null` sichert eine nachfolgende Dereferenzierung, sofern sich `p` in der Zwischenzeit nicht unerwartet ändert. `Objects.requireNonNull(x, "Nachricht")` lehnt `null` unmittelbar mit einer `NullPointerException` ab; `Objects.requireNonNullElse(x, ersatz)` wählt einen nicht-`null` Ersatzwert. `Objects.equals(a,b)` führt einen `null`-sicheren inhaltlichen Vergleich aus. `String.valueOf(obj)` gibt für `null` den Text `"null"` zurück, während `obj.toString()` dann nicht aufgerufen werden kann. Entscheidend ist die Unterscheidung zwischen Referenzgleichheit (`==`: identisches Objekt) und einem sinnvoll definierten Wertevergleich (`equals`).

Bei Referenzvariablen ist zwischen der Identität eines Objekts und der Gleichheit seines Inhalts zu unterscheiden. `a == b` prüft bei zwei Referenzen, ob beide auf dasselbe Objekt zeigen. `a.equals(b)` kann eine inhaltliche Gleichheit prüfen, sofern die Klasse `equals` entsprechend überschreibt. Ohne eigenes `equals` gilt grundsätzlich die aus `Object` geerbte Identitätsprüfung. Für mögliche Nullwerte ist `Objects.equals(a, b)` geeignet.

Eine Zuweisung verändert ausschließlich den Wert der Zielvariablen. Nach `Timer a = new Timer(); Timer b = a;` entstehen nicht zwei Objekte, sondern zwei Referenzen auf ein Objekt. Nach `b = new Timer();` existieren zwei unterschiedliche Objekte und `a` zeigt weiterhin auf das ursprüngliche. `null` ist ebenfalls ein Referenzwert, aber kein Objekt und keine Instanz irgendeiner Klasse. Auf `null` kann keine Instanzmethode aufgerufen werden.

Autoboxing verschleiert manchmal Kosten und Fehlerquellen: `Integer a = 100;` erzeugt eine Wrapper-Repräsentation, und `int b = a;` liest den primitiven Wert heraus. Wird `a` später `null`, wirft das Unboxing eine `NullPointerException`. Zwei Wrapper dürfen nicht allgemein mit `==` auf numerische Inhaltsgleichheit verglichen werden; dafür verwendet man beispielsweise `a.equals(b)` oder nach sinnvoller Nullprüfung die primitiven Werte.

Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
Timer a = new Timer();
Timer b = a;
System.out.println(a == b); // true: dieselbe Instanz
b = new Timer();
System.out.println(a == b); // false: zwei Instanzen
```

Originalfolien mit fachlicher Einordnung

K · Folie 3: KLASSEN

Die `Timer`-Klasse illustriert die drei Rollen einer Klasse: Struktur, Verhalten und Typ. Die Addition muss bei Überschreiten von 59 Sekunden einen Übertrag zu Minuten auslösen und analog einen Übertrag zu Stunden.

KLASSEN

Definition von Bausteinen mit Daten und Operationen

- Datenkomponenten (***Variablen***, ***Felder***)
- Methoden

Schablone für Objekterzeugung

- Erzeugen mit **new**

```
new Timer();
```

Repräsentieren Typen

- für Deklarationen von Variablen, Parameter etc.

```
Timer timer;
```

```
public class Timer {

  int hrs;
  int min;
  int sec;

  void clear() {
    hrs = 0;
    min = 0;
    sec = 0;
  }

  void add(int s) {
    sec += s;
    if (sec > 59) {
       min += sec / 60;
       sec = sec % 60;
    }
    if (min > 59) {
      hrs += min / 60;
      min = min % 60; }
  }
}
```

Klassenkopf mit Name

Daten (Felder)

Methoden

© H. Prähofer, 2026 3

Originalabbildung: 01.Klassen.pdf, Folie 3.

K · Folie 4: VARIABLENDEKLARATION UND OBJEKTERZEUGUNG

Im Speicherschaubild sind Referenzvariablen als eigenständige Werte dargestellt. `t3` bleibt nach der Deklaration uninitialisiert; zwei `new`-Aufrufe erzeugen zwei voneinander unabhängige `Timer`-Instanzen.

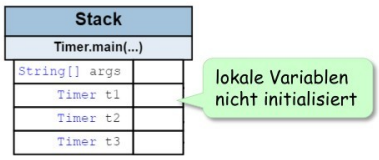
VARIABLENDEKLARATION UND OBJEKTERZEUGUNG

Deklaration von Variablen → Klasse als Typ

- lokale Variablen/Parameter werden am Stack angelegt

Typ der Variablen **t1**, **t2**, **t3**

```
public static void main(String[] args) {
  Timer t1, t2, t3;
```

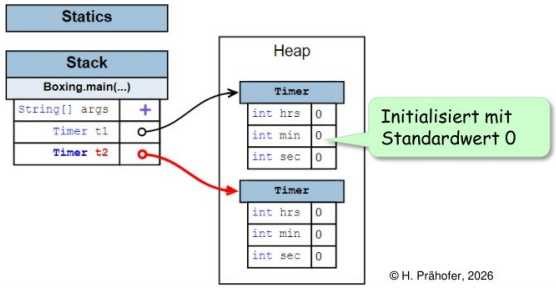


Erzeugung von Objekten → Klasse als Schablone für Objekte

- Objekte werden mit **new** am Heap angelegt!!
- Resultat ist eine Referenz auf Heap-Objekt
- Zugriff auf Heap-Objekte immer über Referenz

Schablone für **Timer**-Objekt

```
  t1 = new Timer();
  t2 = new Timer();
```

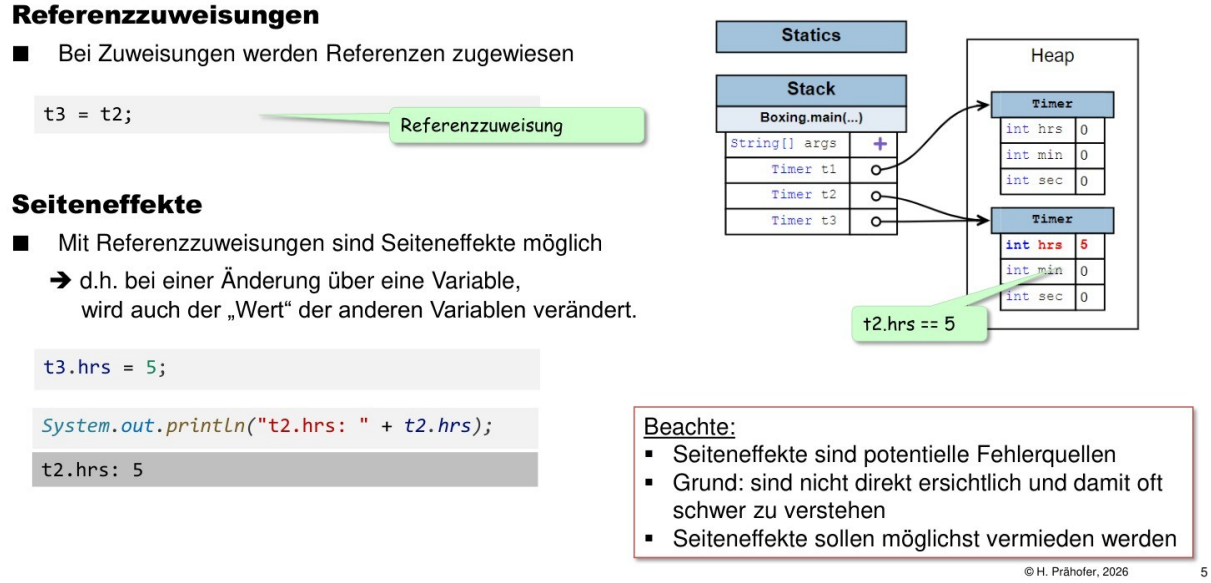


© H. Prähofer, 2026 4

Originalabbildung: 01.Klassen.pdf, Folie 4.

K · Folie 5: REFERENZEN UND REFERENZZUWEISUNGEN

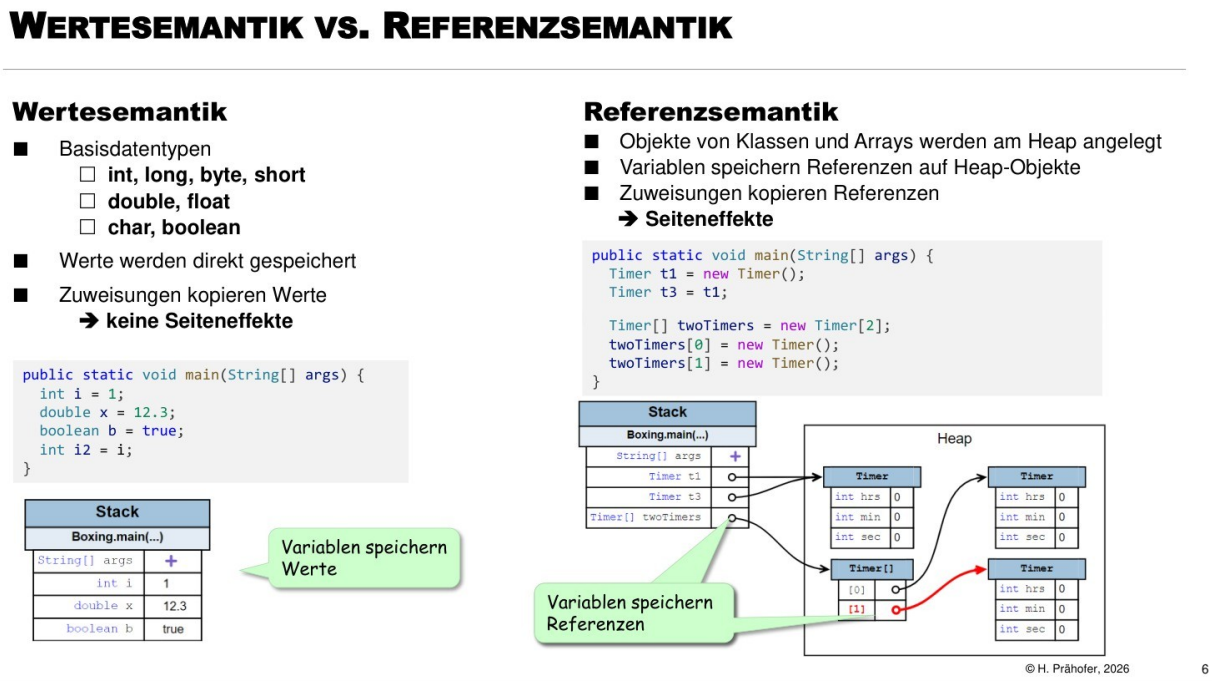
Die beiden Pfeile von `t2` und `t3` enden beim selben Heap-Objekt. Der Seiteneffekt entsteht nur, weil der Objektzustand änderbar ist; das Kopieren der Referenz an sich ändert das Objekt noch nicht.



Originalabbildung: 01.Klassen.pdf, Folie 5.

K · Folie 6: WERTESEMANTIK VS. REFERENZSEMANTIK

Auf der linken Seite kopiert `i2 = i` den Zahlenwert; rechts enthält das `Timer`-Array Referenzen, die nach der Array-Erzeugung zunächst `null` sind. Das Array und die `Timer` sind unterschiedliche Heap-Objekte.



Originalabbildung: 01.Klassen.pdf, Folie 6.

K · Folie 7: WRAPPER-KLASSEN UND BOXING / UNBOXING

Der Tabellenüberblick umfasst alle acht Basisdatentypen und ihre Wrapper. Ein Wrapper ist ein Objekt des Referenztyps, so dass er etwa an einen `Object`-Parameter übergeben werden kann.

Wrapper-Klassen und Boxing / Unboxing

Wrapper-Klassen für Basisdatentypen

- zu den Basisdatentypen gibt es Wrapper-Klassen
- werden eingesetzt, wenn man für Wert eines Basisdatentyps ein Heap-Objekt benötigt

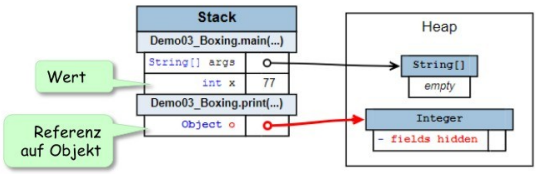
Wertetypen	Wrapper-Klassen
boolean	Boolean
int	Integer
long	Long
byte	Byte
short	Short
char	Character
double	Double
float	Float

Beachte Großbuchstaben

z.B. wenn man eine Methode mit einem **Object**-Parameter mit einem **int**-Wert aufrufen will

```
static void print(Object o) {
  System.out.println(o.toString());
}
int x = 77;
print(Integer.valueOf(x));
```

Parameter Object



© H. Prähofer, 2026 7

Originalabbildung: 01.Klassen.pdf, Folie 7.

K · Folie 8: BOXING UND UNBOXING

Autoboxing gilt auch bei Methodenaufrufen mit passendem Referenzparameter. Beim Unboxing ist insbesondere auf einen möglichen `null`-Wert und die dadurch ausgelöste `Exception` zu achten.

Boxing und Unboxing

- **Boxing**: Automatisches Konvertieren von Werten eines Basisdatentyps in Heap-Objekt

```
int x = 1;
Integer xObj = x;
```

```
static void print(Object o) {
  System.out.println(o.toString());
}
int x = 77;
print(x);
```

automatisches Boxing

- **Unboxing**: Automatisches Konvertieren von Wrapper-Objekt in Wert des Basisdatentyps

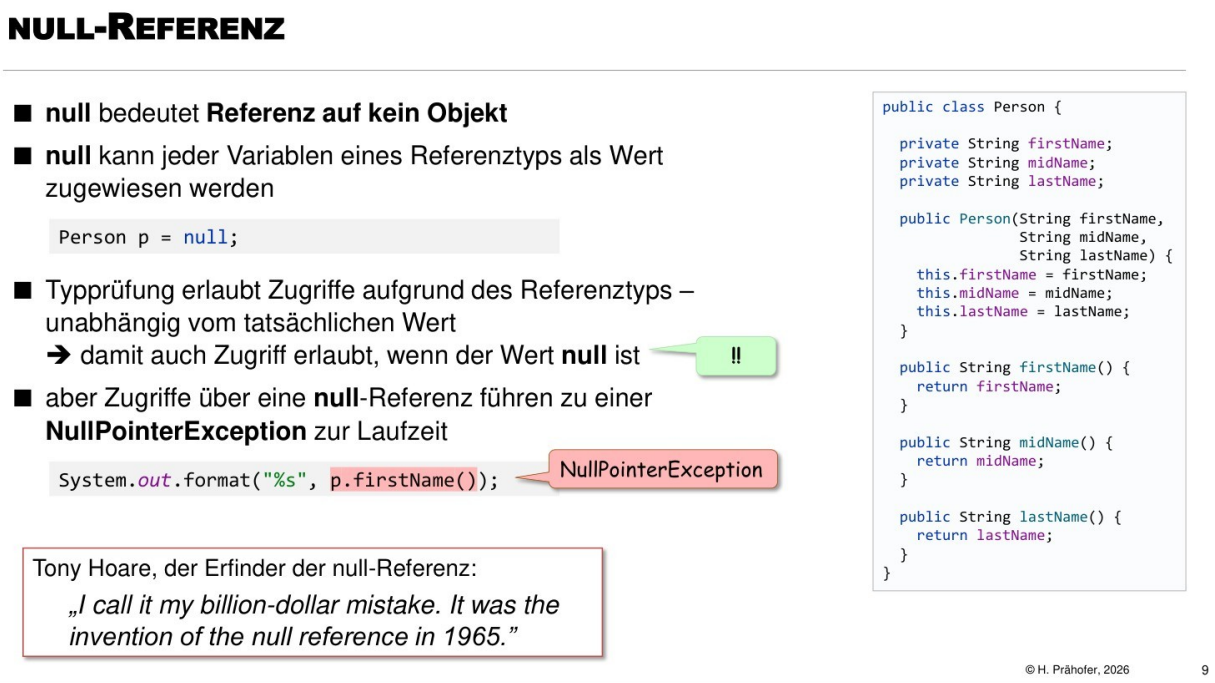
```
int y = xObj;
```

© H. Prähofer, 2026 8

Originalabbildung: 01.Klassen.pdf, Folie 8.

K · Folie 9: NULL-REFERENZ

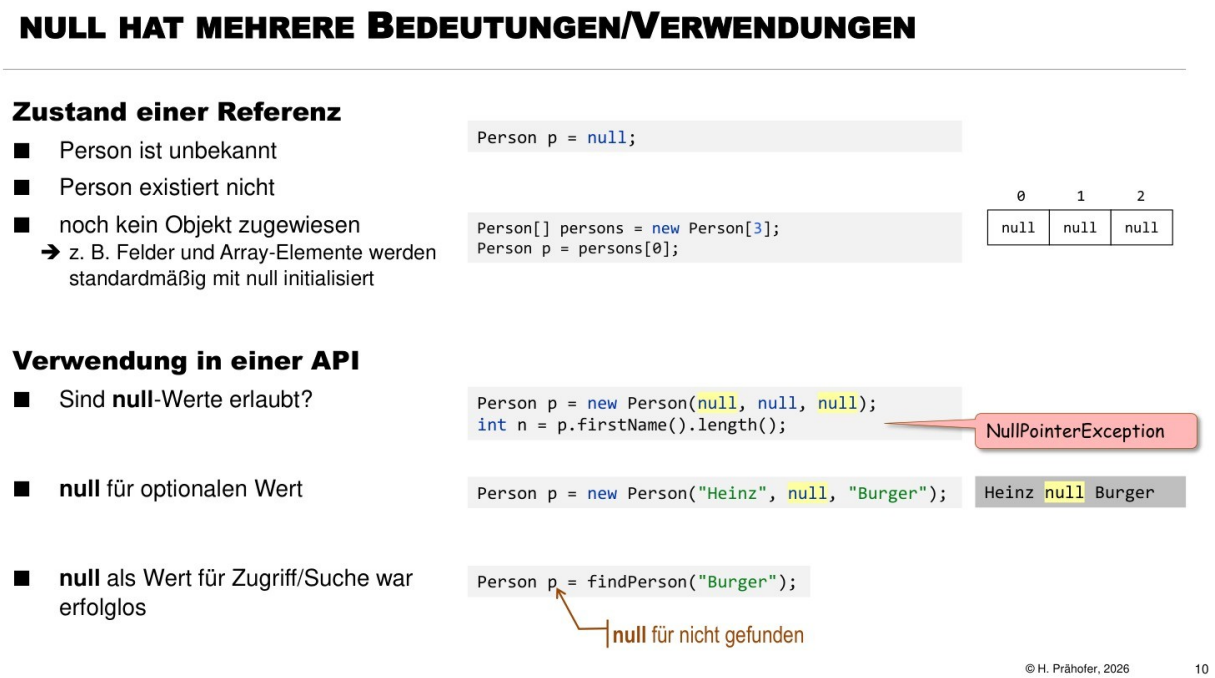
Der Aufruf `p.firstName()` ist typmäßig erlaubt, aber zur Laufzeit fehlerhaft, wenn `p == null` gilt. Dies ist eine zentrale Grenze der statischen Typprüfung in gewöhnlichem Java.



Originalabbildung: 01.Klassen.pdf, Folie 9.

K · Folie 10: NULL HAT MEHRERE BEDEUTUNGEN/VERWENDUNGEN

Die Beispiele unterscheiden fünf Bedeutungen eines fehlenden Wertes. Ohne dokumentierten API-Vertrag kann `null` nicht eindeutig als „nicht gefunden" oder „optional" interpretiert werden.



Originalabbildung: 01.Klassen.pdf, Folie 10.

K · Folie 11: UMGANG MIT NULL-WERTEN: PRÜFEN UND ERSETZEN

Die drei Alternativen entsprechen unterschiedlichen API-Strategien: prüfen, ungültige Eingaben sofort ablehnen oder einen definierten Standardwert wählen. Die Vorbedingungen des Ersatzwertes sind ebenfalls zu beachten.

Umgang mit null-Werten: Prüfen und Ersetzen

- Prüfen: expliziter Vergleich mit **null**

```
if (p != null) {
  System.out.format("%s %s %s%n",
                    p.firstName(), p.midName(), p.lastName());
}
```

- Zurückweisen: **Objects.requireNonNull**
  - wirft bei **null** eine **NullPointerException**

```
public Person(String firstName, String midName, String lastName) {
  Objects.requireNonNull(firstName, "firstName must not be null");
  Objects.requireNonNull(lastName, "lastName must not be null");
  this.firstName = firstName;
  this.midName = midName;
  this.lastName = lastName;
}
```

- Ersetzen: **Objects.requireNonNullElse**
  - alternativer Wert wenn **null**

```
public class Person {

  @Override
  public String toString() {
    return "%s %s %s".formatted(firstName,
        Objects.requireNonNullElse(midName, ""),
        lastName);
  }
}
```

© H. Prähofer, 2026 11

Originalabbildung: 01.Klassen.pdf, Folie 11.

K · Folie 12: UMGANG MIT NULL-WERTEN: NULL-SICHERE OPERATIONEN

`Objects.equals` schützt beide Vergleichsoperanden gegen `null`. `String.valueOf` ist nicht dasselbe wie `p.toString()` und liefert im Nullfall die vier Zeichen `null`.

Umgang mit null-Werten: null-sichere Operationen

- Null-sicherer Wertevergleich
  - **equals** wirft **NullPointerException**, wenn Empfängerobjekt **null**
  - Sicherer Verglich mit **Objects.equals**

NullPointerException, wenn p == null

```
if (p.equals(p2)) { … }
```

```
if (Objects.equals(p, p2)) { … }
```

```
public static boolean equals(Object a, Object b) {
    return (a == b) || (a != null && a.equals(b));
}
```

- Null-sichere String-Konvertierung:
  - **toString** wirft **NullPointerException**, wenn Empfängerobjekt **null**
  - Mit **String.valueOf** wird bei Wert **null** **"null"** ausgegeben

NullPointerException, wenn p == null

```
System.out.println(p.toString());
```

```
System.out.println(String.valueOf(p));
```

```
null
```

```
public static String valueOf(Object obj) {
    return (obj == null) ? "null" : obj.toString();
}
```

- Später:
  - Verwendung von **Optional** ➔ siehe Kapitel **4. Collections und Optional**
  - Annotationen wie **@NonNull** und **@Nullable** ➔ siehe Kapitel **8. Reflection und Annotationen**

© H. Prähofer, 2026 12

Originalabbildung: 01.Klassen.pdf, Folie 12.

1.2 Aufbau, Felder, Methoden und Konstruktoren

Eine Java-Klasse enthält zwei Gruppen von Mitgliedern. **Instanzmitglieder** sind auf ein konkretes Objekt bezogen: Jede Instanz besitzt eigene Instanzfelder, und bei einer Instanzmethode gibt es ein Empfängerobjekt. **Statische Mitglieder**, die mit `static` deklariert werden, gehören zur Klasse und werden

normalerweise über den Klassennamen verwendet. `Timer.maxHrs` bezeichnet beispielsweise ein einziges Klassenfeld; `t1.min` bezeichnet dagegen das Minutenfeld eines bestimmten `Timer`-Objekts. Ein Aufruf `Timer.switchMaxHrs()` benötigt kein `Timer`-Objekt. Ein Aufruf `t1.add(10)` benötigt hingegen das Empfängerobjekt `t1`.

Bei einem Instanzmethodenaufruf ist `this` die Referenz auf das aktuelle Empfängerobjekt. Der Compiler stellt diesen Bezug im Methodenrumpf implizit bereit, so dass `sec += s;` dasselbe Feld anspricht wie `this.sec += s;`, sofern keine gleichnamige lokale Variable oder kein Parameter den Feldnamen verdeckt. Bei `void add(int sec) { this.sec += sec; }` muss `this.sec` verwendet werden, damit das Feld von dem Parameter `sec` unterschieden wird. `this` ist weder ein neu angelegtes Objekt noch eine explizit vom Aufrufer übergebene Referenz, sondern bezeichnet die Instanz, deren Methode gerade ausgeführt wird.

Eine **Methodendeklaration** beschreibt Sichtbarkeit, optionale Modifikatoren, Rückgabetyp, Methodenname, formale Parameter und gegebenenfalls eine `throws`-Klausel. Java unterscheidet bei Überladung Methoden aufgrund ihres Namens und ihrer Parameterliste; bloß unterschiedliche Rückgabetypen oder deklarierte Exceptions reichen nicht aus. Beispiel: `write(String)` und `write(int)` sind verschiedene Überladungen. Bei `Object x = "Text"; write(x);` entscheidet der Compiler anhand des deklarierten Typs `Object`, welche Überladung aufgerufen wird, nicht aufgrund des tatsächlichen `String`-Objekts. Das ist eine wichtige Abgrenzung zum dynamischen Überschreiben, das erst später im Skript behandelt wird.

Ein **Vararg-Parameter** erlaubt beliebig viele Argumente: `void print(String sep, int... numbers)`. Er muss in der Parameterliste ganz am Ende stehen. Innerhalb der Methode ist `numbers` ein `int[]`, also ein Array. Auch `null` Argumente sind möglich; die Methode erhält dann normalerweise ein leeres Array. Die Typen beim Aufruf unterliegen denselben Überladungsregeln wie andere Argumente.

Ein **statischer Initialisierungsblock** ist ein mit `static { ... }` gekennzeichneter Klassenblock. Er wird bei der Klasseninitialisierung in der von Java festgelegten Reihenfolge zusammen mit statischen Feldinitialisierungen abgearbeitet. Er kann beispielsweise Konfigurationsdaten laden; Fehler während der Initialisierung können das Laden der Klasse verhindern. Entgegen der Formulierung auf einer Folie darf eine Java-Klasse **mehrere** statische Initialisierungsblöcke besitzen; sie werden in Textreihenfolge ausgeführt.

Konstruktoren initialisieren frisch erzeugte Objekte. Ihr Name entspricht dem Klassennamen, und sie besitzen keinen Rückgabetyp. Mehrere Konstruktoren können unterschiedliche Parameterlisten anbieten. Ein Konstruktor kann mit `this(...)` einen anderen Konstruktor derselben Klasse aufrufen und so doppelte Initialisierungslogik vermeiden. Vor den eigentlichen Objektoperationen muss der Initialisierungsweg abgeschlossen sein. Hat eine Klasse gar keinen explizit deklarierten Konstruktor, ergänzt der Compiler einen parameterlosen Default-Konstruktor; sobald irgendein Konstruktor selbst deklariert wurde, entfällt diese automatische Ergänzung. Ein Feldinitialisierer wie `boolean male = true` übersteuert dabei den anfänglichen Defaultwert `false`.

Bei der Objekterzeugung laufen vereinfacht diese Schritte ab: Zunächst wird Speicher für das Objekt bereitgestellt und alle Instanzfelder erhalten Standardwerte. Danach wird die Konstruktorverkettung über `super(...)` bis zur Oberklasse abgearbeitet; die Initialisierung der Instanzfelder und Initialisierungsblöcke einer Klasse erfolgt in ihrer festgelegten Reihenfolge, bevor ihr Konstruktorrumpf vollständig ausgeführt wird. Ein expliziter Aufruf `this(...)` verkettet zu einem anderen Konstruktor derselben Klasse und darf nicht zu einer zyklischen Konstruktorfolge führen.

Die Überladung wählt aus mehreren gleichnamigen Methoden anhand der statisch bekannten Argumenttypen und der anwendbaren Konvertierungen eine Signatur aus. Erst danach kann für die ausgewählte Instanzmethodensignatur die dynamische Bindung die konkrete überschreibende Implementierung zur Laufzeit bestimmen. Diese zwei Schritte dürfen nicht verwechselt werden. Methoden gleichen Namens unterscheiden sich nicht allein durch einen anderen Rückgabetyp oder eine `throws`-Klausel.

`static` gehört zur Klasse, `this` gehört zur aktuellen Instanz. Deshalb ist `this` in einer statischen Methode nicht verfügbar. Ein statischer Initialisierungsblock kann nicht auf ein noch gar nicht feststehendes Empfängerobjekt zugreifen. Java erlaubt mehrere statische Initialisierungsblöcke in einer Klasse; die Initialisierung verarbeitet sie in Quelltextreihenfolge zusammen mit statischen Feldinitialisierern. Ein `final`-Feld muss auf jedem zulässigen Initialisierungspfad genau einmal gesetzt werden.

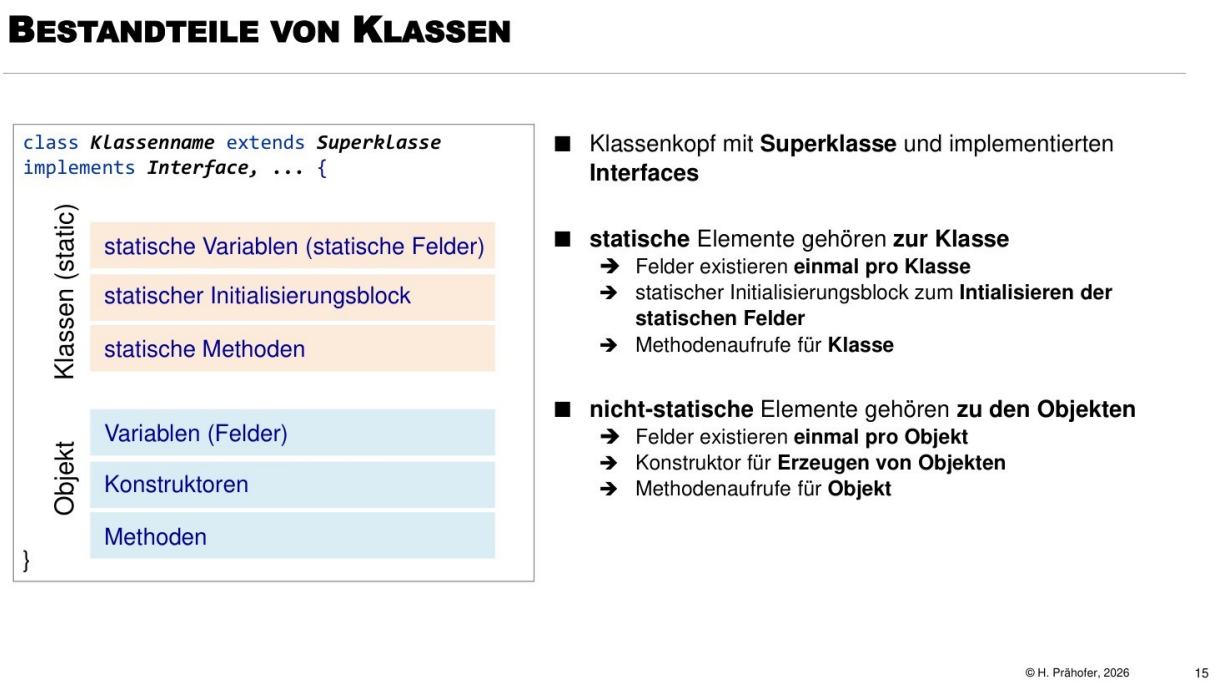
Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
class Counter {
 private int value;
 Counter() { this(0); }
 Counter(int start) { this.value = start; }
 int getValue() { return value; }
 void increment() { value++; }
}
```

Originalfolien mit fachlicher Einordnung

K · Folie 15: BESTANDTEILE VON KLASSEN

Der Klassenkopf trägt `extends` und `implements`; die linke Aufteilung zeigt statische und instanzbezogene Bereiche. Konstruktoren initialisieren Objekte und besitzen keinen Rückgabetyp.



Originalabbildung: 01.Klassen.pdf, Folie 15.

K · Folie 16: FELDER: STATISCHE VS. OBJEKTVARIABLEN

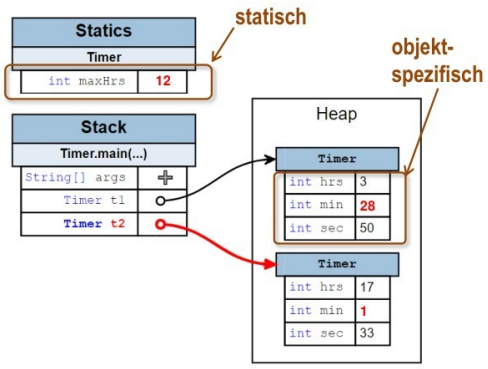
Der statische Wert `maxHrs` ist für beide `Timer` gemeinsam; `hrs`, `min` und `sec` sind pro Objekt getrennt. Die Darstellung verhindert die häufige Verwechslung von Klassenfeld und Instanzfeld.

Felder: Statische vs. Objektvariablen

```
class Timer {
  static int maxHrs = 24;

  int hrs;
  int min;
  int sec;
}
```

Statische Variable (Klassenfeld)
nur einmal pro Klasse vorhanden

Objektvariablen (Felder)
in jedem Objekt vorhanden



Statische Variablen: Zugriff über Klassennamen

```
Timer.maxHrs = 12;
```

```
Timer t1 = new Timer(3, 27, 50);
Timer t2 = new Timer(17, 0, 33);
```

Objektvariablen: Zugriff über Objektreferenz

```
t1.min = t1.min + 1;
t2.min = 1;
```

© H. Prähofer, 2026 16

Originalabbildung: 01.Klassen.pdf, Folie 16.

K · Folie 17: METHODEN: STATISCHE VS. OBJEKTMETHODEN

Die statische Operation wird über `Timer.switchMaxHrs()` aufgerufen. Die Instanzmethoden `clear` und `add` betreffen ausdrücklich die Objekte `t1` und `t2`.

Methoden: Statische vs. Objektmethoden

```
public class Timer {
  static int maxHrs = 24;
  ...
  static void switchMaxHrs() {
    maxHrs = 36 - maxHrs;
  }

  void clear() {...}
  void add(int s) {...}
  ...
}
```

Statische Methode (Klassenmethode)
- wird auf die Klasse selbst angewendet
- Aufruf über Klassennamen

```
Timer.switchMaxHrs();
```

Objektmethoden
- werden auf Objekte der Klasse angewendet
- Aufruf über Objektreferenzen

```
t1.add(10);   t2.add(5);
```

© H. Prähofer, 2026 17

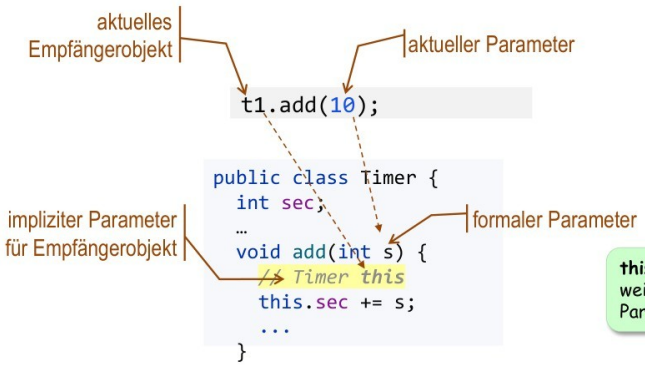
Originalabbildung: 01.Klassen.pdf, Folie 17.

K · Folie 18: OBJEKTMETHODEN: AUFRUF

`this` ist die Empfängerreferenz des gerade aufgerufenen Objekts. `this.sec` ist bei Namensgleichheit von Feld und Parameter erforderlich; ohne Konflikt kann die Qualifikation wegfallen.

OBJEKTMETHODEN: AUFRUF

- **Objektmethoden** werden für **Empfängerobjekt** aufgerufen
- **this** ist impliziter Parameter für Empfängerobjekt
  - ☐ Typ gleich der Klasse
  - ☐ Wert Referenz auf Empfängerobjekt



- **this** ist optional

```
public class Timer {
  int sec;
  …
  void add(int s) {
    sec += s;
    ...
  }
```

this nicht notwendig

- nur notwendig
  - ☐ wenn Zugriff nicht eindeutig

```
public class Timer {
  int sec;
  …
  void add(int sec) {
    this.sec += sec;
    ...
  }
```

this notwendig, weil Konflikt mit Parameter sec

  - ☐ wenn Referenz benötigt

© H. Prähofer, 2026 18

Originalabbildung: 01.Klassen.pdf, Folie 18.

K · Folie 19: METHODEN: METHODENSIGNATUR

Die Folie zeigt die syntaktischen Bestandteile einer Methodendeklaration. **Korrektur:** `private abstract void write(...)` ist in Java unzulässig, weil eine `private` Methode nicht abstrakt überschrieben werden kann. Die genannten Exceptions sind deklarierte Ausnahmen, nicht Teil der Java-Überladungssignatur.

METHODEN: METHODENSIGNATUR

***Sichtbarkeit* [static] [abstract | final] *Typ Name* (*Parameterliste*) [throws *geworfene Exceptions*]**

Beispiele:

```
private abstract void write(String text) throws IOException
public  final  int  readInt()  throws NumberFormatException, IOException
public String readString(String defaultValue)
```

Parameterliste

- keine Parameter — `int readInt()`
- mehrere Parameter — `void writeElems(List<String> elems, String separator)`
- *vararg*-Parameter(als letzter Parameter) — nächste Folie

© H. Prähofer, 2026 19

Originalabbildung: 01.Klassen.pdf, Folie 19.

K · Folie 20: METHODEN: VARARG-PARAMETER

Die drei Punkte gehören zum Typ des letzten Parameters, etwa `int... numbers`. Die Schleife arbeitet mit dem vom Aufruf bereitgestellten Array; ein Aufruf mit `null` Elementen ist möglich.

METHODEN: VARARG-PARAMETER

Letzter Parameter kann Vararg-Parameter sein

- mit Typ und drei Punkten deklariert: ***Type...***

```
void writeNumbers(String separator, int... numbers) {…}
```

- beim Aufruf können beliebig viele Werte angegeben werden

auch keine Werte erlaubt

beliebig viele Argumentwerte

```
writeNumbers(", ", 1, 2, 3, 4);
```

- Innerhalb Methode sind Werte in Array gespeichert

```
void writeNumbers(String separator, int... numbers) {
  for (int i = 0; i < numbers.length; i++) {
    System.out.print(numbers[i]);
    if (i < numbers.length-1) {
      System.out.print(separator);
    }
  }
```

int[] numbers

© H. Prähofer, 2026 20

Originalabbildung: 01.Klassen.pdf, Folie 20.

K · Folie 21: METHODEN: ÜBERLADEN VON METHODEN

Der statische Typ `Object` entscheidet für `write(obj)`, nicht der zur Laufzeit enthaltene `String`. Methoden allein durch einen anderen Rückgabewert oder eine andere `throws`-Liste zu unterscheiden ist nicht zulässig.

METHODEN: ÜBERLADEN VON METHODEN

Überladen von Methoden durch unterschiedliche Parameterlisten

Typen der Parameter und **Anzahl** der Parameter sind wesentlich

Auswahl erfolgt auf Basis der **statischen Typen** der **aktuellen Parameter**

```
void write(String text) {...}          ← write("This is a string");
void write(String text, int n) {...}   ← write("This is a string", 3);
void write(int x) {...}                ← write(1);
void write(Object o) {...}             ← Object obj = "This is a string";
                                         write(obj);
```

Auswahl auf Basis der **statischen** Typen

Anmerkung: Unterschiedliche Rückgabewerte und Exceptions führen zu keinem Überladen von Methoden und sind daher nicht erlaubt

```
void write(String text) {...}
boolean write(String text) { ... }

int readInt() {...}
int readInt() throws IOException
int readInt() throws FormatException
```

© H. Prähofer, 2026 21

Originalabbildung: 01.Klassen.pdf, Folie 21.

K · Folie 22: STATISCHER INITIALISIERUNGSBLOCK

Der Initialisierungsblock führt zusätzliche einmalige Vorbereitung bei der Klasseninitialisierung aus. **Korrektur:** Mehrere `static { ... }`-Blöcke sind erlaubt und werden in Quelltextreihenfolge verarbeitet; die gegenteilige Behauptung der Folie trifft nicht zu.

Statischer Initialisierungsblock

```
public class Timer {
  static int maxHrs = 24;

  static {
    // lies maxHrs aus Datei
    maxHrs = readFromFile("Timer.init");
    ...
  }
  ...
}
```

Statischer Initialisierungsblock
zum Initialisieren der Klassenvariablen

Statischer Initialisierungsblock

- mit Schlüsselwort **static** gekennzeichnet
- hat keinen Namen und keine Parameterliste
- wird automatisch aufgerufen, bevor die Klasse zum ersten Mal benutzt wird
- Jede Klasse hat höchstens 1 statischen Initialisierungsblock
- Anwendung bei:
  - Initialisieren von statischen Feldern
  - Aktionen, die beim Laden der Klasse stattfinden sollen

© H. Prähofer, 2026 22

Originalabbildung: 01.Klassen.pdf, Folie 22.

K · Folie 23: KONSTRUKTOREN

Der parameterlose Konstruktor delegiert mit `this(0,0,0)` an den dreiparametrigen. So erfolgt die Feldinitialisierung über einen gemeinsamen, leichter überprüfbaren Weg.

Konstruktoren

Spezielle Methoden zum Initialisieren von Objekten

- mit gleichem Namen wie die Klasse
- zum Setzen der Felder
- Konstruktoren können überladen werden
- überladene Konstruktoren können mit **this** einen anderen Konstruktor aufrufen

Häufig mehrere überladene Konstruktoren

Best Practice!

```
public class Timer {
  ...
  int hrs, min, sec;

  Timer(int h, int m, int s) {
    hrs = h; min = m; sec = s;
  }

  Timer() {
    this(0, 0, 0);
  }
  void clear() {...}
  void add(int s) {...}
}
```

Aufruf des überladenen Konstruktors

```
Timer t1 = new Timer(3, 25, 10);
```

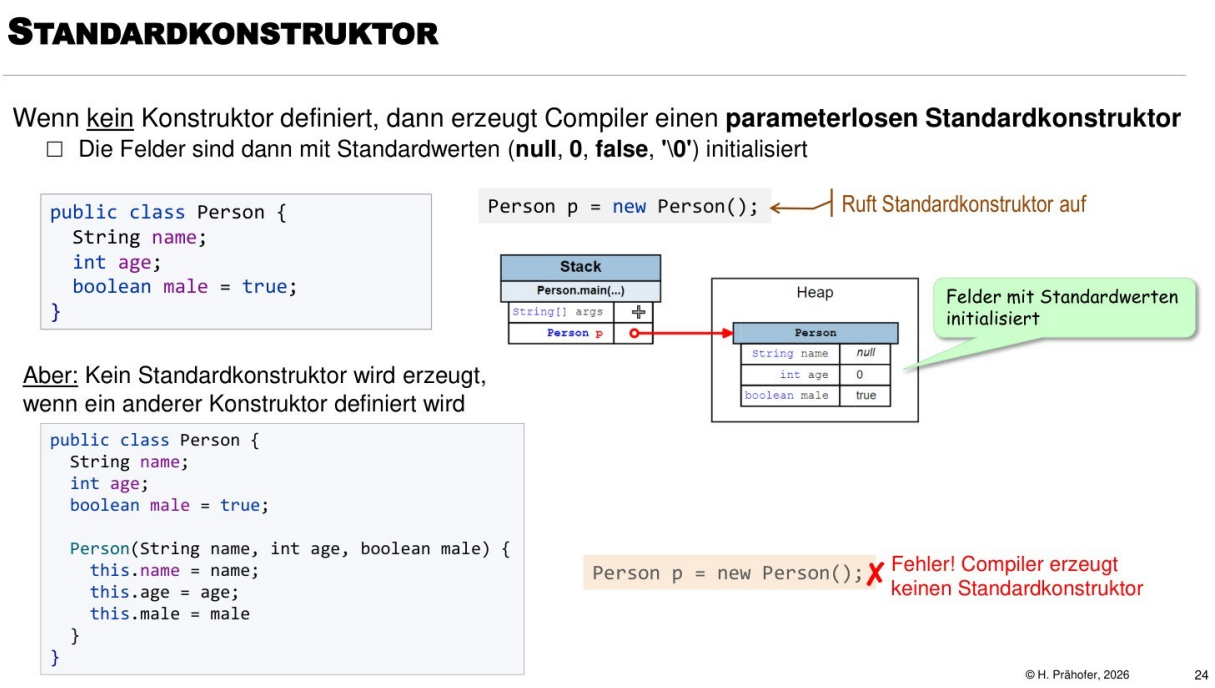
```
Timer t2 = new Timer();
```

© H. Prähofer, 2026 23

Originalabbildung: 01.Klassen.pdf, Folie 23.

K · Folie 24: STANDARDKONSTRUKTOR

Die automatische Konstruktorerzeugung entfällt, sobald die Klasse einen Konstruktor ausdrücklich deklariert. Das Feld `male = true` zeigt, dass Initialisierer den anfänglichen Standardwert überschreiben.



Originalabbildung: 01.Klassen.pdf, Folie 24.

1.3 Packages, qualifizierte Namen und Imports

Ein **Package** bildet einen Namensraum für Java-Typen und dient der logischen Gliederung eines Programms. Die Deklaration `package lesson02.hello;` steht am Anfang einer Java-Quelldatei, vor den Importdeklarationen und der Klassendeklaration. In einer typischen Projektstruktur liegt die Datei in einem entsprechenden Verzeichnis `lesson02/hello`. Unterpackages sind sprachlich selbstständige Packages: `lesson02` und `lesson02.hello` teilen nicht automatisch Package-Zugriffsrechte.

Der **vollqualifizierte Klassenname** verknüpft Package und Typname, beispielsweise `java.util.Date`. Dadurch können `java.util.Date` und `java.sql.Date` parallel existieren, obwohl beide den einfachen Namen `Date` tragen. `import java.util.Date;` ermöglicht die Verwendung des kurzen Namens `Date` innerhalb einer Quelldatei. Ein Import kopiert keine Klassen und lädt auch nicht notwendigerweise bereits bei der Übersetzung Objekte; er vereinfacht ausschließlich die Namensauflösung im Quelltext. `import inout.*;` macht die zugänglichen Typen des einen angegebenen Packages für einfache Namen verfügbar, aber nicht die Typen seiner Unterpackages.

`import static inout.In.readWord;` vereinfacht den Aufruf einer statischen Methode zu `readWord()`. Mit `import static inout.Out.*;` kann man zugängliche statische Mitglieder einer Klasse unqualifiziert verwenden. Solche Importe sollten lesbar bleiben, denn bei Namenskonflikten kann die Herkunft einer Methode unklar werden.

Die Standardbibliothek organisiert ihre Funktionen in bekannten Packages. `java.lang` enthält grundlegende Typen wie `String` und `Object` und steht implizit zur Verfügung. `java.util` stellt `Collections` und Hilfstypen bereit, `java.time` moderne Datums- und Zeittypen, `java.io` und `java.nio` Ein- und Ausgabe, `java.net` Netzwerkfunktionen, `java.util.concurrent` Werkzeuge für nebenläufige Programme, `java.util.function` funktionale Schnittstellen und `java.util.stream` die `Stream`-API. `java.lang.reflect` enthält Reflection-Typen; `java.math`, `java.text` und `java.util.regex` ergänzen

weitere Fachbereiche. Das Java Development Kit (JDK) ist die Entwicklungsumgebung einschließlich Compiler und Bibliotheken.

`package de.example.app;` legt den Paketnamen der Quelldatei fest. Im üblichen Projektlayout liegt die Datei dafür unter `de/example/app/`. Ein Paket ist kein Untertyp eines anderen Pakets: `de.example` und `de.example.app` sind verschiedene Sichtbarkeitsbereiche. Darum umfasst `package-private`-Sichtbarkeit weder automatisch Unterpakete noch beliebige Klassen mit ähnlichem Paketnamen.

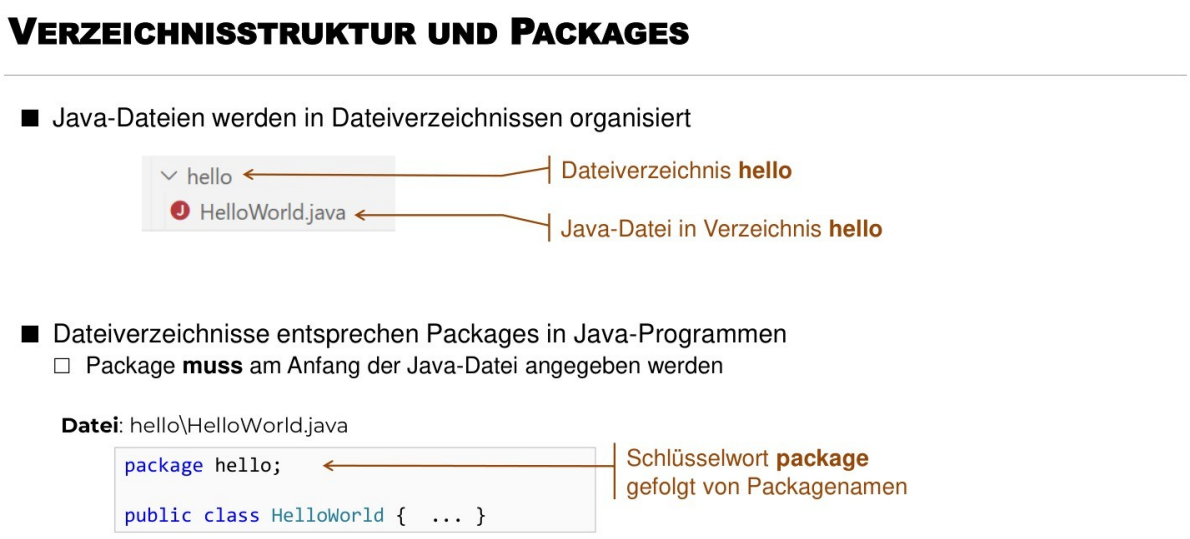
Ein gewöhnlicher `import` macht einen Typnamen im Quelltext kürzer. Die Klasse wird dadurch weder kopiert noch geladen, und der Bytecode erhält keine eigene Implementierung der importierten Klasse. Ein `import x.y.*;` umfasst direkt sichtbare Typen dieses Pakets, aber keine rekursiven Unterpakete. Ein `import static` lässt statische Mitglieder ohne vorangestellten Typnamen referenzieren, soweit keine Namenskonflikte entstehen.

Bei gleichen einfachen Namen kann ein vollständig qualifizierter Name notwendig sein: `java.util.Date` und `java.sql.Date` unterscheiden sich trotz des gemeinsamen Namens `Date`. `java.lang` ist implizit verfügbar. Der Import beeinflusst die Namensauflösung im Compiler, nicht die Zugriffserlaubnis: Ein `private`-Mitglied wird durch keinen Import von außen sichtbar.

Originalfolien mit fachlicher Einordnung

K · Folie 27: VERZEICHNISSTRUKTUR UND PACKAGES

Die Package-Deklaration gehört zum Quelldateianfang und passt üblicherweise zur relativen Verzeichnisstruktur. Eine Java-Klasse kann in mehreren Packages nicht denselben vollqualifizierten Namen besitzen.



Originalabbildung: 01.Klassen.pdf, Folie 27.

K · Folie 28: VERZEICHNISSTRUKTUR UND PACKAGES

Jeder Punkt in `lesson02.hello` entspricht einer weiteren Namensraumebene. Der Unterpackage-Name ist nicht gleichbedeutend mit gemeinsamem Package-Zugriff.

Verzeichnisstruktur und Packages

- Hierarchische Verzeichnisse und Packages
  - Dateiverzeichnisse mit Unterverzeichnissen
  - Package mit Unterpackages
    - ➔ mit Punkt getrennt

© H. Prähofer, 2026 28

Originalabbildung: 01.Klassen.pdf, Folie 28.

K · Folie 29: VOLLQUALIFIZIERTER KLASSENNAME

Zwei Klassen namens `Date` lassen sich über `java.util.Date` beziehungsweise `java.sql.Date` eindeutig unterscheiden. Dies ist ein Beispiel für die Funktion des vollqualifizierten Namens.

Vollqualifizierter Klassenname

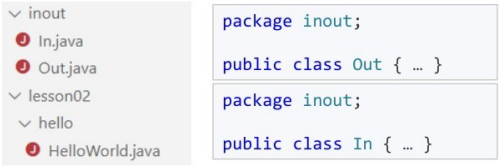
Vollqualifizierter Klassenname = Package plus Klassenname getrennt mit Punkt

➔ erlaubt Identifikation einer Klasse innerhalb des Gesamtsystems

```
lesson02.hello.HelloWorld
```

- Verwendung von Klassen aus anderen Packages über vollqualifizierten Namen

```
package lesson02.hello;
public class HelloWorld {
  public static void main(String[] args) {
    String name = inout.In.readWord();
    inout.Out.println("Hello " + name);
  }
}
```



- Ermöglicht Klassen mit gleichen Namen in unterschiedlichen Packages
  - z.B. **Date** in **java.util** und **Date** in **java.sql**

```
package java.util;

public class Date { … }
```

```
package java.sql;

public class Date { … }
```

```
public class Main {
  public static void main(String[] args) {
    java.util.Date  date;
    java.sql.Date  dbDate;
    …
```

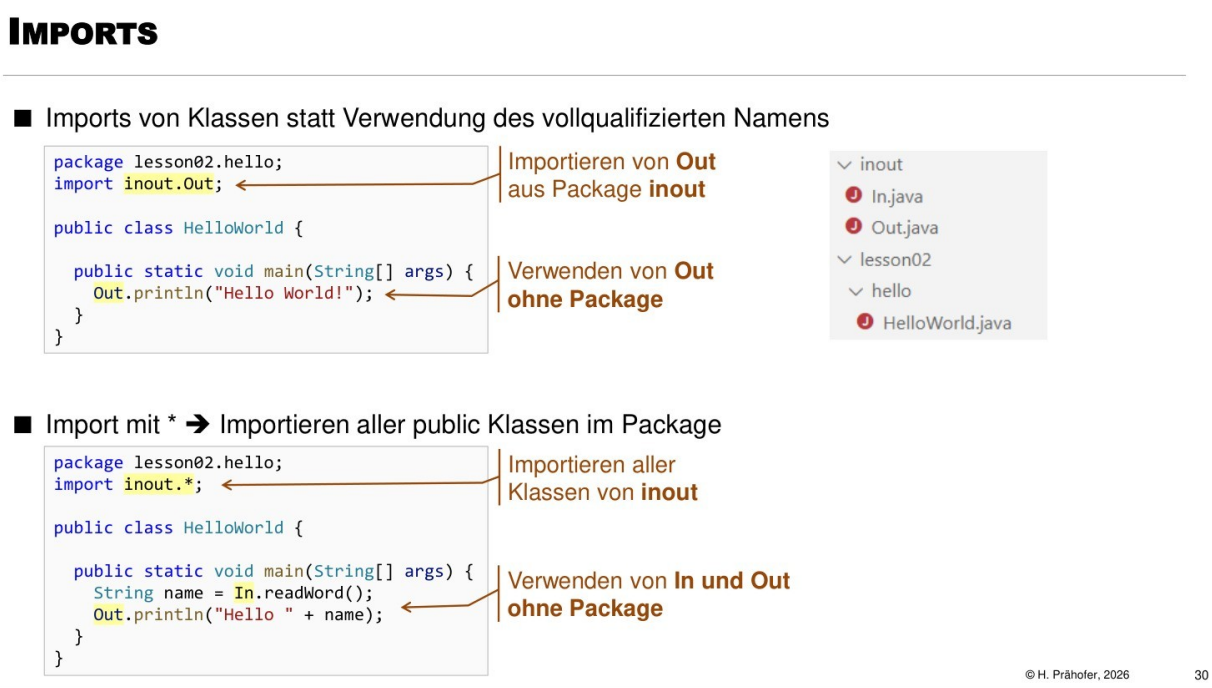
Unterscheidung der Date-Klassen über vollqualifizierten Namen

© H. Prähofer, 2026 29

Originalabbildung: 01.Klassen.pdf, Folie 29.

K · Folie 30: IMPORTS

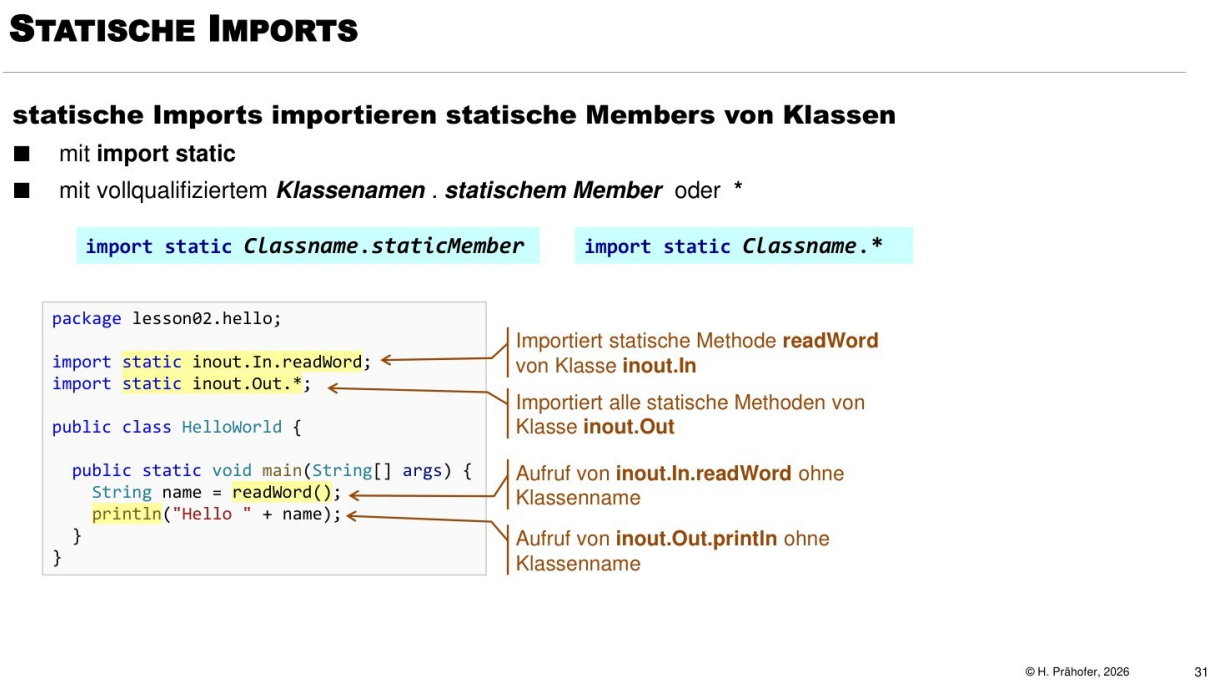
Normaler Import macht Namen aus einem anderen Package lesbarer. Ein Stern-Import umfasst zugängliche Typen im angegebenen Package, aber keine Unterpackages.



Originalabbildung: 01.Klassen.pdf, Folie 30.

K · Folie 31: STATISCHE IMPORTS

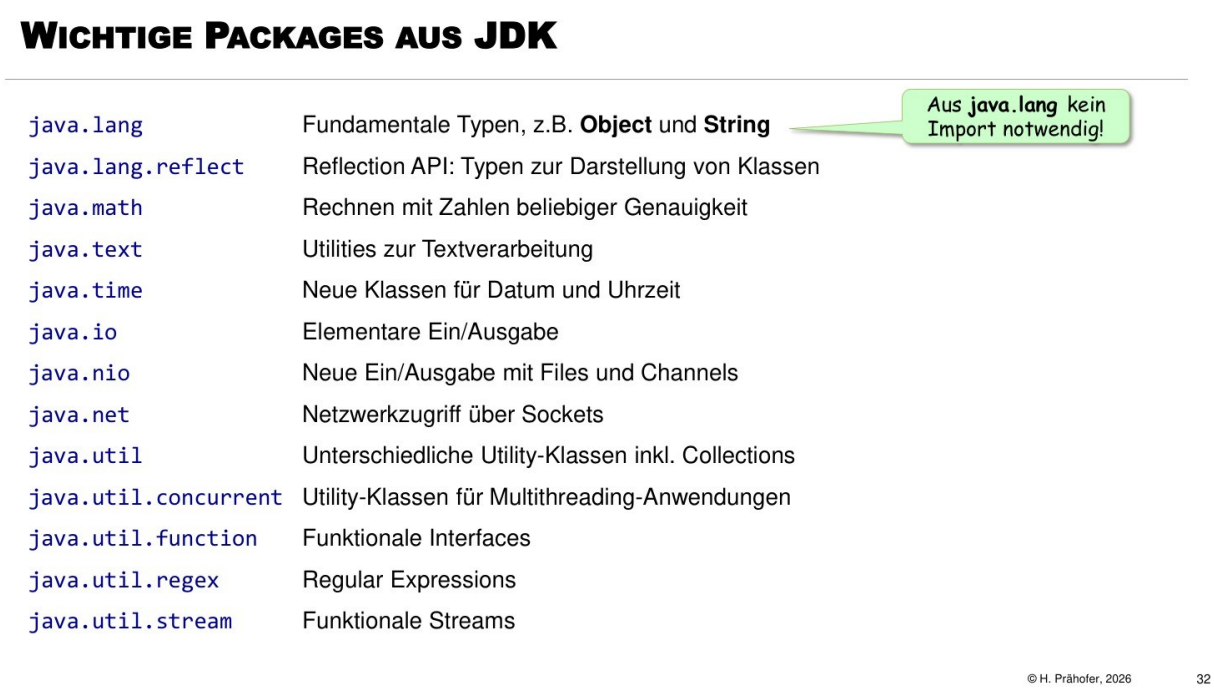
`import static` betrifft direkt statische Felder und Methoden. Die Ausdrücke `readWord()` und `println(...)` können dadurch ohne den Klassennamen geschrieben werden.



Originalabbildung: 01.Klassen.pdf, Folie 31.

K · Folie 32: WICHTIGE PACKAGES AUS JDK

Die Tabelle ordnet Aufgabenfeldern Standardbibliothekspackages zu und gibt damit eine Suchstrategie für APIs. `java.lang` benötigt im Normalfall keinen expliziten Import.



Originalabbildung: 01.Klassen.pdf, Folie 32.

1.4 Sichtbarkeit, Information Hiding und Properties

Information Hiding oder Datenkapselung bezeichnet das Entwurfsprinzip, die interne Darstellung eines Objekts vor beliebigen externen Änderungen zu schützen. Ziel ist nicht nur Geheimhaltung: Wenn ein `Timer` gültige Minutenwerte zwischen 0 und 59 garantieren soll, darf Fremdcode diesen Zustand nicht einfach mit `t.min = 122` verletzen. Stattdessen verarbeitet die öffentliche API Operationen, die gültige Zustandsübergänge erzwingen.

Java kennt vier relevante Zugriffsbereiche. `private` beschränkt den Zugriff auf die deklarierende `Klasse (mit sprachspezifischen Regeln für geschachtelte Klassen)`. Ohne Sichtbarkeitsmodifikator gilt **`package-private`**: zugreifbar innerhalb desselben Packages. `protected` erlaubt Zugriffe im gleichen Package und zusätzlich einen speziell geregelten Zugriff aus Unterklassen, auch packageübergreifend. `public` erlaubt allgemeine Zugriffe, soweit der umgebende Typ erreichbar ist. Die Zugriffe werden durch den Compiler geprüft. Besonders `protected` bedeutet außerhalb des Packages nicht uneingeschränkt „auf jedes Objekt des Oberklassentyps zugreifbar“; der Unterklassenzugriff ist weiter eingeschränkt.

Übliche Richtlinie: Datenfelder als `private`, nach außen nur die tatsächlich benötigten Methoden `public`. Interne Hilfsmethoden ebenfalls `private`, packageinterne Hilfstypen und -methoden gegebenenfalls `package-private`, und `protected` nur für bewusst unterstützte Erweiterungspunkte. Weniger sichtbare Details können später verändert werden, ohne Benutzer der Klasse umzuschreiben.

Ein **Getter** stellt einen lesenden Zugriff bereit, etwa `public int getMin() { return min; }`. Ein **Setter** verändert ein Feld, beispielsweise `public void setMin(int value) { if (value < 0 || value >= 60) throw new IllegalArgumentException(); min = value; }`. Nach der JavaBeans-Konvention heißen Methoden `getX()` und `setX(...)`; für boolesche Werte kann `isX()` verwendet werden. Es gibt auch den Stil `int min()`, der keine JavaBeans-Property im engeren Sinn ist.

Getter und Setter sind keineswegs bloß zusätzliche Syntax um öffentliche Felder. Sie erlauben nur lesende `Eigenschaften (kein Setter)`, berechnete Eigenschaften (Stunden aus einem einzigen Feld `totalSec`), Validierung vor einer Zustandsänderung und weitere kontrollierte Maßnahmen. Dennoch sollte man nicht automatisch für jedes `private` Feld einen frei zugänglichen Setter anbieten: Sonst wäre der Schutz des internen Zustands häufig nur formal. Auch ein Getter kann Kapselung verletzen, wenn er ein internes veränderbares Objekt oder Array ungefiltert zurückgibt. Das ist besonders bei `Collections` relevant.

Eine Klasseninvariante ist eine Zustandsbedingung, die für jedes nach außen verwendbare Objekt gelten soll. Für eine Uhr könnte gelten: Minuten und Sekunden liegen zwischen 0 und 59. Ein ungeprüfter Setter wie `setMin(120)` kann diese Bedingung verletzen. Kapselung verlangt deshalb mehr als nur `private`-Felder: Öffentliche Operationen müssen geeignete Vorbedingungen prüfen und nach jeder erfolgreichen Zustandsänderung wieder einen gültigen Zustand herstellen.

Die Java-Sichtbarkeit `protected` ist feiner als „sichtbar in jeder Unterklasse“: Im selben Paket ist der Zugriff auf geschützte Mitglieder wie Paketzugriff erlaubt. Bei Unterklassen in einem fremden Paket bestehen Einschränkungen, insbesondere beim qualifizierten Zugriff auf eine fremde Oberklasseninstanz. Üblicherweise erfolgt dort der Zugriff über `this`, `super` oder eine Referenz, deren statischer Typ die entsprechende Unterklasse ist. `private`-Elemente bleiben auch für Unterklassen unmittelbar unzugänglich.

Ein Getter ist nicht zwangsläufig eine unproblematische Sicht auf ein Feld. Liefert `getItems()` eine Referenz auf eine interne veränderliche Liste, können Aufrufer das Objekt unter Umgehung seiner Methoden verändern. Je nach API-Vertrag sind eine Kopie, eine nicht modifizierbare Sicht oder eine unveränderliche Repräsentation nötig. Ein Getter muss außerdem nicht ein gespeichertes Feld zurückgeben; eine berechnete Eigenschaft wie `getHours()` kann aus `totalSeconds` ermittelt werden.

Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
public void setMinutes(int minutes) {
 if (minutes < 0 || minutes >= 60) {
 throw new IllegalArgumentException("Minuten außerhalb 0..59");
 }
 this.minutes = minutes;
}
```

Originalfolien mit fachlicher Einordnung

K · Folie 35: SICHTBARKEITSATTRIBUTE

Der Zugriffsschutz nimmt von `private` über `package-private` und `protected` bis `public` zu. Die Feinheiten des `protected`-Zugriffs über Packagegrenzen hinweg werden bei Vererbung vertieft.

Sichtbarkeitsattribute

- **Sichtbarkeitsattribute** erlauben die Sichtbarkeit von Elementen von Klassen einzuschränken
- Sichtbarkeitsattribute unterstützen **Information Hiding**
  - d.h. **inneren Zustand** vor **unerlaubtem Zugriff zu schützen**

konsistenter Zustand kann garantiert werden

Sichtbarkeitsattribute in Java

`private`	Sichtbarkeit auf **Klasse** eingeschränkt
package-private (kein Attribut)	Sichtbar innerhalb des **gleichen Packages** (nicht in Unterpackages)
`protected`	Sichtbar innerhalb des **gleichen Packages** und in den **Unterklassen** (Unterklassen auch aus anderen Packages)
`public`	allgemein sichtbar

Sichtbarkeit erweitert

© H. Prähofer, 2026 35

Originalabbildung: 01.Klassen.pdf, Folie 35.

K · Folie 36: TIMER MIT INFORMATION HIDING

Die `private`-Felder verhindern den unmittelbaren Schreibzugriff, der den `Timer` ungültig machen könnte. **Fehlerhinweis:** Die gezeigte `add(int sec)`-Variante verwendet beim Übertrag teils den Parameter `sec` statt `this.sec`; bei bereits vorhandenen Sekunden liefert sie falsche Ergebnisse. Ein sauberer Entwurf addiert auf den Gesamtzustand und normalisiert danach.

Timer mit Information Hiding

```
public class Timer {
  private int hrs;
  private int min;
  private int sec;

  public Timer(int h, int m, int s) {
    hrs = h; min = m; sec = s;
  }
  public void clear() {
    hrs = 0;
    min = 0;
    sec = 0;
  }
  public void add(int sec) {
    this.sec += sec;
    if (this.sec > 59) { min += sec / 60; this.sec = sec % 60; }
    if (min > 59) { hrs += min / 60; this.min = min % 60; }
  }
  @Override
  public String toString() {
    return String.format("%d:%02d:%02d", hrs, min, sec);
  }
}
```

private: Felder
➔ Man kann von außen nicht auf Datenfelder zugreifen

public: Konstruktor und Methoden
add, **clear**, **toString**

```
Timer t = new Timer(0, 1, 30);
t.add(40);
Out.println(t.toString());
t.clear();
```

kein direkter Zugriff auf Felder
➔ interner Zustand abgesichert

```
t.min = 122;
Out.println(t.hrs);
```

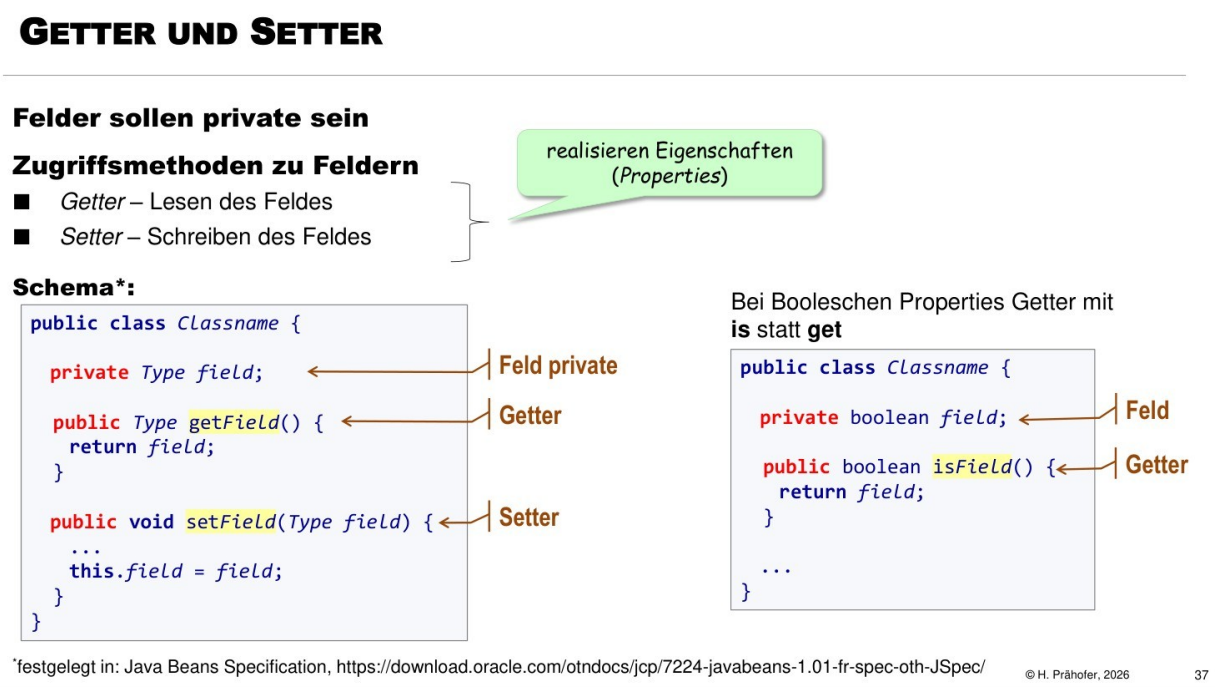
✗ Fehler! Kein Zugriff

© H. Prähofer, 2026 36

Originalabbildung: 01.Klassen.pdf, Folie 36.

K · Folie 37: GETTER UND SETTER

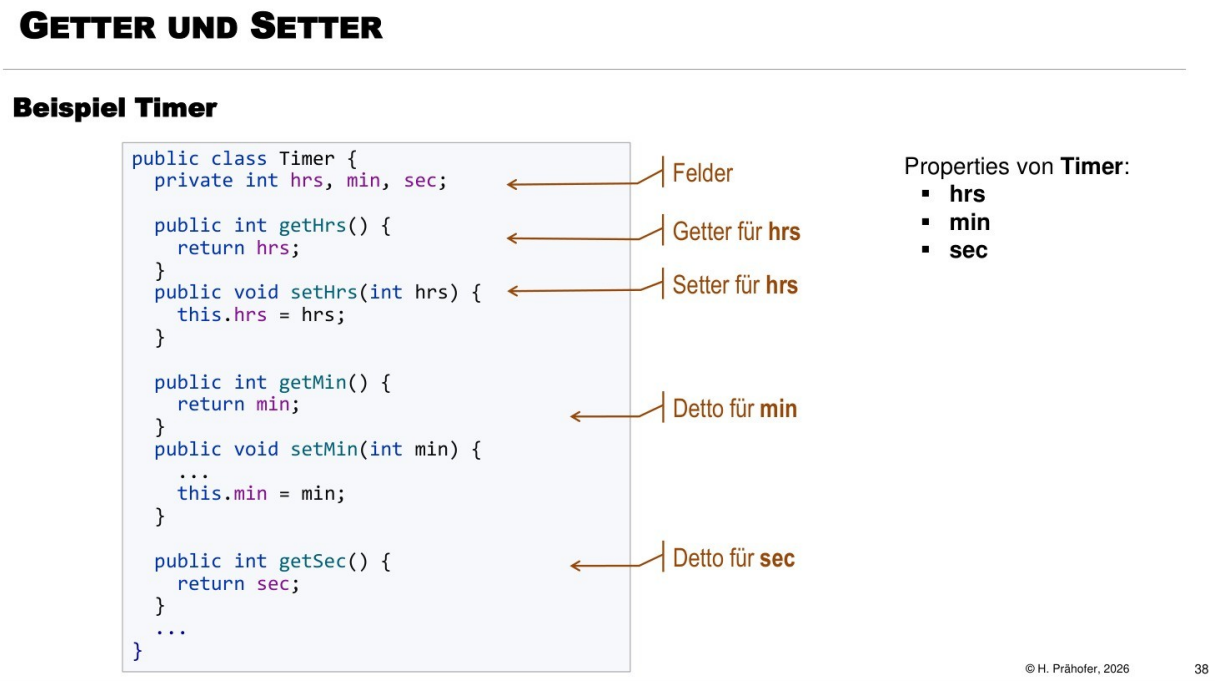
Die JavaBeans-Konvention unterscheidet Getter und Setter über `get`, `set` und bei booleschen Eigenschaften `is`. Freigeben sollte man nur die für die fachliche API notwendigen Zugriffe.



Originalabbildung: 01.Klassen.pdf, Folie 37.

K · Folie 38: GETTER UND SETTER

Die drei Properties des Timers können gelesen beziehungsweise verändert werden. Ein Setter für Minuten sollte den gültigen Bereich kontrollieren, statt beliebige `int`-Werte zu akzeptieren.



Originalabbildung: 01.Klassen.pdf, Folie 38.

K · Folie 39: GETTER UND SETTER: VORTEILE

Die drei Vorteile sind getrennt: schreibgeschützte Sicht, berechnete Property und kontrollierte Zusatzaktion. Später ist außerdem dynamisches Überschreiben dieser Zugriffsmethoden möglich.

Getter und Setter: Vorteile

- read-only-Zugriff, d.h. nur *Getter*, keine *Setter*

```
public class Timer {
  private int hrs, min, sec;
  public int getHrs() { return hrs; }
  public int getMin() { return min; }
  public int getSec() { return sec; }
  ...
}
```

keine Setter definiert

- Properties können auch berechnet werden, z.B. Berechnung von hrs, min, sec aus Feld **totalSec**

```
public class Timer {
  private int totalSec;
  public int getHrs() { return totalSec / 3600; }
  ...
}
```

Berechnung der Properties **hrs**, **min**, **sec** aus **totalSec**

- Zusatzaktionen beim Zugriff möglich (z.B. Logging, Plausibilitätsprüfungen, ...)

```
public class Timer {
  ...
  public void setMin(int min) {
    if (min < 0 || min >= 60) throw new IllegalArgumentException();
    this.min = min;
  }
}
```

Prüfung vor Setzen

- get-/set-Methoden können in Unterklassen überschrieben werden (Felder nicht)

Siehe Kapitel 2. Vererbung

© H. Prähofer, 2026 39

Originalabbildung: 01.Klassen.pdf, Folie 39.

K · Folie 40: GETTER: ALTERNATIVE NAMENSGEBUNG

Der alternative Getter-Stil `min()` ist ein normaler Methodenaufruf und nicht das direkte Lesen eines Feldes. Er entspricht aber nicht automatisch dem JavaBeans-Namensmuster `getMin()`.

Getter: Alternative Namensgebung

Name von Getter gleich Feldnamen

Schema

```
public class Classname {

  private Type field;

  public Type field() {
    return field;
  }

}
```

Feld

Getter

Beachte: Getter ist Methode

Beispiel Timer

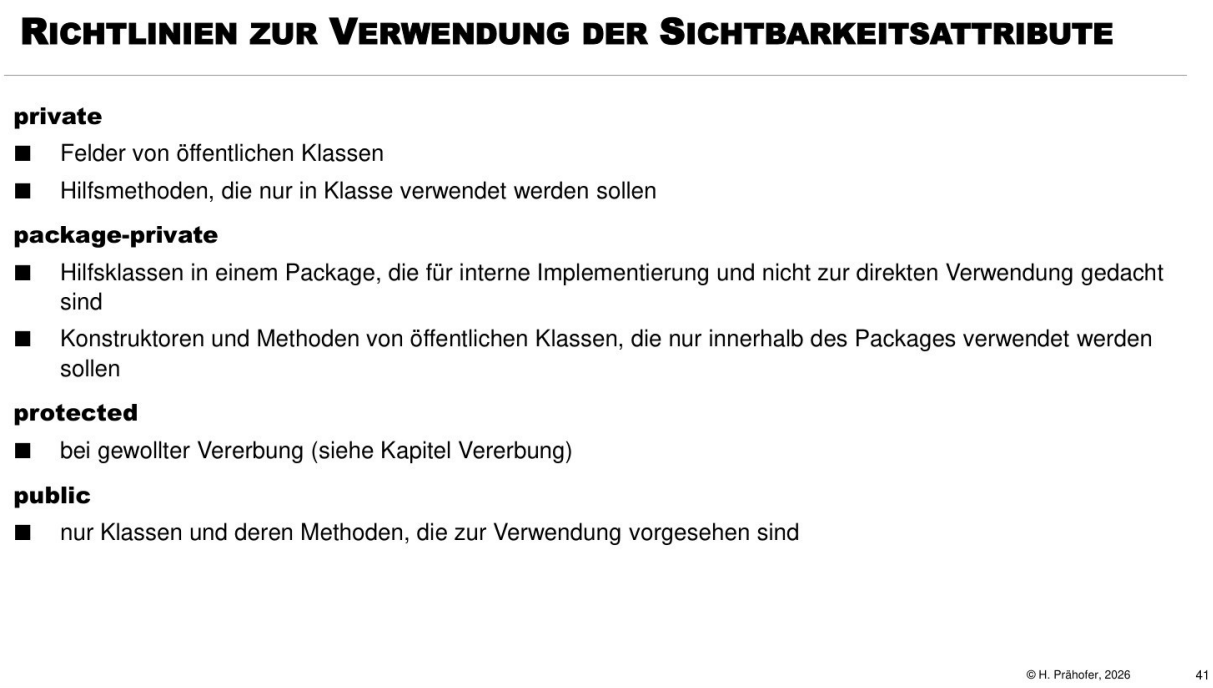
```
public class Timer {
  private int hrs, min, sec;

  public int hrs() {
    return hrs;
  }

  public int min() {
    return min;
  }

  public int sec() {
    return sec;
  }
  ...
}
```

© H. Prähofer, 2026 40

Originalabbildung: 01.Klassen.pdf, Folie 40.

K · Folie 41: RICHTLINIEN ZUR VERWENDUNG DER SICHTBARKEITSATTRIBUTE

Die Folie ist als praxisorientierte Entwurfsrichtlinie zu lesen: interne Details eng sichtbar halten und öffentliche Methoden als bewussten API-Vertrag gestalten.



Originalabbildung: 01.Klassen.pdf, Folie 41.

1.5 final, unveränderliche Objekte und Wertegleichheit

Der Modifikator `final` verbietet nach der Initialisierung eine weitere Zuweisung an die betreffende Variable. Bei `static final` entstehen typischerweise Klassenkonstanten wie `Integer.MAX_VALUE`; die Konvention schreibt solche Konstantennamen in Großbuchstaben mit Unterstrichen. `final`-Instanzfelder müssen in allen Konstruktorpfaden initialisiert werden, etwa direkt bei der Deklaration oder im Konstruktor. Bei einer Referenz `private final Address address;` ist allerdings nur die **Referenz** unveränderlich: Das verwiesene `Address`-Objekt könnte weiterhin veränderbar sein.

Eine **immutable** oder unveränderliche Datenklasse gibt nach ihrer Konstruktion keinen veränderbaren Zustand mehr preis. Bei einer Bruchklasse `Fraction` mit Zähler `num` und Nenner `denom` lassen sich beide als `private` finale Felder speichern. Ein Konstruktor sollte die Invariante `denom != 0` prüfen. Operationen wie Addition und Multiplikation verändern dann nicht die beteiligten Brüche, sondern liefern neue Objekte. Mathematisch gilt für zwei Brüche `a/b` und `c/d`: Multiplikation `(a*c)/(b*d)` und Addition `(a*d+c*b)/(b*d)`, jeweils unter der Voraussetzung `b != 0` und `d != 0`. Bei `int` kann das Zwischenprodukt überlaufen; produktive Implementierungen sollten Wertebereiche, Kürzung und gegebenenfalls `BigInteger` beachten.

Bei Wertobjekten wird **Gleichheit des Inhalts** benötigt, nicht bloß Identität. Die von `Object` geerbte Standardimplementierung von `equals` entspricht im Wesentlichen einem Identitätsvergleich. Eine eigene `equals(Object)`-Implementierung prüft üblicherweise zuerst `this == o`, dann Null und Typkompatibilität, und vergleicht anschließend die für die fachliche Identität relevanten Felder. Bei primitiven Feldern ist `==` passend; bei Referenzfeldern verwendet man `equals` beziehungsweise `null`-sicher `Objects.equals`. Für eine Klasse mit Vererbung kann die Entscheidung zwischen `getClass()` und `instanceof` erheblichen Einfluss auf Symmetrie und Gleichheitsvertrag haben.

`equals` und `hashCode` müssen zusammenpassen. Aus `a.equals(b) == true` folgt zwingend `a.hashCode() == b.hashCode()`. Die Umkehrung ist nicht garantiert: Verschiedene Objekte dürfen denselben Hashwert besitzen. Mit `Objects.hash(num, denom)` kann ein hashCode aus denselben Feldern wie `equals` gebildet werden. Wird ein Objekt nach dem Einfügen in eine Hash-basierte Collection so verändert, dass sich sein Hashwert ändert, kann es dort nicht mehr korrekt gefunden werden. Gerade deshalb sind unveränderliche Wertobjekte gute Schlüssel in Maps und Elemente von Sets.

Im Vergleich demonstrieren `Timer` (veränderlicher Zähler, `clear`/`add` verändern Zustand) und `Time` (unveränderlicher Zeitpunkt, `after` erzeugt eine neue Instanz) zwei bewusst unterschiedliche Semantiken. Die Vorteile unveränderlicher Klassen liegen in fehlenden unbeabsichtigten Mutationen über geteilte Referenzen, einfacherer paralleler Verwendung und leichter verständlichen Zustandsübergängen. Unveränderlichkeit ist jedoch nicht identisch mit `final` allein: Auch die transitiv erreichbaren Datenstrukturen müssen berücksichtigt werden.

`final` hat je nach Deklaration eine andere Wirkung. Bei einer lokalen Variablen oder einem Feld verhindert es die erneute Zuweisung dieser Variablen; es ändert nicht die Änderbarkeit eines referenzierten Objekts. `final List<String> items = new ArrayList<>();` verbietet `items = other;`, aber nicht `items.add("A");`. Bei Methoden verhindert `final` das Überschreiben und bei Klassen die Bildung von Unterklassen.

Eine Klasse ist erst dann tatsächlich unveränderlich, wenn nach dem Konstruktor kein von außen beobachtbarer Zustand mehr geändert werden kann. Dazu gehören Prüfungen in Konstruktoren, keine mutierenden Methoden und die sichere Behandlung veränderlicher referenzierter Objekte. Für `Address` als veränderlichen Feldtyp ist eine defensive Kopie beim Übernehmen und gegebenenfalls beim Herausgeben nötig. Ein `record` garantiert nur die `final`-Eigenschaft seiner Komponentenreferenzen, nicht die tiefe Unveränderlichkeit ihrer Inhalte.

Die Methoden `equals` und `hashCode` erfüllen einen gemeinsamen Vertrag: Sind `a.equals(b)` und damit die beiden Objekte gleich, müssen `a.hashCode()` und `b.hashCode()` denselben Wert liefern. Die Umkehrung gilt nicht. Die Methode `toString()` ist für eine verständliche textuelle Darstellung gedacht, nicht für die Festlegung der Objektgleichheit. Besonders bei Verwendung als Schlüssel in Hash-`Collections` dürfen die für `equals` relevanten Werte nicht unkontrolliert während der Speicherung verändert werden.

Originalfolien mit fachlicher Einordnung

K · Folie 44: FINAL: STATISCHE FELDER

`static final` ergibt eine Klassenkonstante. Die Folie zeigt mit `0x7fffffff` die hexadezimale Darstellung des maximalen positiven `int`-Wertes 2147483647.

FINAL: STATISCHE FELDER

final Variablen sind unveränderbar, d.h., können nur einmal gesetzt werden

final statische Felder

- sind **Programmkonstanten**
- werden mit **Großbuchstaben** geschrieben, mit Underscore als Trennzeichen
- Wert muss **direkt bei Deklaration** oder im **statischen Initialisierungsblock** zugewiesen werden

```
public final class Integer ... {

  public static final int MAX_VALUE = 0x7fffffff;

  ...
}
```

© H. Prähofer, 2026 44

Originalabbildung: 01.Klassen.pdf, Folie 44.

K · Folie 45: FINAL: OBJEKTFELDER

Ein Bruch besitzt finale Zähler- und Nennerfelder, deren Werte nach der Konstruktion feststehen. Für eine sinnvolle Bruchklasse muss `denom != 0` gelten; `final` garantiert diese fachliche Bedingung nicht selbst.

FINAL: OBJEKTFELDER

final Objektvariablen

„immutable"

- müssen **im Konstruktor gesetzt** werden (oder direkt bei Deklaration)
- können **dann nicht mehr verändert** werden

Beispiel: Klasse **Fraction** für Bruchzahlen

```
public class Fraction {
    private final int num;
    private final int denom;

    public Fraction(int num, int denom) {
        if (denom == 0) throw new IllegalArgumentException();
        this.num = num;
        this.denom = denom;
    }
    public int num() {
        return num;
    }
    public int denom() {
        return denom;
    }
   ...
}
```

Werte der final Variablen müssen im Konstruktor "fixiert" werden!

nur Getter (Setter nicht möglich)

```
Fraction f1 = new Fraction(1, 2);
Out.println(f1.num() + "/" + f1.denom());
```

1/2

```
Fraction f2 = new Fraction(2, 3);
Out.println(f2.num() + "/" + f2.denom());
```

2/3

© H. Prähofer, 2026 45

Originalabbildung: 01.Klassen.pdf, Folie 45.

K · Folie 46: UNVERÄNDERLICHE DATENKLASSEN

Unveränderliche Datenklassen dienen als Wertobjekte. Objekte mit `final`-Referenzfeldern sind nicht automatisch tief unveränderlich, wenn sie auf mutierbare Unterobjekte zeigen.

Unveränderliche Datenklassen

- Objekte mit nur **unveränderlichen** Feldern
- Sollen **equals** und **hashCode** für Wertevergleich implementieren

```
public class Fraction {
  private final int num, denom;

  public Fraction(int num, int denom) {
    if (denom == 0) throw new IllegalArgumentException();
    this.num = num; this.denom = denom;
  }

  public int num() { return num; }
  public int denom() { return denom; }

  @Override
  public boolean equals(Object o) { ...}

  @Override
  public int hashCode() { ... }

  @Override
  public String toString() { return num + "/" + denom; }
}
```

Vorteile unveränderlicher Objekte:
- verhindern Seiteneffekte
- besitzen ähnliche Eigenschaften wie Standarddatentypen

© H. Prähofer, 2026 46

Originalabbildung: 01.Klassen.pdf, Folie 46.

K · Folie 47: IMPLEMENTIERUNG EQUALS UND HASHCODE

Die `equals(Object)`-Vorlage prüft Identität, Null und Klasse und vergleicht anschließend relevante Felder. `getClass()` erfordert exakte Laufzeittypgleichheit; dies ist bewusst eine der möglichen Entwurfsvarianten.

Implementierung equals und Hashcode

equals für Wertevergleich

- Vergleich der internen Feldwerte auf Gleichheit

```
public class Fraction {
  private final int num;
  private final int denom;
  ...
  @Override
  public boolean equals(Object o) {
    if (this == o) {
      return true;
    }
    if (o == null || getClass() != o.getClass()) {
      return false;
    }
    Fraction fraction = (Fraction) o;
    return num == fraction.num &&
           denom == fraction.denom;
  }
}
```

Schema zur Implementierung von equals

equals aus **Object** überschreiben
immer mit Parameter von Typ **Object** — **nie** anders

zuerst auf **Referenzgleichheit** prüfen
referenzgleiche Objekte sind immer gleich !

dann Parameter auf **null** und **Klassen vergleichen**
wenn **null** oder Klassen nicht ident dann nicht gleich

dann **Typecast** des Arguments

dann **ausgewählte Felder** vergleichen
für Referenztypen wieder mit **equals**

© H. Prähofer, 2026 47

Originalabbildung: 01.Klassen.pdf, Folie 47.

K · Folie 48: IMPLEMENTIERUNG EQUALS UND HASHCODE

Hashcodes sind nicht eindeutige IDs. Nur die Richtung „equals impliziert gleichen hashCode“ ist zwingend; gleiche Hashcodes implizieren ausdrücklich keine Wertegleichheit.

Implementierung equals und Hashcode

Bedingungen zur Implementierung hashCode

- **equals** und **hashCode** immer zusammen
- immer so, dass Folgendes gilt:

$$a.equals(b) == true \Rightarrow a.hashCode() == b.hashCode()$$

umgekehrt nicht unbedingt notwendig

d.h., sind zwei Objekte **a** und **b equals**, so müssen auch ihre **hashCode**s gleich sein

Schema zur Implementierung von hashCode

- für **hashCode** am besten gleiche Felder wie bei **equals** verwenden
- am besten **Objects.hash** verwenden

Näheres bei Abschnitt zu Collections

```
import java.util.Objects;

public class Fraction {
  ...
@Override
  public int hashCode() {
    return Objects.hash(num, denom);
  }
}
```

© H. Prähofer, 2026 48

Originalabbildung: 01.Klassen.pdf, Folie 48.

K · Folie 49: IMPLEMENTIERUNG EQUALS UND HASHCODE

Beim Wertevergleich von Referenzfeldern braucht man deren `equals`-Methoden. Falls Vornamen oder Nachnamen `null` sein dürfen, muss auch die Implementierung `null`-sicher gestaltet sein.

Implementierung equals und Hashcode

Beispiel Person

```
public class Person {
  private final String firstName;
  private final String lastName;
  private final int age;
  ...
  @Override
  public boolean equals(Object o) {
    if (this == o) {
      return true;
    }
    if (o == null || getClass() != o.getClass()) {
      return false;
    }
    Person person = (Person) o;
    return firstName.equals(person.firstName) &&
           lastName.equals(person.lastName) &&
           age == person.age;
  }
  @Override
  public int hashCode() {
    return Objects.hash(firstName, lastName, age);
  }
}
```

Vergleich von Referenztypen mit **equals**

© H. Prähofer, 2026 49

Originalabbildung: 01.Klassen.pdf, Folie 49.

K · Folie 50: UNVERÄNDERLICHE DATENOBJEKTE: OPERATIONEN

`mult` und `add` erzeugen neue `Fraction`-Objekte. **Zusatz:** Ganzzahlige Zwischenrechnungen können überlaufen, und Nenner müssen vor der Operation ungleich `null` sein.

Unveränderliche Datenobjekte: Operationen

- Operationen mit unveränderlichen Objekten erzeugen immer neue Objekte

```
public class Fraction {
  private final int num, denom;

  public Fraction(int num, int denom) {
    if (denom == 0) throw new IllegalArgumentException();
    this.num = num; this.denom = denom;
  }
  public int num() { return num; }
  public int denom() { return denom; }

  public Fraction mult(Fraction f) {
    return new Fraction(this.num * f.num, this.denom * f.denom);
  }

  public Fraction add(Fraction f) {
    return new Fraction(this. num * f.denom + f.num * this. denom, this. denom * f.denom);
  }

  ...
}
```

Objekte selbst werden nie verändert

Operationen erzeugen und liefern neue Objekte

© H. Prähofer, 2026 50

Originalabbildung: 01.Klassen.pdf, Folie 50.

K · Folie 51: VERÄNDERLICHE VS. UNVERÄNDERLICHE KLASSEN: BEISPIEL TIMER UND TIME

`Timer` ändert seinen Zustand, `Time` liefert bei einer Operation eine neue Instanz. Diese semantische Entscheidung beeinflusst Gleichheit, Nebenwirkungen und API-Gestaltung.

Veränderliche vs. unveränderliche Klassen: Beispiel Timer und Time

Timer erlaubt zählen → veränderlich

- Methoden verändern das Objekt
- kein **equals** und **hashCode**

```
public class Timer {
  private int hrs, min, sec;

  public Timer(int hrs, int min, int sec) { ... }

  public void clear() {
    hrs = 0; min = 0; sec = 0;
  }
  public void add(int s) {
    this.sec += s;
    if (sec > 59) {
      min += sec / 60;
      sec = sec % 60;
    }
    if (min > 59) {
      hrs += min / 60;
      min = min % 60;
    }
  }
}
```

Methoden ändern Objekt

Time stellt Zeitpunkt dar → unveränderlich

- Methoden liefern neue Objekte
- **equals** und **hashCode** implementiert

```
public class Time {
  private final int hrs, min, sec;

  public Time(int hrs, int min, int sec) { ... }

  ...
  @Override
  public boolean equals(Object o) { ... }

  @Override
  public int hashCode() { ... }

  public Time after(int sec) {
    int totalSec = ...;
    return new Time(totalSec / 3600,
                    (totalSec % 3600) / 60,
                    totalSec % 60);
  }
}
```

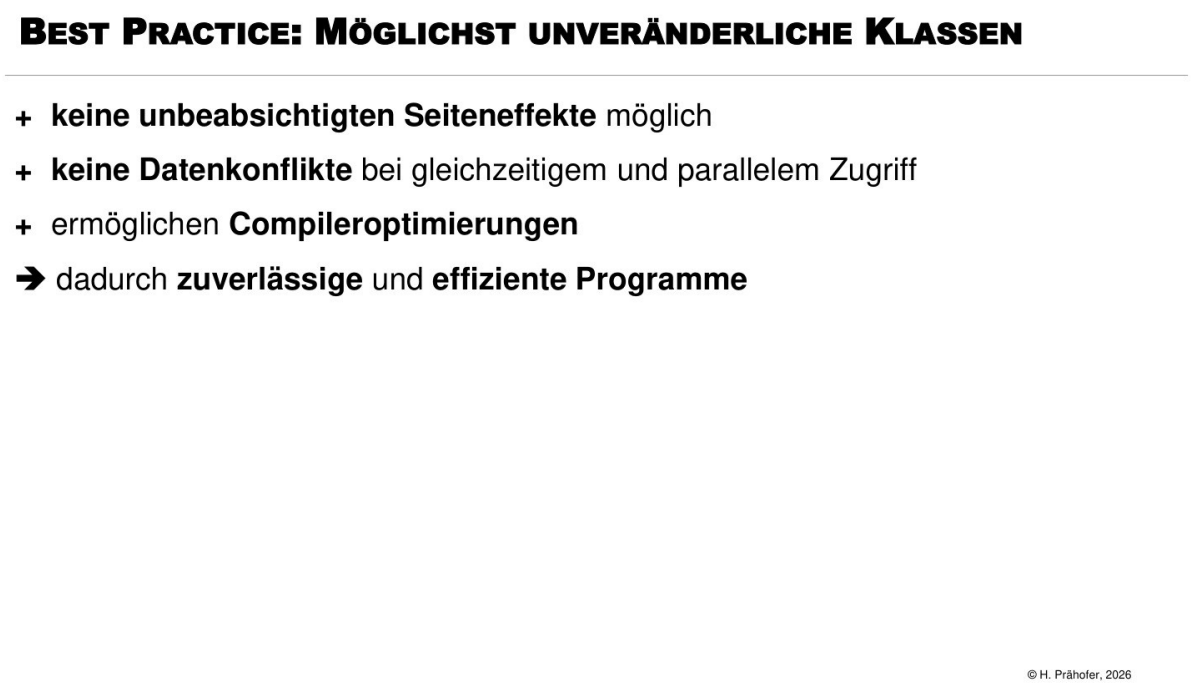
Methoden erzeugen und liefern neue Objekte

© H. Prähofer, 2026 51

Originalabbildung: 01.Klassen.pdf, Folie 51.

K · Folie 52: BEST PRACTICE: MÖGLICHST UNVERÄNDERLICHE KLASSEN

Fehlende veränderbare gemeinsame Zustände erleichtern Parallelverarbeitung und reduzieren Fehlerquellen. Dennoch können das Erzeugen vieler neuer Objekte und große Kopien Kosten verursachen.



Originalabbildung: 01.Klassen.pdf, Folie 52.

1.6 Entwurfsrichtlinien und Fallstudie Telefonbuch

Beim Entwurf größerer objektorientierter Programme ist die Unterscheidung zwischen **Datenklassen** und **API-Klassen** hilfreich. Datenklassen repräsentieren Fachobjekte und deren Werte. API-Klassen bieten die Operationen an, mit denen ein Programmsystem benutzt wird. Datenobjekte sollten möglichst vollständig beim Erzeugen initialisiert werden und, wenn fachlich sinnvoll, unveränderlich sein. Öffentliche Konstruktoren sind eine langfristige Festlegung auf einen Erzeugungsweg; statische Fabrikmethoden können später flexibler sein, etwa wenn Untertypen oder zwischengespeicherte Objekte zurückgegeben werden sollen.

Eine gute öffentliche API ist **vollständig**, weil sie die fachlich geforderten Operationen verfügbar macht; **elementar**, weil sie unnötige kombinatorische Varianten vermeidet; und **einfach und präzise**, weil Methodennamen, Parameter, Rückgabewerte und Fehlersituationen die Bedeutung eindeutig machen. Sie soll alle Invarianten schützen: Jeder zulässige Methodenaufruf muss das Objekt wieder in einem fachlich gültigen Zustand hinterlassen. Die Kapselung betrifft daher nicht nur einzelne Feldzugriffe, sondern auch Referenzen auf interne Datenstrukturen, die als Rückgabewerte herausgegeben werden.

Die Telefonbuch-Fallstudie übersetzt diese Regeln in eine konkrete Klassenarchitektur. `Entry` repräsentiert Name, Vorwahl und Telefonnummer; `PhoneBook` stellt die operationale Schnittstelle für Einfügen, Suchen, Aktualisieren und Navigation bereit. `EntryList` und `Node` implementieren die interne verkettete Speicherung. `PhoneBookIndex` bietet zusätzlich eine Abkürzung zu Einträgen, die mit einem bestimmten Anfangsbuchstaben beginnen beziehungsweise nach diesem liegen. `PhoneBookApplication` bildet die Konsolenanwendung; eine spätere grafische Benutzeroberfläche könnte dieselbe API nutzen.

Der Index ist keine zweite vollständige Sammlung sämtlicher Kontaktdaten: Er verweist auf Positionen in der sortierten Eintragsfolge. Beim Einfügen, Entfernen oder Umbenennen muss diese Verweisstruktur konsistent aktualisiert werden. Als Entwurfsaufgabe sind geeignete Sichtbarkeiten, `final`-Felder und

Methodensignaturen für `PhoneBook` zu bestimmen. Sinnvolle API-Entscheidungen betreffen etwa die Behandlung doppelter Namen, die Sortierreihenfolge, die Rückgabe für einen erfolglosen Suchvorgang sowie mögliche Exceptions bei ungültigen Angaben. Diese Fragen sind in den Vorlesungsfolien als Gestaltungsaufgabe formuliert; die Folien legen dafür nicht alle Verhaltensentscheidungen fest.

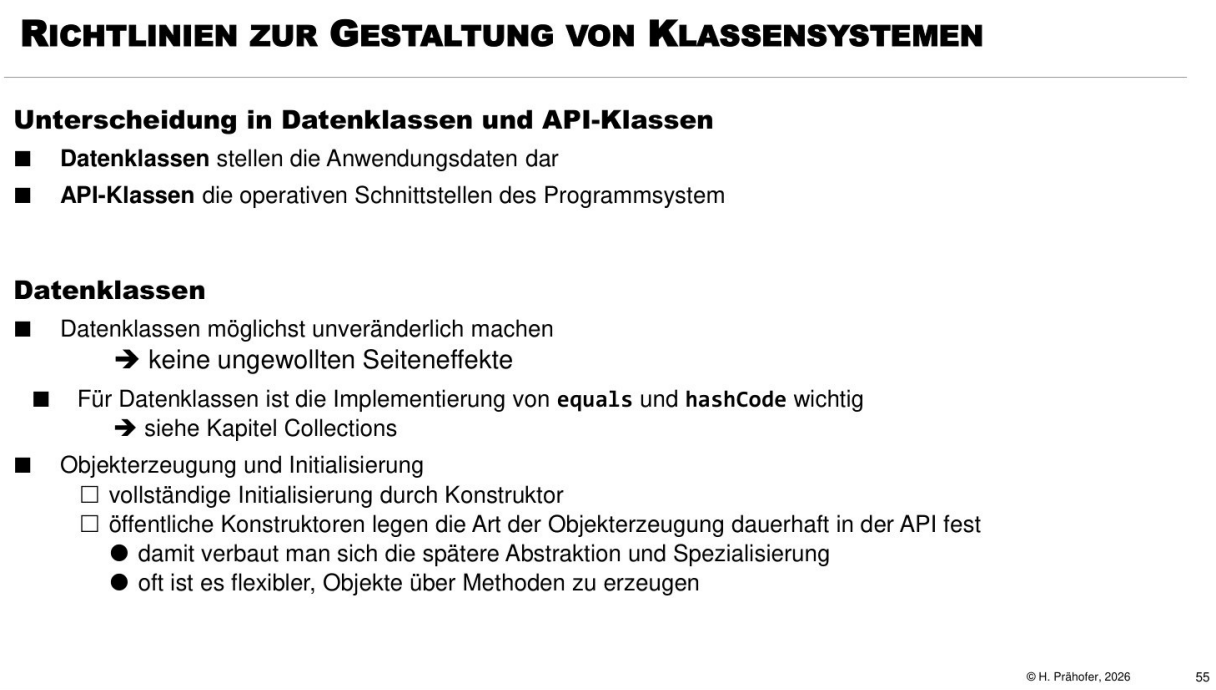
Die Trennung von `PhoneBook`, `Entry` und einer internen `EntryList` schützt das öffentliche Modell vor Änderungen der Speicherdarstellung. Eine Suchmethode `find(String name)` sollte vor ihrer Implementierung dokumentieren, ob bei Nichtfinden `null`, ein `Optional<Entry>` oder eine `Exception` vorgesehen ist; diese drei Verträge haben unterschiedliche Folgen für aufrufende Programme. Das Vorlesungsmaterial lässt diese konkrete Entscheidung offen. Sie darf im Entwurf nicht stillschweigend vorausgesetzt werden.

Ein sortiertes Telefonbuch benötigt eine festgelegte Ordnung für Einträge, beispielsweise nach Nachname und Vorname. Jede Einfügeoperation muss die Sortierung erhalten; ein Index auf Anfangsbuchstaben ist anschließend nur dann korrekt, wenn er auf die aktuelle Eintragsfolge verweist. Für eine API-Prüfung betrachtet man daher nicht nur einzelne Methodennamen, sondern jeden möglichen Zustandsübergang: Welche Angaben werden geprüft, welche Objekte werden verändert, und welche Zusicherungen gelten danach weiterhin?

Originalfolien mit fachlicher Einordnung

K · Folie 55: RICHTLINIEN ZUR GESTALTUNG VON KLASSENSYSTEMEN

Die Unterscheidung von Datenklassen und API-Klassen ist ein Entwurfswerkzeug. Öffentliche Konstruktoren definieren einen später schwer veränderbaren Erzeugungsvertrag; Factory-Methoden schaffen Alternativen.



Originalabbildung: 01.Klassen.pdf, Folie 55.

K · Folie 56: RICHTLINIEN ZUR GESTALTUNG VON KLASSENSYSTEMEN

Die API muss korrekte Zustandsübergänge erzwingen. Besonders Rückgaben von internen veränderlichen `Collections` wären eine mögliche Umgehung der Kapselung.

Richtlinien zur Gestaltung von Klassensystemen

API-Klassen

- API-Methoden zur Implementierung der operationalen Schnittstelle
  - **vollständig**: Operationen müssen alle Anforderungen abdecken
  - **elementar**: Operationen vermeiden, die man durch Kombination von elementaren Operationen realisieren kann
  - **einfach und klar**: Die Funktion einer Operation soll sich klar aus Schnittstelle ergeben
    ➔ Namenswahl ist entscheidend

- **Information Hiding**: keine ungewollten Seiteneffekte, die das System korrumpieren könnten
  - jeder Aufruf muss von einem korrekten Zustand zu einem nächsten korrekten Zustand führen
  - Aufrufe müssen abgesichert sein, d.h. es darf nie ein inkorrekter Zustand entstehen können
  - Keine ungewollten Seiteneffekte bei zurückgegebenen Objekten
    - Z.B. Keine direkten Veränderungen von intern verwendeten Datenstrukturen
      ➔ siehe Kapitel Collections
    - Für Rückgabewerte am besten unveränderliche Objekte verwenden

© H. Prähofer, 2026 56

Originalabbildung: 01.Klassen.pdf, Folie 56.

K · Folie 57: FALLBEISPIEL TELEFONBUCH

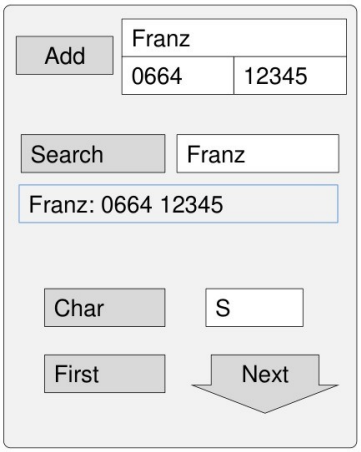
Die GUI-Skizze zeigt die fachlichen Anwendungsfälle: Anlegen, Suchen, Namensnavigation und sequenzielles Blättern. Diese Anforderungen sollen zu klaren Methoden führen.

Fallbeispiel Telefonbuch

- Einfaches Programmsystem zur Verwaltung von Telefonbucheinträgen
- Einträge mit Namen, Vorwahl und Nummer
- Einträge nach Namen sortiert
- Funktionen:
  - Anfügen von Einträgen
  - Suchen mit Namen
  - Prüfen, ob Eintrag enthalten
  - Blättern von einem zum nächsten
  - Springen zum nächsten Eintrag mit gegebenem Buchstaben

➔ Programmsystem könnte Basis für GUI-Anwendung sein

Entwurf GUI:



© H. Prähofer, 2026 57

Originalabbildung: 01.Klassen.pdf, Folie 57.

K · Folie 58: FALLBEISPIEL TELEFONBUCH: KLASSEN

Die Aufteilung macht `Entry` zur Datenklasse und `PhoneBook` zur öffentlichen Steuerungsschicht; `EntryList`, `Node` und `PhoneBookIndex` bleiben interne Strukturen.

Fallbeispiel Telefonbuch: Klassen

- **Entry**: Datenklasse für einen Eintrag im Telefonbuch mit Namen, Vorwahl und Nummer
- **PhoneBook**: Klasse mit operationalen Schnittstelle zur Verwaltung der Einträge
  - ☐ Anfügen von Einträgen
  - ☐ Update von Nummer und Name
  - ☐ Suchen mit Namen
  - ☐ Springen zum ersten Eintrag
  - ☐ Springen zum nächsten Eintrag mit gegebenem Buchstaben
  - ☐ Blättern von einem zum nächsten Eintrag
- **EntryList** und **Node**: verkettete Liste mit den Einträgen, Verkettung mit Klasse Node
- **PhoneBookIndex**: Indexstruktur von Buchstaben zum ersten Knoten mit oder nach dem Buchstaben
- **PhoneBookApplication**: Konsolenapplikation zur Verwendung des Telefonbuchs

© H. Prähofer, 2026 58

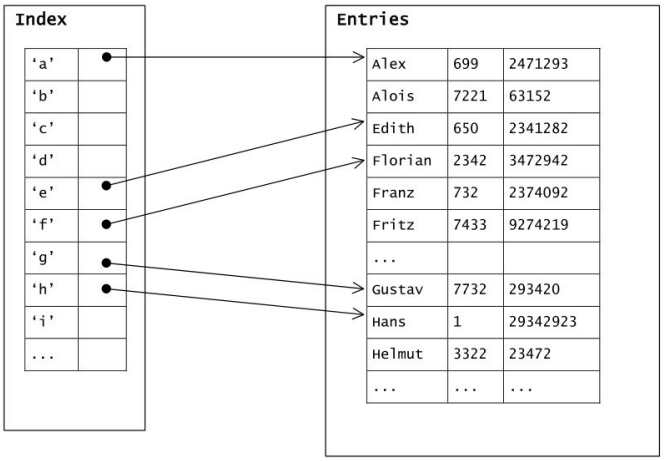
Originalabbildung: 01.Klassen.pdf, Folie 58.

K · Folie 59: FALLBEISPIEL TELEFONBUCH: KLASSEN

Der Index verbindet Anfangsbuchstaben mit Positionen in einer alphabetisch sortierten Liste. Die Konsistenz von Index und Listenspeicher ist beim Einfügen oder Entfernen zu bewahren.

Fallbeispiel Telefonbuch: Klassen

- **PhoneBookIndex**

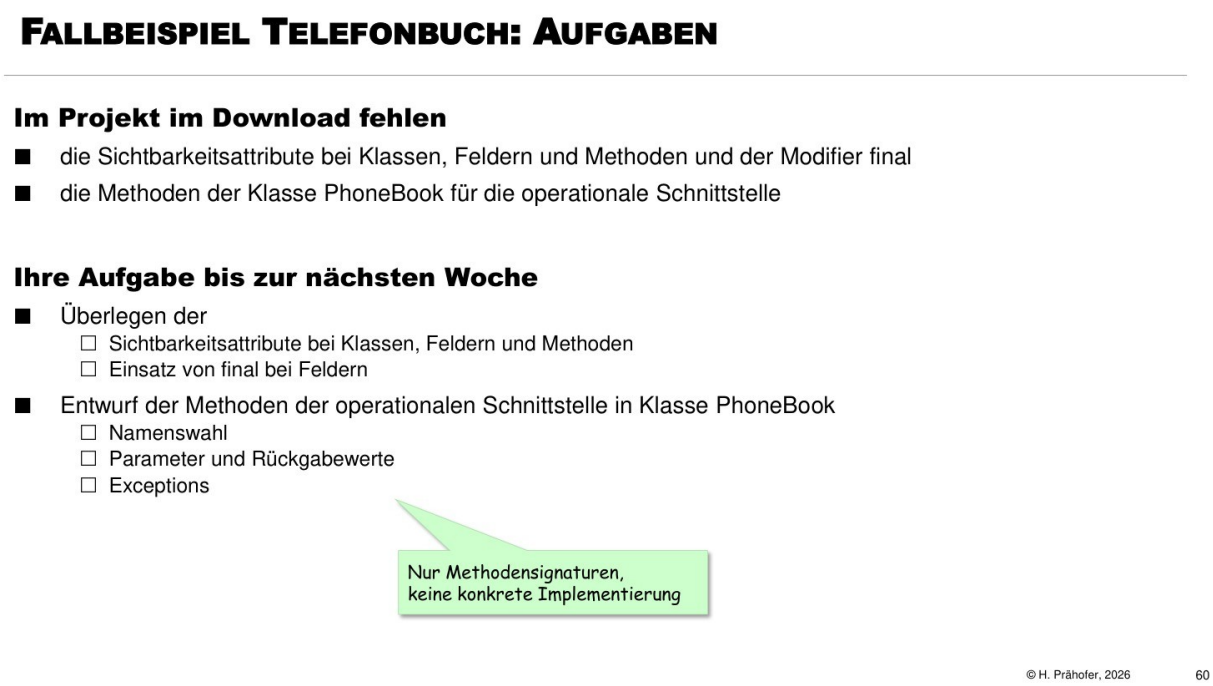


© H. Prähofer, 2026 59

Originalabbildung: 01.Klassen.pdf, Folie 59.

K · Folie 60: FALLBEISPIEL TELEFONBUCH: AUFGABEN

Die Folie enthält bewusst eine offene Entwurfsaufgabe und keinen bereits festgelegten vollständigen Lösungscode. Sichtbarkeiten, finale Felder sowie Signaturen einschließlich Exceptions sind selbst herzuleiten.



Originalabbildung: 01.Klassen.pdf, Folie 60.

2 Vererbung und dynamische Bindung

2.0 Statische und dynamische Betrachtung; UML

Statisch bezeichnet Eigenschaften, die aus dem Programmtext bestimmt werden, beispielsweise den deklarierten Typ einer Variablen oder die Auswahl einer überladenen Methode. **Dynamisch** bezeichnet Eigenschaften, die erst bei der Programmausführung feststehen, etwa den tatsächlichen Typ des Objekts an einer Referenz. Java ist statisch typisiert: Der Compiler prüft, welche Methoden aufgrund der Deklarationen erreichbar sind. Zusätzliche Laufzeitprüfungen können dennoch erforderlich werden, insbesondere bei Referenzcasts. Ein statisch abgelehnter Ausdruck führt zu einem Übersetzungsfehler; eine nicht bestandene Laufzeitprüfung wirft eine `Exception`.

Die Unified Modeling `Language (UML)` stellt Klassen, Beziehungen und Abläufe grafisch dar. Ein Klassendiagramm enthält typischerweise den Klassennamen, Felder und Methoden; UML schreibt Feldtypen als `name: Typ` und Methodensignaturen als `m(x: Typ): Rückgabetyp`. Für das Lesen der weiteren Folien sind außerdem die offenen Dreieckspfeile der Generalisierung, gestrichelte Realisierungspfeile bei Interfaces und Assoziationskanten wichtig. Die Vorlesungsfolien verwenden bewusst teils verkürzte Klassendiagramme, um die jeweils wichtigen Elemente hervorzuheben.

Der statische Typ stammt aus der Deklaration einer Variablen beziehungsweise aus der statischen Typanalyse eines Ausdrucks. Der dynamische Typ eines nicht-`null` Referenzwerts ist die tatsächliche Klasse des referenzierten Objekts. Bei `Shape s = new Circle(...);` ist der statische Typ `Shape` und die Objektklasse `Circle`. Welche Methoden nach dem Punkt zugänglich sind, entscheidet die statische Prüfung; welche überschreibende Instanzmethode ausgeführt wird, entscheidet die dynamische Bindung.

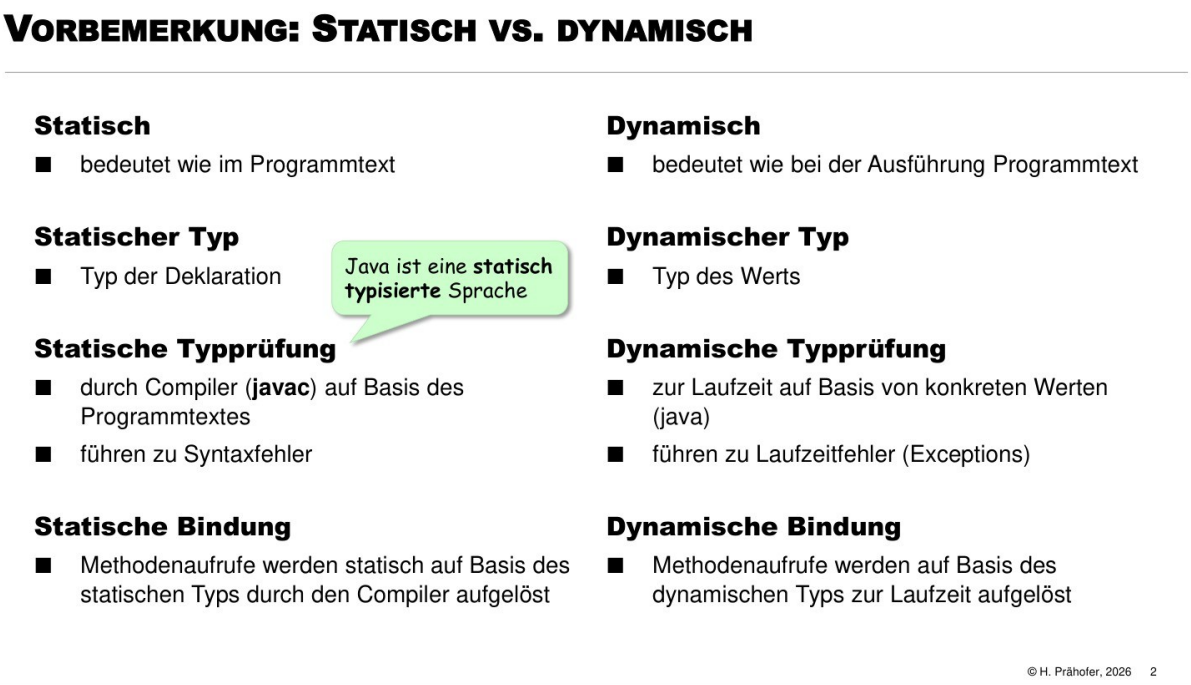
In einem UML-Klassendiagramm beschreibt das Rechteck eine Klasse samt Attributen und Operationen. Eine durchgezogene Linie mit hohlem Dreieck steht für eine Generalisierung (`extends` zwischen Klassen), eine

gestrichelte Realisierungslinie zu einem Interface für `implements`. UML ist eine Modellnotation und kein ausführbarer Java-Code. Die Diagramme der Folien vereinfachen Sichtbarkeiten, Generics oder Parameterlisten teilweise; für die genauen Sprachregeln ist der gezeigte Java-Code maßgeblich.

Originalfolien mit fachlicher Einordnung

V · Folie 2: VORBEMERKUNG: STATISCH VS. DYNAMISCH

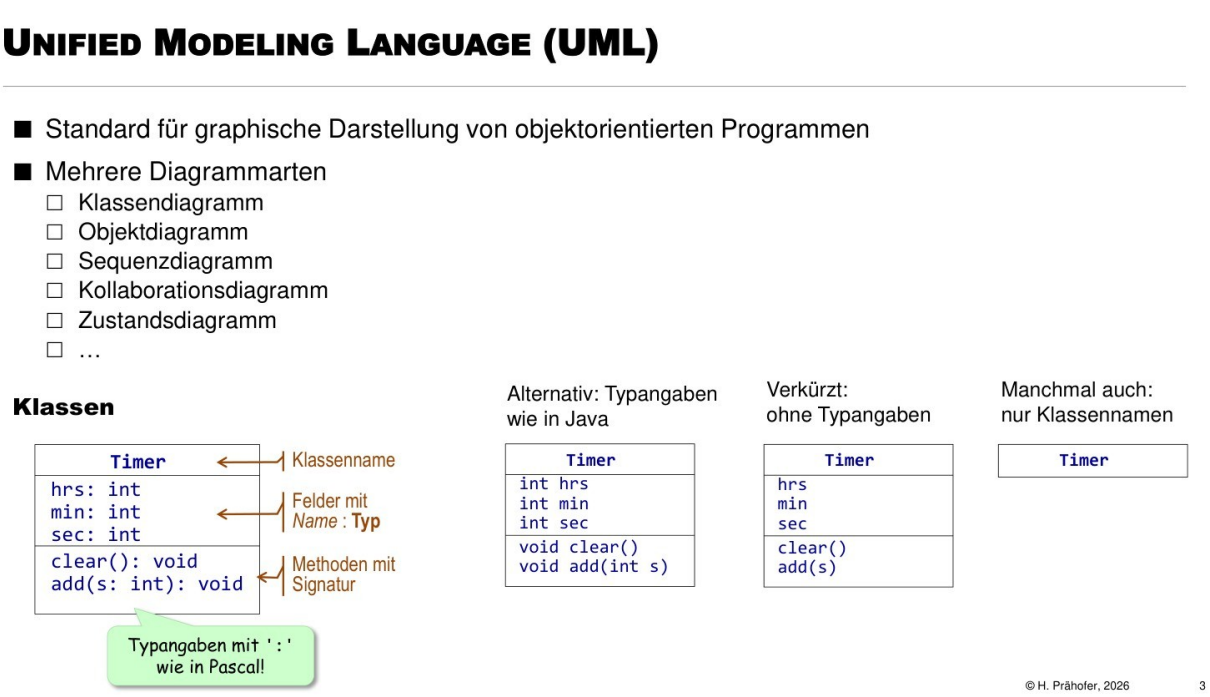
Die Unterscheidung beschreibt Zeitpunkt und Informationsbasis der Entscheidung. Compilerfehler betreffen statische Prüfungen, Exceptions können aus dynamischen Prüfungen entstehen.



Originalabbildung: 02.Vererbung.pdf, Folie 2.

V · Folie 3: UNIFIED MODELING LANGUAGE (UML)

UML stellt gemeinsame Begriffe für Klassen-, Objekt-, Zustands- und Ablaufdarstellungen bereit. Die Skizzen zeigen verschiedene Detailgrade desselben `Timer`-Modells.



Originalabbildung: 02.Vererbung.pdf, Folie 3.

2.1 Vererbung, super, abstract und final

Vererbung ist in Java die Ableitung einer Unterklasse von einer Oberklasse. `class ClockTimer extends Timer` definiert einen Subtyp, der die zugänglichen vererbbaren Eigenschaften seines Supertyps übernimmt und neue Mitglieder hinzufügen oder vererbbare Methoden überschreiben darf. Jenseits aller expliziten Oberklassen steht `java.lang.Object` an der Wurzel der Klassenhierarchie. Klassenkonstruktoren werden allerdings **nicht vererbt**. Deshalb initialisiert ein Unterklassenkonstruktor den Oberklassenteil des neuen Objekts durch einen Aufruf `super(...)`. Ist ein passender parameterloser Oberklassenkonstruktor vorhanden, kann Java den Aufruf automatisch ergänzen; andernfalls muss der Entwickler den geeigneten Aufruf ausdrücklich formulieren.

Das **Überschreiben** einer Methode bedeutet, dass eine Unterklasse eine neue Implementierung für eine geerbte, überschreibbare Instanzmethode bereitstellt. Die Parameterliste muss zur überschriebenen Methode passen; Rückgabetyp und geprüfte Exceptions unterliegen zusätzlichen Kompatibilitätsregeln. Die Annotation `@Override` ist optional, aber dringend empfehlenswert: Der Compiler prüft damit, ob wirklich eine passende Methode überschrieben wurde. Ein versehentliches `add()` statt `add(int)` führt dadurch zu einem erkennbaren Fehler. Der Aufruf `super.add(s)` innerhalb der überschreibenden Methode wählt ausdrücklich die Oberklassenimplementierung und ist nützlich, wenn zunächst das allgemeine Verhalten ausgeführt und danach spezialisiert werden soll.

Ein `ClockTimer` kann die bereits geerbte Zeitaddition um eine obere Stundengrenze `maxHrs` ergänzen. Bei einem `maxHrs` von 24 soll `hrs` beispielsweise nach 23 wieder auf 0 springen; bei 12 folgt eine andere Begrenzung. Eine robuste Implementierung muss gewährleisten, dass `maxHrs` positiv ist und Stunden/Minuten/Sekunden durchweg konsistent normalisiert werden. Der Zugriff auf `protected`-Felder der Oberklasse ist in Unterklassen grundsätzlich vorgesehen, aber gezielt begrenzte geschützte Methoden sind oft stabiler als direkter Feldzugriff.

`Object` stellt grundlegende Methoden bereit. `toString()` erzeugt eine Zeichenkettendarstellung, `equals(Object)` definiert Vergleichsverhalten und `hashCode()` einen Hashwert. Die ersten drei werden für eigene Wertobjekte häufig sinnvoll überschrieben. `getClass()`, `wait`, `notify` und `notifyAll` sind dagegen `final`; sie können nicht überschrieben werden. `clone()` ist geschützt und verlangt für einen öffentlichen Kopiermechanismus besondere Behandlung. `finalize()` gilt als veraltet und darf nicht als normale Ressourcenverwaltung benutzt werden.

Die vier Sichtbarkeitsbereiche gelten auch in Klassenhierarchien: `protected` erlaubt das bewusste Angebot eines Erweiterungspunkts. Beim Überschreiben darf die Sichtbarkeit erweitert werden, etwa von `protected` auf `public`, aber nicht eingeschränkt. Sonst könnte ein Client, der den öffentlichen Vertrag des Oberklassentyps erfüllt sieht, diesen Vertrag bei Verwendung des Untertyps plötzlich nicht mehr nutzen.

Eine **abstrakte Methode** besitzt nur eine Deklaration und keinen Methodenkörper, beispielsweise `public abstract void draw();`. Eine Klasse, die solche Methoden enthält, ist abstrakt, sie trägt den Modifikator `abstract` und kann nicht unmittelbar mit `new` instanziiert werden. Ein konkreter Subtyp muss alle geerbten abstrakten Methoden implementieren. Bei `Shape` definieren `draw()` und `getWidth()` daher den gemeinsamen Vertrag für `Circle` und `Rectangle`, während diese konkreten Klassen die zeichnungs- und formabhängige Implementierung liefern. Referenzen und `Arrays` dürfen trotzdem den abstrakten Typ `Shape` verwenden.

`final` hat hier zwei andere Bedeutungen als bei Feldern: Eine `final`-Klasse darf keine Unterklassen besitzen, und eine `final`-Methode darf nicht überschrieben werden. Abstrakte und finale Klassen wären begrifflich widersprüchlich und können deshalb nicht kombiniert werden. Im Gegensatz zu Methoden können **Felder nicht polymorph überschrieben** werden: Definiert eine Unterklasse ein gleichnamiges Feld, entsteht ein zweites Feld; der sichtbare Feldname wird anhand des statischen Referenztyps aufgelöst. Diese Feldüberdeckung erschwert die Lesbarkeit und sollte vermieden werden.

Eine abgeleitete Klasse führt neue Mitglieder hinzu und übernimmt die geerbte Schnittstelle der Basisklasse. Konstruktoren werden allerdings nicht vererbt: Der Unterklassenkonstruktor muss die Initialisierung der Oberklasse über `super(...)` sicherstellen. Fehlt ein expliziter Superkonstruktor-Aufruf, versucht der Compiler grundsätzlich einen geeigneten parameterlosen Oberklassenkonstruktor. Ist dieser nicht zugänglich oder nicht vorhanden, muss ein passender Aufruf angegeben werden.

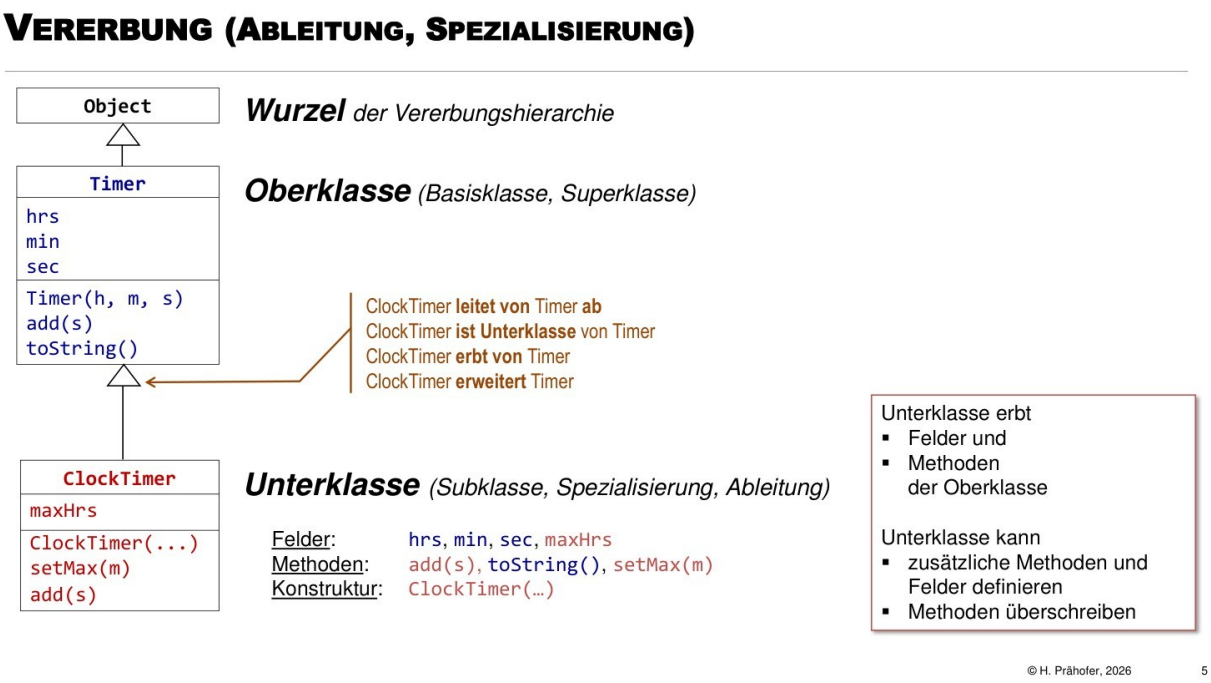
Beim Überschreiben einer Methode sind Name, Parameterliste, Rückgabetyp-Kompatibilität und Sichtbarkeit zu beachten. `@Override` ist selbst keine Änderung der Bindung, sondern veranlasst eine zusätzliche Compilerprüfung. Ein Aufruf `super.add(s)` umgeht für diesen Aufruf die überschreibende Implementierung und verwendet die geerbte Methode; eine anschließend ausgeführte Zeile kann das Ergebnis weiter verändern. `private`-Methoden sind nicht virtuell überschreibbar, statische Methoden können nur verdeckt werden.

Eine abstrakte Klasse kann gemeinsame Felder und fertige Methoden bereitstellen und zugleich abstrakte Operationen verlangen. Von ihr selbst darf kein Objekt mit `new` erzeugt werden; eine konkrete Unterklasse muss alle noch abstrakten Methoden implementieren. `final` schützt entweder eine Methode vor einer erneuten Überschreibung oder eine Klasse vor weiterer Ableitung. Das Verdecken gleichnamiger Felder ist etwas anderes als Methodenüberschreibung: Ein Feldzugriff wird statisch aufgelöst und sollte deshalb nicht absichtlich in einer Unterklasse wiederholt werden.

Originalfolien mit fachlicher Einordnung

V · Folie 5: VERERBUNG (ABLEITUNG, SPEZIALISIERUNG)

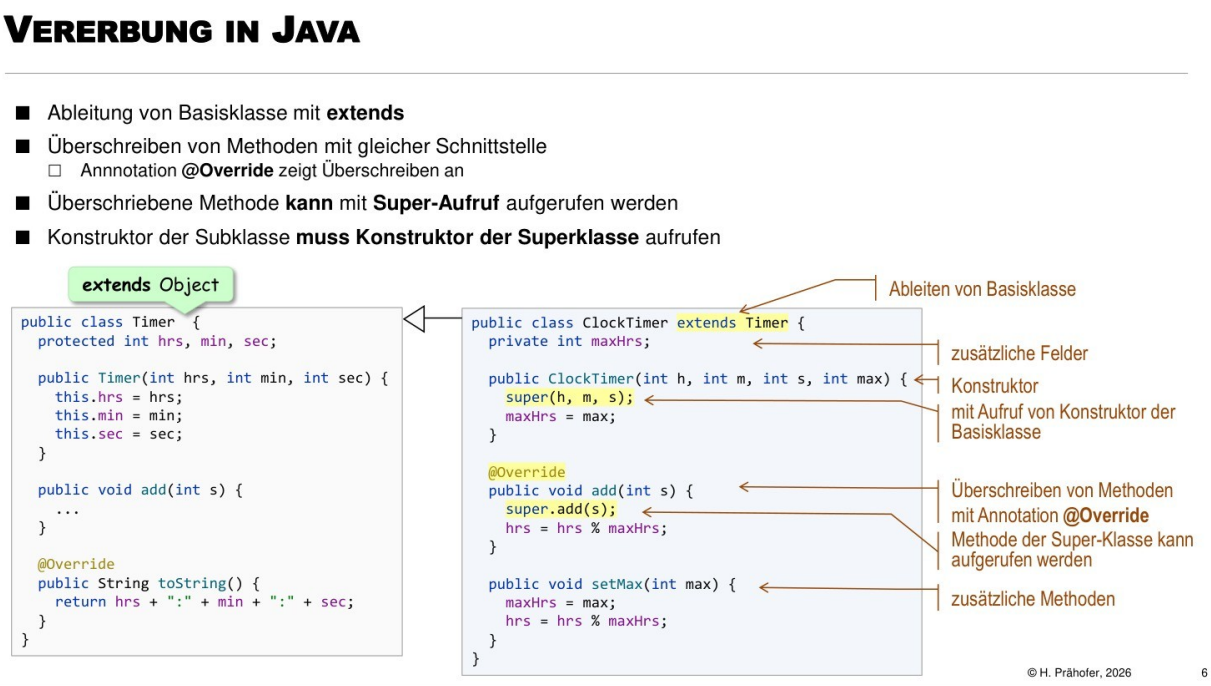
Die Generalisierungspfeile zeigen die is-a-Beziehung von `ClockTimer` zu `Timer`. Konstruktoren werden nicht geerbt, während zugängliche Felder und Methoden den Untertyp prägen.



Originalabbildung: 02.Vererbung.pdf, Folie 5.

V · Folie 6: VERERBUNG IN JAVA

`extends`, `super(...)`, `@Override` und `super.add(s)` erfüllen unterschiedliche Rollen. Insbesondere `super(...)` initialisiert den Oberklassenanteil und `super.add(s)` ruft eine konkrete Oberklassenmethode auf.



Originalabbildung: 02.Vererbung.pdf, Folie 6.

V · Folie 7: DIE KLASSE OBJECT

Die `Object`-Methoden sind Basis jeder Klassenhierarchie. `equals`, `hashCode` und `toString` können sinnvoll spezialisiert werden, `getClass` und die Monitoroperationen sind dagegen `final`.

Die Klasse Object

Klasse Object ist Wurzel der Vererbungshierarchie

- alle Klassen erben direkt oder indirekt von **Object**
- erben damit die Methoden von **Object**

in VM implementiert!

Referenzgleichheit!

```
public class Object {

  public native int hashCode();
  public boolean equals(Object obj) { return (this == obj); }
  public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
  }

  public final native Class<?> getClass();
  public final native void notify();
  public final native void notifyAll();
  public final native void wait(long timeout) throws InterruptedException;
  public final void wait(long timeout, int nanos) throws InterruptedException { ... }
  public final void wait() throws InterruptedException { ... }

  protected native Object clone() throws CloneNotSupportedException;

  @Deprecated
  protected void finalize() throws Throwable { }
}
```

- Methoden **toString**, **hashCode**, **equals** können/sollen überschrieben werden
- **getClass**, **wait**, **notify** können nicht überschrieben werden (siehe **final**)
- **clone** muss überschrieben werden, wenn man Kopien erzeugen will
- **finalize** ist "deprecated" und soll nicht mehr verwendet werden

© H. Prähofer, 2026 7

Originalabbildung: 02.Vererbung.pdf, Folie 7.

V · Folie 8: ÜBERSCHREIBEN VON METHODEN UND SUPER-AUFRUFE

Die Überschreibung ersetzt die normale Implementierung bei dynamischer Bindung; mit einem expliziten `super`-Aufruf kann die Unterklasse dennoch die geerbte Logik weiterverwenden.

Überschreiben von Methoden und Super-Aufrufe

- Überschreibende Methoden können mit Super-Aufrufen überschriebene Methode aufrufen

```
public class Timer  {
  protected int hrs, min, sec;
  public Timer(int hrs, int min, int sec) {
    this.hrs = hrs; this.min = min; this.sec = sec;
  }

  public void add(int s) {
       sec += s;
       if (sec > 59) { min += sec/60; sec %= 60; }
       if (min > 59) { hrs += min/60; min %= 60; }
  }
  ...
}
```

```
public class ClockTimer extends Timer {
  public ClockTimer (int h, int m, int s, int max) {
    super(h, m, s);
    maxHrs = max;
  }

  @Override
  public void add(int s) {
    super.add(s);
    hrs = hrs % maxHrs;
  }
  ...
}
```

Geerbte Methode kann **überschrieben** werden, indem man sie mit **derselben Signatur** neu deklariert.
➔ **verdeckt die geerbte Methode!**

Super-Aufruf
überschriebene Methode der Basisklasse kann mit **super** aufgerufen werden

© H. Prähofer, 2026 8

Originalabbildung: 02.Vererbung.pdf, Folie 8.

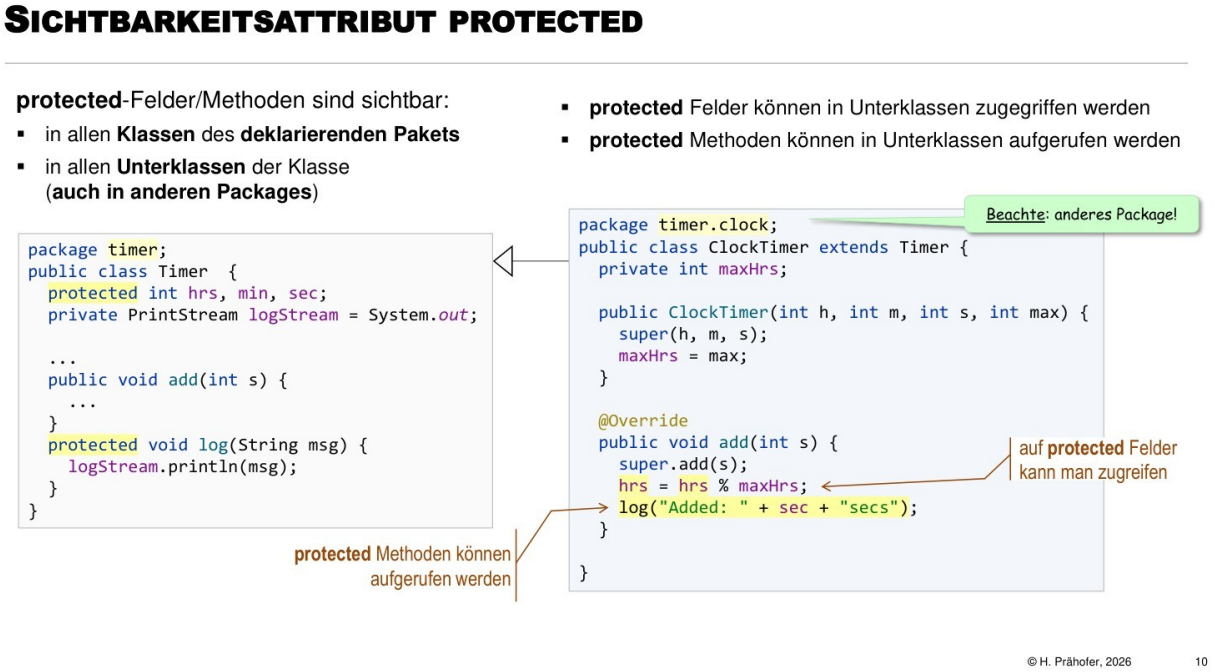
V · Folie 9: ANNOTATION @OVERRIDE

`@Override` bewirkt eine Compilerkontrolle. Ein Methodenname oder eine Parameterliste mit Tippfehler erzeugt sonst unbemerkt nur eine neue Methode.

Originalabbildung: 02.Vererbung.pdf, Folie 9.

V · Folie 10: SICHTBARKEITSATTRIBUT PROTECTED

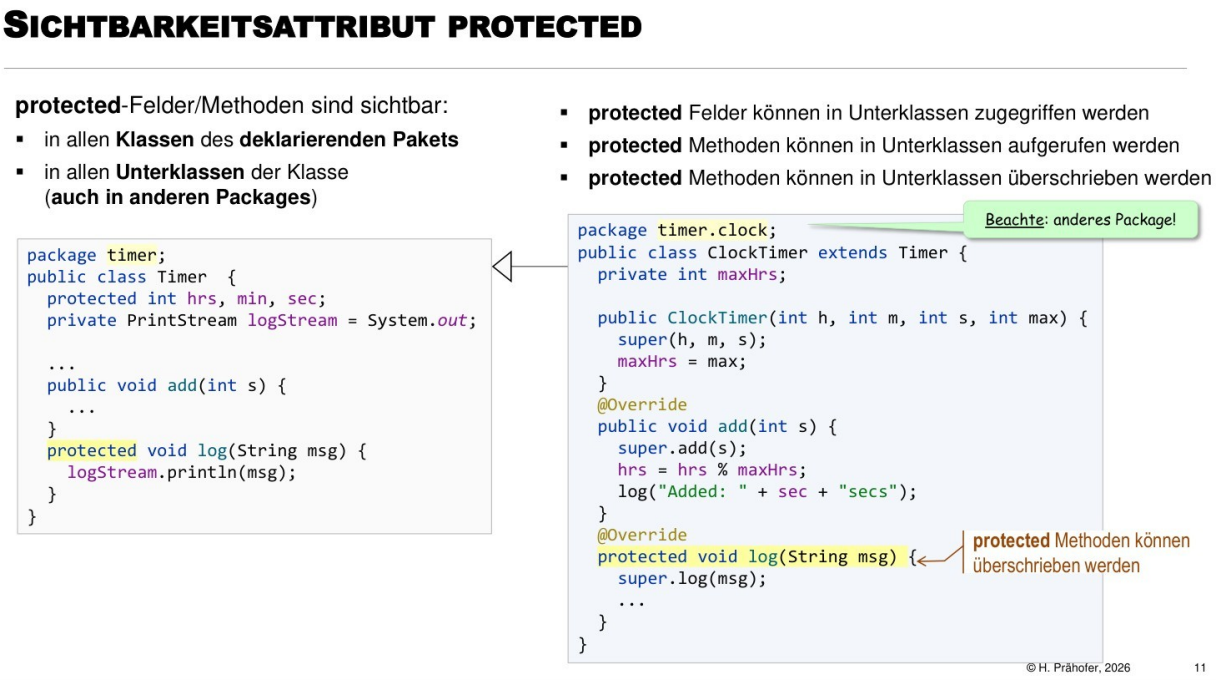
Eine Unterklasse in anderem Package darf ihre geerbten geschützten Felder und Methoden unter den Java-Zugriffsregeln verwenden. Dieses Recht ist nicht gleichbedeutend mit allgemeinem Zugriff auf beliebige Oberklassenobjekte.



Originalabbildung: 02.Vererbung.pdf, Folie 10.

V · Folie 11: SICHTBARKEITSATTRIBUT PROTECTED

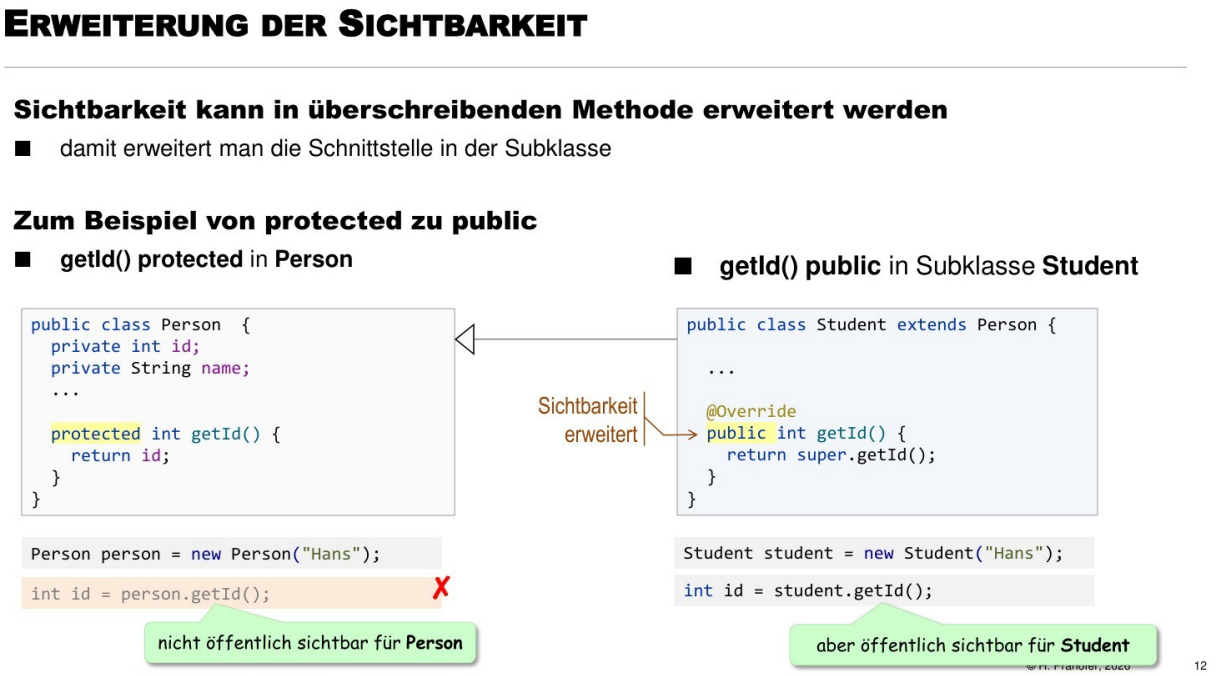
Der Aufruf `super.log(msg)` bindet die geerbte Implementierung ausdrücklich ein. Die überschreibende Methode darf weitere Aktionen ergänzen und ihre Zugänglichkeit nicht reduzieren.



Originalabbildung: 02.Vererbung.pdf, Folie 11.

V · Folie 12: ERWEITERUNG DER SICHTBARKEIT

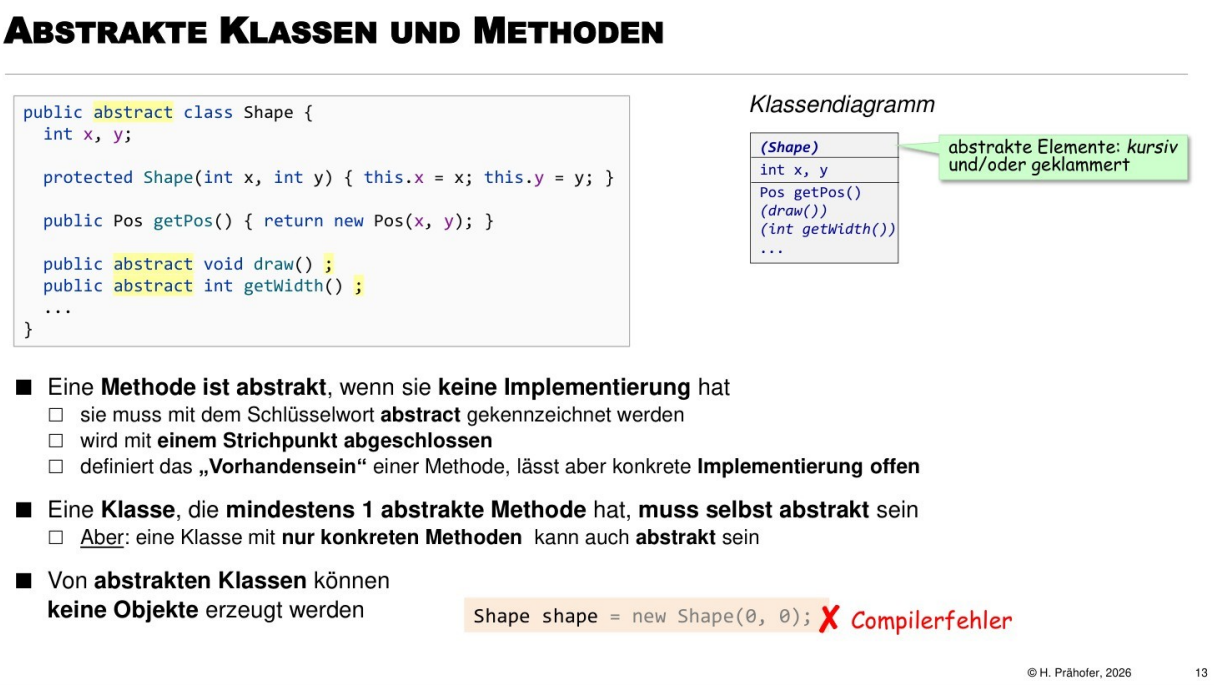
Die Methoden-Sichtbarkeit kann beim Überschreiben erweitert, niemals verengt werden. Damit bleibt der Vertrag der Oberklasse für polymorphe Clients verwendbar.



Originalabbildung: 02.Vererbung.pdf, Folie 12.

V · Folie 13: ABSTRAKTE KLASSEN UND METHODEN

Eine abstrakte Methode enthält keinen Rumpf und schließt mit `;`. **Korrektur:** Der fragmentierte Code der Folie enthält einen Satz-/Layoutfehler bei `new Pos(x,y);` das Prinzip „abstrakter Vertrag ohne Implementierung“ bleibt davon unberührt.



Originalabbildung: 02.Vererbung.pdf, Folie 13.

V · Folie 14: ABLEITUNG VON ABSTRAKTER KLASSE

`Circle` und `Rectangle` machen die abstrakten `Shape`-Operationen konkret. `ColoredShape` bleibt abstrakt, weil die geerbten abstrakten Methoden noch nicht implementiert sind.

Originalabbildung: 02.Vererbung.pdf, Folie 14.

V · Folie 15: ABSTRAKTE KLASSEN ALS TYPEN

Obwohl `Shape` nicht instanziiert werden kann, ist es ein gültiger Referenz- und Arrayelementtyp. Die Schleife funktioniert, wenn jedes Element eine konkrete, nicht-`null` Instanz enthält.

Abstrakte Klassen als Typen

■ Eine **abstrakte Klasse** kann **als Typ** in Deklarationen verwendet werden:

```
Shape shape = new Circle(100, 100, 10);
```

```
Shape[] shapes = new Shape[2];
shapes[0] = new Rectangle(10, 20, 30, 10);
shapes[1] = new Circle(10, 20, 30);
```

■ Die abstrakte Klasse bestimmt, **welche Methoden vorhanden** sind
- ☐ Damit kann man auch **alle Methoden der abstrakten Klasse aufrufen** (auch abstrakte)

```
shape.draw();
int w = shape.getWidth();
```

draw, **getWidth** in **Shape** abstrakt, aber in Klassen **Rectangle** und **Circle** konkret

```
for (Shape shape : shapes) {
  shape.draw();
}
```

© H. Prähofer, 2026 15

Originalabbildung: 02.Vererbung.pdf, Folie 15.

V · Folie 16: FINAL KLASSEN

`final` am Klassennamen sperrt die Unterklassenbildung. Es schützt damit unter anderem gegen spezialisierte Überschreibungen von sicherheits- oder vertragskritischem Verhalten.

final Klassen

Von **Klassen**, die als **final** deklariert sind, dürfen **keine Unterklassen** abgeleitet werden
➔ Sperren von Bilden von Subklassen

```
public final class Account {
  int balance;
  public void deposit (int value) {
    balance = balance + value;
  }
  public void withdraw (int value) {
    balance = balance - value;
  }
}
```

Beachte: **final** Klassen können nicht **abstract** sein!

```
public class FakeAccount extends Account {
  public void deposit (int value) {
    balance = balance + value / 2;
  }
}
```

✗ final Klasse Account kann nicht erweitert werden

Wegen dynamischer Bindung von

```
account.deposit(value);
```

könnte sonst die Semantik dieser Operation geändert werden.

Anwendung: Für Klassen, deren Semantik **unabänderlich** sein soll (z.B. **String**)

© H. Prähofer, 2026 16

Originalabbildung: 02.Vererbung.pdf, Folie 16.

V · Folie 17: FINAL METHODEN

`final` an Methoden verhindert die Überschreibung dieser Methoden, erlaubt aber die Bildung von Unterklassen, die andere Methoden spezialisieren.

FINAL METHODEN

Methoden, die als **final** deklariert sind, dürfen **in Unterklassen nicht überschrieben** werden
➔ **Schutz vor Überschreiben**

```
public class Account {
  protected int balance;

  public final void deposit(int value) {
    balance = balance + value;
  }
  public final void withdraw(int value) {
    balance = balance - value;
  }
  public void print() {
    System.out.println(balance);
  }
}
```

```
public class MyAccount extends Account {

  @Override
  public void print () {
    System.out.println(balance + ",00");
  }

  public void deposit(int value) { }
  public void withdraw(int value) { }
}
```

✗ final Methoden können nicht überschrieben werden

Bilden einer Unterklasse ist erlaubt:
- **print** darf auch überschrieben werden,
- **deposit** und **withdraw** aber nicht

© H. Prähofer, 2026 17

Originalabbildung: 02.Vererbung.pdf, Folie 17.

V · Folie 18: ÜBERDECKEN VON FELDERN

Gleichnamige Felder existieren als zwei verschiedene Speicherstellen. Die Auswahl erfolgt anhand des statischen Referenztyps, nicht der Laufzeitklasse des Objekts.

ÜBERDECKEN VON FELDERN

Unterklasse könn(t)en Datenfelder in der Oberklasse überecken

```
class A {
  int x;
  A() {
    x = 1;
  }
}
```

Typen überdeckender Felder können verschieden sein

```
class B extends A {
  String x;
  B() {
    x = "abc";
    super.x = 3;
  }
}
```

überdeckt x aus A

Zugriff auf überdecktes Feld mit **super**

```
A a = new A();
int xA = a.x;
```

```
B b = new B();
String xB = b.x;
```

Aber beachte:

```
A a = new B();
int x = a.x;
```

ist **x** von **A**

Felder sind statisch gebunden!

Regel: Überdecken von Feldern vermeiden!
- kaum nützlich und Fehlerquelle
- besser anderen Name wählen

© H. Prähofer, 2026 18

Originalabbildung: 02.Vererbung.pdf, Folie 18.

2.2 Subtypen, dynamische Bindung, Typtests und Exceptions

Eine Klassenhierarchie erzeugt zugleich eine **Subtyphierarchie**. `Circle extends Shape` bedeutet, dass ein `Circle` dort eingesetzt werden kann, wo eine `Shape`-Referenz erlaubt ist. Das nennt man Substituierbarkeit beziehungsweise polymorphe Zuweisung. Bei `Shape shape = new Circle(10, 10, 5);` ist der **statische Typ** `Shape`, der **dynamische Typ** des bezeichneten Objekts `Circle`. Der Compiler erlaubt deshalb nur die durch `Shape` bereitgestellten sichtbaren Methoden. Die Ausführung einer überschriebenen Instanzmethode, beispielsweise `shape.draw()`, erfolgt aber gemäß dem konkreten Laufzeittyp `Circle`.

Diese Trennung ist der Kern der **dynamischen Bindung**: Eine Sammlung `Shape[]` kann Instanzen von `Circle` und `Rectangle` enthalten, und `shape.draw()` führt für jedes Element die jeweilige Implementierung aus. Das Programm muss die Formen dafür nicht vorab mit einer `if`-Kette unterscheiden. Die Arrayelemente müssen allerdings tatsächlich mit Objekten belegt sein: Eine Schleife, die auch uninitialisierte `null`-Positionen mit `shape.draw()` verarbeitet, würde eine `NullPointerException` auslösen. Statische Methoden werden nicht dynamisch überschrieben, sondern verborgen; `private` und finale Methoden nehmen ebenfalls nicht an normaler virtueller Überschreibung teil. **Überladung** wird bereits bei der Übersetzung anhand der statischen Argumenttypen entschieden.

Mit `instanceof` kann die Laufzeitzugehörigkeit geprüft werden: `shape instanceof Circle` ist wahr, wenn die Referenz auf eine Instanz des Typs `Circle` oder eines Untertyps verweist. Ein **Typecast** wie `(Circle) shape` ändert nicht das Objekt selbst, sondern erlaubt es, die Referenz unter einem engeren statischen Typ zu verwenden. Ist der Typzusammenhang bereits bei der Übersetzung unmöglich, folgt ein Compilerfehler; ist der Cast grundsätzlich zulässig, aber der aktuelle Laufzeittyp falsch, entsteht eine `ClassCastException`. Die sicherere Kombination lautet `if (shape instanceof Circle circle) { circle.getRadius(); }`. Diese seit Java 16 verfügbare Pattern-Matching-Form verknüpft Typprüfung und Variablenbindung.

Ein **`switch`-Ausdruck** liefert einen Wert: `int days = switch(month) { case 1,3,5 -> 31; ... default -> 30; };`. Anders als die klassische fall-through-Anweisung benötigen Pfeil-Cases kein `break`. Mehrere Anweisungen innerhalb eines Case-Blocks können mit `yield` einen gemeinsamen Ergebniswert liefern. Der Ausdruck muss hinsichtlich aller möglichen Fälle vollständig sein, etwa durch ein `default`, wenn der geprüfte Typ nicht ohnehin vollständig abgedeckt ist. Seit Java 21 können Switch-Cases auch Typ-Patterns enthalten, beispielsweise `case Circle c -> ...`. Auf diese Weise lässt sich ein Ergebnis je nach dynamischem Typ eines Objekts berechnen. Bei Flächenberechnungen der Folien ist die Variable `r` ein Radius; für den Kreis gilt $A = \pi\, r^2$, für das Rechteck $A = \text{Breite} \times \text{Höhe}$.

Exceptions sind Objekte einer Hierarchie mit Wurzel `Throwable`. Direkt darunter stehen `Error` (schwerwiegende Laufzeitprobleme) und `Exception`. Innerhalb von `Exception` bilden `RuntimeException` und ihre Unterklassen die sogenannten unchecked Exceptions; hierzu gehören `NullPointerException`, `IllegalArgumentException`, `IndexOutOfBoundsException` und `ArithmeticException`. Zu den typischen **checked Exceptions** zählen `IOException` und `FileNotFoundException`, die eine Methode im Allgemeinen selbst behandeln oder in ihrer `throws`-Klausel weiterreichen muss. `NumberFormatException` ist eine `RuntimeException` und muss nicht deklariert werden.

Bei mehreren `catch`-Blöcken wird die `Exception` anhand ihres dynamischen Typs einer passenden Klausel zugeordnet. Spezielle Typen müssen vor allgemeineren Typen aufgeführt werden, andernfalls wären einzelne `catch`-Klauseln unerreichbar. `finally` bezeichnet den Block, der im üblichen Kontrollfluss nach `try` und

`catch` ausgeführt wird und oft der Freigabe von Ressourcen dient; für schließbare Ressourcen ist `try-with-resources` meist die bessere Wahl. Die Schreibweise `throws IOException, FormatException` teilt aufrufenden Methoden mit, welche geprüften Ausnahmen berücksichtigt werden müssen.

Vorsicht ist bei **virtuellen Methodenaufrufen in Konstruktoren** nötig: Wird von einem Oberklassenkonstruktor eine überschreibbare Methode aufgerufen, kann bereits die Unterklassenimplementierung ausgeführt werden, obwohl deren Felder noch nicht vollständig initialisiert sind. Dieses Problem lässt sich vermeiden, indem man aus Konstruktoren keine veränderbaren Erweiterungspunkte aufruft. Die in der Vorlesung erwähnten Template-Methoden nutzen dynamische Bindung bewusst, allerdings normalerweise an vollständig konstruierten Objekten.

Polymorphe Verarbeitung entsteht durch eine gemeinsame Obertyp-Schnittstelle. Eine Methode `void drawAll(Shape[] shapes)` darf für jedes Element `shape.draw()` aufrufen, wenn `draw` in `Shape` deklariert ist. Bei einem `Circle` wird die `Circle`-Implementierung ausgeführt, bei einem `Rectangle` die entsprechende `Rectangle`-Implementierung. Eine nur in `Circle` deklarierte Methode wie `getRadius()` lässt sich über eine Variable vom statischen Typ `Shape` nicht ohne geeignete Typprüfung und Typanpassung aufrufen.

`instanceof Circle` prüft einen Referenzwert zur Laufzeit und ist bei `null` immer falsch. Ein Cast `((Circle) shape)` erweitert nicht die tatsächliche Fähigkeit des Objekts, sondern nur die statische Sicht des Compilers; bei einem inkompatiblen Objekt folgt eine `ClassCastException`. Mit `if (shape instanceof Circle circle) { ... }` führt Java seit Version 16 Typprüfung und lokale Typbindung gemeinsam durch.

Die `Exception`-Hierarchie beginnt bei `Throwable`. `Error` steht im Wesentlichen für schwerwiegende Probleme, `Exception` für Ausnahmefälle. Geprüfte `Ausnahmen (checked exceptions)` müssen gefangen oder mit `throws` deklariert werden; `RuntimeException` und ihre Unterklassen sowie `Error` zählen nicht dazu. Mehrere `catch`-Klauseln müssen von spezielleren zu allgemeineren Typen geordnet sein. Ein `finally`-Block wird gewöhnlich beim Verlassen eines `try` abgearbeitet, wobei die exakte Kontrolle bei Prozessabbruch und ähnlichen Sonderfällen nicht garantiert ist.

Vorsicht beim Aufruf überschreibbarer Methoden aus Konstruktoren: Die dynamische Bindung kann bereits eine Methode der Unterklasse erreichen, obwohl deren eigene Felder noch nicht vollständig initialisiert sind. Der Konstruktor einer Oberklasse sollte deshalb insbesondere keine fremd überschreibbaren Methoden aufrufen, deren Ergebnis von Unterklasseninvarianten abhängt.

Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
Shape shape = obtainShape(); // liefert ein Shape-Objekt
shape.draw(); // dynamische Methodenauswahl
if (shape instanceof Circle c) {
 System.out.println(c.getRadius());
}
```

Originalfolien mit fachlicher Einordnung

V · Folie 21: TYPEN, SUBTYPEN UND TYPHIERARCHIEN

Die Hierarchie ist transitiv: Wenn `ClockTimer` ein `Timer` ist und `Timer` ein `Object`, ist `ClockTimer` auch ein `Object`.

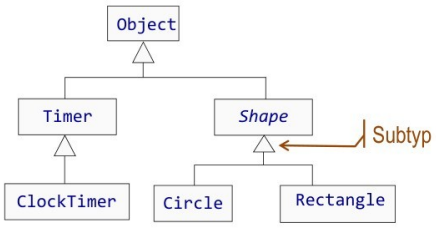
Typen, Subtypen und Typhierarchien

Typ

- eine **Klasse** führt einen **Typ** ein
- Typ wird für die **Deklaration von Variablen**, **Parameter** etc. verwendet
- Der Typ bestimmt, **welche Operationen verfügbar** sind

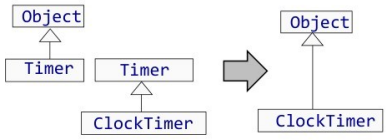
Subtyp

- Mit **Vererbung** wird eine **Subtyp-Beziehung** eingeführt
- Ein **Subtyp** ist mit seinem **Supertyp kompatibel**



Typhierarchie

- **Subtyp-Beziehung** ist **transitiv**
- Mit einer **Klassenhierarchie** ist eine **Typhierarchie** verbunden



© H. Prähofer, 2026 21

Originalabbildung: 02.Vererbung.pdf, Folie 21.

V · Folie 22: TYPKOMPATIBILITÄT UND POLYMORPHE ZUWEISUNG

Die polymorphe Zuweisung ist nur eine Konversion einer Referenz zu einem breiteren Typ. Das Objekt selbst bleibt unverändert, und seine Laufzeitklasse wird nicht in die Oberklasse verwandelt.

Typkompatibilität und polymorphe Zuweisung

Typkompatibilität

- **Subklasse** ist mit **Superklasse kompatibel**

Vom Compiler geprüft!

Eine Subklasse muss die Schnittstelle der Superklasse mindestens erfüllen

- man kann auf **alles zugreifen**, was **Superklasse erlaubt**
- Überschreibende Methoden müssen bezüglich **Rückgabetyp** und geworfenen **Exceptions kompatibel** sein

siehe Abschnitt zu Kovarianz bei Rückgabewerten und Exceptions

Polymorphe Zuweisung

- **Typkompatibilität** ermöglicht eine **Subklasse immer dort einzusetzen**, wo **Superklasse verlangt wird**
- Einer Variablen eines **Supertyps** kann ein **Objekt eines Subtyps** zugewiesen werden
- **Subtyp-Beziehung** ist **transitiv**

Supertyp

```
Shape shape;
shape = new Circle(10, 10, 5);
shape = new Rectangle(10, 10, 20, 10);
```

Subtyp

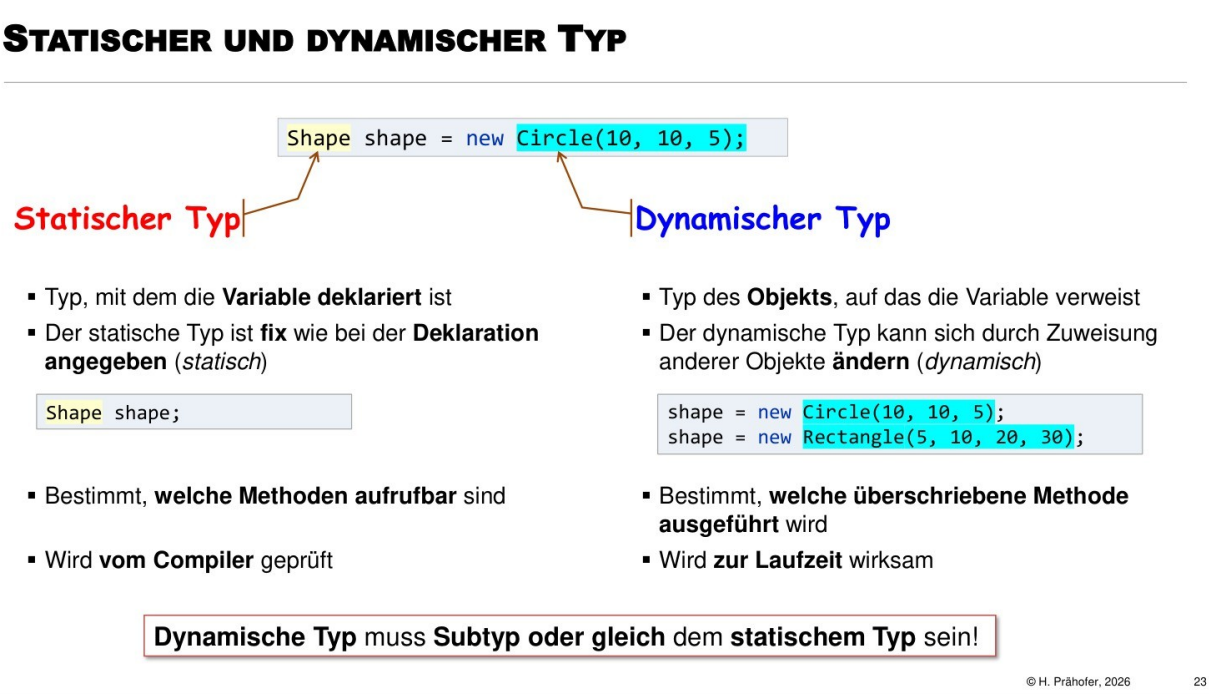
```
Object object;
object = new Circle(10, 10, 5);
object = new Rectangle(10, 10, 20, 10);
```

© H. Prähofer, 2026 22

Originalabbildung: 02.Vererbung.pdf, Folie 22.

V · Folie 23: STATISCHER UND DYNAMISCHER TYP

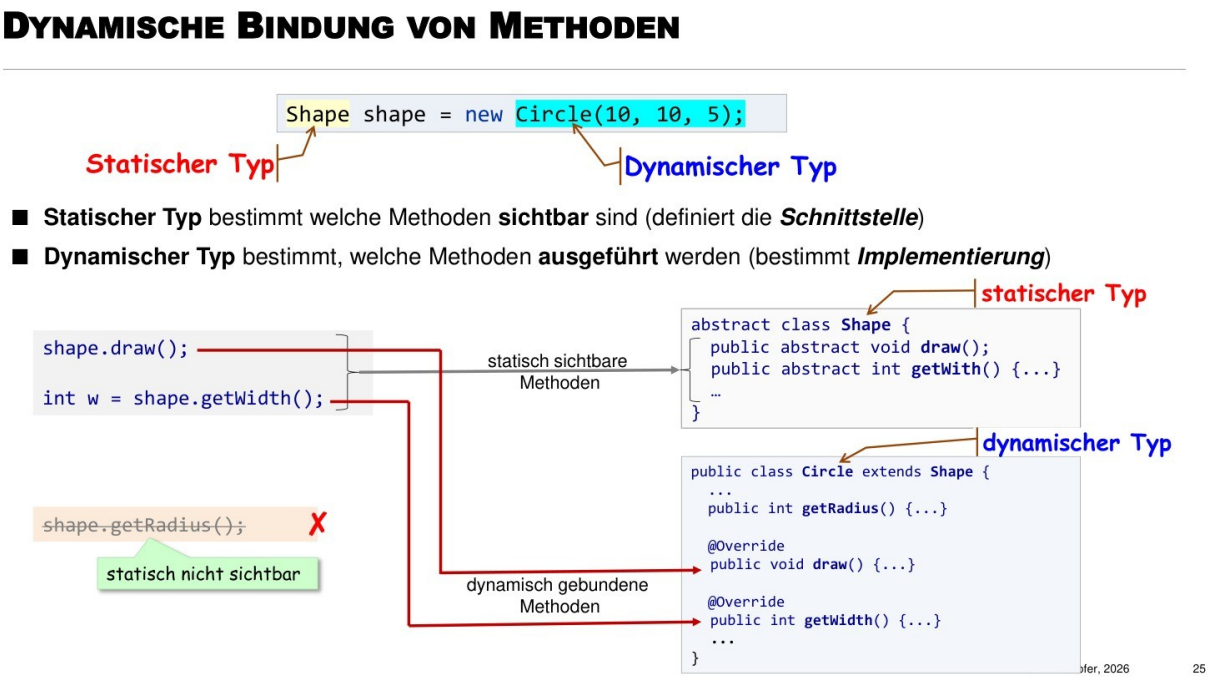
In `Shape shape = new Circle(...)` beschränkt `Shape` statisch die aufrufbaren Methoden; `Circle` legt die Implementierung überschreibbarer Instanzmethoden fest.



Originalabbildung: 02.Vererbung.pdf, Folie 23.

V · Folie 24: DYNAMISCHE BINDUNG VON METHODEN

Die nicht zu `Shape` gehörende Methode `getRadius()` ist über die `Shape`-Referenz nicht unmittelbar aufrufbar. Ein geeigneter Typtest und Cast schaffen eine sichere engere Sicht.



Originalabbildung: 02.Vererbung.pdf, Folie 24.

V · Folie 25: POLYMORPHE VERARBEITUNG

Das Array kann heterogene `Shape`-Untertypen halten. **Korrektur:** Im Beispiel mit Arraylänge 10 sind nur zwei Elemente initialisiert; eine for-each-Schleife über alle zehn Elemente benötigt deshalb eine Nullprüfung oder eine vollständig belegte Struktur.

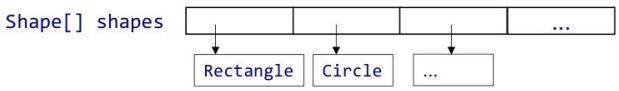
Polymorphe Verarbeitung

■ **Unterschiedliche Objekte** können auf **einheitliche Weise** verarbeitet werden

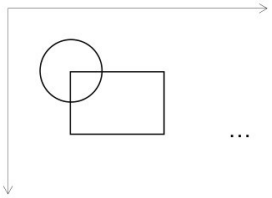
Beispiel: Array mit Objekten unterschiedlicher **Shape**-Unterklassen

```
Shape[] shapes = new Shape[10];
shapes[0] = new Rectangle(20, 20, 30, 20);
shapes[1] = new Circle(20, 20, 10);
...
```

Polymorphe Zuweisungen




```
for (Shape shape: shapes) {
    shape.draw();
}
```



© H. Prähofer, 2026 26

Originalabbildung: 02.Vererbung.pdf, Folie 25.

V · Folie 26: LAUFZEIT-TYPTEST

`instanceof` fragt den dynamischen Typ ab. Ein Kreis ist zugleich `Circle`, `Shape` und `Object`, aber nicht `Rectangle` oder `ColoredShape`.

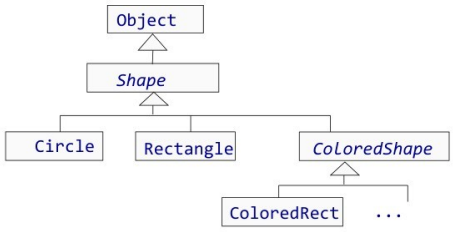
Laufzeit-Typtest

```
Shape shape = …
```

Prüfen des Typ des Objekts zur Laufzeit

```
if (shape instanceof Circle) ...
```

true, wenn **shape** vom dynamischen Typ **Circle** (oder einer **Subklasse** davon) ist.



Beispiele

```
Shape shape = new Circle(10, 10, 20);
```

```
if (shape instanceof Object)...      true
if (shape instanceof Shape)...       true
if (shape instanceof Circle) ...     true
if (shape instanceof Rectangle)...   false
if (shape instanceof ColoredShape)   false
...
```

© H. Prähofer, 2026 27

Originalabbildung: 02.Vererbung.pdf, Folie 26.

V · Folie 27: TYPUMWANDLUNG (TYPECAST)

Ein Cast ändert nur die statische Sicht. Er scheitert mit `ClassCastException`, wenn das konkrete Objekt nicht zum Zieltyp passt.

Typumwandlung (Typecast)

Mit einem Typecast kann die statische Sicht auf einen Wert geändert werden

```
Shape shape = new Circle(5, 5, 10);
  (Circle) shape
```

Sicht des Wertes von **shape** wird auf **Circle** erweitert !

- damit kann der Wert als **Circle** verwendet werden

```
((Circle) shape).getRadius();
```

Bedingungen

- Zieltyp des Casts muss zum **statischen Typ kompatibel** sein ➔ sonst **Compiler-Fehler**

```
... (Timer) shape ...
```

✗ Compiler-Fehler: **Shape** inkompatibel mit Typ **Timer**

- Objekt zur Laufzeit muss vom Zieltyp oder Subtyp sein ➔ sonst **ClassCastException**

```
... (Rectangle) shape ...
```

↯ Laufzeitfehler: **ClassCastException**, Wert ist nicht vom Typ **Rectangle** (sondern Circle)

© H. Prähofer, 2026 28

Originalabbildung: 02.Vererbung.pdf, Folie 27.

V · Folie 28: KOMBINATION TYPTEST UND TYPECAST

Pattern Matching vereinigt `instanceof`, Cast und deklarierte lokale Variable. Die Pattern-Variable ist nur im logisch zulässigen Bereich verfügbar.

Kombination Typtest und Typecast

- Sicherer Zugriff: Typtest gefolgt von Typecast

```
if (shape instanceof Rectangle) {
  Rectangle rect = (Rectangle) shape;
  rect.rotate();
}
```

häufiger Anwendungsfall

- Neuer instanceof-Test mit "Pattern Matching"
  - ☐ kombiniert Typtest, Typecast und Variablenbindung

seit Java 16

Typtest auf **Rectangle**

Variablenbindung an "Pattern-Variable" vom Typ **Rectangle**

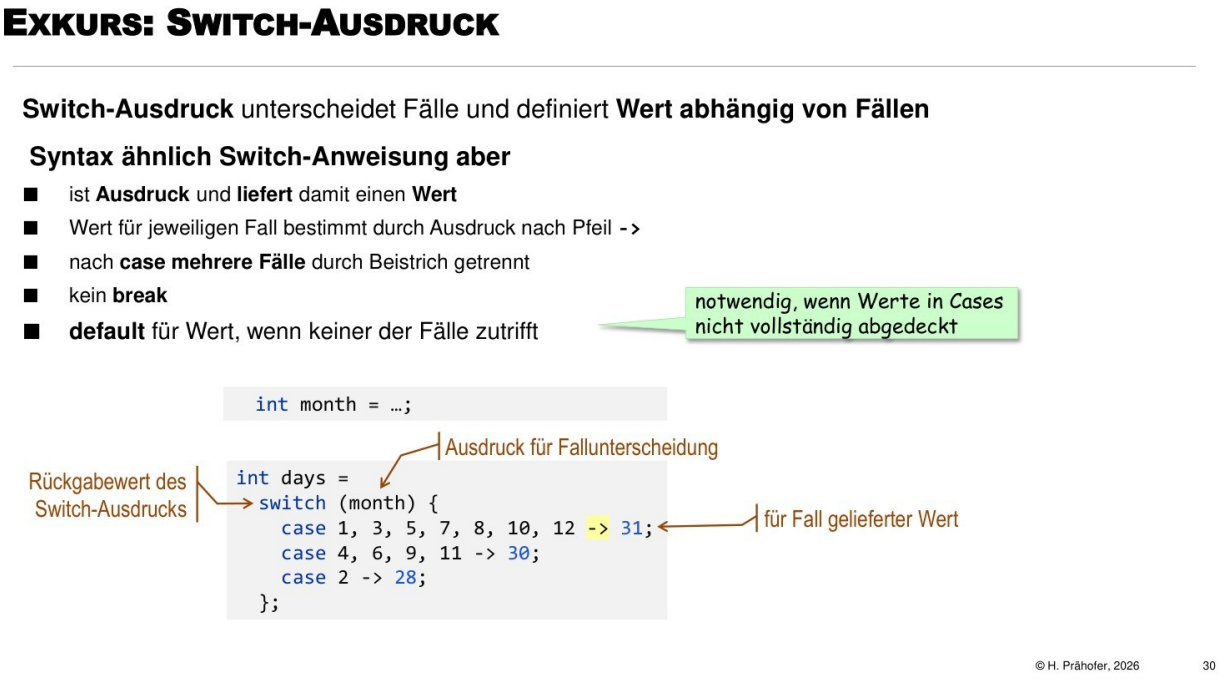
```
if (shape instanceof Rectangle rect) {
  rect.rotate();
}
```

© H. Prähofer, 2026 29

Originalabbildung: 02.Vererbung.pdf, Folie 28.

V · Folie 29: EXKURS: SWITCH-AUSDRUCK

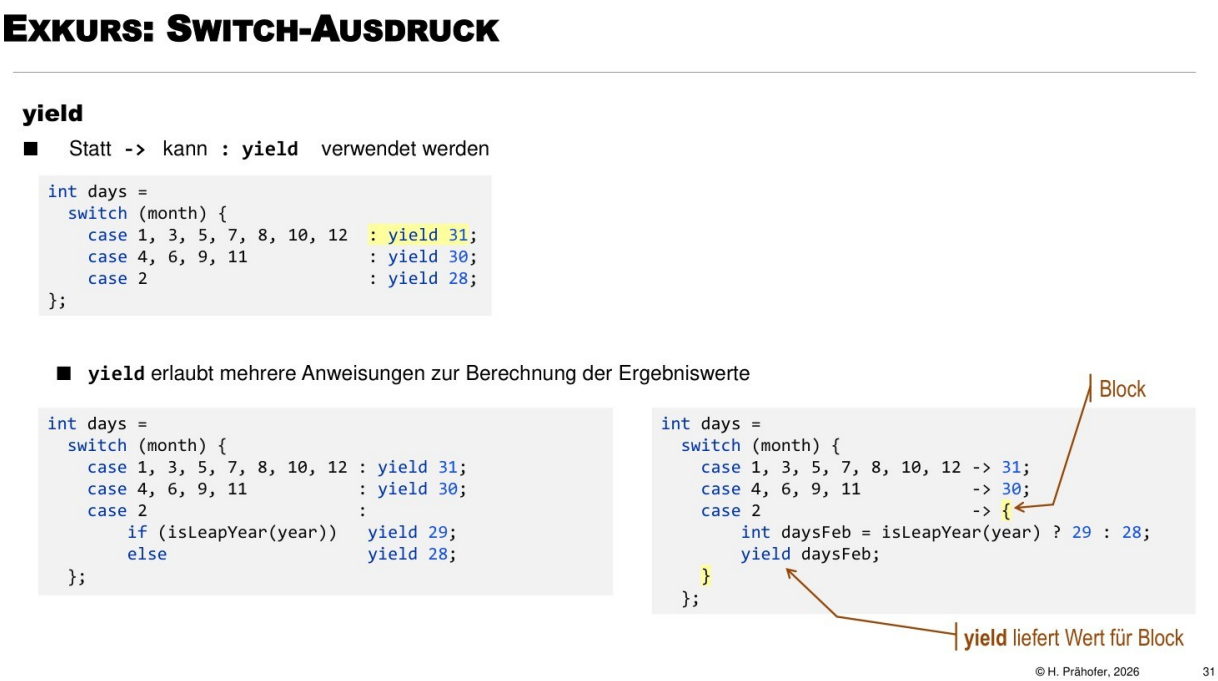
Ein `switch`-Ausdruck muss einen Ergebniswert für sämtliche möglichen Fälle liefern. **Korrektur:** Das gezeigte Monatsbeispiel ohne `default` ist für einen beliebigen `int month` nicht vollständig und kompiliert als Ausdruck so nicht.



Originalabbildung: 02.Vererbung.pdf, Folie 29.

V · Folie 30: EXKURS: SWITCH-AUSDRUCK

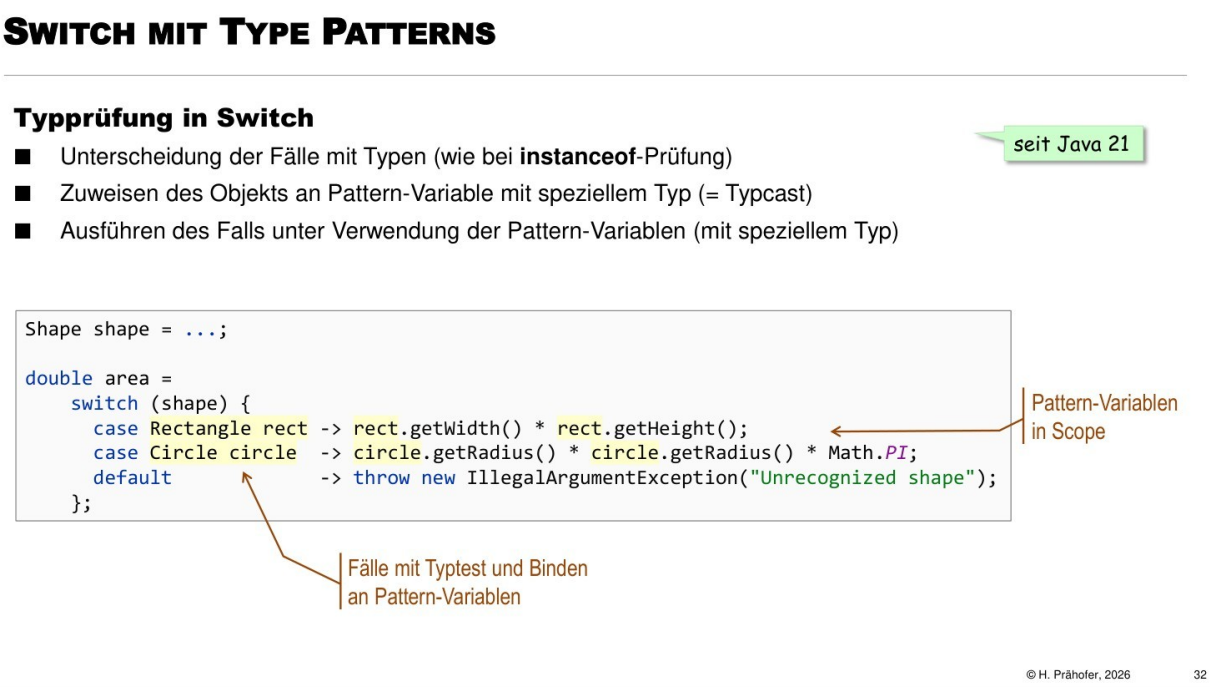
`yield` liefert das Ergebnis eines Switch-Case-Blocks, wenn dieser mehrere Anweisungen enthält. Das ist nicht mit `return` aus der umgebenden Methode zu verwechseln.



Originalabbildung: 02.Vererbung.pdf, Folie 30.

V · Folie 31: SWITCH MIT TYPE PATTERNS

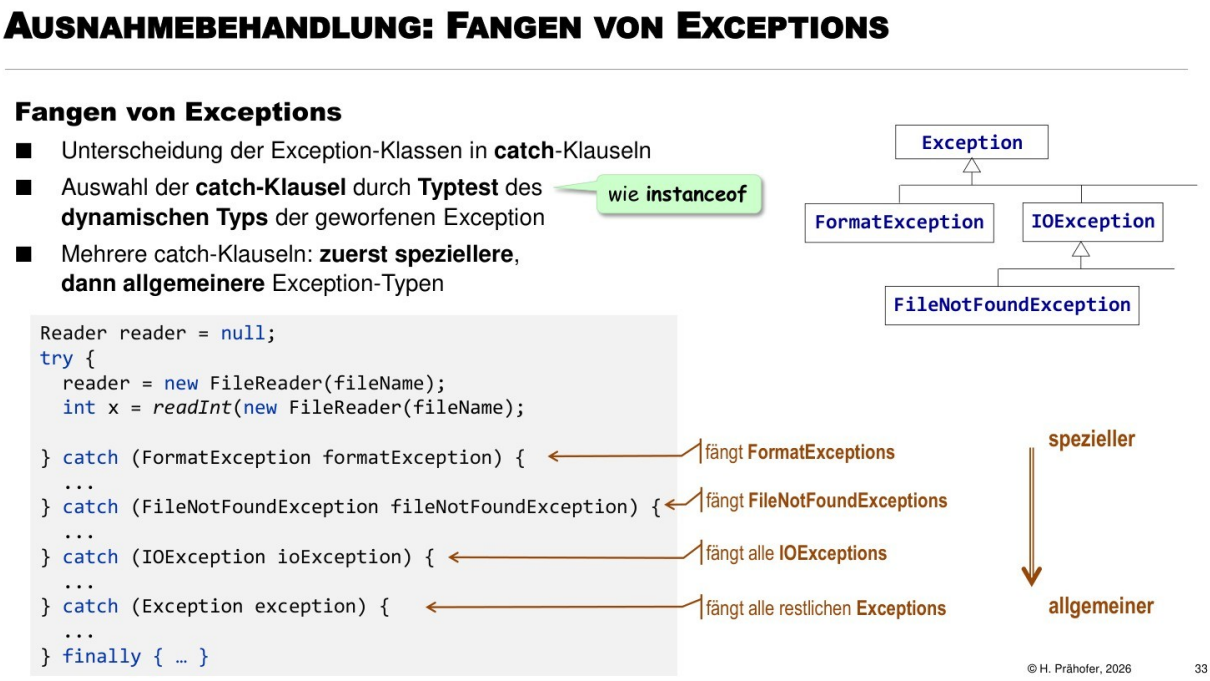
Typmuster im `switch` wählen je nach Laufzeittyp den passenden Case. Das Beispiel berechnet Flächen verschiedener `Shape`-Varianten und fängt andere Typen im `default` ab.



Originalabbildung: 02.Vererbung.pdf, Folie 31.

V · Folie 32: AUSNAHMEBEHANDLUNG: FANGEN VON EXCEPTIONS

Die speziellere `Exception` muss in einer mehrstufigen `catch`-Kette vor dem allgemeineren Typ stehen. **Korrektur:** In der Folie fehlt im Beispielaufruf von `readInt(new FileReader(fileName))` eine schließende Klammer.

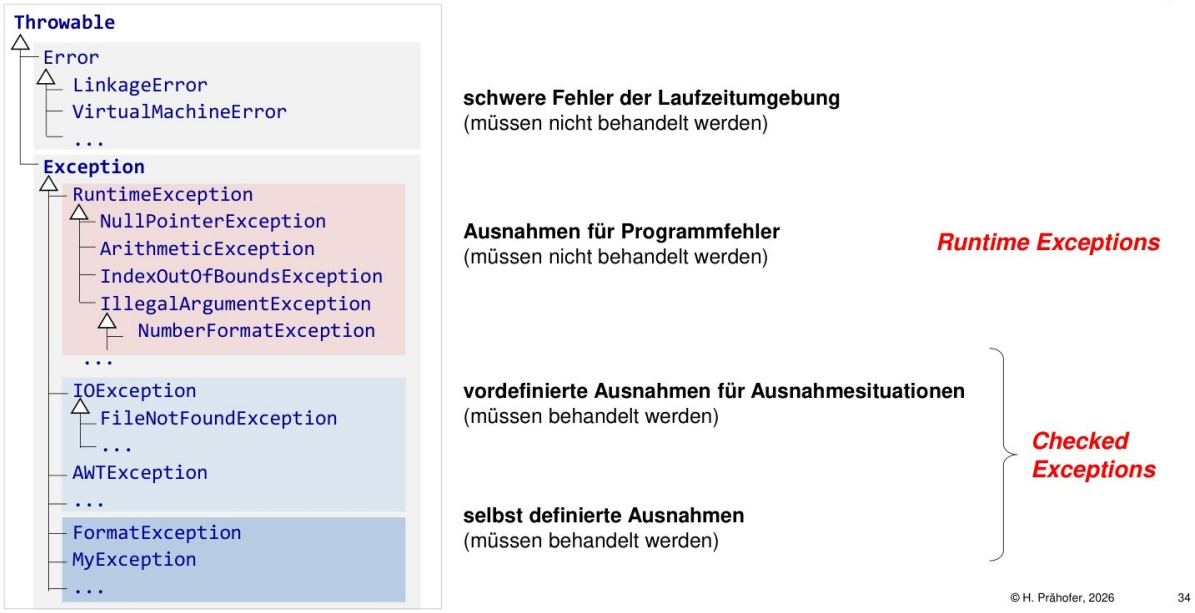


Originalabbildung: 02.Vererbung.pdf, Folie 32.

V · Folie 33: HIERARCHIE DER EXCEPTION-KLASSEN

Die `Exception`-Hierarchie erklärt Checked/Unchecked: `RuntimeException`-Unterklassen brauchen keine `throws`-Deklaration; checked `Exception`-Unterklassen müssen behandelt oder deklariert werden.

Hierarchie der Exception-klassen

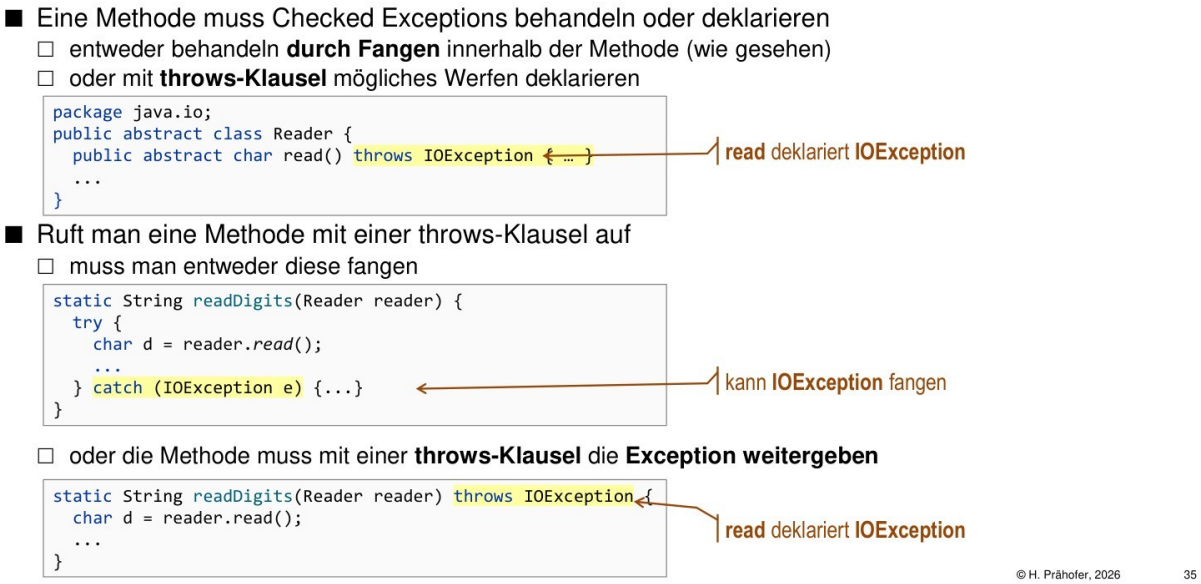


Originalabbildung: 02.Vererbung.pdf, Folie 33.

V · Folie 34: THROWS-KLAUSEL BEI METHODEN

Korrektur: Eine `abstract`-Methode besitzt keinen Körper; die Schreibweise `public abstract char read() throws IOException { ... }` ist ungültig und muss als abstrakte Deklaration mit Semikolon stehen.

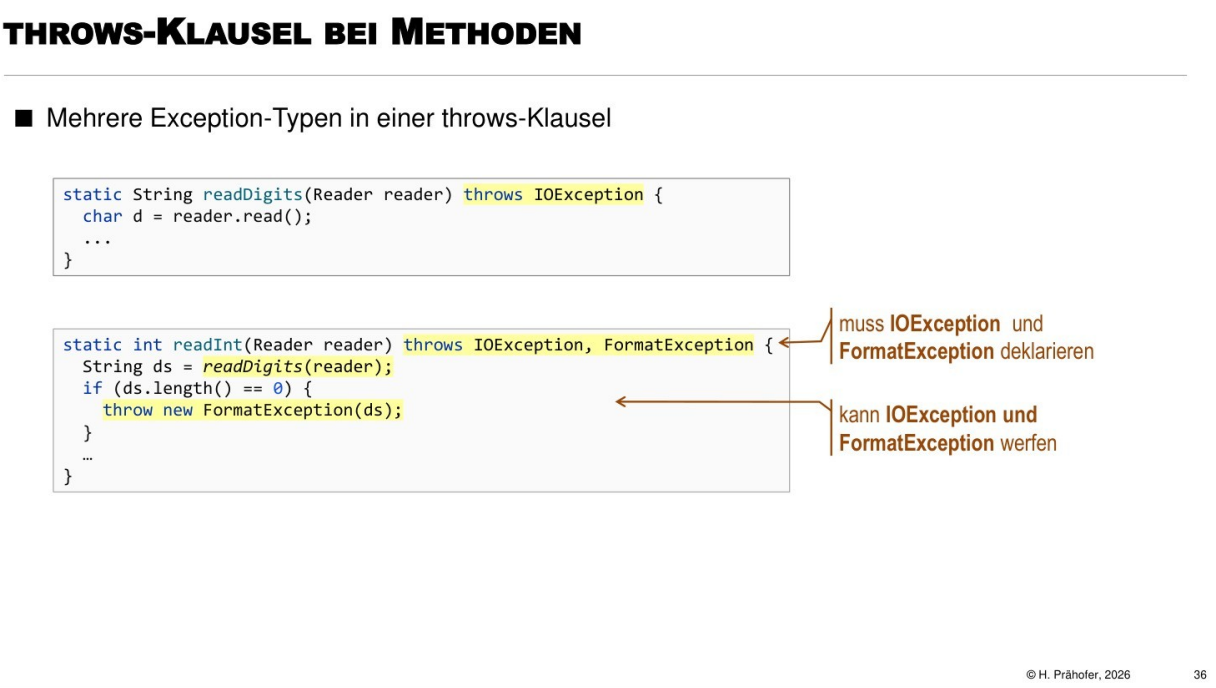
throws-Klausel bei Methoden



Originalabbildung: 02.Vererbung.pdf, Folie 34.

V · Folie 35: THROWS-KLAUSEL BEI METHODEN

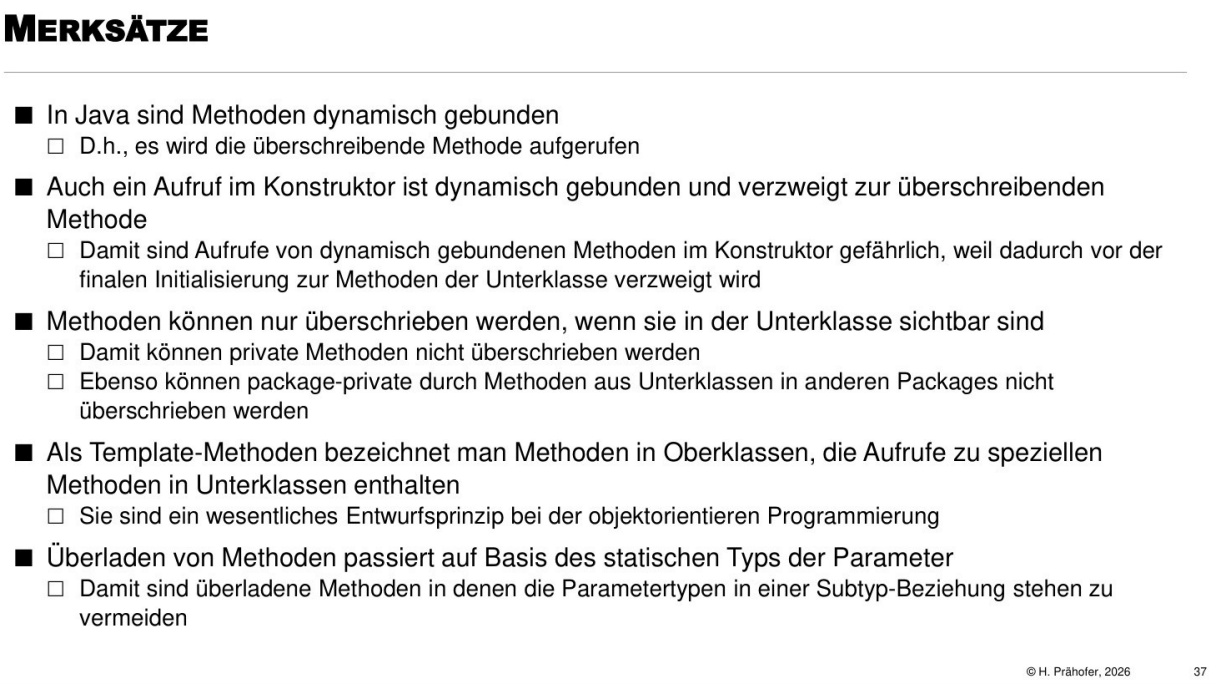
Eine Methode kann mehrere checked Exceptions weiterreichen. Aufrufer müssen die deklarierten Typen über `try/catch` oder eine eigene `throws`-Klausel abdecken.



Originalabbildung: 02.Vererbung.pdf, Folie 35.

V · Folie 36: MERKSÄTZE

Die Merksätze verknüpfen dynamische Bindung, Kontrakte und Entwurfsmuster. **Präzisierung:** Nicht alle Java-Methoden werden virtuell gebunden; `static` und `private` sind keine überschreibbaren virtuellen Methoden.



Originalabbildung: 02.Vererbung.pdf, Folie 36.

2.3 Kovariante Rückgabetypen und geprüfte Exceptions

Beim Überschreiben einer Methode muss die Schnittstelle der Unterklasse für Benutzer des Oberklassentyps kompatibel bleiben. Bei **kovarianten Rückgabetypen** darf eine überschreibende Methode einen spezielleren Referenz-Rückgabetyp deklarieren. Beispielsweise verspricht `Person.myWork()` ein Ergebnis vom Typ `Work`;

`Student.myWork()` darf `Study` zurückliefern, wenn `Study` ein Untertyp von `Work` ist. Eine Variable `Work work = person.myWork();` bleibt dann korrekt, unabhängig davon, ob `person` zur Laufzeit ein `Student` oder `Professor` ist. Für primitive Rückgabetypen gilt diese freie Verengung nicht.

Eine ähnliche Regel betrifft **checked Exceptions** in der `throws`-Klausel. Wenn die Oberklassenmethode `doWork(Work)` eine `WorkException` deklariert, darf die Überschreibung dieselbe, weniger oder speziellere geprüfte Exceptions angeben. Sie darf aber keine zusätzlichen allgemeineren checked Exceptions verlangen, die die Aufrufer der Oberklassenmethode vorher nicht behandeln mussten. Unterklassen können sogar ganz ohne geprüfte `Exception` implementieren. Die Deklaration des statischen Typs bleibt für den Aufrufer verbindlich: Bei `Person p = new Genius(); p.doWork(w);` muss der Compiler weiterhin die in `Person` deklarierte geprüfte `Exception` berücksichtigen, obwohl die konkrete `Genius`-Methode möglicherweise keine auslöst. Unchecked Exceptions unterliegen dieser einschränkenden Regel nicht.

Die kovariante Rückgaberegel gilt nur für Referenztypen. Ist `Work` der Rückgabetyp der Basisklasse, darf eine überschreibende Methode `Study` zurückgeben, sofern `Study` ein Untertyp von `Work` ist. Dagegen kann man `int` nicht durch `long` ersetzen, obwohl Java Zahlen bei einem gewöhnlichen Zuweisungsausdruck in diese Richtung konvertieren kann. Ebenso begrenzt eine Oberklassenmethode mit `throws IOException` ihre Überschreibungen: Diese dürfen beispielsweise `FileNotFoundException`, aber keine zusätzliche allgemeinere geprüfte `Exception` deklarieren.

Der Compiler orientiert sich an der aufgerufenen Methodendeklaration des statischen Empfängertyps. Bei `Base x = new Derived();` muss ein Aufruf `x.work()` diejenigen checked Exceptions berücksichtigen, die `Base.work()` deklariert. Dass die konkrete Methode in `Derived` bei diesem Aufruf keine solche `Exception` wirft, beseitigt die Pflichten für Quelltext mit statischem Typ `Base` nicht.

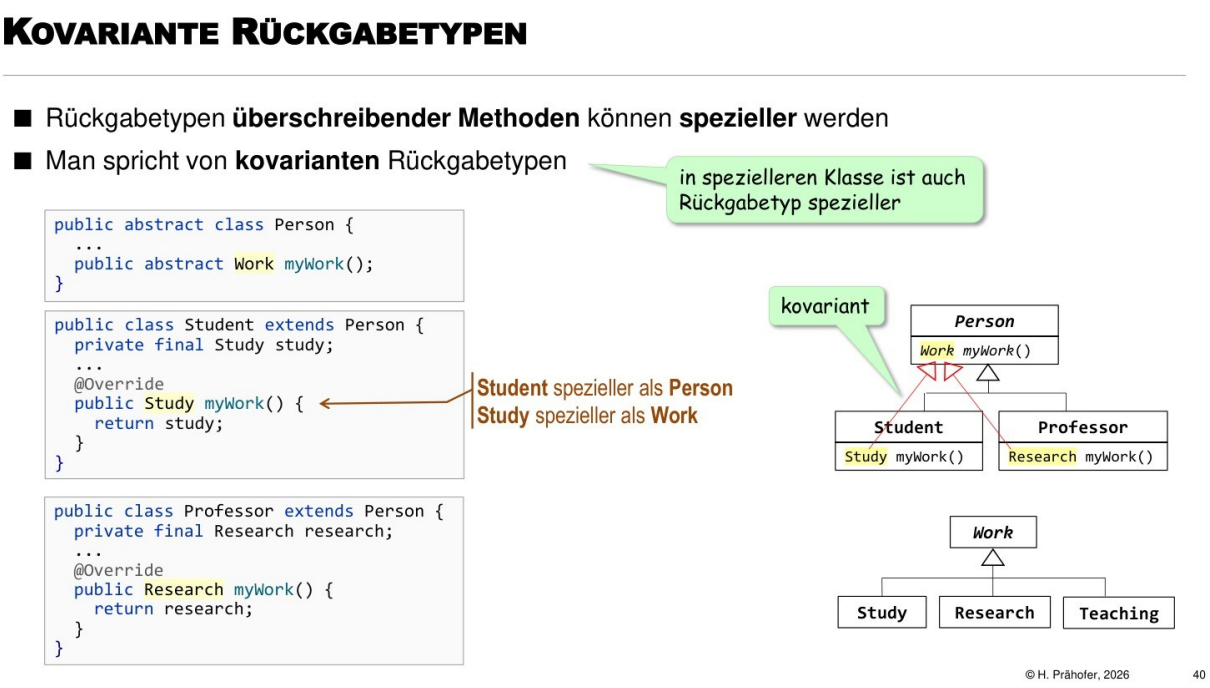
Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
class Base {
 Number value() throws java.io.IOException { return 1; }
}
class Derived extends Base {
 @Override Integer value() { return 2; }
}
```

Originalfolien mit fachlicher Einordnung

V · Folie 39: KOVARIANTE RÜCKGABETYPEN

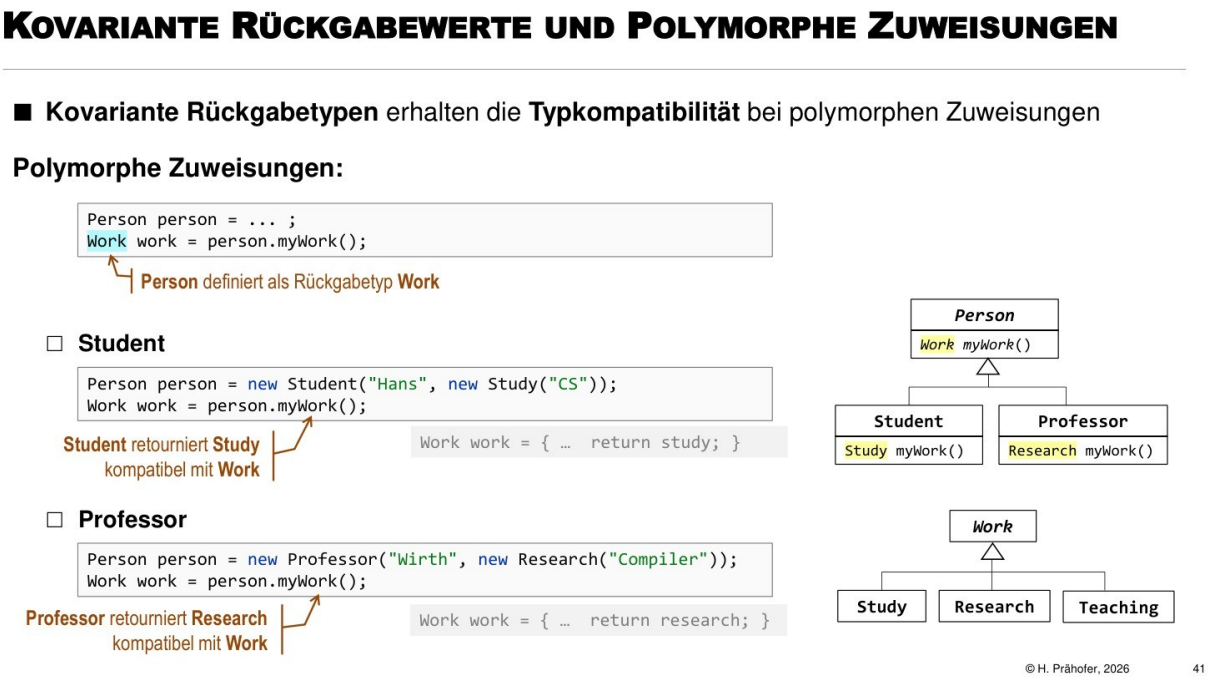
Die Unterklassen `Student` und `Professor` dürfen speziellere Untertypen von `Work` zurückgeben. So bleibt die Rückgabe für alle Aufrufer mit statischem Typ `Person` nutzbar.



Originalabbildung: 02.Vererbung.pdf, Folie 39.

V · Folie 40: KOVARIANTE RÜCKGABEWERTE UND POLYMORPHE ZUWEISUNGEN

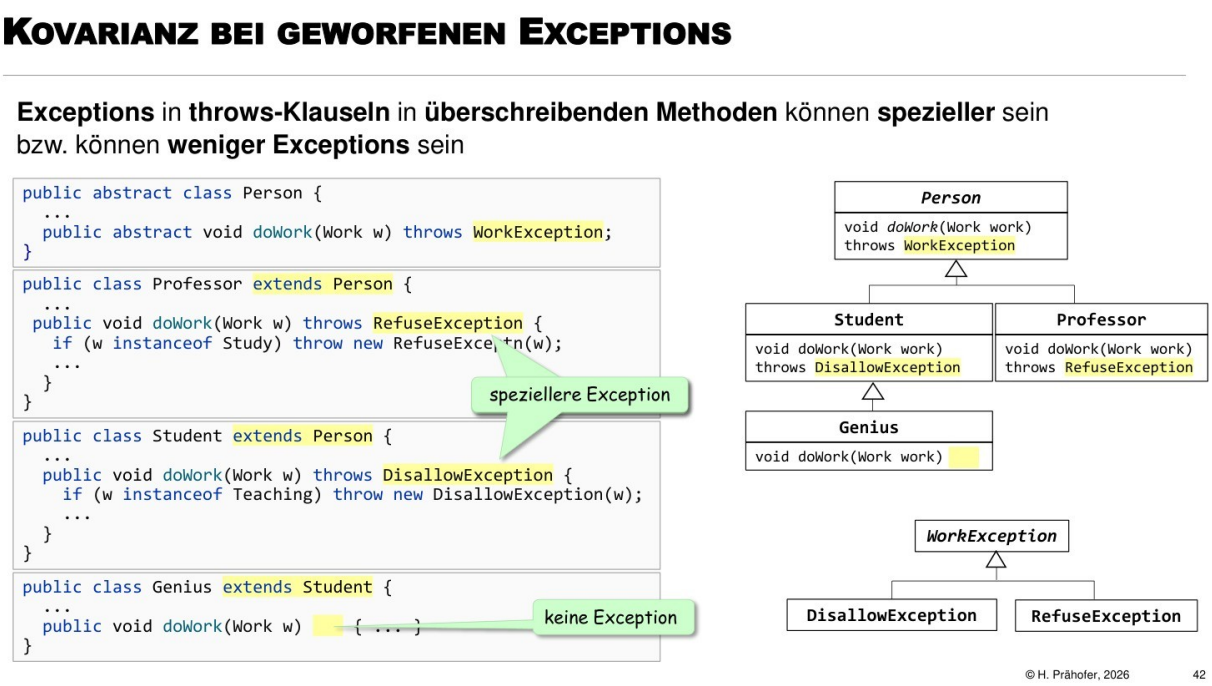
Die polymorphe Zuweisung ändert weder das zurückgegebene Objekt noch dessen dynamischen Typ. Der Rückgabewert der Unterklasse ist nur zusätzlich als Oberklassen-Rückgabetyp verwendbar.



Originalabbildung: 02.Vererbung.pdf, Folie 40.

V · Folie 41: KOVARIANZ BEI GEWORFENEN EXCEPTIONS

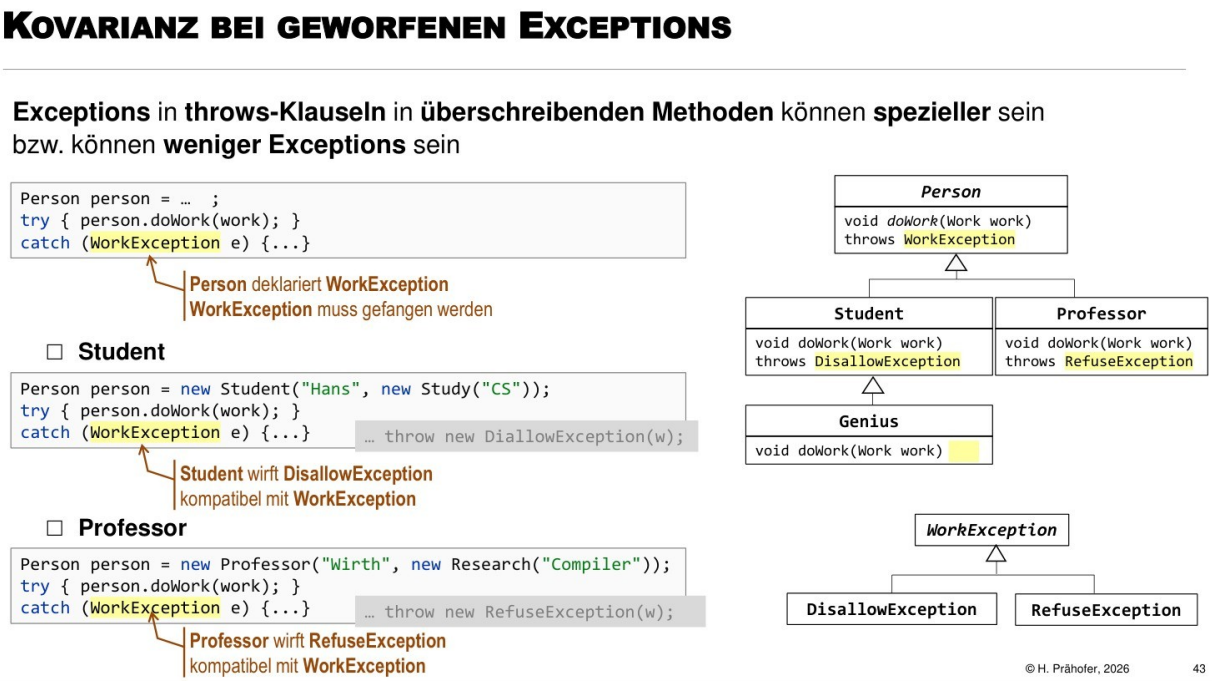
Die Regeln für `throws` beziehen sich auf geprüfte Exceptions. Die Unterklasse darf eine spezielle `RefuseException` statt der allgemeineren `WorkException` deklarieren.



Originalabbildung: 02.Vererbung.pdf, Folie 41.

V · Folie 42: KOVARIANZ BEI GEWORFENEN EXCEPTIONS

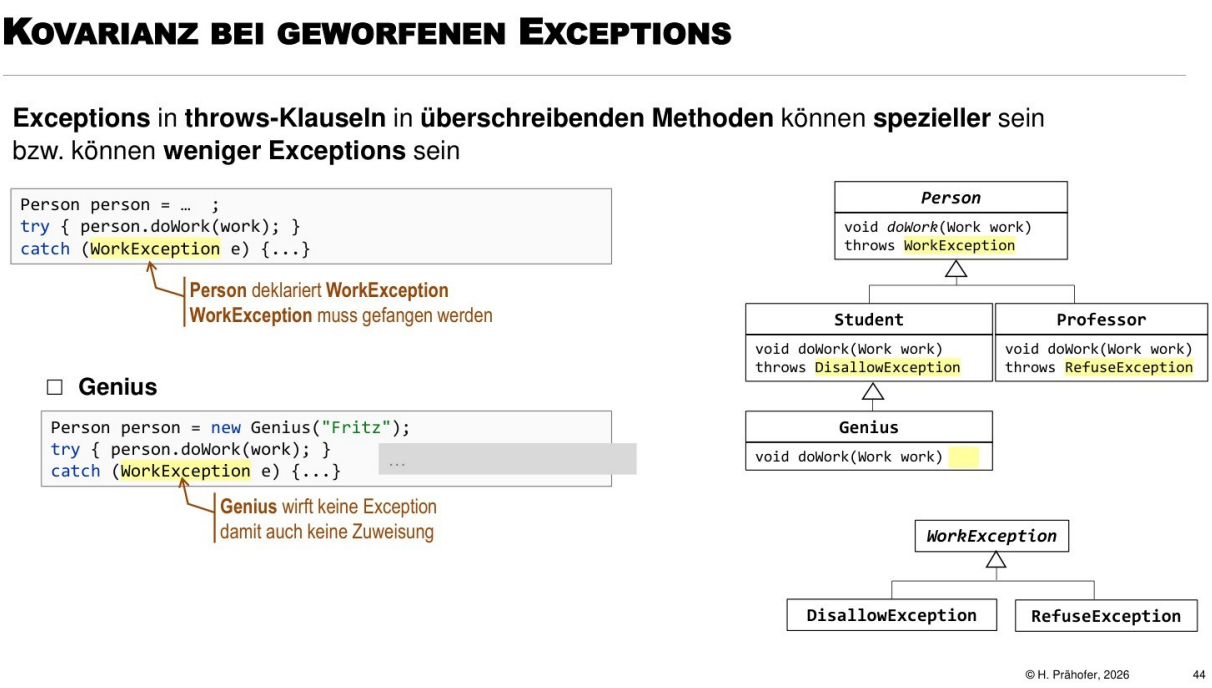
Auch wenn das Objekt zur Laufzeit ein `Professor` ist, entscheidet der deklarierte `Person`-Vertrag, welche geprüften Ausnahmen der Aufrufer behandeln muss.



Originalabbildung: 02.Vererbung.pdf, Folie 42.

V · Folie 43: KOVARIANZ BEI GEWORFENEN EXCEPTIONS

Eine Überschreibung darf sogar keine geprüfte `Exception` werfen. Trotzdem muss der Client bei einem Methodenaufruf über `Person` die Möglichkeit einer `WorkException` berücksichtigen.



Originalabbildung: 02.Vererbung.pdf, Folie 43.

2.4 Fallstudie List: Basisklasse und Implementierungen

Die Fallstudie `List` verfolgt einen Entwurf in mehreren Evolutionsstufen. Eine Liste ist eine geordnete Folge von Elementen, auf die etwa über einen Index zugegriffen wird. Die öffentliche Schnittstelle stellt Operationen wie `size()`, `get(idx)`, `indexOf(o)`, `contains(o)`, `add(elem)`, `add(idx,elem)`, `remove(...)` und `clear()` bereit. Eine erste abstrakte Klasse kann gemeinsame Operationen wie `contains` über bereits vorhandene abstrakte Methoden implementieren: `contains(o)` ist wahr, wenn `indexOf(o) >= 0` ist. Auf diese Weise müssen Implementierungen nicht alle Methoden duplizieren.

`LinkedList` und `ArrayList` unterscheiden sich nicht primär durch ihr äußeres Verhalten, sondern durch ihre **Speicherrepräsentation**. Eine doppelt verkettete Liste hält Knoten, die neben dem Element Referenzen auf Vorgänger und Nachfolger enthalten. Einfügen oder Entfernen bei bereits bekanntem Knoten kann lokal stattfinden; für den Zugriff auf ein beliebiges Element ist typischerweise ein Durchlaufen der Verkettung notwendig. Ein Array speichert Elemente an aufeinanderfolgenden Indexpositionen; Lesen anhand eines gültigen Index ist direkt möglich, während Einfügen in der Mitte eine Verschiebung erfordert. Ein wachsendes Array muss bei Kapazitätserschöpfung häufig durch ein größeres Array ersetzt werden.

Der erste `List`-Entwurf benutzt Elemente vom allgemeinen Typ `Object`. Das ist zwar flexibel, erzwingt beim Lesen aber häufig Casts und bietet keine statische Garantie, dass alle Elemente dem erwarteten Typ entsprechen. Später werden Generics diese Schwäche beheben. Außerdem führt zusätzliche Funktionalität wie Logging zunächst zu einem Vererbungsproblem: Soll für jede konkrete Speicherrepräsentation eine Logging-Unterklasse angelegt werden, wachsen die Kombinationen schnell. Ab hier motiviert die Fallstudie Wrapper und Decorators.

Wichtig ist die Abgrenzung zur echten Java-`Collections`-API: Die in der Vorlesung selbst entworfenen Klassen `List`, `ArrayList` und `LinkedList` sind **didaktische Modelltypen**. Die tatsächlichen Typen `java.util.List`, `java.util.ArrayList` und `java.util.LinkedList` haben eine umfangreichere,

teilweise andere API und Hierarchie. Das Skript verwendet die Folienbezeichnungen im Kontext der Fallstudie und kennzeichnet Unterschiede, statt die Skizzen als vollständige Originalimplementierungen der Java-Standardbibliothek auszugeben.

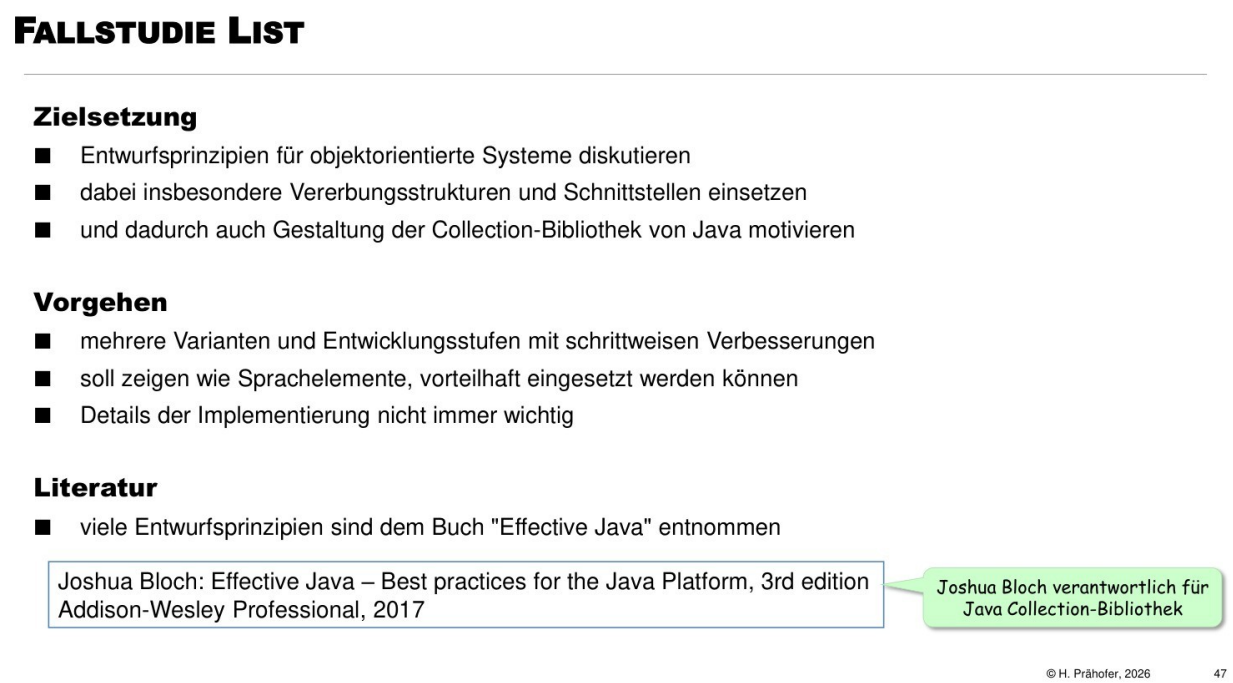
Die Laufzeitkosten ergeben sich aus den erforderlichen Speicherzugriffen. Bei einer verketteten Liste benötigt `get(i)` im einfachen Vorwärtsmodell bis zu `i` Folgeschritte und damit im schlechtesten Fall `O(n)` Zeit. Ein lineares Durchlaufen mit `get(0)`, `get(1)` bis `get(n-1)` kostet deshalb insgesamt $O(n^2)$. Ein `Iterator`, der den aktuellen Knoten hält und jeweils die nächste Referenz verfolgt, durchläuft die Liste dagegen in `O(n)` Zeit.

Bei einer Arrayliste kann `get(i)` das Element direkt adressieren und benötigt für einen gültigen Index `O(1)` Zeit. Das Verschieben nach einem Einfügen oder Entfernen im mittleren Bereich erfordert dagegen `O(n)` Zeit. Das Anfügen kann bei genügend Kapazität `O(1)` kosten; eine Kapazitätserweiterung benötigt gelegentlich `O(n)` Kopierarbeit. Diese Unterschiede motivieren später getrennte Interfaces für sequentielle und indexbasierte Operationen.

Originalfolien mit fachlicher Einordnung

V · Folie 46: FALLSTUDIE LIST

Die Fallstudie modelliert schrittweise eine Sammlung und motiviert die Java-`Collections`-Architektur. Die hier gezeigten Typen sind didaktische Entwürfe, keine vollständigen Originalklassen aus `java.util`.



Originalabbildung: 02.Vererbung.pdf, Folie 46.

V · Folie 47: FALLSTUDIE LIST

Neben Array- und Link-Repräsentation geht es um Zusatzverhalten sowie die Integration externer Strukturen. Diese Anforderungen führen später zu Decorator und Adapter.

Fallstudie List

Aufgabenstellung

- Implementierung von Listen mit Objektwerten
- mit Varianten in Implementierung: **LinkedList** und **ArrayList**
- mit Zusatzfunktionen, wie z.B. Logging
- mit Einbindung von ähnlichen Strukturen, z.B. Arrays
- Verwendung von Vererbung, Interfaces und schließlich „Entwurfsmuster“

© H. Prähofer, 2026

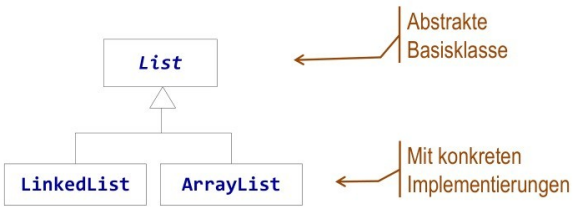
Originalabbildung: 02.Vererbung.pdf, Folie 47.

V · Folie 48: INTERFACE LIST UND IMPLEMENTIERENDE KLASSEN

Eine abstrakte Listenklasse verbindet gemeinsamen Vertrag und Basisimplementierung. Erst spätere Folien trennen die Rolle der Schnittstelle explizit in ein Interface aus.

Interface List und Implementierende Klassen

- Abstrakte Basisklasse **List**
  - ☐ fixiert Schnittstelle
  - ☐ wird als Typ verwendet
  - ☐ für Gemeinsamkeiten in Implementierung
- Konkrete Klassen
  - ☐ **LinkedList**
  - ☐ **ArrayList**
  - ☐ ...



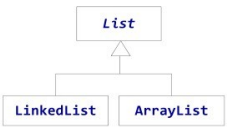
© H. Prähofer, 2026

Originalabbildung: 02.Vererbung.pdf, Folie 48.

V · Folie 49: ListBASISKLASSE LIST

Die Basisklasse kann `contains` mithilfe von `indexOf` implementieren. Die genaue Wirkung von `clear` und `remove` hängt von den definierten Invarianten und Implementierungen ab.

Basisklasse List



Öffentliche Schnittstelle mit

- gemeinsamer Eigenschaft **size**
- abstrakten Methoden
- konkreten Methoden (unter Verwendung von abstrakten Methoden)

```
public abstract class List {
  private int size = 0;

  protected List() {}

  public int size() { return size; }
  abstract Object get(int idx);
  public abstract int indexOf(Object o);
  public boolean contains(Object o) { return indexOf(o) >= 0; }
  public void clear() {…}
  public abstract void add(Object elem);
  public abstract void add(int idx, Object elem);
  public void addAll(Object... elems) {…}
  public abstract Object remove(int idx);
  public boolean remove(Object elem) {…}
  @Override
  public String toString() {…}

  // protected ----------------------------------------
  protected final void incrSize() { size++; }
  protected final void decrSize() { size--; }
  protected final void resetSize() { size = 0; }
  protected final void assertValidIndex(int idx) {…}
}
```

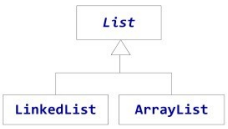
© H. Prähofer, 2026 50

Originalabbildung: 02.Vererbung.pdf, Folie 49.

V · Folie 50: ListKONKRETE KLASSE: LINKEDLIST

Die doppelte Verkettung speichert Vorgänger- und Nachfolgerreferenzen. Besonders sorgfältig sind Änderungen an `head`, `tail` und der `size`-Zählung.

Konkrete Klasse: LinkedList



- als doppelt-verkettete Liste
- mit Knotenklasse **Node**

```
public final class LinkedList extends AbstractList {
  private Node head;
  private Node tail;

  public LinkedList(Object... initial) {
    super();
    addAll(initial);
  }
  @Override
  public Object get(int idx) {
    assertValidIndex(idx);
    Node n = head;
    for (int i = 0; i < idx; i++) {
      n = n.next;
    }
    return n.elem;
  }
  ...
```

- Anmerkungen
  - ☐ Klasse ist **final**
  - ☐ Knotenklasse package-private und **final**
  - ☐ Verkettung schwierig
    - ➔ Hilfsmethoden **link** und **unlink**

```
final class Node {
  Object elem;
  Node next, prev;

  Node(Object elem) {
    this.elem = elem;
  }
}
```

```
@Override
public void add(int idx, Object elem) {
  Node node = new Node(elem);
  if (idx == size()) {  // at end
    link(node, tail, null);
  } else {
    assertValidAddIndex(idx);
    Node p = head;
    for (int i = 0; i < idx; i++) { p = p.next; }
    link(node, p.prev, p);
  }
  incrSize();
}
...
private void link(Node node, Node prev, Node next) { … }
private void unlink(Node node) {…}
}
```

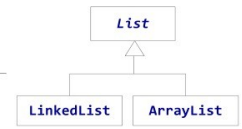
© H. Prähofer, 2026 51

Originalabbildung: 02.Vererbung.pdf, Folie 50.

V · Folie 51: ListKONKRETE KLASSE: ARRAYLIST

Der Array-basierte Speicher bietet schnellen Indexzugriff; für Einfügen und Entfernen müssen Elemente unter Umständen verschoben und die Kapazität erweitert werden.

Konkrete Klasse: ArrayList



- Array zum Speichern der Elemente
- Füllung des Arrays gleich **size**

- Anmerkungen
  - Klasse is **final**
  - Vergrößern des Arrays, wenn voll

```
public final class ArrayList extends AbstractList {

  private static final int CAP = 16;
  private Object[] elems;

  public ArrayList(Object... initial) {
    super();
    elems = new Object[CAP];
    addAll(initial);
  }

  @Override
  public Object get(int idx) {
    assertValidIndex(idx);
    return elems[idx];
  }
  ...
```

```
  @Override
  public void add(int idx, Object elem) {
    assertValidIndex(idx);
    if (elems.length == size()) {
      Object[] resized = new Object[size() * 2];
      System.arraycopy(elems, 0, resized, 0, size());
      elems = resized;
    }
    System.arraycopy(elems, idx, elems, idx + 1, size() - idx);
    elems[idx] = elem;
    incrSize();
  }

  @Override
  public void clear() {
    elems = new Object[CAP];
    resetSize();
  }
  ...
}
```

© H. Prähofer, 2026 52

Originalabbildung: 02.Vererbung.pdf, Folie 51.

V · Folie 52: PROBLEMANALYSE

Wenn die Liste beliebige `Object`-Werte speichert, ist ein Cast beim Lesen nötig. Das kann erst zur Laufzeit fehlschlagen; Generics lösen später diese Typunsicherheit.

Problemanalyse

Implementierung hat noch eine Reihe von Problemen

- Man kann beliebige Objekte speichern, beim Lesen sind aber **Typecasts** notwendig
  - Typcasts sind eine häufige Fehlerquelle

```
public class LinkedList extends AbstractList {
  protected Node head;
  ...
  public boolean add(Object elem) {...}
  public Object get(int idx) {...}
  ...
}
```

Parametertyp **Object** erlaubt beliebige Objekte

Rückgabetyp garantiert nur **Object**

```
final class Node {
  public Object elem;
  ...
}
```

Knotenwert von Typ **Object**

```
List list = new LinkedList();
```

add von beliebigen Objekten

```
list.add(new Circle(10, 10, 20));
list.add(new Rectangle(10, 10, 20, 30));
list.add(new Timer());
list.add("Hallo");
```

get mit Typcasts

```
Circle sp = (Circle) list.get(0);
Rectangle r = (Rectangle) list.get(1);
Timer t = (Timer) list.get(2);
String s = (String) list.get(3);
```

Fehlerquelle Typecasts: Wenn falscher Zugriff, dann ClassCastException
➔ **Abhilfe durch Generics**

© H. Prähofer, 2026 53

Originalabbildung: 02.Vererbung.pdf, Folie 52.

V · Folie 53: PROBLEMANALYSE

Die nächste Problemanalyse betrifft die öffentliche API und Erweiterbarkeit. Mehr Komfortfunktionen sollten nicht zu einer Explosion spezialisierter Listenklassen führen.

PROBLEMANALYSE

Implementierung hat noch eine Reihe von Problemen

- Man kann beliebige Objekte speichern, beim sind Lesen aber **Typecasts** notwendig
  - ☐ Typcasts sind eine häufige Fehlerquelle

➔ **Lösung: Generics (siehe Kapitel 6)**

- Spezialisierungen sind oft schwierig
  - ☐ Detailwissen der Basisklasse ist notwendig
  - ☐ Erweiterungen sind stark von Basisklasse abhängig
  - ☐ Basisklasse muss für Vererbung korrekt gestaltet sein
    - ➔ Eingreifen in Verhalten der Basisklasse nicht möglich

➔ **Lösung**:
- ☐ Klassen mit einer **vollständigen Schnittstelle** schaffen, sodass weitere Methoden überflüssig
  - ➔ konkrete Klassen dann **final** machen
- ☐ Zusätze bei Methoden dann **nicht durch Vererbung**, sondern durch **Delegation** realisieren
  - ➔ **siehe Entwurfsmuster**

© H. Prähofer, 2026 55

Originalabbildung: 02.Vererbung.pdf, Folie 53.

V · Folie 54: PROBLEMANALYSE: IMPLEMENTIERUNG VON ZUSÄTZEN

Logging lässt sich nicht elegant für jede `List`-Repräsentation als separat programmierte Variante pflegen. Die gewünschte Zusatzfunktion ist orthogonal zur Speicherstrategie.

PROBLEMANALYSE: IMPLEMENTIERUNG VON ZUSÄTZEN

Beispiel

- Listen mit Logging der Methodenaufrufe

Probleme bei Zusätzen durch Vererbung

- Zusätze müssten sowohl für **LinkedList** als auch für **ArrayList** implementiert werden
- Vererbung ist oft schwierig und es kann zu unerwarteten Interaktionen zwischen Superklasse und Subklasse kommen

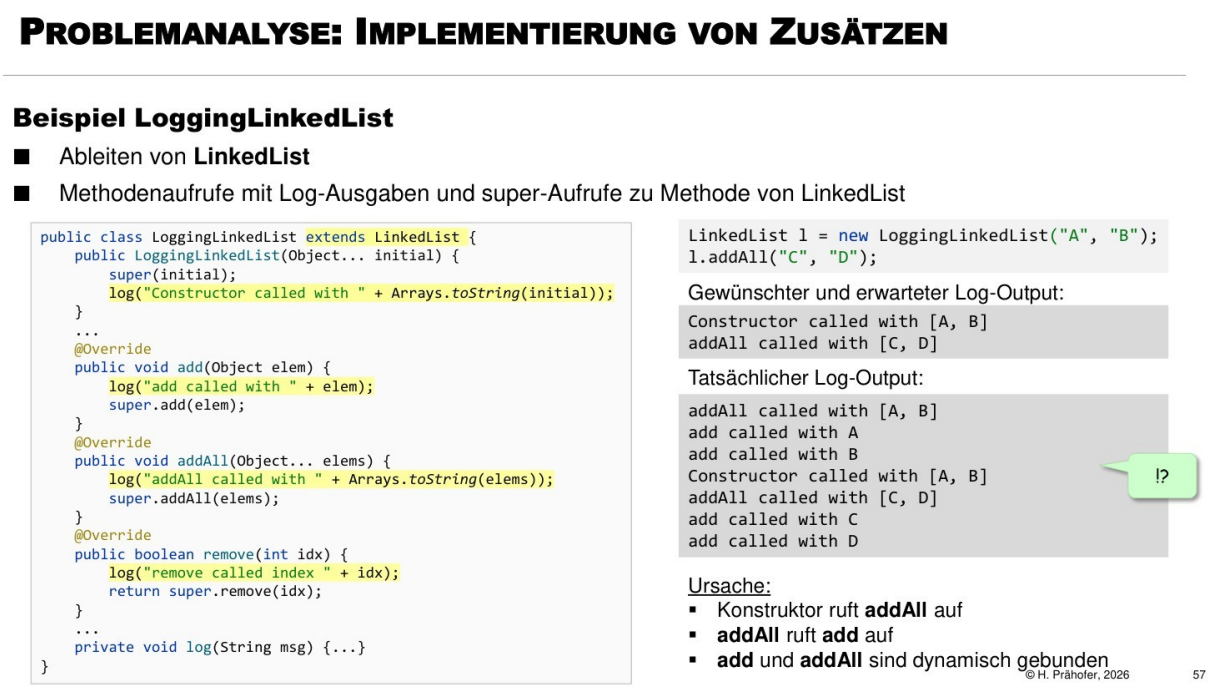
➔ siehe nächste Folie

© H. Prähofer, 2026 56

Originalabbildung: 02.Vererbung.pdf, Folie 54.

V · Folie 55: PROBLEMANALYSE: IMPLEMENTIERUNG VON ZUSÄTZEN

`LoggingLinkedList` koppelt Protokollierung an eine konkrete Datenstruktur. Für eine `ArrayList` wäre eine zweite Logging-Unterklasse nötig; ein Wrapper vermeidet diese Duplikation.



Originalabbildung: 02.Vererbung.pdf, Folie 55.

2.5 Entwurfsmuster: Wrapper, Decorator, Adapter, Factory, Template Method und Strategy

Entwurfsmuster (Design Patterns) sind wiederverwendbare Beschreibungen bewährter Lösungsstrukturen für wiederkehrende Entwurfsprobleme. Sie sind keine fertigen Codebibliotheken, sondern benennen Rollen, Beziehungen, Variationspunkte und Konsequenzen. Die Vorlesung verweist auf das Buch *Design Patterns: Elements of Reusable `Object`-Oriented Software* und erläutert unter anderem Musternamen, Problem, Lösung, Varianten und Auswirkungen. Die UML-Diagramme unterscheiden dabei Generalisierung beziehungsweise Realisierung von Assoziation: Ein Objekt kann ein anderes nur besitzen oder referenzieren, ohne dessen Unterklasse zu sein.

Ein **Wrapper** speichert eine Referenz auf ein anderes Objekt und ergänzt dessen Verwendung. So kann `LoggingListWrapper` eine Liste halten, Aufrufe protokollieren und sie an die eingepackte Liste delegieren. Erreicht der Wrapper dieselbe Schnittstelle wie das Original, kann er für dessen Benutzer transparent einsetzbar sein. Wichtig ist die Unterscheidung zwischen zusätzlichem Verhalten vor und nach einem Aufruf und der tatsächlich weitergeleiteten Operation.

Das Muster **Decorator** ist ein spezieller struktureller Wrapper, der dieselbe Abstraktion wie das dekorierte Objekt anbietet. Ein abstrakter `ListDecorator` hält eine Referenz auf eine `List` und delegiert die Grundoperationen. Konkrete Decorators wie `LoggingListDecorator`, `NullCheckDecorator` oder `UnmodifiableListDecorator` ändern gezielt das Verhalten. Mehrere Decorators können geschachtelt werden: `new LoggingListDecorator(new NullCheckDecorator(list))`. Die Reihenfolge ist dabei relevant, weil die äußere Schicht Aufrufe vor der inneren Schicht sieht. Ein Decorator ist keine Kopie der ursprünglichen Daten: Änderungen durch andere Referenzen auf die dekorierte Sammlung bleiben möglich.

Ein **Adapter** macht eine bereits vorhandene Struktur mit einer anderen erwarteten Schnittstelle verwendbar. In der Fallstudie bietet `ArrayToListAdapter` einem Array eine `List`-Sicht. `SingleToListAdapter`

präsentiert ein einzelnes Objekt als Liste der Länge eins. Die Adapter besitzen die adaptierten Objekte und übersetzen `size()` und `get(idx)` auf deren jeweilige Repräsentation. Ein Adapter muss definieren, welche Änderungen unterstützt werden und ob sein Verhalten eine Live-Sicht oder eine Kopie darstellt. Das Baumknoten-Beispiel illustriert dieselbe Entwurfsidee für unterschiedliche Strukturklassen.

Eine **Factory-Methode** kapselt die Erzeugungsentscheidung. Statt dass Clientcode die genaue konkrete Klasse kennen muss, ruft er beispielsweise `List.of(array)` oder `List.of(single)` auf. Der Rückgabetyp ist die abstrakte `List`; intern entscheidet die Factory, welchen Adapter oder welche Implementierung sie erzeugt. Das erleichtert späteren Wechsel der Implementierung. Der Begriff wird in den Folien etwas breit für statische Fabrikmethoden verwendet; im engeren GoF-Factory-Method-Muster wird die Erzeugungsmethode häufig polymorph von Unterklassen implementiert. Beide Ideen teilen die Entkopplung der Clients von der konkreten Objektklasse.

Das Verhaltensmuster **Template Method** definiert in einer Oberklasse den stabilen Ablauf eines Algorithmus und lässt einzelne Schritte von Unterklassen konkretisieren. In `DataProcessor.process(source)` bilden `loadData`, `parseContent` und `writeInfo` den Ablauf; eine Unterklasse wie `XMLDataProcessor` oder `CSVDataProcessor` liefert die formatspezifische Verarbeitung `parseContent`. Der Gesamtalgorithmus kann `final` sein, damit seine Reihenfolge unverändert bleibt. Die Vorlesungsfolie `List.copyTo(from,to)` zeigt verwandte Wiederverwendung: Durch Aufrufe der `List`-Schnittstelle funktioniert dieselbe Kopieroperation unabhängig von den konkreten Implementierungen.

Beim Muster **Strategy** wird ein austauschbares Verfahren in ein eigenes Objekt ausgelagert. Ein `SortingStrategy` kann beispielsweise `sort(int[])` anbieten; konkrete Implementierungen sind `BubbleSort` und `QuickSort`. Der Kontext besitzt eine Referenz auf die aktuell ausgewählte Strategie und delegiert die Berechnung. Ein anderes Beispiel wählt zwischen `FileLoader` und `NetLoader` für das Laden von Daten. `Comparator<T>` dient als Strategie für die Reihenfolge bei einer Sortierung: Seine Methode `compare(a,b)` liefert einen negativen, `null` oder positiven Wert je nach Ordnung. Die vorgeführte `List`-Sortierung nutzt einen `Comparator` für Vergleiche in einem einfügeartigen Sortierverfahren. Dieselbe Strategieidee wird auch für Suche, Serialisierung, Kompression, Logging und andere austauschbare Vorgehensweisen eingesetzt.

Für die Musteranalyse sollten stets drei Fragen beantwortet werden: Welche Verantwortung wird vom Hauptobjekt entkoppelt? Welche Objekte besitzen oder erweitern welche anderen Objekte? Was kann nachher ausgetauscht oder ergänzt werden, ohne den Client ändern zu müssen? Diese Fragen helfen, Wrapper, Decorator, Adapter, Factory, Template Method und Strategy nicht bloß anhand ihrer UML-Form zu verwechseln.

Das Strategy-Muster kapselt einen austauschbaren Algorithmus in einem Objekt mit gemeinsamer Schnittstelle. Ein Kontextobjekt hält eine Referenz auf die Strategie und delegiert einen Berechnungsschritt an sie. Darin unterscheidet sich Strategy von Template Method: Template Method legt eine Ablaufstruktur typischerweise durch Vererbung fest; Strategy variiert den gewählten Algorithmus durch Komposition, also das Halten einer anderen Objektinstanz.

Beim Vergleich von Adapter und Decorator sind ihre öffentlichen Verträge entscheidend. Der Adapter übersetzt eine bestehende API in eine benötigte andere API. Der Decorator implementiert dieselbe Abstraktion wie das eingepackte Objekt und ergänzt oder kontrolliert dessen Verhalten. Eine statische

Fabrikmethode wie `List.of(array)` legt die konkrete Implementierung fest, ohne diese im Rückgabetyp zu veröffentlichen. Das präzise GoF-Muster „Factory Method“ ist von einer beliebigen statischen Factory zu unterscheiden.

Originalfolien mit fachlicher Einordnung

V · Folie 57: ENTWURFSMUSTER (DESIGN PATTERNS)

Ein Muster beschreibt eine Lösungsstruktur und Rollen, nicht den einzigen richtigen Code. Die Anwendungsbedingungen bestimmen, ob es ein gutes Design ist.

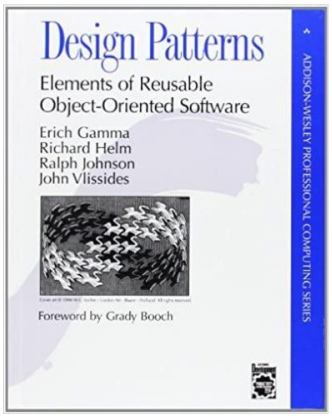
Originalabbildung: 02.Vererbung.pdf, Folie 57.

V · Folie 58: DESIGN PATTERNS: ELEMENTS OF REUSABLE OBJECT-ORIENTED SOFTWARE

Der Band von Gamma, Helm, Johnson und Vlissides prägte die klassische Design-Patterns-Terminologie. Die Unterscheidung struktureller, erzeugender und verhaltensbezogener Muster hilft beim Einordnen.

Design Patterns: Elements of Reusable Object-Oriented Software

- Buch, das den Begriff "Design Pattern" prägte
- entstand im Kontext von Frameworks für graphische Oberflächen



erschienen 1995
ein Bestseller der Informatik-Literatur

enthält 23 Muster
klassifiziert in
- Erzeugende Muster
- Strukturmuster
- Verhaltensmuster

© H. Prähofer, 2026 60

Originalabbildung: 02.Vererbung.pdf, Folie 58.

V · Folie 59: BESTANDTEILE EINES MUSTERS

Name, Problem, Kontext, Lösung, Konsequenzen und Varianten machen ein Muster kommunizierbar. Ohne die Wirkungen und Grenzen bleibt ein UML-Bild bloße Form.

Bestandteile eines Musters

1. **Name**: Schafft Begriffe und definiert ein Vokabular
   z.B. "Dekorator", "Adapter", "Fabrik", ...

Vokabular des Softwareentwicklers

2. **Zweck**: Wann soll man das Muster verwenden?
   z.B. Flexibles Hinzufügen von Zusätzen ohne Klassen ableiten zu müssen

3. **Lösung**: Objektstrukturen und Interaktionen
   z.B. Klassenhierarchie mit Decorator-Klassen (siehe folgend)

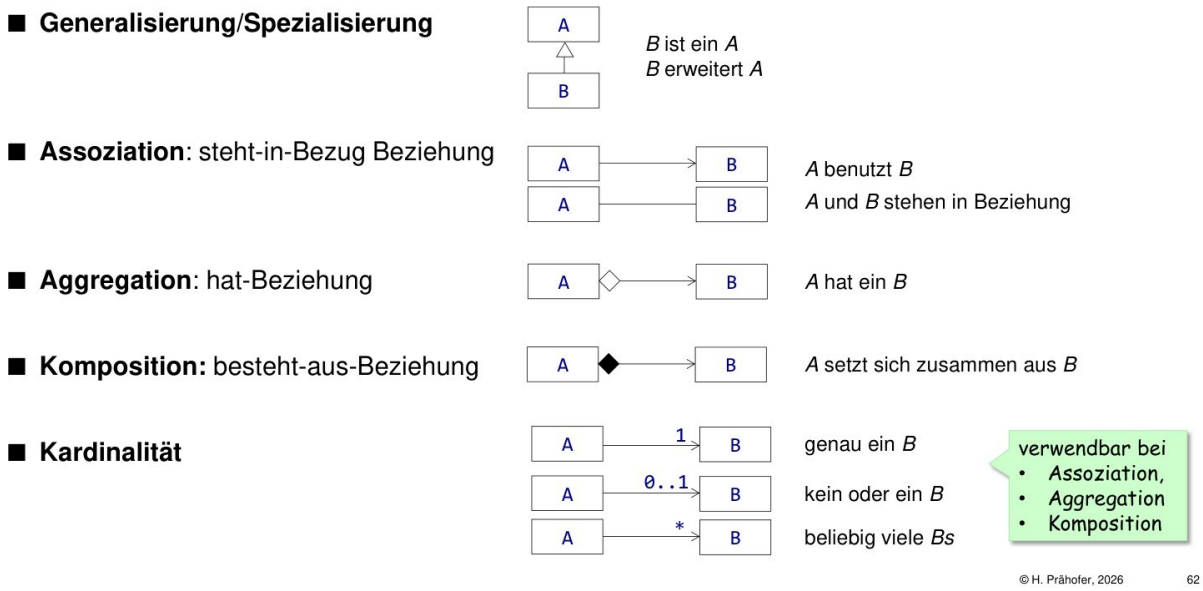
© H. Prähofer, 2026 61

Originalabbildung: 02.Vererbung.pdf, Folie 59.

V · Folie 60: EXKURS: UML NOTATION

Die UML-Linien erklären das Zusammenspiel von „ist ein“ (Vererbung) und „hat ein“ (Referenz/Assoziation). Bei Wrappern ist insbesondere die gehaltene Referenz entscheidend.

Exkurs: UML Notation

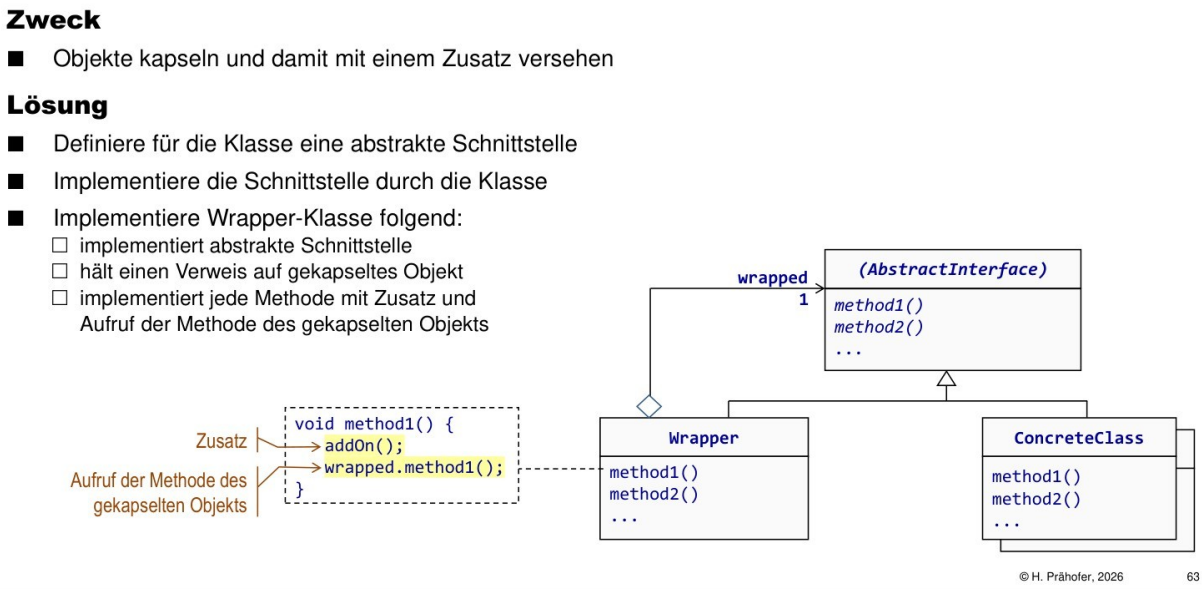


Originalabbildung: 02.Vererbung.pdf, Folie 60.

V · Folie 61: ENTWURFSMUSTER WRAPPER

Ein Wrapper enthält eine Referenz auf das umhüllte Objekt und übersetzt oder ergänzt Aufrufe. Entscheidend ist, dass Daten und Zustandsverantwortung im ursprünglichen Objekt bleiben können.

Entwurfsmuster Wrapper



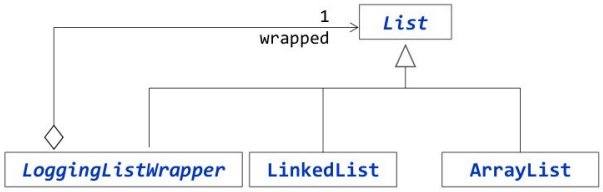
Originalabbildung: 02.Vererbung.pdf, Folie 61.

V · Folie 62: ENTWURFSMUSTER WRAPPER: LOGGINGLISTWRAPPER

Die LoggingListWrapper-Skizze erläutert die Beziehung zwischen Wrapper und eingepackter `List`; beim Aufruf werden die Parameter an die `List` delegiert.

Entwurfsmuster Wrapper: LoggingListWrapper

Beispiel: LoggingListWrapper



© H. Prähofer, 2026 64

Originalabbildung: 02.Vererbung.pdf, Folie 62.

V · Folie 63: ENTWURFSMUSTER WRAPPER: LOGGINGLISTWRAPPER

Beim Delegieren muss jede benötigte Methode übernommen oder sinnvoll weitergeleitet werden. Sonst verhält sich der Wrapper unvollständig und verletzt Erwartungen seiner Clients.

Entwurfsmuster Wrapper: LoggingListWrapper

Beispiel: LoggingListWrapper

```
public class LoggingListWrapper extends List {

  private final List wrapped;

  public LoggingListWrapper(List wrapped) {
    log("Constructor called ");
    this.wrapped = wrapped;
  }
  ...
  @Override
  public void add(Object elem) {
    log("add called with element " + elem );
    wrapped.add(elem);
  }
  @Override
  public void addAll(Object... elems) {
    log("addAll called with elements %s".formatted(elems));
    wrapped.addAll(elems);
  }

  ...
}
```

gekapseltes Objekt

Zusatz log

```
List l = new LoggingListWrapper(
                    new LinkedList());

l.addAll("A", "B", "C");
l.add("D");
l.remove("C");
```

```
Constructor called
addAll called with [A, B, C]
add called with D
remove called with C
```

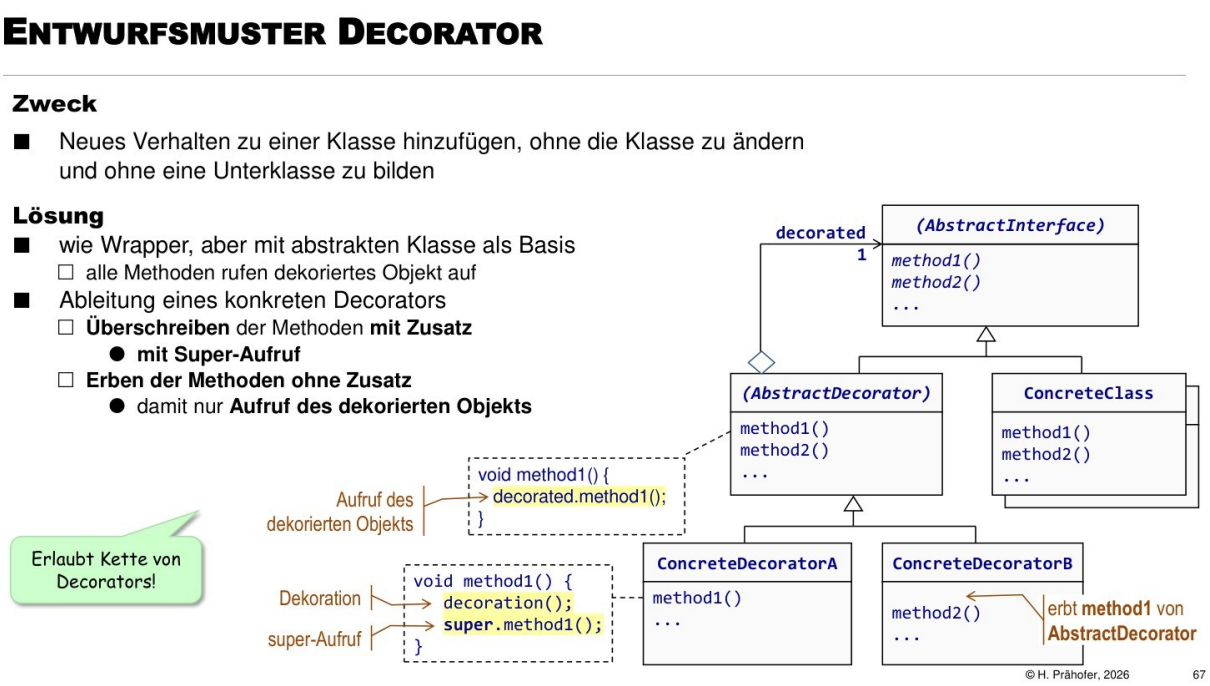
```
List l = new LoggingListWrapper(
                    new ArrayList());
...
```

© H. Prähofer, 2026 65

Originalabbildung: 02.Vererbung.pdf, Folie 63.

V · Folie 65: ENTWURFSMUSTER DECORATOR

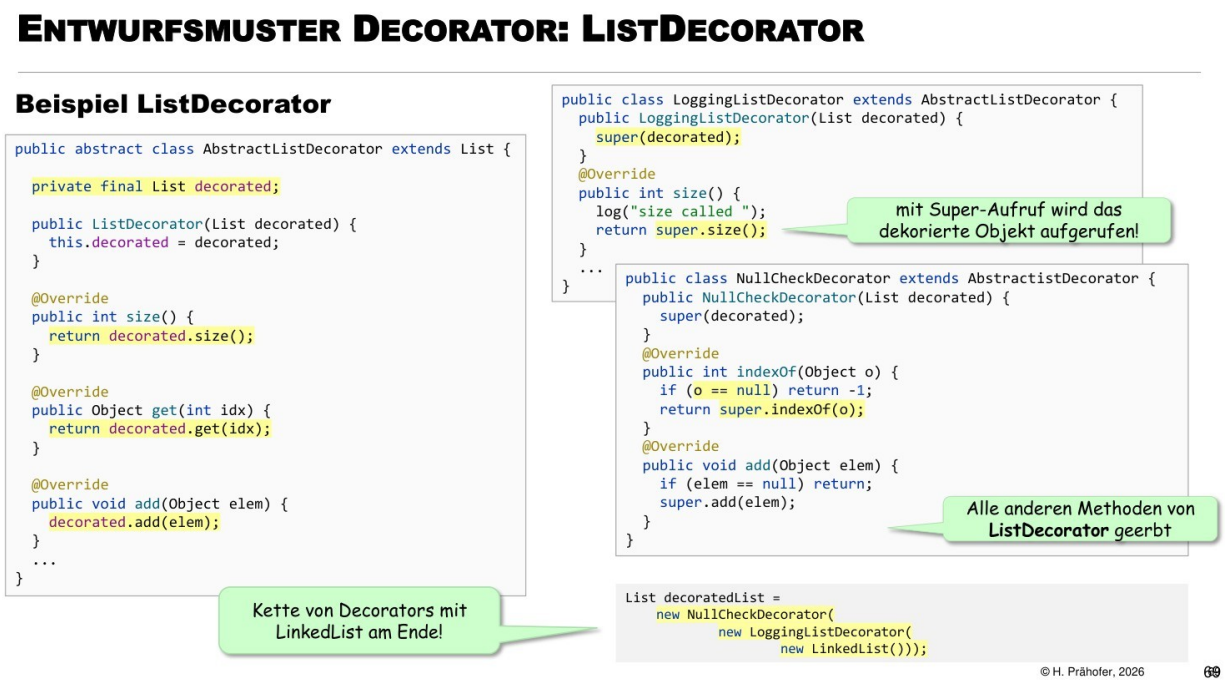
Ein Decorator erfüllt selbst denselben Vertrag wie das dekorierte Objekt und kann daher transparent eingesetzt sowie mehrfach geschachtelt werden.



Originalabbildung: 02.Vererbung.pdf, Folie 65.

V · Folie 66: ENTWURFSMUSTER DECORATOR: LISTDECORATOR

Der abstrakte ListDecorator implementiert die Weiterleitung, während konkrete Decorators wie Logging und NullCheck die eigentliche Änderung hinzufügen.



Originalabbildung: 02.Vererbung.pdf, Folie 66.

V · Folie 67: ENTWURFSMUSTER DECORATOR: UNMODIFIABLELIST

Ein Unmodifiable-Decorator kann über Mutationsaufrufe eine `UnsupportedOperationException` auslösen. Das verhindert direkte Änderungen durch diese Sicht, nicht zwangsläufig Änderungen über andere Referenzen.

Entwurfsmuster Decorator: UnmodifiableList

Beispiel UnmodifiableList

- Decorator, um Veränderungen zu verhindern

```
public class UnmodifiableListDecorator extends AbstractistDecorator {

  public UnmodifiableListDecorator(List decorated) {
    super(decorated);
  }

  @Override
  public void add(Object elem) {
    throw new UnsupportedOperationException("forbidden");
  }

  @Override
  public void add(int idx, Object elem) {
    throw new UnsupportedOperationException("forbidden");
  }

  @Override
  public Object remove(int idx) {
    throw new UnsupportedOperationException("forbidden");
  }

  ...
}
```

Anwendung: **nur lesender** Zugriff auf interne Daten

```
public class MyClass {

    private List myPrivateListData = new ArrayList();

    public List getListData() {
        return new UnmodifiableListDecorator(myPrivateListData);
    }

    ...
}
```

Nur lesende Zugriffe !

```
MyClass myObject = new MyClass();
List data = myObject.getListData();

data.remove(...); // throws UnsupportedOperationException
```

Problematisch: **List** bietet **add**, **remove** etc. an, und **UnmodifiableList** verhindert diese !?

© H. Prähofer, 2026 70

Originalabbildung: 02.Vererbung.pdf, Folie 67.

V · Folie 68: ENTWURFSMUSTER ADAPTER

Der Adapter übersetzt eine fremde bestehende Schnittstelle in die erwartete `List`-Schnittstelle. Anders als der Decorator steht die Schnittstellenanpassung im Vordergrund.

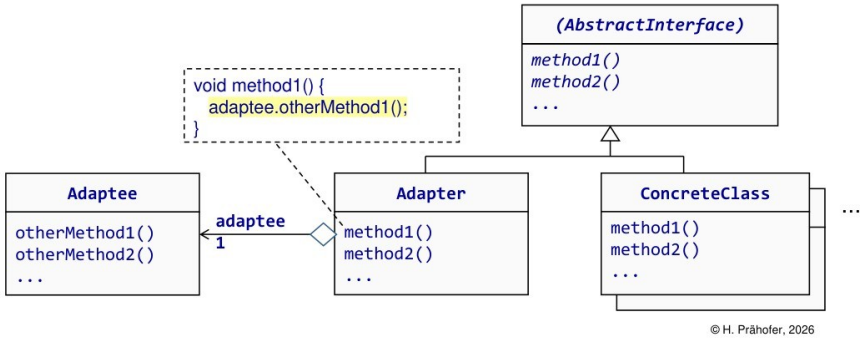
Entwurfsmuster Adapter

Zweck

- Fremde Klasse zu einem bestehenden Klassensystem kompatibel machen

Lösung

- Gemeinsames Interface
- Klasse, die Interface implementiert und Aufrufe in Aufrufe an die fremde Klasse umsetzt (Forwarding)



© H. Prähofer, 2026 71

Originalabbildung: 02.Vererbung.pdf, Folie 68.

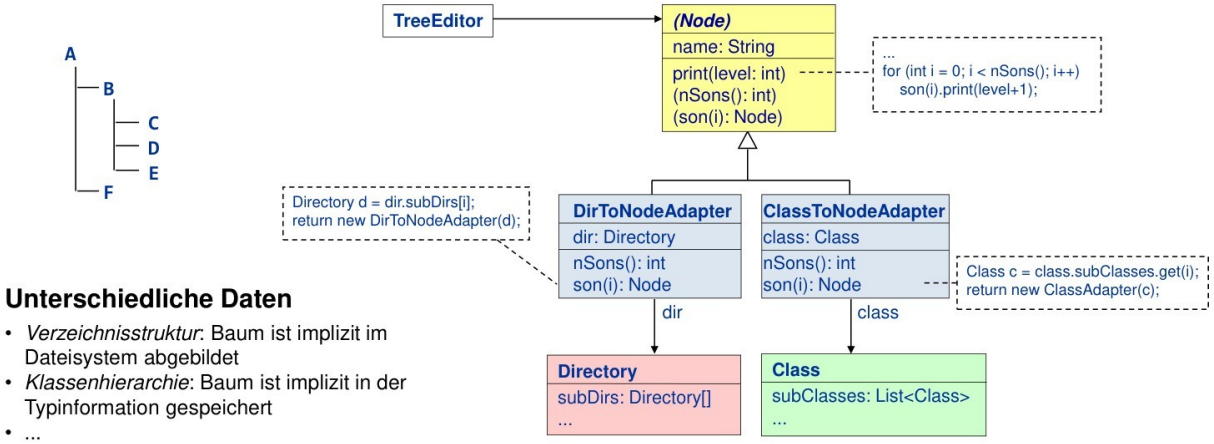
V · Folie 69: ENTWURFSMUSTER ADAPTER

Im Baumknoten-Beispiel wird eine vorhandene Knotendarstellung hinter einer einheitlichen Ziel-API verborgen. Dies erleichtert Clients, die den ursprünglichen Knotentyp nicht kennen sollen.

ENTWURFSMUSTER ADAPTER

Beispiel Adapter: Klassendiagramm für Baumknoten-Adapter

- Einheitliche Schnittstelle für verschiedene Baumstrukturen



Unterschiedliche Daten

- *Verzeichnisstruktur*: Baum ist implizit im Dateisystem abgebildet
- *Klassenhierarchie*: Baum ist implizit in der Typinformation gespeichert
- ...

© H. Prähofer, 2026 72

Originalabbildung: 02.Vererbung.pdf, Folie 69.

V · Folie 70: ENTWURFSMUSTER: ADAPTER [3/3]

Die erweiterte Adapterdarstellung trennt adaptiertes Objekt, Adapter und Zielschnittstelle. Eine Vererbung der fremden Klasse ist dafür nicht grundsätzlich nötig.

ENTWURFSMUSTER: ADAPTER [3/3]

Beispiel Adapter: Klassendiagramm für Baumknoten-Adapter

- Implementierung

```
class TreeEditor {
  Node tree;
  TreeEditor(Node tree) {
    this.tree = tree;
  }
  void printTree() {
    tree.print(0);
  }
}
```

```
abstract class Node {
  String name;
  abstract int nSons();
  abstract Node son(int i);
  void print(int level) {
    ... write name at level ...
    for (int i = 0; i < nSons(); i++)
      son(i).print(level+1);
  }
}
```

```
class DirToNodeAdapter extends Node {
  Directory dir;
  DirAdapter(Directory dir) {
    this.dir = dir;
    name = d.getName();
  }
  int nSons() {return dir.subDirs.length;}
  Node son(int i) {
    Directory sub = dir.subDirs[i];
    return new DirAdapter(sub);
  }
}
```

```
class Directory {
  Directory[] subDirs;
  String getName() {...}
  ...
}
```

- Benutzung

```
TreeEditor editor =
  new TreeEditor(
    new DirToNodeAdapter(
      Directory.rootDirectory()
    )
  );
...
editor.printTree();
```

© H. Prähofer, 2026 73

Originalabbildung: 02.Vererbung.pdf, Folie 70.

V · Folie 71: ENTWURFSMUSTER FACTORY-METHODE [1/2]

Fabrikmethoden verbergen konkrete Klassennamen und bündeln die Entscheidung, welche Implementierung erzeugt werden soll.

Entwurfsmuster Factory-Methode [1/2]

Zweck

- Abstrahiere von der Erzeugung von konkreten Objekten
- Verberge, wie Objekte erzeugt werden

Lösung

- Verberge den Konstruktor der konkreten Objekte
- Stelle für die Erzeugung von Objekten Methoden bereit
- Optional: Führe abstrakte Schnittstellen / abstrakte Klassen ein, um die konkreten Implementierungen zu verbergen

Konsequenzen

- Implementierungen können ausgetauscht werden
- erzeugte Strukturen können variiert und optimiert werden
- Factory-Methoden können flexibel auf Anforderungen reagieren

© H. Prähofer, 2026 74

Originalabbildung: 02.Vererbung.pdf, Folie 71.

V · Folie 72: ENTWURFSMUSTER FACTORY-METHODE [1/2]

Statische Erzeuger können anhand der Parameter zwischen Implementierungen wählen. **Begriffshinweis:** Dies sind vor allem statische Factory-Methoden; das klassische GoF-Factory-Method-Muster benutzt eine polymorphe Erzeugungsoperation.

Entwurfsmuster Factory-Methode [1/2]

Implementierung in Java

- Factory-Methode als statische Methoden
- oft mit Namen **of**

Beispiel: List.of

- überladene Methoden **List.of** erzeugen optimierte Listenstrukturen mit fixer Länge

```
List<String> none = List.of();
List<String> one = List.of("A");
List<String> two = List.of("A", "B");
List<String> many = List.of("A", "B", "C", "D", "E");
```

Singleton-Objekt
Objekt mit einem Feld
Objekt mit zwei Feldern
Objekt mit Array fixer Länge
hochoptimiert!

© H. Prähofer, 2026 75

Originalabbildung: 02.Vererbung.pdf, Folie 72.

V · Folie 73: FACTORY-METHODE

Das Shape-Beispiel zeigt, dass die Aufrufstelle den Erzeugungsweg über eine Methode erhalten kann, statt direkt überall new aufzurufen.

Factory-Methode

Beispiel Shapes

- Factory-Methoden für konkrete Typen von **Shape**
- mit Klassenmethoden entweder in Basisklasse **Shape**

```
public abstract class Shape {

  public static Circle circle(int x, int y, int r) {
    return new Circle(x, y, r);
  }
  public static Rectangle rect(int x, int y, int w, int h) {
    return new Rectangle(x, y, w, h);
  }
  …
}
```

```
public class Circle extends Shape {
  private int radius;

  public Circle(int x, int y, int r) {
    super(x, y);
    this.radius = r;
  }
  …
}
```

```
public class Rectangle extends Shape {
  private int w, h;

  public Rectangle(int x, int y, int w, int h) {
    super(x, y);
    this.w = w; this.h = h;
  }
  …
}
```

- oder in eigenen Utility-Klasse **Shapes**

Mehrzahl !

```
public class Shapes {

  public static Circle circle(int x, int y, int r) {…}

  public static Rectangle rect(int x, int y, int w, int h) {…}
}
```

© H. Prähofer, 2026 76

Originalabbildung: 02.Vererbung.pdf, Folie 73.

V · Folie 74: ENTWURFSMUSTER ADAPTER: ARRAYTOLISTADAPTER

Der Adapter präsentiert ein Array über die `List`-Methoden `size` und `get`. Eine Factory `List.of(array)` verbirgt den eigentlichen Adaptertyp.

Entwurfsmuster Adapter: ArrayToListAdapter

Beispiel: ArrayToListAdapter

- Array als **List** verwenden

```
final class ArrayToListAdapter extends List {
    private final Object[] array;

    public ArrayToListAdapter(Object[] array) {
        this.array = array;
    }
    @Override
    public int size() {
        return array.length;
    }
    @Override
    public Object get(int idx) {
        if (idx < 0 || idx >= array.length)
            throw new IndexOutOfBoundsException(idx);
        return array[idx];
    }
    @Override
    public int indexOf(Object o) {
        for (int i = 0; i < array.length; i++) {
            if (array[i].equals(o)) return i;
        }
        return -1;
    }
    ...
}
```

- Erzeugen mit Factory-Methode **of**

```
public class List {

    public static List of(Object[] array) {
      Objects.requireNonNull(array, "no null");
      return new ArrayToListAdapter(array);
    }

    public int size();
    ...
}
```

Factory-Methode, die Adapter für array erzeugt

```
Object[] array = {"A", "B", "C"};
List listFromArray = List.of(array);

System.out.println(listFromArray.get(0));
```

© H. Prähofer, 2026 77

Originalabbildung: 02.Vererbung.pdf, Folie 74.

V · Folie 75: ENTWURFSMUSTER ADAPTER: SINGLE2LISTADAPTER

Das einzelne Objekt wird als Ein-Element-Liste dargestellt. **Korrektur:** Die Adapterbenennung im Factory-Code ist uneinheitlich (`Array2ListAdapter`/`ArrayToListAdapter`); entscheidend ist die Rückgabe des zur konkreten Struktur passenden Adapters.

Entwurfsmuster Adapter: Single2ListAdapter

Beispiel Adapter: Single2ListAdapter

- einzelnes Objekt als **List** verwenden
- Erzeugen mit Factory-Methode **of**

```
final class SingleToListAdapter extends List {
    private final Object single;

    public SingleToListAdapter(Object object) {
        this.single = object;
    }

    @Override
    public int size() {
        return 1;
    }

    @Override
    public Object get(int idx) {
        if (idx == 0) return single;
        else throw new IndexOutOfBoundsException(idx);
    }

    @Override
    public int indexOf(Object o) {
        if (o.equals(single)) return 0;
        else return -1;
    }
    ...
}
```

```
public class List {

    public static List of(Object[] array) {
        Objects.requireNonNull(array, "no null");
        return new Array2ListAdapter(array);
    }

    public static List of(Object single) {
      Objects.requireNonNull(single, "no null");
      return new Single2ListAdapter(single);
    }

    public int size();
    public Object get(int idx);
    public default boolean contains(Object o) {
        return indexOf(o) >= 0;
    }
    public int indexOf(Object o);
}
```

Factory-Methode, die Adapter für Objekt erzeugt

```
List listWithX = List.of("X");
System.out.println(listFromArray.get(0));
```

© H. Prähofer, 2026 78

Originalabbildung: 02.Vererbung.pdf, Folie 75.

V · Folie 76: TEMPLATE METHOD (SCHABLONENMETHODE)

Die Template Method führt einen festen Algorithmus aus und überlässt einzelne Schritte der dynamischen Bindung. Die Oberklasse koordiniert, die Unterklassen spezialisieren.

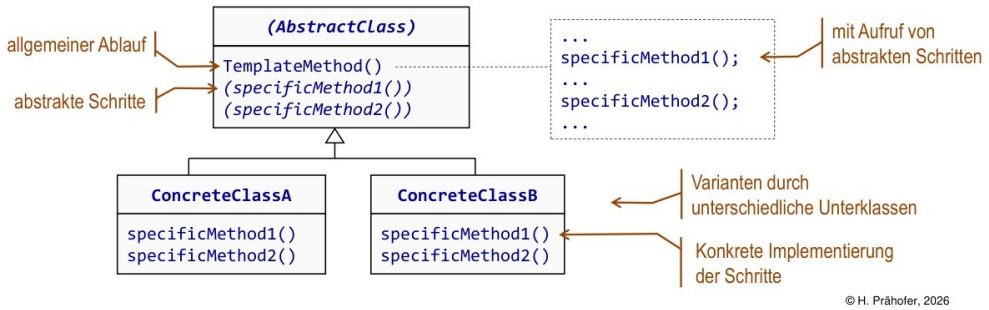
Template Method (Schablonenmethode)

Zweck

- Gerüst eines Algorithmus, bei dem die Schritte unterschiedlich ausgeprägt sein können

Lösung

- Definiere eine Methode für den allgemeinen Ablauf (Template) in abstrakten Basisklasse
- mit Schritten, die in Unterklassen speziell implementiert werden sollen
- Unterschiedliche Unterklassen erlauben allgemeinen Ablauf unterschiedlich auszuprägen



© H. Prähofer, 2026 79

Originalabbildung: 02.Vererbung.pdf, Folie 76.

V · Folie 77: TEMPLATE METHOD (SCHABLONENMETHODE)

`DataProcessor.process` lädt Daten, verarbeitet sie und schreibt das Ergebnis. `parseContent` ist dabei der austauschbare abstrakte Schritt.

Template Method (Schablonenmethode)

Beispiel Template Methode: DataProcessor

- konkreter Ablauf **process** mit Schritten Laden, Verarbeiten und Schreiben der Daten
- abstrakte Methode **parseContent** zum Verarbeiten der Daten

```
public abstract class DataProcessor {

  public final void process(String source) {
    String content = loadData(source);
    Info info = parseContent(content);
    writeInfo(info);
  }

  protected String loadData(String source) { ... }

  protected abstract Info parseContent(String data);

  protected void writeInfo(Info info) {
    info.write();
  }
}
```

abstrakten Basisklasse mit

konkreter Methode für allgemeinen Ablauf

konkreter Methode zum Laden der Daten

abstrakter Methode zur Verarbeitung

konkreter Methode zum Schreiben der Ergebnisse

© H. Prähofer, 2026 80

Originalabbildung: 02.Vererbung.pdf, Folie 77.

V · Folie 78: TEMPLATE METHOD (SCHABLONENMETHODE)

`XMLDataProcessor` und `CSVDataProcessor` überschreiben nur die erforderliche Verarbeitung; die Reihenfolge der anderen Prozessschritte bleibt unverändert.

Template Method (Schablonenmethode)

Beispiel Template Method: DataProcessor

- Ausprägung der Methode **parseContent** in Unterklassen

```
public abstract class DataProcessor { ... }
```

```
public class XMLDataProcessor extends DataProcessor {

  @Override
  protected Info parseContent(String data) { ... }

}
```

Konkrete Ausprägung zur Verarbeitung von XML Daten

mit konkreter Methode zur Verarbeitung der XML Daten

```
public class CSVDataProcessor extends DataProcessor {

  @Override
  protected Info parseContent(String data) { ... }

}
```

Konkrete Ausprägung zur Verarbeitung von CSV Daten

mit konkreter Methode zur Verarbeitung der CSV Daten

© H. Prähofer, 2026 81

Originalabbildung: 02.Vererbung.pdf, Folie 78.

V · Folie 79: TEMPLATE METHODE COPYTO BEI LIST

`copyTo` ist ein Beispiel für die Wiederverwendung einer Operation über einen allgemeinen `List`-Vertrag; die dynamisch gebundenen `List`-Operationen übernehmen die konkrete Arbeit.

Template Methode CopyTo bei List

- Methode **copyTo** verwendet **size**, **add** und **get** unabhängig von konkreten Implementierung der beiden Listen **from** und **to**

```
public abstract class List {
  …
  public static void copyTo(List from, List to) {
    for (int i = 0; i < from.size(); i++) {
      to.add(from.get(i));
    }
  }
  …
}
```

© H. Prähofer, 2026 82

Originalabbildung: 02.Vererbung.pdf, Folie 79.

V · Folie 80: STRATEGY (STRATEGIE)

Die Strategy-Rolle modelliert eine ganze Familie austauschbarer Algorithmen. Der Kontext besitzt eine Strategy-Referenz statt die Varianten als `if`-Zweige fest einzubauen.

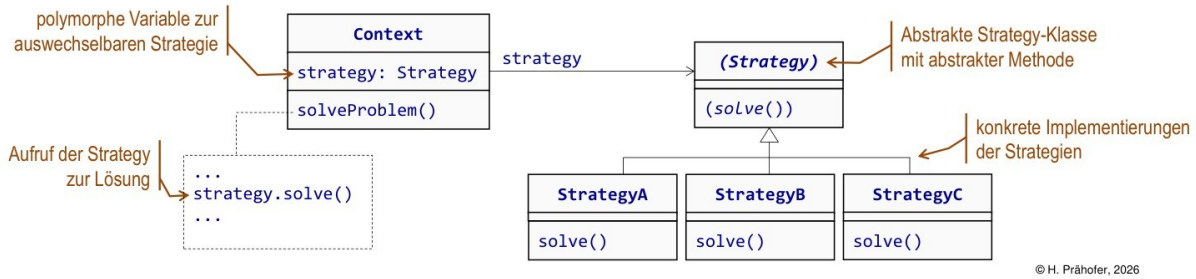
Strategy (Strategie)

Zweck

- Definiere eine Familie von Verfahren, die man beliebig austauschen kann

Lösung

- Lagere das Verfahren in ein eigenes Strategie-Objekt aus
- Definiere eine abstrakte Klasse mit der Schnittstelle für das Verfahren
- Implementiere unterschiedliche Ausprägungen des Verfahrens in Unterklassen
- Verwende eine Variable im Kontext, mit der man die Variante des Verfahren definieren kann

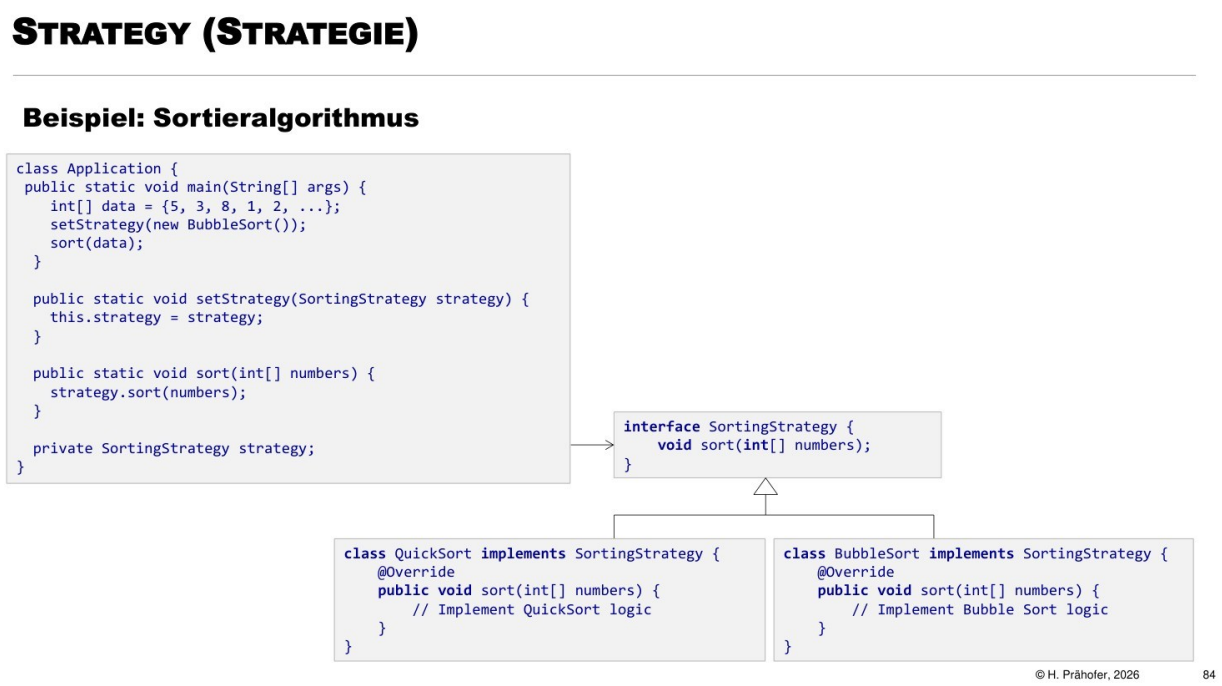


© H. Prähofer, 2026 83

Originalabbildung: 02.Vererbung.pdf, Folie 80.

V · Folie 81: STRATEGY (STRATEGIE)

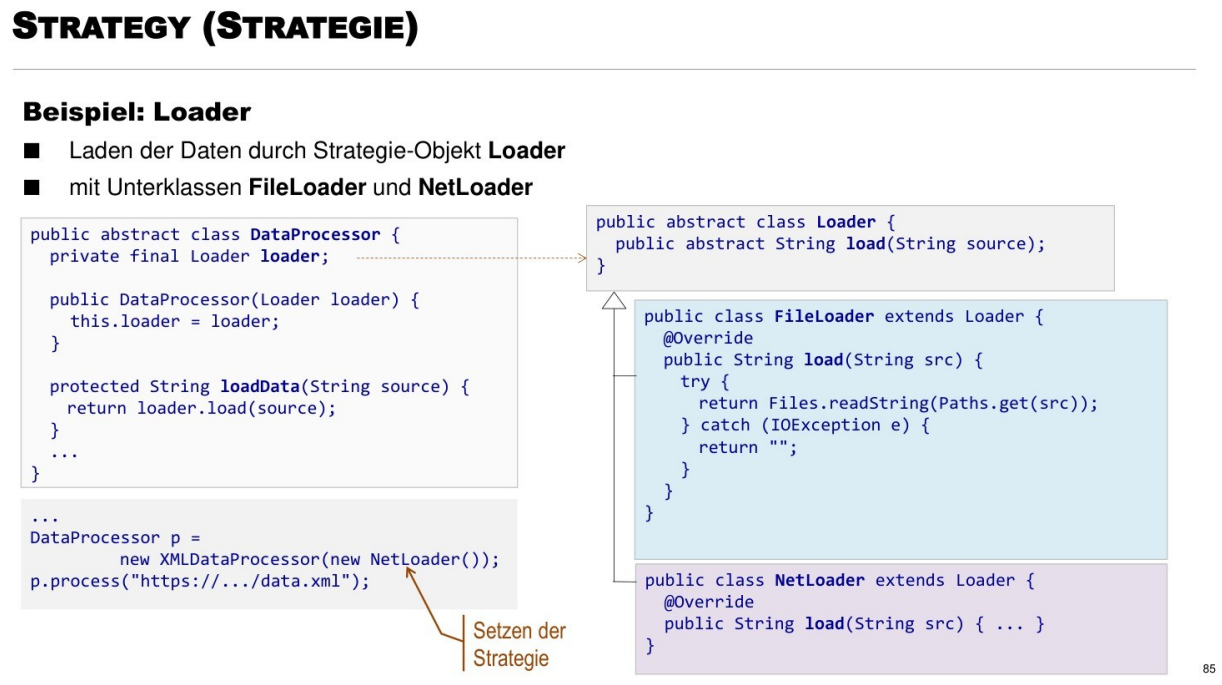
Das BubbleSort/QuickSort-Beispiel ersetzt den Sortieralgorithmus durch eine andere Strategy. **Korrektur:** Der Ausschnitt mischt `static main`/statische Methoden mit `this.strategy`; `this` ist in einer statischen Methode unzulässig. Entweder Instanzkontext oder eine entsprechend statische Variable verwenden.



Originalabbildung: 02.Vererbung.pdf, Folie 81.

V · Folie 82: STRATEGY (STRATEGIE)

Der Loader kapselt die Datenquelle unabhängig vom Verarbeitungsablauf. Die Fehlerbehandlung sollte explizit sein; `IOException` einfach durch den leeren `String` zu ersetzen kann Fehler verschleiern.



Originalabbildung: 02.Vererbung.pdf, Folie 82.

V · Folie 83: LIST: METHODE SORT MIT STRATEGIE FÜR SORTIERUNG

Ein `Comparator` definiert die Ordnung, während die Listenmethode den Algorithmus ausführt. Im Beispiel ähnelt die Suche nach der Einfügeposition einer Einfügesortierung.

List: Methode Sort mit Strategie für Sortierung

- Sortierung mit Sortierstrategie **sortingStrategy** als Parameter

```java
public interface Comparator<T> {
   int compare(T o1, T o2);
   …
}
```

```java
public abstract class List {
  …
  public <T> void sort(Comparator<Object> sortingStrategy) {
    for (int i = 1; i < size(); i++) {
      Object element = get(i);
      int insertionIndex = i;
      while (insertionIndex > 0 &&
                sortingStrategy.compare(get(insertionIndex - 1), element) > 0) {
        insertionIndex--;
      }
      if (insertionIndex != i) {
        remove(i);
        add(insertionIndex, element);
      }
    }
  }
  …
}
```

© H. Prähofer, 2026 86

Originalabbildung: 02.Vererbung.pdf, Folie 83.

V · Folie 84: STRATEGY (STRATEGIE)

Die Beispiele reichen von Sortierung über Kompression bis Authentifizierung. Das gemeinsame Merkmal ist das Austauschen eines Verfahrens über eine einheitliche Schnittstelle.

Strategy (Strategie)

Anwendungsbeispiele

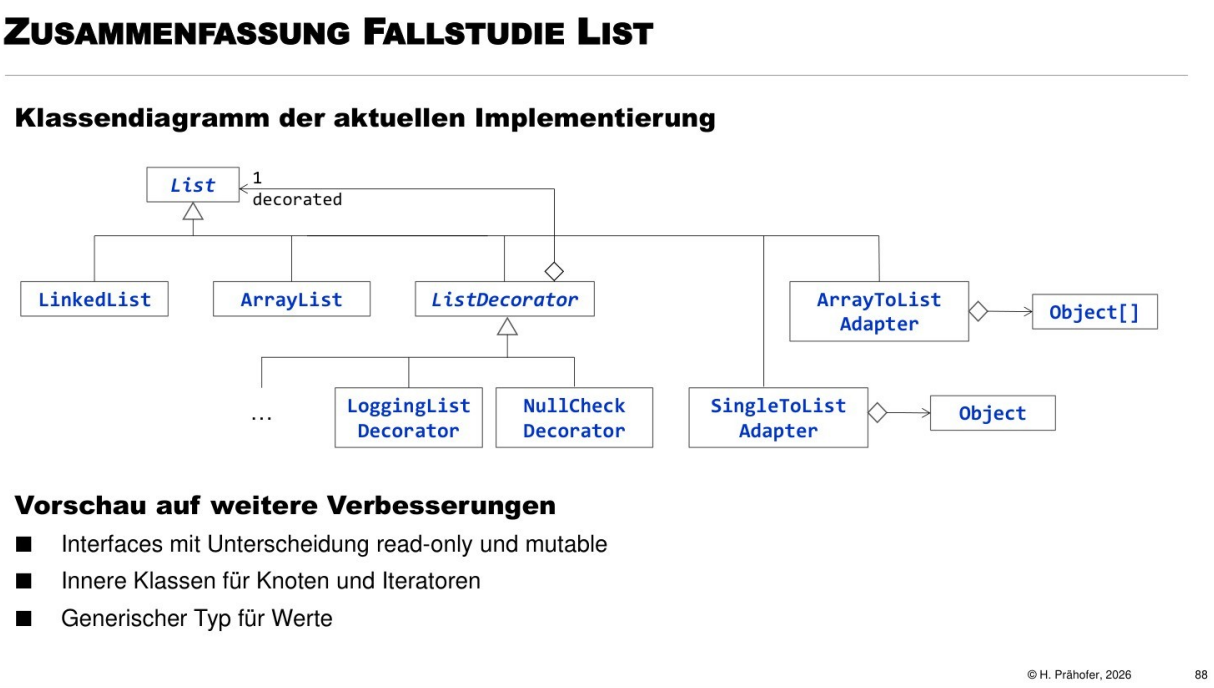
- Algorithmen
  - ☐ Sortieralgorithmen (z. B. QuickSort, MergeSort, BubbleSort)
  - ☐ Suchalgorithmen (z. B. Binäre Suche, Lineare Suche)
  - ☐ Pfadfindungsstrategien (z. B. A*, Dijkstra's Algorithmus)
  - ☐ …
- Serialisierungs-Strategien: JSON, XML, YAML, ...
- Datenkomprimierung: ZIP, RAR, GZIP, 7z
- Logging-Strategien: Protokollierung in Konsole, Datei, Datenbank, Cloud-Dienste
- Authentifizierungsmechanismen: OAuth, JWT, LDAP, SAML
- Textverarbeitung
  - ☐ Rechtschreibprüfung (z. B. Hunspell, LanguageTool)
  - ☐ Textkodierungsformate (z. B. Base64, UTF-8, ASCII)
- uvm.

© H. Prähofer, 2026 87

Originalabbildung: 02.Vererbung.pdf, Folie 84.

V · Folie 85: ZUSAMMENFASSUNG FALLSTUDIE LIST

Das Klassendiagramm fasst Liste, Decorators und Adapter zusammen. Die Vorschau nennt generische Typen, Iteratoren und eine getrennte read-only/mutable-Sicht als nächste Stufe.



Originalabbildung: 02.Vererbung.pdf, Folie 85.

3 Interfaces

3.0 Vorschau: generische Typen und Methoden

Ein **generischer Typ** enthält einen oder mehrere Typparameter als Platzhalter für konkrete Referenztypen. In `List<E>` steht `E` für den Elementtyp. Wird `List<Student>` gebildet, behandeln Methoden dieses Typs `Student` als Elementtyp: `add(Student)` nimmt passende Elemente an, und `get(int)` liefert statisch `Student`. Eine versehentliche Übergabe eines unabhängigen Typs wie `Professor` wird zur Übersetzungszeit erkannt. Der Buchstabe `E` ist nur eine Konvention; andere Typparameter heißen häufig `T`, `K` oder `V`.

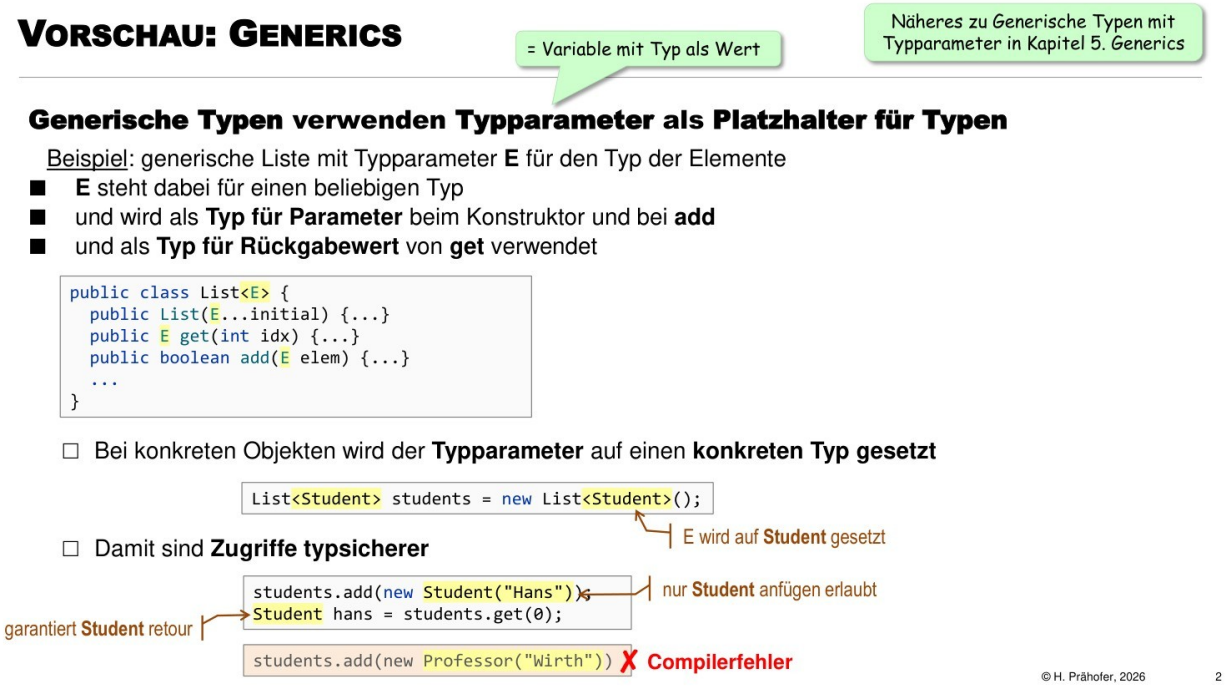
Auch Methoden können unabhängig von ihrer Klasse generisch sein: `static <T> T find(List<T> list, T item)`. Hier steht `<T>` vor dem Rückgabetyp, und der Compiler kann `T` aus den Argumenten ableiten. Werden eine `List<String>` und ein `String` übergeben, ist `T` ein `String`; bei einer Personenliste entsteht ein entsprechender Personenrückgabetyp. Die `List`-Beispiele auf den Folien sind vorbereitende Modellbeispiele; die ausführlichen Generics-Regeln werden erst in einem späteren Vorlesungskapitel behandelt.

Ein Typparameter ist ein in der Deklaration eingeführter Typplatzhalter wie `E` oder `T`. Eine Klasse `Box<E>` verwendet `E` beispielsweise für das Feld und die Rückgabe der Methode `get()`. Beim parametrisierten Typ `Box<String>` ist das Typargument `String`. Eine generische Methode kann unabhängig von einer generischen Klasse einen eigenen Typparameter einführen, zum Beispiel `static <T> T identity(T value)`. Die formale Unterscheidung zwischen Typparameter und Typargument wird in Kapitel 5 ausführlich behandelt.

Originalfolien mit fachlicher Einordnung

I · Folie 2: VORSCHAU: GENERICS TypparameterNäheres zu Generischein Kapitel 5.TypenGenericsmit = Variable mit Typ als

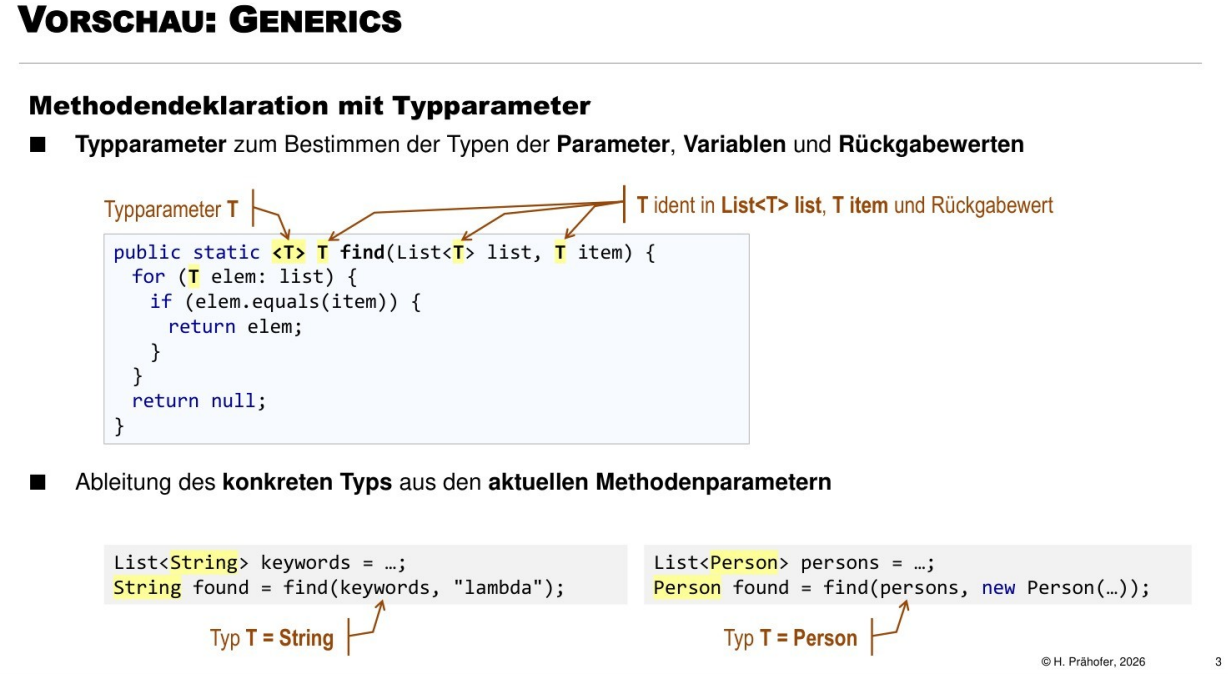
Der Typparameter `E` verhindert unpassende Elemente bereits bei der Übersetzung. Das gezeigte `List`-Modell ist ein Ausblick auf das gesonderte Generics-Kapitel.



Originalabbildung: 03-Interfaces.pdf, Folie 2.

I · Folie 3: VORSCHAU: GENERICS

Der Typparameter `T` der Methode wird aus den konkreten Argumenttypen abgeleitet. Daher können `String`- und `Person`-Listen dieselbe generische Methode verwenden.



Originalabbildung: 03-Interfaces.pdf, Folie 3.

3.1 Motivation: Einfach- und Mehrfachvererbung

Java unterstützt bei **Klassen** Einfachvererbung: Jede Klasse außer `Object` besitzt genau eine direkte Oberklasse. Ein `DayTimer` kann `ClockTimer` erweitern, dieser wiederum `Timer`; zwei direkte Klassenoberklassen sind aber nicht zulässig. Die theoretische Mehrfachvererbung einer Klasse aus `Haus` und `Boot` würde zwar beide Merkmalsgruppen verbinden, erzeugt jedoch Konfliktfragen, sobald gleichnamige Felder oder Methoden vorkommen.

Bei einer **Rautenstruktur** teilen sich zwei Oberklassen wiederum eine gemeinsame Oberklasse; dann stellt sich die Frage, ob deren Zustand einmal oder mehrfach im abgeleiteten Objekt auftaucht. Java vermeidet diese Problemklasse bei gewöhnlichen Klassen durch Einfachvererbung und ermöglicht Mehrfachtypisierung stattdessen über Interfaces. Mehrere Interfaces können Anforderungen an dieselbe Klasse formulieren, ohne dass das Objekt zugleich Instanzzustände aus mehreren unabhängigen Basisklassen übernehmen müsste.

Java erlaubt bei Klassen nur eine direkte Oberklasse. Mehrere unabhängige Fähigkeiten werden dagegen über Interfaces beschrieben. Ein konkreter Typ kann `extends Base` verwenden und gleichzeitig `implements Reader, Writer` angeben. Die Mehrfachimplementierung mehrerer Interfaces ist nicht gleichzusetzen mit einer mehrfachen Vererbung von Instanzfeldern und Konstruktoren.

Zwei Interfaces können eine gleichnamige abstrakte Methode mit kompatibler Signatur deklarieren. Eine einzelne passende Implementierung erfüllt dann beide Verträge. Bei konkurrierenden gleichnamigen Default-Methoden muss die implementierende Klasse gegebenenfalls ausdrücklich überschreiben und den Konflikt auflösen. Das erklärt, weshalb Interface-Mehrfachvererbung trotz möglicher Namenskonflikte andere Regeln als Klassen-Mehrfachvererbung hat.

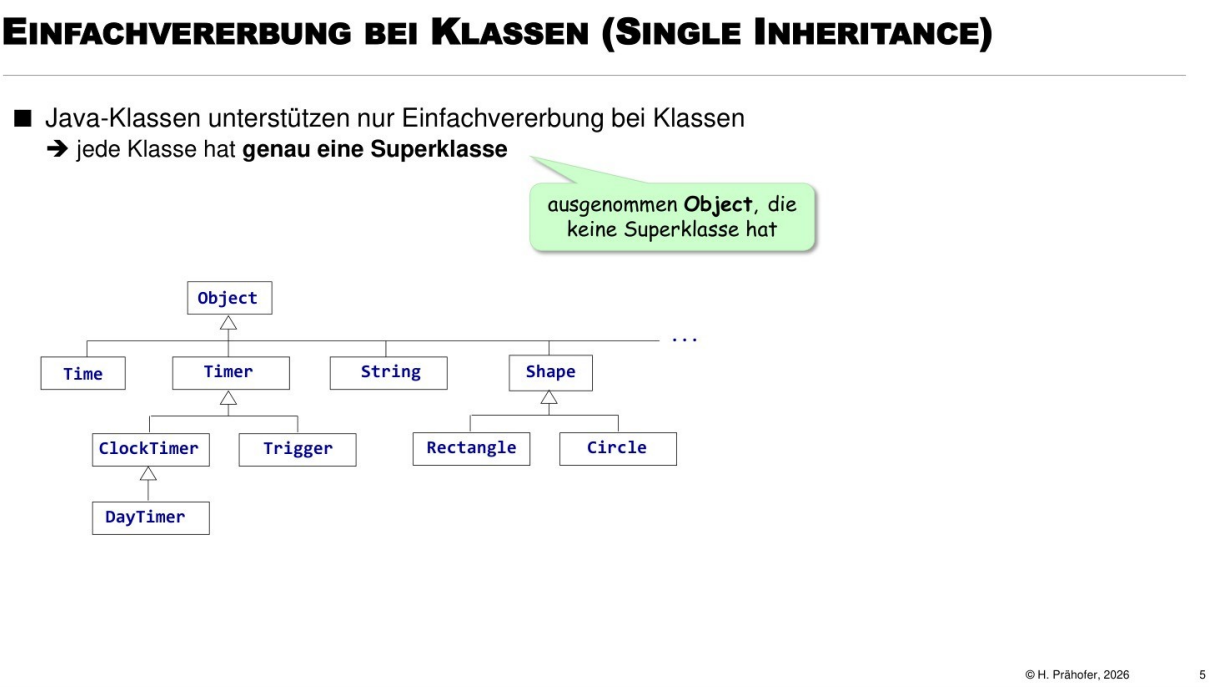
Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
interface A { default String name() { return "A"; } }
interface B { default String name() { return "B"; } }
class C implements A, B {
 @Override public String name() { return A.super.name(); }
}
```

Originalfolien mit fachlicher Einordnung

I · Folie 5: EINFACHVERERBUNG BEI KLASSEN (SINGLE INHERITANCE)

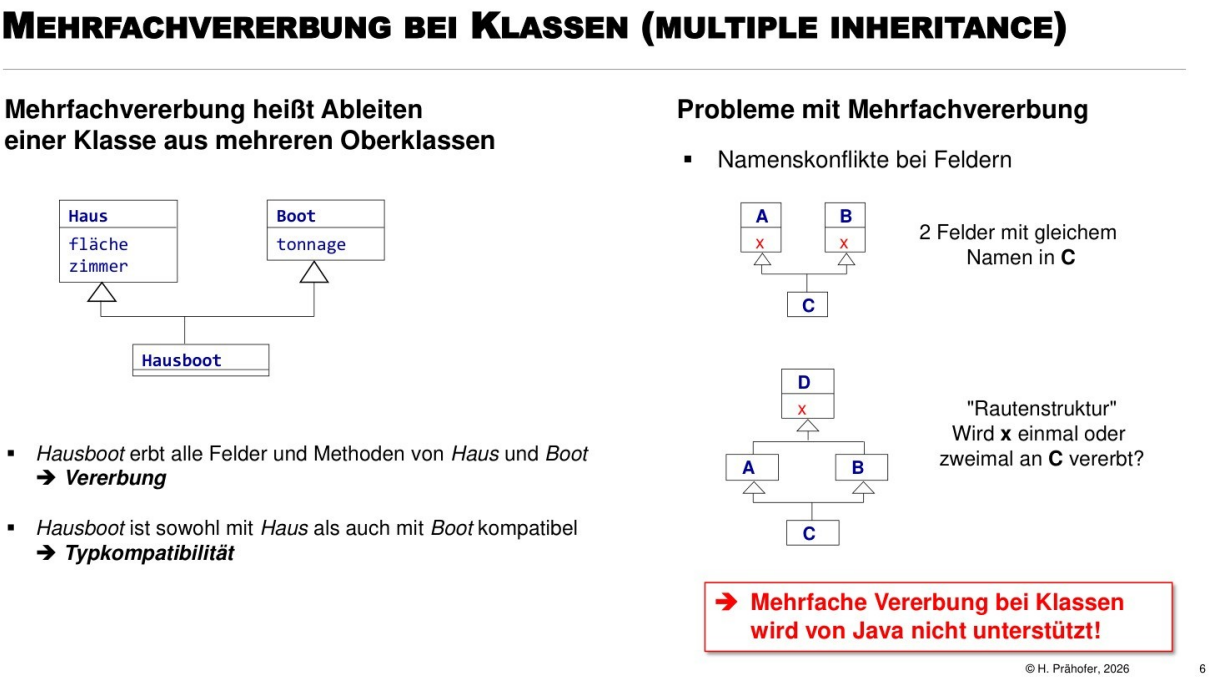
Bei Java-Klassen führt jede Klasse, außer `Object`, genau eine direkte Oberklasse auf. Die Hierarchie kann trotzdem mehrere Ebenen tief sein.



Originalabbildung: 03-Interfaces.pdf, Folie 5.

I · Folie 6: MEHRFACHVERERBUNG BEI KLASSEN (MULTIPLE INHERITANCE)

Die Namenskonflikte und Rautenstruktur illustrieren die Probleme multipler Zustandsvererbung. Interfaces ermöglichen dagegen mehrere gemeinsame Verträge.



Originalabbildung: 03-Interfaces.pdf, Folie 6.

3.2 Interfaces als Verträge und Typen

Ein **Interface** deklariert eine verwendbare Schnittstelle, also welche Operationen Objekte eines Typs anbieten. `public interface Closable { void close(); }` sagt zunächst nichts darüber aus, wie ein bestimmtes Objekt geschlossen wird. Methoden ohne Körper sind in einem Interface implizit `public`

`abstract`; eine konkrete Klasse implementiert sie mit `implements`. Eine Klasse kann nur eine Superklasse erweitern, aber beliebig viele Interfaces implementieren: `class File extends OSObject implements Reader, Writer`. Werden gleichlautende abstrakte Anforderungen gestellt, reicht normalerweise eine passende Implementierung, die beide Verträge erfüllt.

Interfaces selbst können andere Interfaces mit `extends` erweitern, auch mehrere. In UML erscheint ein Interface als Klasse mit Stereotyp `«interface»`, während Implementierungen durch gestrichelte Realisierungskanten mit leerem Dreieck und Klassenerweiterungen durch durchgezogene Generalisierungskanten dargestellt werden. Die Folien unterscheiden dies im Diagramm um `Reader`, `Writer`, `Closable`, `File` und `OSObject`.

Interfaces sind echte Referenztypen. `Writer w = new File();` und `Writer w = new Window();` sind polymorphe Zuweisungen. Der Compiler erlaubt über `w` nur die im `Writer`-Vertrag sichtbaren Operationen, beispielsweise `open()` und `write(char)`, nicht aber allein in `File` vorhandenes `flush()` oder allein in `Window` vorhandenes `resize()`. Beim Aufruf wird zur Laufzeit die konkrete Implementierung gewählt. Ein und dasselbe `File`-Objekt kann gleichzeitig über eine `Reader`-Referenz zum Lesen und eine `Writer`-Referenz zum Schreiben verwendet werden. Die beiden Referenzen beschreiben unterschiedliche Sichten auf das gleiche Objekt; sie erzeugen keine zwei Kopien.

Ein Interface ist ein Referenztyp und kann wie eine Klasse als Variablentyp oder Parametertyp verwendet werden. Bei `Writer w = new FileWriter();` erlaubt der statische Typ `Writer` nur die in dieser Schnittstelle angebotenen Operationen. Ob das konkrete Objekt darüber hinaus andere Methoden besitzt, ändert diese statische Zugriffsgrenze nicht. Aufrufe überschreibbarer Interface-Methoden werden dynamisch an die konkrete Implementierung gebunden.

Interface-Methoden ohne Rumpf sind implizit `public abstract`; Interface-Konstanten sind implizit `public static final`. Bei einer implementierenden Klasse muss die Sichtbarkeit einer öffentlich geforderten Operation mindestens `public` sein. Ein Interface besitzt keine eigenen veränderlichen Instanzfelder und keine Konstruktoren. Seine konkrete Implementierung darf dagegen eigene Felder und Konstruktoren besitzen.

Originalfolien mit fachlicher Einordnung

I · Folie 8: INTERFACES IN JAVA

Interfaces können abstrakte, Default-, statische und `private` Methoden haben, außerdem Konstanten. Sie besitzen keine Instanzfelder und keine Konstruktoren.

INTERFACES IN JAVA

Interfaces erlauben Mehrfachvererbung
- Klassen können mehrere Interfaces implementieren

Interfaces können haben
- abstrakte Methoden (**public abstract**)
- konkrete Default-Methoden (**public default**)
- private Methoden (**private**)

- statische Konstanten (**public static final**)
- statische Methoden (**public static**)
- private statischen Methoden (**private static**)

Unterschied zu (abstrakten) Klassen
- **keine** Objektfelder
- **keine** Konstruktoren
- **keine** normalen nicht-abstrakten Methoden

nur Default-Methoden

Interfaces sind Typen

© H. Prähofer, 2026 8

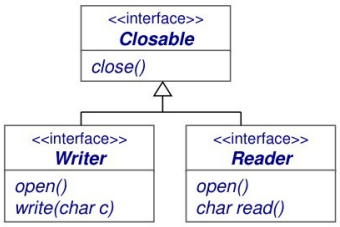
Originalabbildung: 03-Interfaces.pdf, Folie 8.

I · Folie 9: <<interface>>INTERFACES IN JAVA Closable

`Reader` und `Writer` erweitern `Closable`. Abstrakte Methoden sind implizit `public`; identische geerbte Forderungen werden nicht mehrfach implementiert.

INTERFACES IN JAVA

- Schlüsselwort **interface**
- abstrakte Methoden
  - implizit **public abstract**




```
public interface Closable {
  void close();
}
```

```
public interface Reader extends Closable {
  char read();
  void open();
  void close();
}
```

```
public interface Writer extends Closable {
  void write(char c);
  void open();
  void close();
}
```

automatisch
public abstract

(Default-, private und static Methoden später)

© H. Prähofer, 2026 9

Originalabbildung: 03-Interfaces.pdf, Folie 9.

I · Folie 10: MEHRFACHVERERBUNG BEI INTERFACES IN JAVA

`implements Writer, Reader` macht eine File-Instanz zu beiden Typen. Die Klasse darf gleichzeitig genau eine konkrete Oberklasse mit `extends` angeben.

Mehrfachvererbung bei Interfaces in Java

- Klassen können Interfaces implementieren
  - Schlüsselwort **implements**
  - eine Klasse kann **mehrere Interfaces implementieren** (aber nur eine Klasse erweitern)

Beispiel: Klasse **File** erweitert **OSObject** und implementiert Interfaces **Reader** und **Writer**

© H. Prähofer, 2026 10

Originalabbildung: 03-Interfaces.pdf, Folie 10.

I · Folie 11: UNIFIED MODELING LANGUAGE (UML)

Die gestrichelte Realisierung steht für Implementierung eines Interfaces, die durchgezogene Generalisierung für Klassen- oder Interface-Erweiterung.

Unified Modeling Language (UML)

Interface in UML

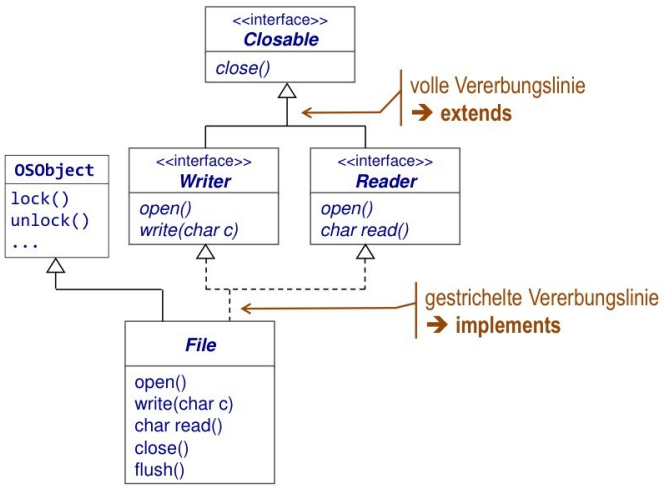
- Name kursiv
- abstrakte Methoden kursiv
- optional mit Annotation **<<interface>>**
- keine Felder, nur Methoden

Erweiterung von Interfaces

- wie bei Klassenvererbung

Implementierung von Interfaces durch Klassen

- mit gestrichelten Linie

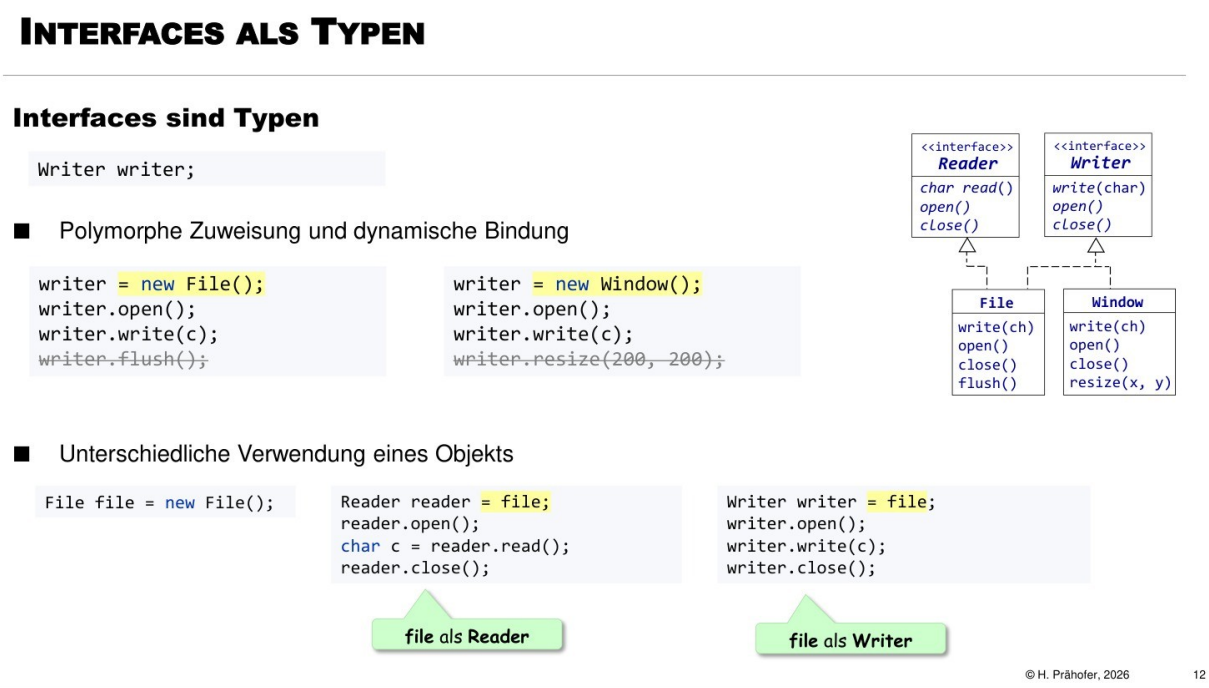


© H. Prähofer, 2026 11

Originalabbildung: 03-Interfaces.pdf, Folie 11.

I · Folie 12: INTERFACES ALS TYPEN

Dasselbe File-Objekt kann als `Reader` oder `Writer` verwendet werden. Der statische Interface-Typ grenzt jeweils die erlaubten Methoden ein.



Originalabbildung: 03-Interfaces.pdf, Folie 12.

3.3 Default-Methoden und die Weiterentwicklung von Interfaces

Seit Java 8 dürfen Interfaces **Default-Methoden** mit einem Methodenrumpf enthalten. Eine Default-Methode ist eine öffentliche Instanzmethode, die bereits eine Implementierung bereitstellt und von implementierenden Klassen geerbt werden kann. Sie wird mit `default` deklariert, etwa `default void writeLn() { write('\n'); }`. Besonders sinnvoll ist es, eine umfangreichere komfortable API aus wenigen abstrakten Operationen aufzubauen. So kann ein Interface `Writer` abstrakt `write(char)`, `open()` und `close()` verlangen und daraus `write(String)` und `writeLn(String)` implementieren. Die konkrete Klasse muss nur die erforderlichen Basisoperationen umsetzen.

Dieses Prinzip erzeugt **rich interfaces**: Eine schmale abstrakte Schnittstelle stellt eine reichhaltige Menge ableitbarer Operationen bereit. `Ordered<T>` kann `Comparable<T>` erweitern und auf dessen abstraktes `compareTo(T)` die Default-Methoden `less`, `lessEqual`, `greater` und `greaterEqual` stützen. Entscheidend ist das Vorzeichen von `compareTo`: ein negatives Ergebnis bedeutet „kleiner“, `null` „gleichgeordnet“, ein positives „größer“. Das im Folienbeispiel verwendete `Fraction.compareTo` enthält eine Vorzeichenumkehr und muss fachlich korrigiert `werden (siehe Fehlerhinweise)`.

Default-Methoden erlauben auch das vorsichtige **Erweitern bestehender Interfaces**: Eine später ergänzte konkrete Methode kann sofort von bestehenden Implementierungen geerbt werden, ohne deren Quellcode zu ändern. Das erklärt die Einführung zusätzlicher Operationen in `Collection<E>`, etwa `removeIf` und Streams. Mehrfachvererbung von Default-Methoden kann jedoch Konflikte auslösen, die die implementierende Klasse ausdrücklich auflösen muss. Eine Interface-Default-Methode darf nicht `final` sein und darf die öffentlichen Methoden von `Object` wie `equals`, `hashCode` und `toString` nicht als Default-Implementierung ersetzen. Die Folien verwenden `Collection` stellenweise als „Basisklasse“; korrekt ist `java.util.Collection` ein **Interface**.

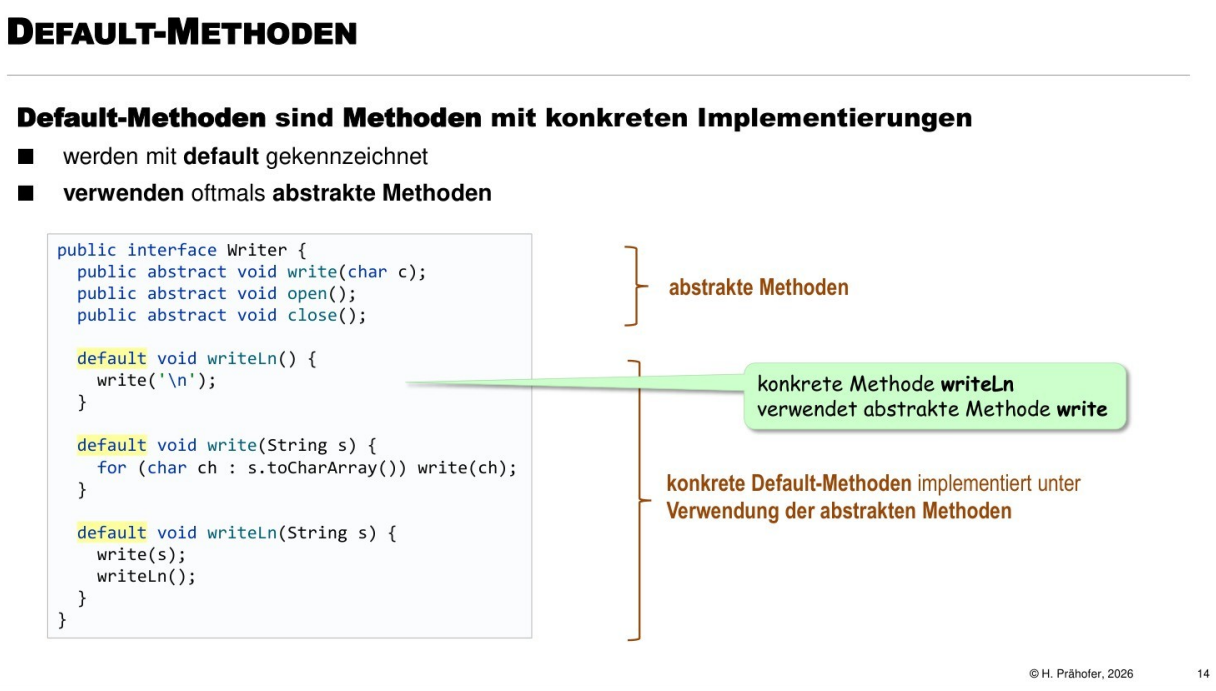
Eine Default-Methode hat einen Methodenrumpf im Interface. Sie kann abstrakte Interface-Operationen verwenden, etwa wenn `writeLn()` auf `write(char)` zurückgreift. Dadurch kann eine bestehende Schnittstelle um ein gemeinsames Verhalten erweitert werden, ohne dass jede implementierende Klasse den neuen Methodenrumpf selbst wiederholen muss. Eine implementierende Klasse kann eine Default-Methode überschreiben; eine in einer Oberklasse vorhandene konkrete Methode hat im Konfliktfall Vorrang vor einer Interface-Default-Methode.

Bei zwei voneinander unabhängigen Interfaces mit konkurrierenden Default-Methoden muss die implementierende Klasse eine eindeutige eigene Methode definieren. Soll gezielt die Default-Implementierung eines bestimmten Interfaces aufgerufen werden, kann dies mit `InterfaceName.super.method()` geschehen, soweit die erforderlichen Vererbungsbeziehungen vorliegen. `default` ist keine Alternative zu `static`: Default-Methoden werden an Objekten aufgerufen; statische Interface-Methoden am Interfacenamen.

Originalfolien mit fachlicher Einordnung

I · Folie 14: DEFAULT-METHODEN

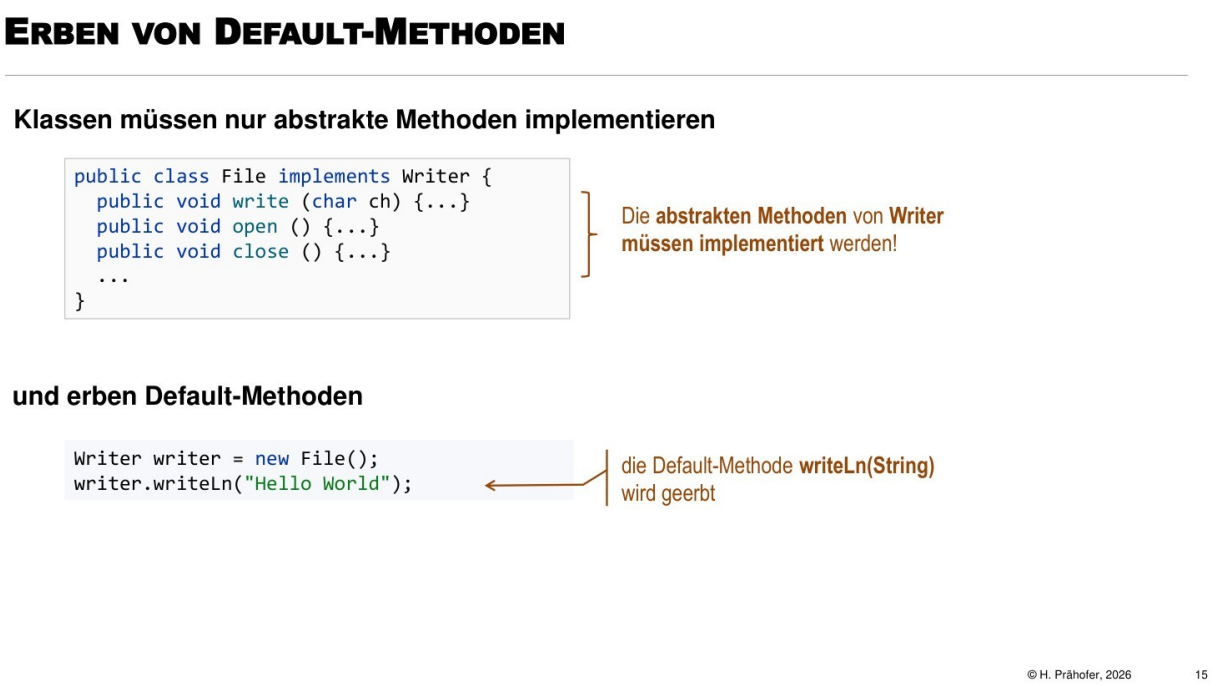
Eine Default-Methode baut auf abstrakten Basisoperationen auf. Sie stellt sofort verfügbaren Standardcode bereit, ohne der implementierenden Klasse neue Pflichtmethoden aufzuerlegen.



Originalabbildung: 03-Interfaces.pdf, Folie 14.

I · Folie 15: ERBEN VON DEFAULT-METHODEN

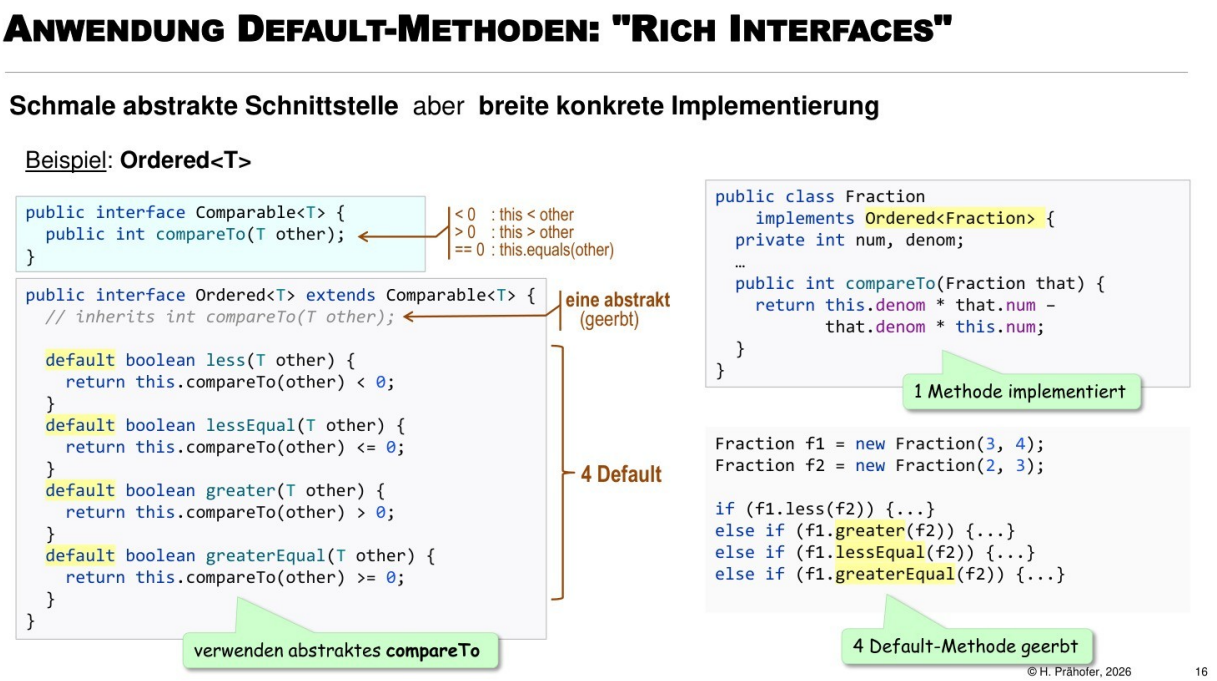
Eine File-Klasse implementiert die abstrakten `Writer`-Methoden und erbt `writeLn` automatisch. Die Implementierung bleibt über die dynamisch gebundenen abstrakten Operationen anpassbar.



Originalabbildung: 03-Interfaces.pdf, Folie 15.

I · Folie 16: ANWENDUNG DEFAULT-METHODEN: "RICH INTERFACES"

`Ordered<T>` leitet vier Vergleichsoperationen aus `compareTo` ab. **Korrektur:** Das angegebene `Fraction.compareTo` berechnet die Differenz mit umgekehrtem Vorzeichen und vertauscht dadurch die Ordnung. Richtig ist beispielsweise `this.num that.denom - that.num this.denom`, bei großen Werten vorzugsweise mit `long` oder `BigInteger`.



Originalabbildung: 03-Interfaces.pdf, Folie 16.

I · Folie 17: ANWENDUNG DEFAULT-METHODEN: SCHNITTSTELLENERWEITERUNGEN

Zusätzliche Collection-Default-Methoden erweitern die API bestehender Klassen. **Präzisierung:** `Collection` ist in Java ein Interface und keine Basisklasse.

Anwendung Default-Methoden: Schnittstellenerweiterungen

Default-Methoden zur Erweiterung bestehender Interfaces (ohne die Klassen ändern zu müssen)

Beispiel: Interface **Collection**

- **Collection** ist Basisklasse für alle Collection-Klassen in Java-Bibliothek
- Erweiterung von Collection um Default-Methoden zur Unterstützung der funktionalen Programmierung (siehe Kapitel Streams)

```
public interface Collection<E> extends Iterable<E> {
  int size();
  boolean isEmpty();
  boolean contains(Object o);
  boolean add(E e);
  boolean remove(Object o);
  ...
  default <T> T[] toArray(IntFunction<T[]> generator) {...}
  default boolean removeIf(Predicate<? super E> filter) {...}
  default Stream<E> stream() {...}
  default Stream<E> parallelStream() {...}
}
```

13 abstrakte Methoden

4 zusätzliche Default-Methoden

➔ Damit **erben** alle **implementierenden Klassen** diese **Default-Methoden** und müssen **nicht selbst erweitert** werden!

© H. Prähofer, 2026 17

Originalabbildung: 03-Interfaces.pdf, Folie 17.

I · Folie 18: EINSCHRÄNKUNGEN BEI DEFAULT-METHODEN

Eine Default-Methode ist öffentlich und kann nicht `final` sein. Die Methoden `equals`, `hashCode` und `toString` von `Object` dürfen nicht durch Interface-Default-Methoden ersetzt werden.

Einschränkungen bei Default-Methoden

- Nur **public** Default-Methoden
- Eine Default-Methode kann nicht **final** deklariert werden.
- Eine Default-Methode darf keine public Methode von **Object** überschreiben. Das betrifft:
  - ☐ **toString()**
  - ☐ **equals(Object)**
  - ☐ **hashCode()**

© H. Prähofer, 2026 18

Originalabbildung: 03-Interfaces.pdf, Folie 18.

3.4 Statische Konstanten, Fabriken und private Hilfsmethoden

Interfaces dürfen **Konstanten** und **statische Methoden** enthalten. Ein Feld im Interface ist implizit `public static final`, beispielsweise `char EOL = '\n';`. Es gehört nicht zu einer einzelnen Instanz, sondern zum Interface und wird gewöhnlich als `Writer.EOL` angesprochen. Eine statische Fabrik wie

`Writer.fromFile(file)` kann ein passendes konkretes Objekt erzeugen und dennoch nach außen den abstrakten Typ `Writer` liefern. Statische Interface-Methoden sind nicht wie gewöhnliche Default-Methoden über eine Instanz polymorph gebunden.

Private Interface-Methoden, seit Java 9 verfügbar, dienen als wiederverwendbare Implementierungsdetails für Default- oder statische Methoden. Enthalten beispielsweise zwei Default-Methoden denselben Zeilenende-Schreibvorgang, kann eine `private` Methode `writeEOL()` die gemeinsame Logik kapseln. Private Methoden gehören nicht zum öffentlichen Vertrag des Interfaces und können von implementierenden Klassen nicht als Schnittstellenoperation aufgerufen oder überschrieben werden. Private statische Hilfsmethoden sind ebenfalls zulässig.

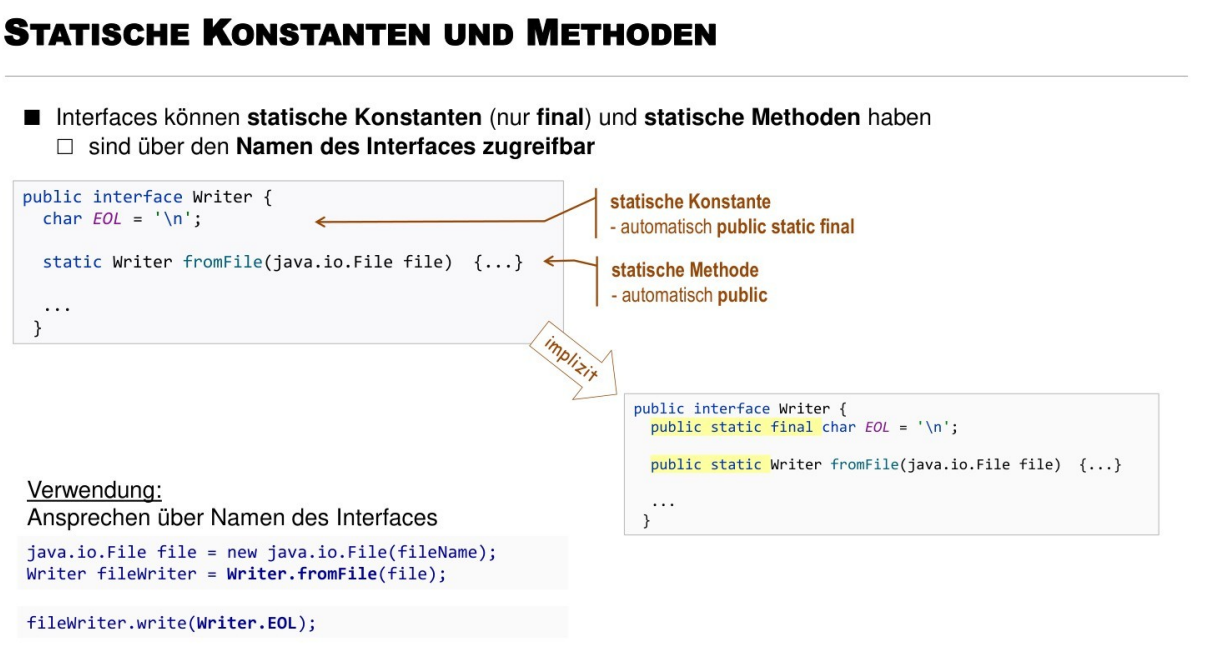
Statische Interface-Methoden bilden keine Instanzschnittstelle: `Writer.fromFile(file)` ist zulässig, `writer.fromFile(file)` dagegen nicht als entsprechender statischer Interface-Aufruf. Anders als normale statische Methoden einer Klasse werden sie nicht als geerbte statische Methoden über einen implementierenden Klassennamen aufgerufen. Private Interface-Methoden sind interne Hilfsmethoden, die etwa mehrfach benötigten Code mehrerer Default-Methoden gemeinsam aufnehmen.

Die in den Folien vorkommenden Namen `Reader`, `Writer`, `File` und `Window` bezeichnen zunächst eigene Lehrbeispieltypen. Sie dürfen nicht unbesehen mit `java.io.Reader`, `java.io.Writer` oder `java.io.File` gleichgesetzt werden. Die Standardbibliothek besitzt andere konkrete Signaturen und teilweise andere Oberklassen.

Originalfolien mit fachlicher Einordnung

I · Folie 20: STATISCHE KONSTANTEN UND METHODEN

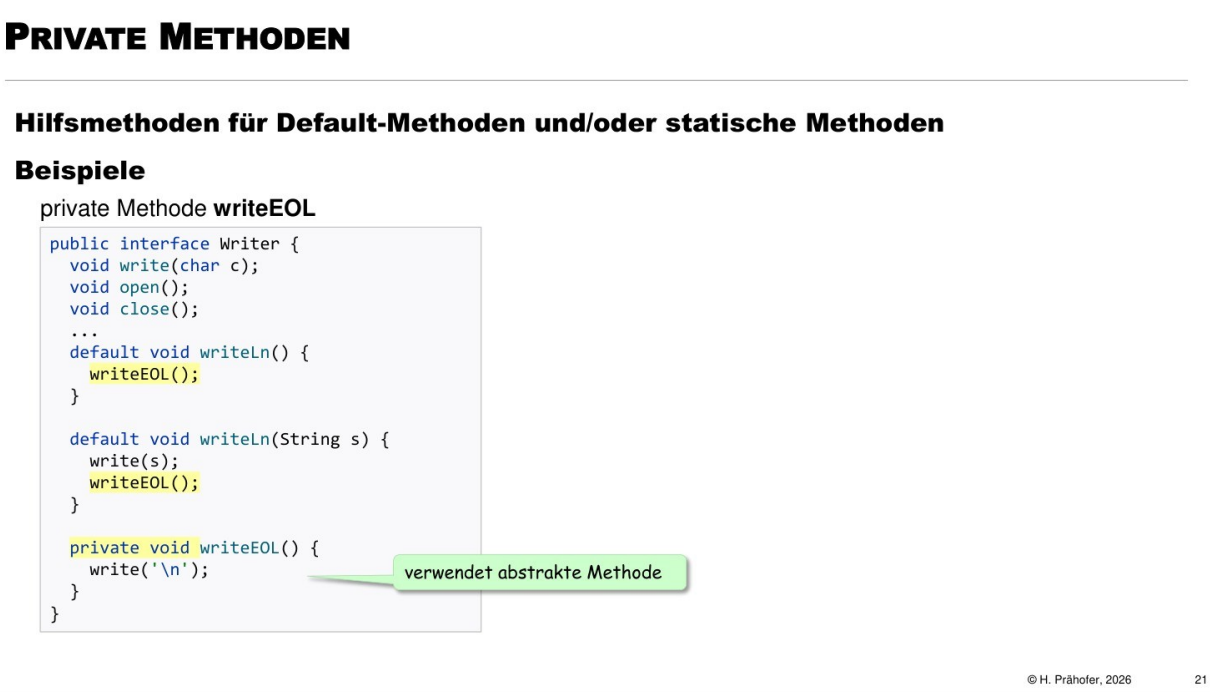
Felder in Interfaces sind automatisch `public static final`, statische Methoden werden über den Interfacenamen verwendet. Die Klasse des erzeugten Writers kann verborgen bleiben.



Originalabbildung: 03-Interfaces.pdf, Folie 20.

I · Folie 21: PRIVATE METHODEN

Eine `private` Methode wie `writeEOL()` bündelt Hilfslogik, die von mehreren Default-Methoden verwendet wird. Der Client sieht diese Hilfsmethode nicht.



Originalabbildung: 03-Interfaces.pdf, Folie 21.

3.5 Bibliotheksinterfaces: Runnable, Iterable, Ordnung und Kopien

Das Standardinterface **`Runnable`** beschreibt eine ausführbare Aufgabe mit der abstrakten Methode `void run()`. Die Klasse `Thread` kann ein solches Objekt erhalten und mit `start()` einen neuen `Thread` starten, der dessen `run()` ausführt. `run()` direkt aufzurufen startet dagegen **keinen** neuen `Thread`; die Methode läuft dann im aktuellen Ausführungskontext. Das Beispiel `Clock extends Timer implements Runnable` erhöht periodisch eine Uhrzeit und wartet mit `Thread.sleep(1000)` ungefähr eine Sekunde; echte Uhrzeitmessung erfordert wegen Laufzeitverzögerungen zusätzlich eine geeignete Zeitquelle. Bei `InterruptedException` sollte ein `Thread` Interrupts im Allgemeinen nicht gedankenlos ignorieren.

`Iterator<E>` legt die schrittweise Verarbeitung von Elementen fest. `hasNext()` fragt, ob ein weiteres Element verfügbar ist, `next()` liefert es oder wirft am Ende eine `NoSuchElementException`. `Iterable<T>` stellt die Methode `iterator()` bereit und ermöglicht dadurch die erweiterte for-Schleife `for (T x : iterable)`. Der `Iterator` kapselt die konkrete Traversierungsstrategie der Sammlung. Viele Iteratoren ermöglichen optional `remove`; wird die Operation nicht unterstützt, kann eine `UnsupportedOperationException` auftreten.

`Comparable<T>` definiert eine natürliche Ordnung durch `int compareTo(T other)`. Das Ergebnis muss nicht genau -1, 0 oder 1 sein, nur das Vorzeichen ist festgelegt. Im Beispiel `Person` wird zunächst nach Nachname, dann Vorname und schließlich Alter sortiert. Die Implementierung führt dazu die Vergleiche nacheinander aus und kehrt zurück, sobald ein Vergleich nicht `null` ist.

`Comparator<T>` ist eine auswechselbare externe Vergleichsstrategie mit `int compare(T a,T b)`. Ein `Comparator` kann dieselbe Personenkategorie anders anordnen als die natürliche Ordnung, beispielsweise

zuerst nach Alter und dann nach Namen. Eine konkrete `PersonComparator`-Klasse benötigt dafür keinen Eingriff in `Person`. Die Default-Methode `reversed()` kehrt die Ordnung eines Comparators um. Für beide Schnittstellen müssen Vergleichsregeln konsistent sein, insbesondere Transitivität der Ordnung; sonst können Sortieralgorithmen unerwartet reagieren.

Marker-Interfaces sind Interfaces ohne eigene Methoden. `Serializable` markiert Klassen für Java-Serialisierung, `Cloneable` beeinflusst die Zulässigkeit von `Object.clone()`. Die Folien weisen darauf hin, dass Annotationen heute in vielen Fällen eine flexiblere Kennzeichnung bieten, während die genannten Marker in der bestehenden Standardbibliothek weiterhin reale Bedeutung haben.

Das geschützte `Object.clone()` erzeugt im Standardfall eine **flache Kopie**: Die direkten Feldwerte werden kopiert; Referenzfelder verweisen anschließend auf dieselben Unterobjekte wie im Original. Zur öffentlichen Unterstützung von `clone()` implementiert eine Klasse üblicherweise `Cloneable`, überschreibt die Methode öffentlich, bietet einen kovarianten Rückgabetyp und verarbeitet `CloneNotSupportedException`. Alternativ nimmt ein **Copy-Konstruktor** ein vorhandenes Objekt entgegen, etwa `Person(Person other)`. Durch `new Address(other.address)` kann er ein verschachteltes veränderbares Objekt ausdrücklich mitkopieren und so eine tiefere Kopie ermöglichen. Ob ein Kopiervorgang wirklich vollständig tief ist, hängt von allen beteiligten Unterobjekten ab.

`Iterable<E>` definiert die Methode `Iterator<E> iterator()` und ermöglicht damit die for-each-Schleife. Ein `Iterator<E>` stellt mit `hasNext()` fest, ob ein weiterer Wert vorhanden ist, und liefert ihn mit `next()`. Wird `next()` nach Erreichen des Endes aufgerufen, muss ein ordnungsgemäßer `Iterator` eine `NoSuchElementException` auslösen. Die Methode `remove()` ist optional und kann eine `UnsupportedOperationException` werfen.

`Comparable<T>` definiert die natürliche Ordnung eines Typs über `compareTo(T)`. Das Ergebnis ist negativ, `null` oder positiv – nicht notwendig genau -1, 0 oder +1. `Comparator<T>` ist ein eigenständiges Vergleichsobjekt, das zwei Werte über `compare(a,b)` ordnet und dadurch alternative Sortierungen erlaubt. Eine Ordnung sollte konsistent, transitiv und möglichst mit `equals` verträglich sein; bei abweichender Gleichheit können sortierte Mengen sonst überraschende Ergebnisse liefern.

Das Markerinterface `Cloneable` erklärt lediglich eine Fähigkeit für die spezielle `Object.clone()`-Implementierung. Ein Aufruf von `clone()` erzeugt standardmäßig nur eine flache Kopie; referenzierte veränderliche Unterobjekte werden nicht automatisch dupliziert. Ein Copy-Konstruktor kann die gewünschte Kopiertiefe ausdrücklich festlegen. Das Interface `Serializable` kennzeichnet die Teilnahme am Java-Serialisierungsmechanismus; es garantiert weder Datensicherheit noch Kompatibilität über beliebige Versionen hinweg.

Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
java.util.List<String> words = java.util.List.of("A", "B");
for (String word : words) System.out.println(word);
java.util.Comparator<String> byLength =
 java.util.Comparator.comparingInt(String::length);
```

Originalfolien mit fachlicher Einordnung

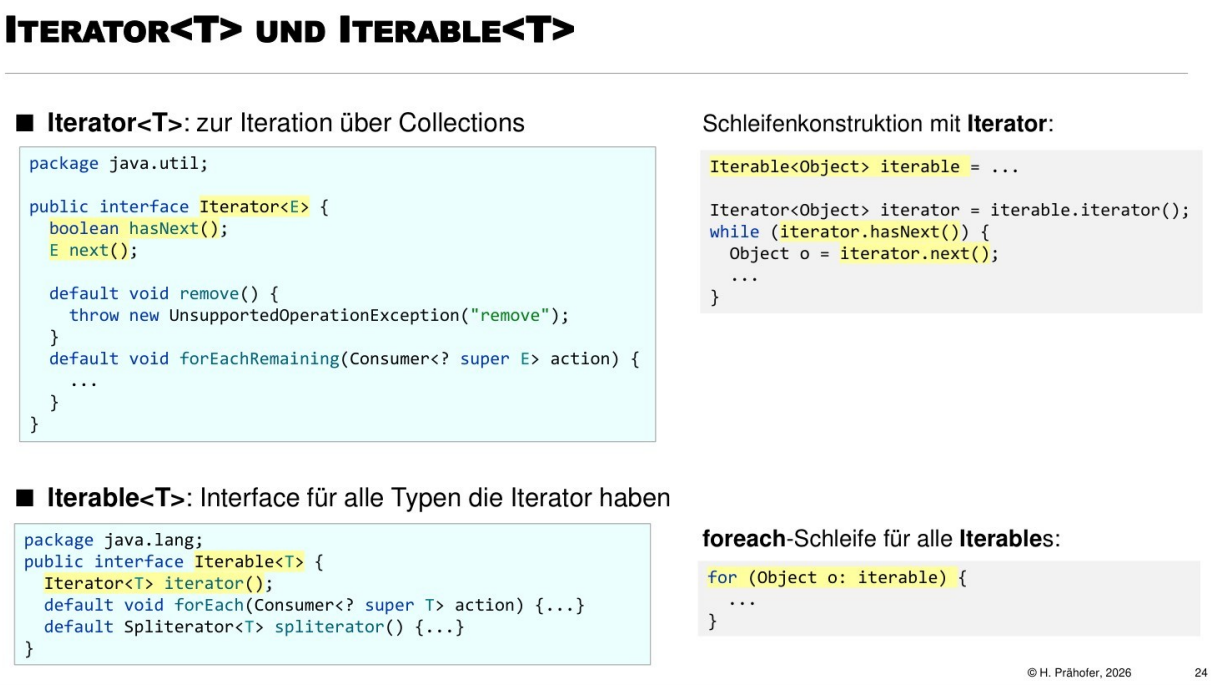
I · Folie 23: INTERFACE JAVA.LANG.RUNNABLE

`new Thread(runnable).start()` startet einen `Thread`, der `run()` ausführt. Ein direkter Aufruf `run()` würde nur im selben `Thread` laufen.

Originalabbildung: 03-Interfaces.pdf, Folie 23.

I · Folie 24: ITERATOR<T> UND ITERABLE<T>

`Iterable<T>` liefert einen `Iterator`; die for-each-Schleife ist eine bequemere Schreibweise für das schrittweise Iterieren mit `hasNext` und `next`.



Originalabbildung: 03-Interfaces.pdf, Folie 24.

I · Folie 25: COMPARABLE<T>

`Comparable<Person>` definiert die natürliche Vergleichsregel innerhalb von `Person`. Die Vergleichskaskade folgt Nachname, Vorname und Alter.

COMPARABLE<T>

■ Für Ordnungsrelation

```
public interface Comparable<T> {
  public int compareTo(T other);
}
```

```
 < 0  : this < other
== 0  : this.equals(other)
 > 0  : this > other
```

■ Implementierung durch Klassen

```
public class Person implements Comparable<Person> {
  private String firstName;
  private String lastName;
  private int age;
  ...

  public int compareTo(Person other) {
    int c = this.lastName.compareTo(other.lastName);
    if (c != 0) return c;
    c = this.firstName.compareTo(other.firstName);
    if (c != 0) return c;
    return Integer.compare(this.age, other.age);
  }
}
```

© H. Prähofer, 2026 25

Originalabbildung: 03-Interfaces.pdf, Folie 25.

I · Folie 26: COMPARATOR<T>

`Comparator<Person>` hält eine alternative Ordnung außerhalb von `Person`. Mit `reversed()` lässt sich diese Ordnung umdrehen.

COMPARATOR<T>

■ Zum Vergleich von zwei Objekten

```
package java.util;
public interface Comparator<T> {
  int compare(T o1, T o2);

  default Comparator<T> reversed() {
    return Collections.reverseOrder(this);
  }
  ...
}
```

mehrere Default-Methoden

■ Erzeugen von Comparator-Objekten (unabhängig von eigentlicher Klasse)

```
import java.util.Comparator;

public class PersonComparator implements Comparator<Person> {
  public int compare(Person p1, Person p2) {
    int c = Integer.compare(p1.age(), p2.age());
    if (c != 0) return c;
    c = p1.getLastName().compareTo(p2.getLastName());
    if (c != 0) return c;
    return p1.getFirstName().compareTo(p2.getFirstName());
  }
}
```

Mehr zu Comparable und Comparator bei Kapitel Klassenbibliothek!

© H. Prähofer, 2026 26

Originalabbildung: 03-Interfaces.pdf, Folie 26.

I · Folie 27: SONDERFALL: TAGGING INTERFACES Achtung: Statt Tagging Interfaces werden heute Annotationen eingesetzt!

Marker-Interfaces enthalten keine Methoden. `Serializable` und `Cloneable` sind historische und weiterhin wirksame Standardbibliotheksbeispiele.

SONDERFALL: TAGGING INTERFACES

Achtung: Statt Tagging Interfaces werden heute Annotationen eingesetzt!

- Tagging Interfaces haben **keine Methode**
- werden ausschließlich zur **Kennzeichnung von Klassen** verwendet
  ➔ Klasse ist durch den **Interface-Typ** gekennzeichnet

Bespiele:

- **Serializable**: für Klassen, die über Serialisierungsmechanismus gespeichert werden können

```
package java.io;

public interface Serializable { }
```

- **Cloneable**: für Klassen, die man mit **clone()** kopieren können soll

```
package java.lang;

public interface Cloneable { }
```

© H. Prähofer, 2026 27

Originalabbildung: 03-Interfaces.pdf, Folie 27.

I · Folie 28: FALLBEISPIEL: INTERFACE CLONEABLE UND METHODE CLONE

`Object.clone()` ist geschützt und erzeugt grundsätzlich eine flache Feldkopie. Ohne `Cloneable` führt der Standardaufruf im Regelfall zu `CloneNotSupportedException`.

FALLBEISPIEL: INTERFACE CLONEABLE UND METHODE CLONE

- Tagging-Interface **Cloneable** zur Kennzeichnung von Klassen, die **clone** unterstützen sollen

```
public interface Cloneable { }
```

- Methode **clone** in **Object**
  - native Methode für Speicherkopie des aufgerufenen Objekts
  - wirft eine **CloneNotSupportedException**, wenn die Klasse **Cloneable** nicht implementiert
  - ist **protected**, kann daher nur in Unterklassen aufgerufen werden
  - Rückgabetyp ist **Object**, zurückgegebenes Objekt ist vom gleichem Typ wie geclontes Objekt

```
public class Object {
  protected native Object clone() throws CloneNotSupportedException;
  ...
}
```

- Klasse, die **clone** unterstützen soll, muss
  - **Cloneable** implementieren
  - **clone** von **Object** überschreiben ➔ siehe nächste Folie

© H. Prähofer, 2026 28

Originalabbildung: 03-Interfaces.pdf, Folie 28.

I · Folie 29: FALLBEISPIEL: INTERFACE CLONEABLE UND METHODE CLONE

Das Überschreiben von `clone` erweitert die Sichtbarkeit und spezialisiert den Rückgabetyp auf `Timer`. Der Catch-Zweig verarbeitet die checked `Exception` des `super.clone`-Aufrufs.

Fallbeispiel: Interface Cloneable und Methode clone

Klasse, die clone unterstützen soll, muss clone von Object folgend überschreiben

- Sichtbarkeit auf **public** erweitern
- Typ des Rückgabewerts kovariant auf Klasse spezialisieren
- **CloneNotSupportedException** abfangen und keine Exception werfen
- mit Super-Call Kopie erstellen
- Kopie auf Klasse casten und zurückgeben

```
public class Object {
  protected native Object clone() throws CloneNotSupportedException;
```

```
public class Timer implements Cloneable {

  @Override
  public Timer clone() {
    try {
      return (Timer) super.clone();
    } catch (CloneNotSupportedException e) {
      throw new InternalError();
    }
  }
  ...
```

Kovarianter Rückgabetyp
Erweiterung Sichtbarkeit auf **public**
Typcast zur speziellen Klasse
Interface **Cloneable** implementieren
clone überschreiben ohne **throws CloneNotSupportedException**
super-Aufruf von **clone** von **Object** erstellt Kopie
CloneNotSupportedException fangen

© H. Prähofer, 2026 29

Originalabbildung: 03-Interfaces.pdf, Folie 29.

I · Folie 30: ALTERNATIVE ZU CLONE: COPY-KONSTRUKTOREN

Ein Copy-Konstruktor kann Unterobjekte ausdrücklich kopieren. `new Address(other.address)` vermeidet die gemeinsame Benutzung desselben veränderbaren Address-Objekts.

Alternative zu clone: Copy-Konstruktoren

- Als Alternative zu **clone**, werden heute oft Copy-Konstruktoren verwendet
  - ☐ Konstruktor mit dem zu kopierenden Objekt als Parameter
  - ☐ Initialisieren der Felder mit Werten von zu kopierendem Objekt
  - ☐ Möglichkeit mit Copy-Konstruktoren der Felder eine tiefe Kopie zu erstellen

```
public class Person {
  private final String name;
  private final Address address;

  public Person(String name, Address address) {
    this.name = name;
    this.address = address;
  }

  public Person(Person other) {
    this.name = other.name;
    this.address = new Address(other.address);
  }

  public String name() { return name; }

  public Address address() { return address; }
}
```

Copy-Konstruktor mit zu kopierendem Objekt als Parameter
Setzen der Werte aus kopiertem Objekt
Tiefe Kopie durch Copy-Konstruktor von Feldwert

© H. Prähofer, 2026 30

Originalabbildung: 03-Interfaces.pdf, Folie 30.

3.6 Fallstudie List und MutableList: lesende versus ändernde Zugriffe

Die `List`-Fallstudie wird nun von einer abstrakten Basisklasse auf eine **Interface-basierte Architektur** umgebaut. Das Interface `List` beschreibt die verwendbare Schnittstelle; eine abstrakte Klasse `AbstractList` kann wiederverwendbare Basisimplementierungen anbieten; `LinkedList` und `ArrayList` bleiben konkrete Speicherstrukturen. Adapter und Decorators können dieselbe `List`-Schnittstelle implementieren, ohne in derselben Klassenvererbungslinie zu liegen. Genau das ist ein großer Vorteil von Interfaces: Unterschiedliche Realisierungen können unter einem gemeinsamen Vertrag zusammenarbeiten.

Zunächst enthält das vorgestellte `List`-Interface sowohl lesende Methoden (`size`, `get`, `contains`, `indexOf`) als auch verändernde Methoden (`add`, `remove`, `clear`). Ein `UnmodifiableListDecorator` kann Änderungen zwar mit `UnsupportedOperationException` blockieren, bietet aber dennoch formal eine Schnittstelle an, die diese Operationen enthält. Für den Benutzer ist das eine Schwäche des Vertrages: Man kann syntaktisch `add` aufrufen und erfährt erst zur Laufzeit, dass die konkrete Variante es nicht erlaubt.

Eine alternative Architektur trennt deshalb **`List`** mit ausschließlich lesenden Operationen von **`MutableList extends List`** mit mutierenden Operationen. Eine Funktion, die nur lesen soll, erhält den schmalen Typ `List`; die Anwendung darf intern weiterhin eine `MutableList` besitzen. Das ist eine bewusste Einschränkung der statischen Zugriffsmöglichkeiten und ähnelt dem Modell, das die Folien als Kotlin-Ansatz präsentieren.

Ein bloßer Rückgabetyp `List` schützt allerdings nicht garantiert vor Änderungen: Wenn das zugrunde liegende Objekt tatsächlich `MutableList` implementiert, kann ein Client bei bekanntem Laufzeittyp einen Cast `(MutableList) list` versuchen und anschließend die Mutationsmethoden aufrufen. Besser ist ein eigenständiges **Read-only-Wrapperobjekt**, das tatsächlich nur das lesende Interface implementiert und intern auf die veränderliche Liste delegiert. Ein Cast dieses Wrappers zu `MutableList` schlägt zur Laufzeit fehl. Dennoch ist der Wrapper damit lediglich eine **unveränderbare Zugriffssicht**: Wird die zugrunde liegende Liste über eine andere Referenz geändert, sieht die Wrapper-Sicht den geänderten Inhalt. Eine absolut unveränderliche Liste erfordert zusätzliche Maßnahmen wie Kopieren beziehungsweise unveränderliche interne Datenstrukturen.

Im abschließenden Klassendiagramm erscheinen die Beziehungen gemeinsam: `List` als allgemeiner lesender Vertrag, `MutableList` als Erweiterung, `AbstractList`, konkrete Listen, Decorators sowie Array- und Einzelobjekt-Adapter. Die Vorlesung nennt als weitere Verbesserungen geschachtelte Klassen für Listenknoten, Iteratoren für sequentielles Traversieren und generische Elementtypen; diese späteren Themen sind in den vorliegenden Foliensätzen nur als Ausblick enthalten.

Eine als read-only deklarierte Schnittstelle verhindert lediglich schreibende Operationen über diese Referenz. Wenn das reale Objekt gleichzeitig `MutableList` implementiert, kann ein geeigneter Cast oder eine andere Referenz die Daten ändern. Ein Wrapper, dessen tatsächliche Klasse ausschließlich das lesende Interface implementiert, verhindert zwar den Cast auf `MutableList`, kann aber weiterhin eine lebende Sicht auf eine intern veränderliche Liste anbieten. Weder read-only noch ein solcher Wrapper bedeuten automatisch „unveränderlicher Snapshot“.

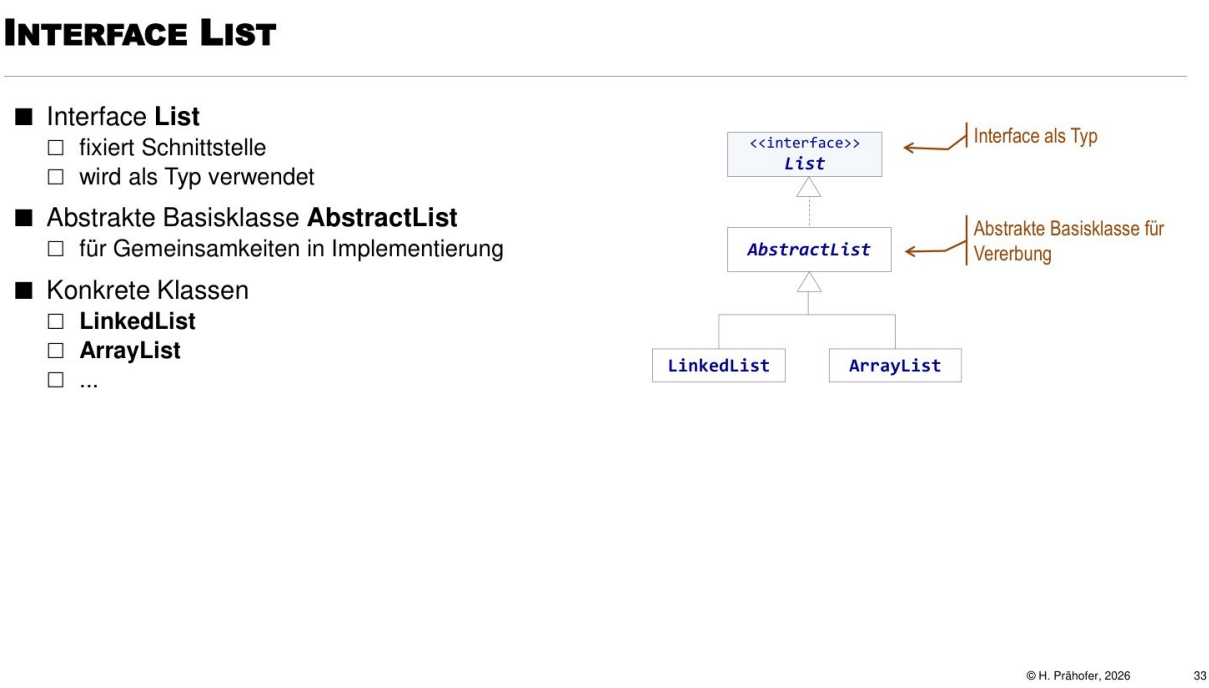
Die Fallstudie macht einen grundlegenden API-Unterschied sichtbar: Ein Interface beschreibt, welche Operationen zugänglich sind; eine Implementierung beschreibt, wie sie ausgeführt werden. Wenn die Fähigkeit zur Änderung von einer lesenden Schnittstelle getrennt wird, können Methoden gezielter minimale

Fähigkeiten anfordern. Der Schutz interner Daten hängt aber zusätzlich davon ab, ob Rückgaben kopieren, delegieren oder nur eine lebende Sicht erzeugen.

Originalfolien mit fachlicher Einordnung

I · Folie 33: INTERFACE LIST

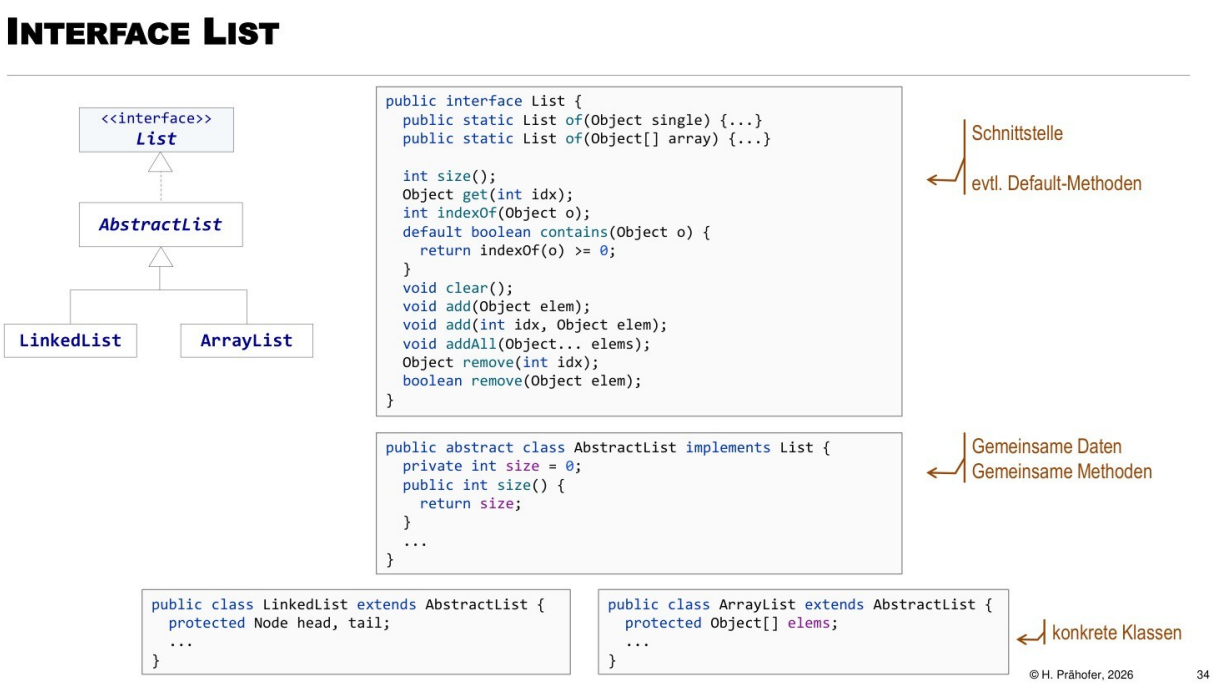
Die UML-Hierarchie trennt gemeinsame Schnittstelle (`List`), abstrakte Wiederverwendung (`AbstractList`) und zwei konkrete Speicherrepräsentationen.



Originalabbildung: 03-Interfaces.pdf, Folie 33.

I · Folie 34: INTERFACE LIST

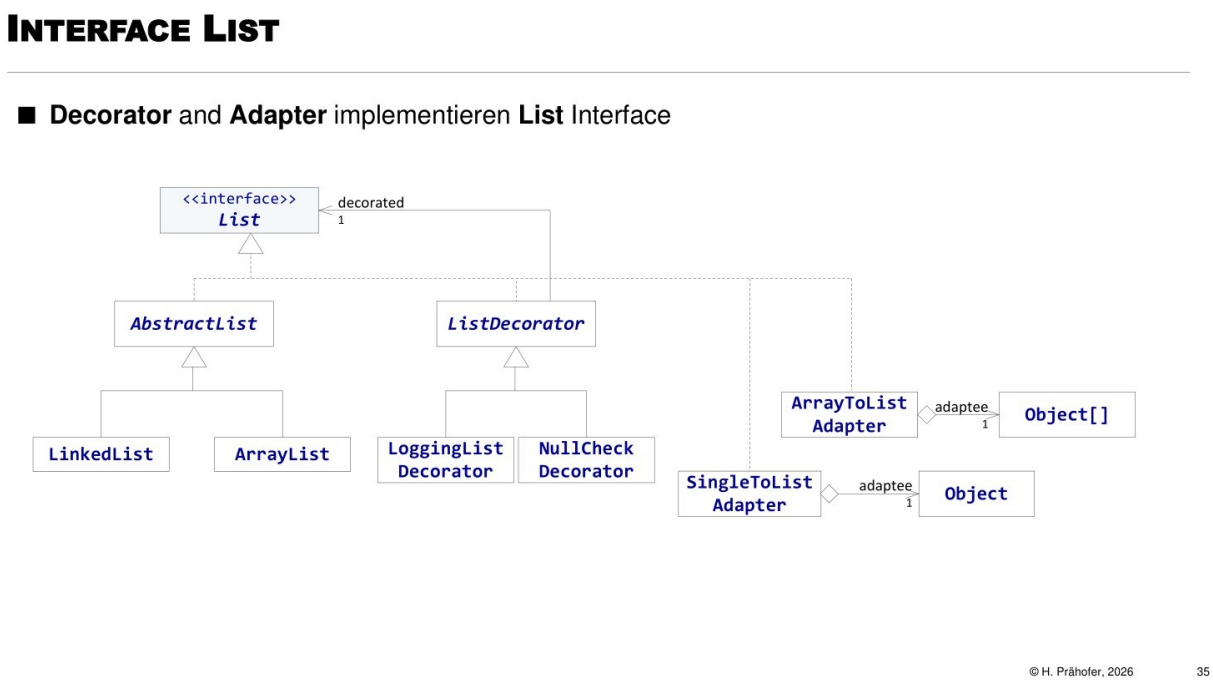
Die gezeigte `List` ist eine eigens entworfene Modell-API. Sie ist nicht identisch mit allen Signaturen von `java.util.List`.



Originalabbildung: 03-Interfaces.pdf, Folie 34.

I · Folie 35: INTERFACE LIST

Decorators und Adapter erfüllen denselben `List`-Vertrag, obwohl sie keine Unterklassen von AbstractList sein müssen. Dadurch können unabhängige Implementierungslinien zusammenspielen.



Originalabbildung: 03-Interfaces.pdf, Folie 35.

I · Folie 37: UNMODIFIABLELISTDECORATOR: PROBLEMANALYSE

Ein UnmodifiableListDecorator blockiert Mutation zur Laufzeit. Das Interface selbst enthält jedoch weiterhin mutierende Operationen und signalisiert diese Einschränkung nicht statisch.

UnmodifiableListDecorator: Problemanalyse

UnmodifiableListDecorator wirft UnsupportedOperationException

- Decorator, um Veränderungen zu verhindern

```
public class UnmodifiableListDecorator extends ListDecorator {

  public UnmodifiableListDecorator(List decorated) {
    super(decorated);
  }

  @Override
  public void add(Object elem) {
    throw new UnsupportedOperationException("forbidden");
  }

  @Override
  public void add(int idx, Object elem) {
    throw new UnsupportedOperationException("forbidden");
  }

  @Override
  public Object remove(int idx) {
    throw new UnsupportedOperationException("forbidden");
  }

  ...
}
```

Anwendung: **nur lesender** Zugriff auf interne Daten

```
public class MyClass {

    private List myPrivateListData = new ArrayList();

    public List getListData() {
        return new UnmodifiableListDecorator(myPrivateListData);
    }
    ...
}
```

Nur lesende Zugriffe !

```
MyClass myObject = new MyClass();
List data = myObject.getListData();

data.remove(...); // throws UnsupportedOperationException
```

Problematisch: Interface **List** bietet **add**, **remove** etc. an, und **UnmodifiableList** verhindert dies mit einer Exception!?

© H. Prähofer, 2026 37

Originalabbildung: 03-Interfaces.pdf, Folie 37.

I · Folie 38: Approach von Kotlin!INTERFACES LIST UND MUTABLELIST

Die getrennten Interfaces `List` und `MutableList` helfen bereits beim Kompilieren, unzulässige Methodenaufrufe abzuweisen. **Korrektur:** Die gezeigte `remove(Object)`-Default-Methode braucht einen Sonderfall, falls `indexOf` den Wert -1 liefert.

Interfaces List und MutableList

Approach von Kotlin!

- Trennung der Interfaces
  - **List**: nur lesende Zugriffe (*read-only*)
  - **MutableList**: Methoden mit Veränderungen

```
public interface List {
  public int size();
  public Object get(int idx);
  public default boolean contains(Object o) {
    return indexOf(o) >= 0;
  }
  public int indexOf(Object o);
}
```

```
public interface MutableList extends List {
  public void add(Object elem);
  public default void addAll(Object...elems) {
    for (Object elem: elems) { add(elem); }
  }
  public boolean remove(int idx);
  public default boolean remove(Object elem) {
    return remove(indexOf(elem));
  }
  public default void clear() {...}
}
```

```
public class LinkedList implements MutableList { ... }
public class ArrayList implements MutableList { ...}
```

Verwendung bei Datenkapselung:

```
public class MyApplication {
    private MutableList myPrivateListData = new ArrayList();
    public void addData(String data) {
        myPrivateListData.add(data);

    public List getListData() {
        return myPrivateListData;
    }
}
```

Rückgabe als read-only Interface

intern als mutable

Problem: mit Typecast kann Anwender Zugriffsschutz umgehen

```
MyClass myObject = new MyClass();
List list = myObject.getListData();
MutableList mutList = (MutableList) list;
mutList.clear();
```

Client kann mit Typcast vollen Zugriff erlangen

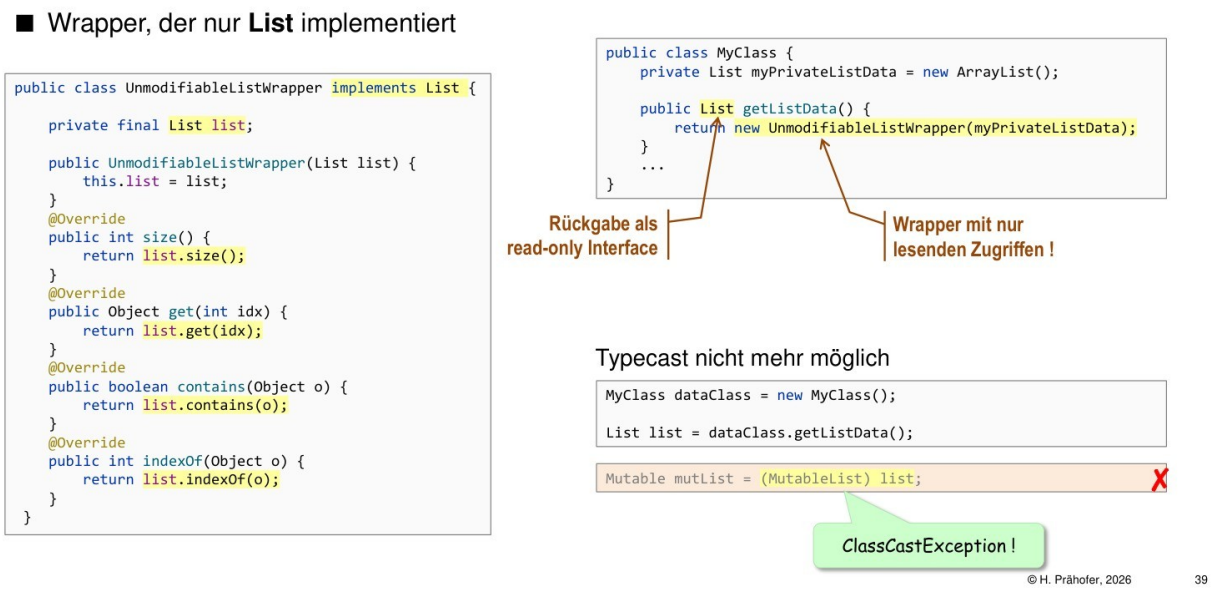
© H. Prähofer, 2026 38

Originalabbildung: 03-Interfaces.pdf, Folie 38.

I · Folie 39: WIRKLICH UNVERÄNDERLICHE LISTE

Der nur `List` implementierende Wrapper kann nicht erfolgreich als `MutableList` gecastet werden. Trotzdem bleibt die zugrunde liegende Liste über andere Referenzen veränderbar: Das ist eine read-only-Sicht, keine notwendigerweise tief immutable Liste.

Wirklich unveränderliche Liste

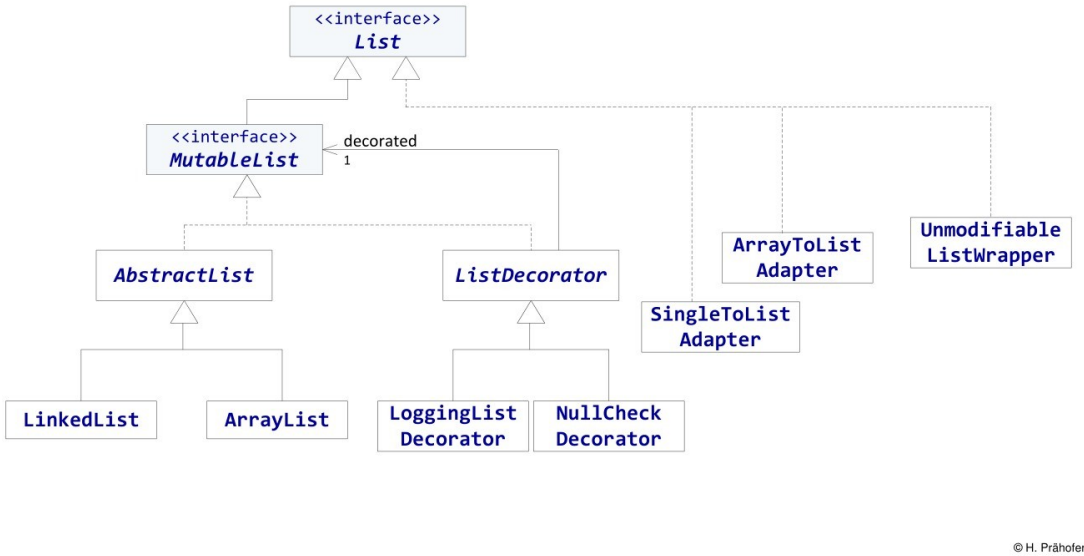


Originalabbildung: 03-Interfaces.pdf, Folie 39.

I · Folie 40: KLASSENDIAGRAMM LIST UND MUTABLELIST

Das Diagramm zeigt, welche Klassen `List` oder `MutableList` realisieren und welche Objekte sie adaptieren beziehungsweise dekorieren.

Klassendiagramm List und MutableList



Originalabbildung: 03-Interfaces.pdf, Folie 40.

I · Folie 42: WEITERE VERBESSERUNGEN

Die noch offenen Verbesserungen sind innere Knotenklassen, Iteratoren und Generics. Sie werden als Ausblick genannt und können ohne weitere Folien nicht vollständig aus diesem Material hergeleitet werden.

Weitere Verbesserungen

- Innere Klassen für Knoten ➔ 5. Innere Klasse
- Iterator für sequentiellen Zugriff ➔ 5. Innere Klasse
- Generische Elementtypen ➔ 6. Generics

© H. Prähofer, 2026

Originalabbildung: 03-Interfaces.pdf, Folie 42.

4 Spezielle Klassen

Das vierte Themengebiet baut auf Klassen, Vererbung und Interfaces auf und behandelt besondere Arten von Klassendeklarationen: Records als kompakte, wertorientierte Datenbehälter, Enum-Klassen für eine feste Menge benannter Instanzen sowie geschachtelte, lokale und anonyme Klassen. Anschließend werden diese Sprachmittel in einer zusammenhängenden Fallstudie auf Listen, Iteratoren und Adapter angewandt. Die Beispiele `List`, `MutableList` und `IndexedList` sind eigene Lehrbeispiele und dürfen nicht mit gleichnamigen Typen der Java-Standardbibliothek verwechselt werden. Wo für das Verständnis eine exakte Java-Regel erforderlich ist, wird sie zusätzlich zur Aussage der Folie erklärt.

4.1 Record-Klassen

Eine `Record-Klasse (Schlüsselwort record, standardisiert seit Java 16)` beschreibt einen unveränderlichen Satz benannter Komponenten, beispielsweise Zähler und Nenner eines Bruchs. Die Kopfzeile `record Fraction(int num, int denom)` legt bereits die Struktur des Datensatzes fest. Der Compiler stellt für diese Komponenten `private`, endgültig zugewiesene Felder, einen passenden Konstruktor und öffentliche Lesezugriffe bereit. Anders als bei gewöhnlichen JavaBeans heißen diese Zugriffe `num()` und `denom()`, nicht `getNum()` und `getDenom()`. Außerdem werden `equals(Object)`, `hashCode()` und `toString()` automatisch anhand der Komponenten realisiert.

Bei der Erzeugung `Fraction f = new Fraction(1, 2)` laufen die Argumente durch den kanonischen Konstruktor. Dieser übernimmt alle Komponenten in derselben Reihenfolge und mit denselben Typen wie im `Record`-Kopf. Fehlt eine eigene Implementierung, erstellt der Compiler einen entsprechenden Konstruktor. Soll ein ungültiger Zustand verhindert werden, kann die Konstruktion selbst validieren: Ein Nenner von `null` ist bei einer Bruchdarstellung nicht zulässig. Ein kompakter Konstruktor `public Fraction { ... }` verzichtet auf die eigene Parameterliste; die Komponentenparameter stehen dennoch im Block zur Verfügung und werden nach erfolgreicher Ausführung den Feldern zugewiesen. Normierungen der Parameter sind möglich, direkte Zuweisungen `this.num = ...` hingegen sind in einem kompakten Konstruktor unzulässig.

Wichtig ist der genaue Begriff der Unveränderlichkeit. Die automatisch erzeugten Felder eines Records sind `final`: Ihre Referenzen beziehungsweise primitiven Werte können nach der Konstruktion nicht neu gesetzt werden. Enthält eine Komponente jedoch ein veränderbares Objekt, etwa ein Array oder eine Liste, ist auch der `Record` nicht automatisch tief unveränderlich. Der Aufrufer kann das referenzierte Objekt unter Umständen weiterhin verändern. Für echte Wertobjekte müssen veränderbare Eingaben defensiv kopiert und beim Zugriff gegebenenfalls ebenfalls kopiert oder durch unveränderliche Sichten ersetzt werden.

Records sind implizit `final` und leiten sich direkt von `java.lang.Record` ab. Sie können keine zusätzliche Instanzfelder deklarieren oder von einer eigenen Superklasse erben. Erlaubt sind hingegen statische Felder, Instanz- und Klassenmethoden, implementierte Interfaces, zusätzliche Konstruktoren mit einem Aufruf eines anderen Konstruktors sowie selbst definierte Zugriffs-, Vergleichs- und Textmethoden. Wer `equals` und `hashCode` überschreibt, muss beide Verträge konsistent halten; bei Records ist das meistens unnötig. Zwei Brüche wie `1/2` und `2/4` werden ohne vorherige Normalisierung nicht als gleich erkannt, weil die Komponentenwerte unterschiedlich sind, obwohl beide denselben mathematischen Wert darstellen.

Für die Brucharithmetik gilt `(a/b) + (c/d) = (a*d + c*b)/(b*d)` und `(a/b)*(c/d) = (a*c)/(b*d)`. Die Ganzzahlmultiplikationen in den Codebeispielen können bei großen Operanden überlaufen; dies ist bei `int` unabhängig von der `Record`-Syntax. Bei realer Implementierung gehören eine Prüfung auf Nullnenner, eine einheitliche Vorzeichenregel, optional Kürzen mittels größtem gemeinsamen Teiler sowie eine Entscheidung über Überlaufbehandlung oder BigInteger zu einem sauberen Modell.

Ein `Record` deklariert explizit die Komponenten seines Zustands, etwa `record Fraction(int num, int denom) {}`. Der Compiler erzeugt insbesondere `private` finale Komponentenfelder, Zugriffsmethoden `num()` und `denom()`, einen kanonischen Konstruktor sowie `equals`, `hashCode` und `toString` auf Komponentenbasis. Die generierte Zugriffsmethode heißt nicht `getNum()`. Ein `Record` ist implizit `final` und erweitert die spezielle Oberklasse `java.lang.Record`.

Ein kompakter Konstruktor `public Fraction { ... }` prüft oder normalisiert die Parameter, bevor die impliziten Zuweisungen an Komponentenfelder erfolgen. Beispielsweise muss bei einer Bruchklasse `denom == 0` abgefangen werden. Ein zusätzlicher Konstruktor wie `Fraction(int n) { this(n, 1); }` muss letztlich zum kanonischen Konstruktor delegieren. Records sind flach unveränderlich: Enthält eine Komponente eine veränderliche Liste, bleibt deren Inhalt ohne defensive Kopie veränderbar.

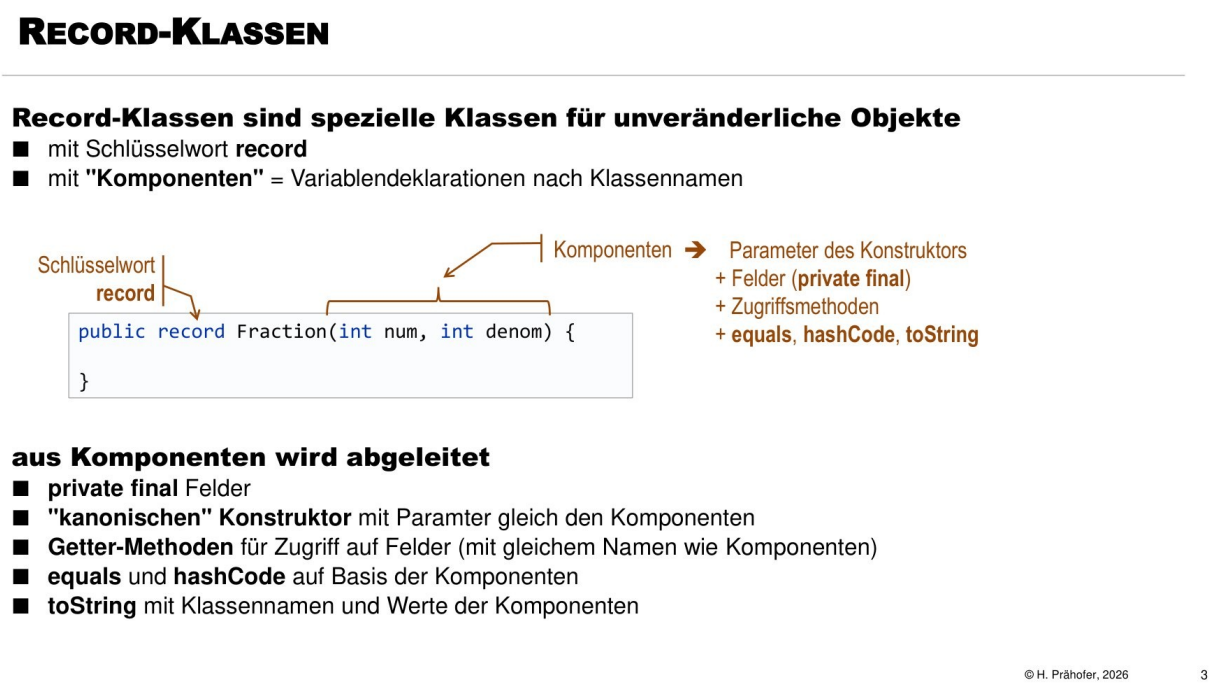
Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
public record Fraction(int num, int denom) {
 public Fraction {
 if (denom == 0) throw new IllegalArgumentException("Nenner 0");
 }
 public Fraction add(Fraction other) {
 return new Fraction(num * other.denom + other.num * denom,
 denom * other.denom);
 }
}
```

Originalfolien und ausführliche Einordnung

S · Folie 3: RECORD-KLASSEN Record-Klassen sind spezielle Klassen für unveränderliche Objekte

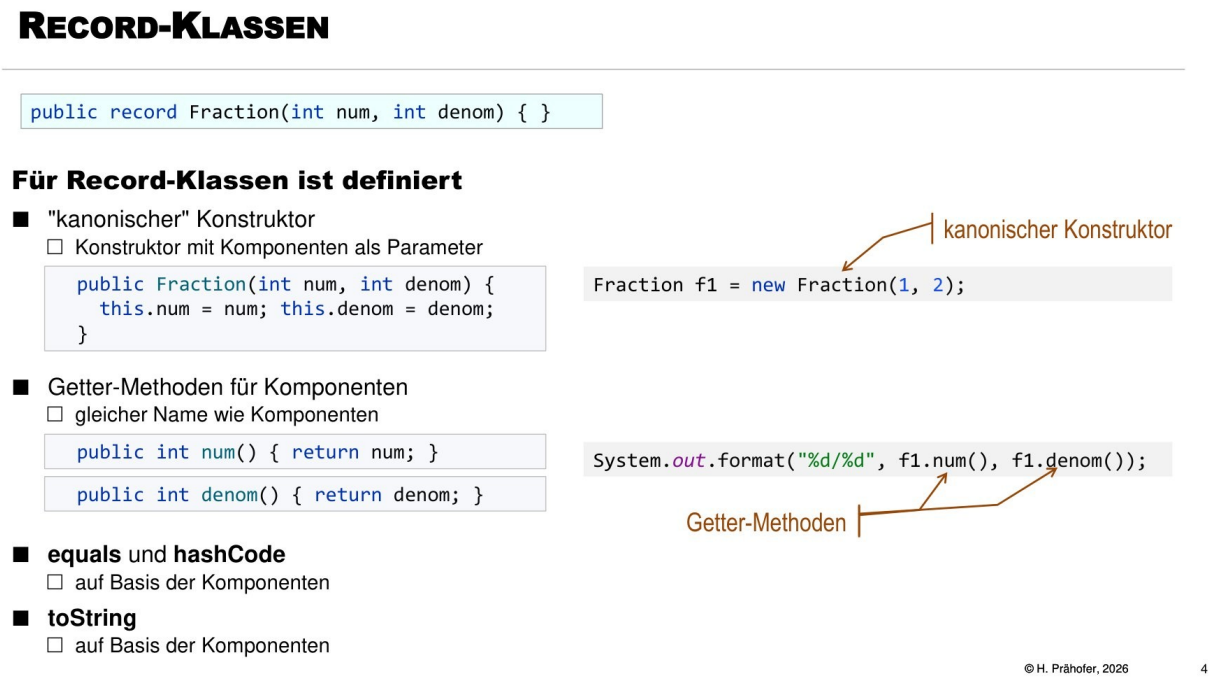
In der Kopfzeile des Records stehen die beiden Komponenten num und denom. Daraus erzeugt der Compiler je ein privates finales Instanzfeld, den kanonischen Konstruktor und die beiden gleichnamigen Zugriffsmethoden. equals, hashCode und toString sind keine frei erfundenen Hilfsfunktionen, sondern gehören zur von Java vorgeschriebenen `Record`-Semantik. Das gezeichnete „unveränderlich“ meint zunächst die Bindung der Komponenten, nicht zwingend die Tiefenunveränderlichkeit jedes referenzierten Objekts.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 3.

S · Folie 4: RECORD-KLASSEN Für Record-Klassen ist definiert

Der Aufruf `new Fraction(1, 2)` ordnet die Argumente den Konstruktorparametern num und denom zu. Anschließend lassen sich diese Werte über `f1.num()` und `f1.denom()` lesen. Die Folie trennt Konstruktion und Zugriffsmethoden; gerade diese Kopplung zwischen `Record`-Kopf und öffentlich zugänglichen Komponenten macht den Datenträger transparent.

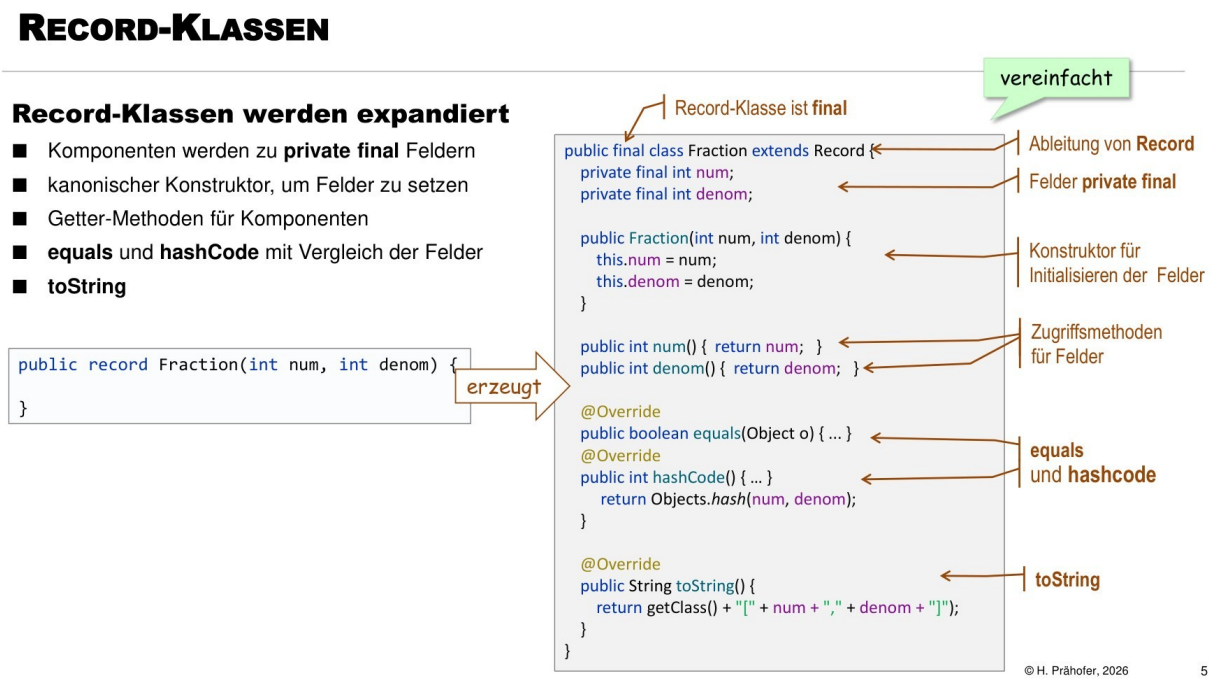


Originalabbildung: 04-Spezielle Klassen.pdf, Folie 4.

S · Folie 5: Record-Klassen werden expandiert

Die rechte Klassendeklaration ist als vereinfachtes Denkmodell gekennzeichnet, nicht als wortwörtlicher Compiler-Output. Sie veranschaulicht die geerbte Basisklasse `Record`, finale Felder und vom Compiler implementierte Methoden.

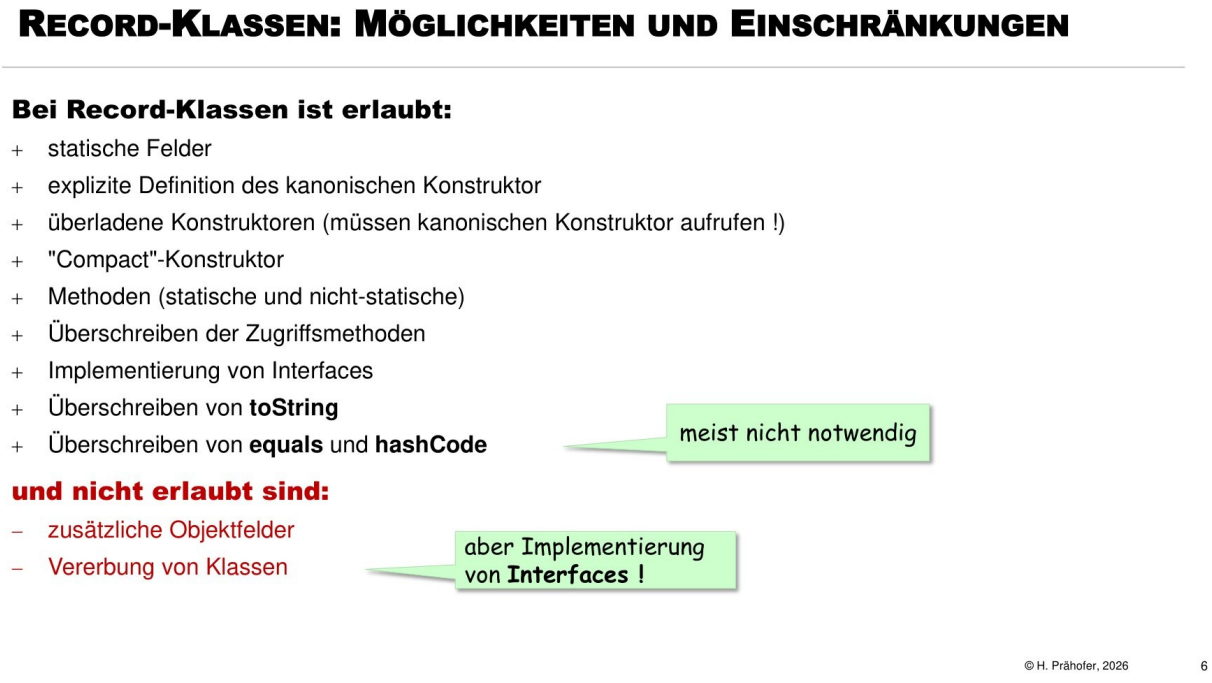
Der Ausdruck `getClass()` im skizzierten toString ist nicht gleichbedeutend mit der üblichen Darstellung `Fraction`[num=1, denom=2]. Die konkrete automatisch erzeugte Zeichenkette enthält normalerweise Komponentennamen und -werte.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 5.

S · Folie 6: RECORD-KLASSEN: MÖGLICHKEITEN UND EINSCHRÄNKUNGEN Bei Record-Klassen ist erlaubt

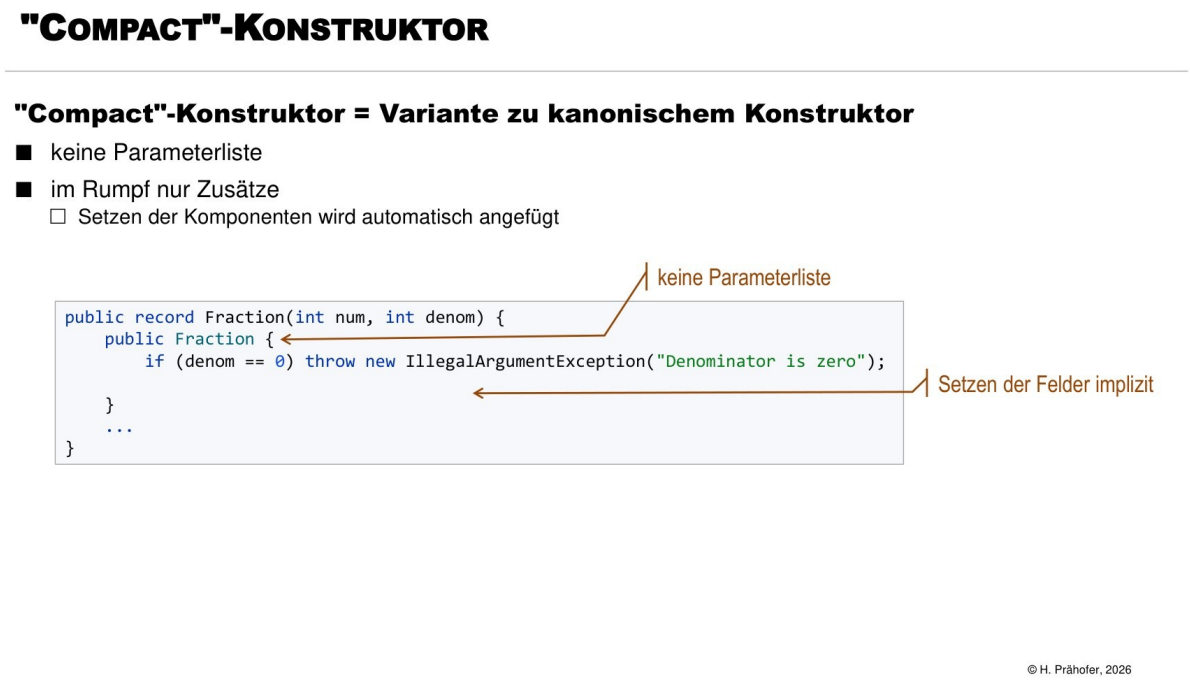
Die Plus- und Minusliste grenzt Records gegenüber regulären Klassen ab. Zusätzliche Instanzfelder sind verboten, weil sonst die transparente, im Kopf definierte Datenträgerstruktur unvollständig wäre. Eigene Methoden, statische Elemente und implementierte Interfaces sind erlaubt. Ein weiterer Konstruktor muss mit `this(...)` zu einem anderen Konstruktor desselben Records delegieren; dessen Kette endet beim kanonischen Konstruktor.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 6.

S · Folie 7: "COMPACT"-KONSTRUKTOR "Compact"-Konstruktor = Variante zu kanonischem Konstrukto

Im kompakten Konstruktor `Fraction` steht nach dem Namen keine Parameterliste. Der Rumpf validiert den Nenner und meldet bei `null` eine `IllegalArgumentException`. Erst wenn der Rumpf erfolgreich endet, werden die endgültigen Felder automatisch mit den eventuell zuvor normalisierten Komponentenparametern initialisiert. Dieser Mechanismus vermeidet doppelt geschriebene Zuweisungen.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 7.

S · Folie 8: BEISPIEL: RECORD-KLASSE FRACTION

Die Folie führt einen expliziten kanonischen Konstruktor, einen überladenen Einparameterkonstruktor und zwei arithmetische Methoden vor. mult multipliziert Zähler mit Zähler und Nenner mit Nenner. Für add muss der Ergebnisnenner denom * `f.denom` lauten; das dort stehende n ist undefiniert und daher ein Compilerfehler. Das Ergebnis eines Methodenaufrufs ist ein neues `Record`-Objekt: Die ursprünglichen Brüche ändern sich nicht.

Beispiel: Record-Klasse Fraction

```java
public record Fraction(int num, int denom) {

    public Fraction(int num, int denom) {
        if (denom == 0) throw new IllegalArgumentException("Denominator is zero");
        this.num = num;
        this.denom = denom;
    }

    public Fraction(int n) {
        this(n, 1);
    }

    public Fraction mult (Fraction f) {
        return new Fraction(num * f.num, denom * f.denom);
    }
    public Fraction add (Fraction f) {
        return new Fraction(num * f.denom + f.num * denom, n * f.denom);
    }

    @Override
    public String toString() {
        return String.format("%d/%d", num, denom);
    }
}
```


© H. Prähofer, 2026

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 8.

Korrigiertes und validierendes Java-Beispiel

Die folgende Variante zeigt einen kompakt validierenden Konstruktor und eine korrekte Bruchaddition. Sie ist eine zusätzliche didaktische Implementierung; die Originalfolie verwendet einen expliziten kanonischen Konstruktor und enthält in add einen Schreibfehler.

```
public record Fraction(int num, int denom) {
 public Fraction {
 if (denom == 0) throw new IllegalArgumentException("Nenner 0");
 }
 public Fraction(int n) { this(n, 1); }
 public Fraction add(Fraction other) {
 return new Fraction(
 num * other.denom + other.num * denom,
 denom * other.denom
 );
 }
 public Fraction mult(Fraction other) {
 return new Fraction(num * other.num, denom * other.denom);
 }
}
```

Beispielrechnung: `new Fraction(1, 2).add(new Fraction(1, 3))` ergibt den `Record Fraction`[num=5, denom=6]. Die Rückgabe wird nicht gekürzt; für 2/4 gegenüber 1/2 entscheidet die unveränderte `Record`-Gleichheit anhand der Komponenten und daher auf ungleich.

4.2 Enumerationsklassen

Eine Enum-`Klasse (enum)` definiert einen eigenen Referenztyp mit einer vorgegebenen, endlichen Menge benannter Konstanten. `enum Month { JAN, FEB, ... }` erzeugt für jeden Namen eine Instanz dieses Typs. Programmcode benutzt diese vorhandenen Instanzen wie `Month.JUL`, darf aber nicht beliebige neue Month-Objekte mit `new Month()` erzeugen. Damit kann ein Parameter ausschließlich einen der erlaubten

Monatswerte oder, wie jeder Referenztyp, `null` annehmen. Im Vergleich zu frei gewählten Zeichenketten entdeckt der Compiler viele ungültige Werte früh.

Jede Konstante ist ein eindeutig identifizierbares Objekt; Vergleiche mit `==` und `!=` sind daher für Enumwerte passend. Die vom Compiler bereitgestellte Methode `values()` liefert ein neues Array aller Konstanten in Deklarationsreihenfolge. `valueOf(String)` sucht nach dem exakten Konstantennamen und wirft bei unbekannten Namen eine `IllegalArgumentException`. `name()` gibt den konstanten Bezeichner zurück; das standardmäßig geerbte `toString()` liefert zunächst denselben Text, darf aber überschrieben werden. `ordinal()` zählt von `null` an und beschreibt die aktuelle Position. Er ist nicht als dauerhafter Geschäftscode, Datenbankschlüssel oder Speicherformat geeignet, weil das Einfügen oder Umordnen von Konstanten die Zahlen verändert.

Enums sind spezielle Klassen, keine bloßen Zahlenkonstanten. Nach der Konstantenliste und einem Semikolon lassen sich Felder, Konstruktoren und Methoden definieren. Bei `JAN(31)` ruft die Laufzeit während der Initialisierung den nur innerhalb der Enumklasse zugänglichen Konstruktor mit dem Wert 31 auf. Das Feld `days` kann `private final` sein; dann steht für jede Konstante die angegebene Zahl fest. Die Folie vereinfacht die Länge des Februars auf 28 Tage: Ein korrektes Kalendermodell berücksichtigt Schaltjahre und sollte gegebenenfalls `java.time.Month` beziehungsweise `YearMonth` verwenden.

Die Enumklasse erbt von `java.lang.Enum<E>` und implementiert dadurch bereits `Comparable<E>` und `Serializable`. Die Vergleichsreihenfolge von `compareTo` entspricht der Deklarationsreihenfolge. Man darf deshalb weder eine andere Superklasse angeben noch `equals`, `hashCode` oder `compareTo` für eine alternative Gleichheits- oder Sortierordnung überschreiben. Enums können eigene Interfaces implementieren und weitere Methoden definieren oder überschreiben, soweit dies zulässig ist. Eigene Konstruktoren dürfen nicht `public` oder `protected` sein: `private` ist erlaubt; ohne Sichtbarkeitsmodifier gelten sie in Enumklassen ebenfalls als `private`.

Eine `enum`-Deklaration definiert eine endliche Menge benannter Instanzen derselben Enum-Klasse. `Month.JAN` bezeichnet genau die deklarierte Instanz. `values()` liefert ein neues Array der Enumwerte in Deklarationsreihenfolge, `valueOf("JAN")` sucht die Instanz mit genau diesem Namen, und `ordinal()` liefert die nullbasierte Position. Die Zahl von `ordinal()` eignet sich im Allgemeinen nicht als dauerhaft gespeicherter fachlicher Code: Eine Umordnung der Enumkonstanten würde ihren Wert ändern.

Enums können eigene finale oder veränderliche Felder, Methoden und `private` Konstruktoren besitzen. Die Konstanten werden bei der Initialisierung der Enum-Klasse erzeugt; normale Benutzerprogramme dürfen keine weiteren Instanzen per `new Month(...)` erzeugen. Der Vergleich mit `==` ist bei Enumwerten sinnvoll, weil jede Konstante eine eindeutige Instanz ist. Ein Monat mit fester Tageszahl ist als allgemeines Kalenderobjekt unzureichend, weil Februar vom Jahr abhängt; für vollständige Kalenderregeln ist `java.time` geeignet.

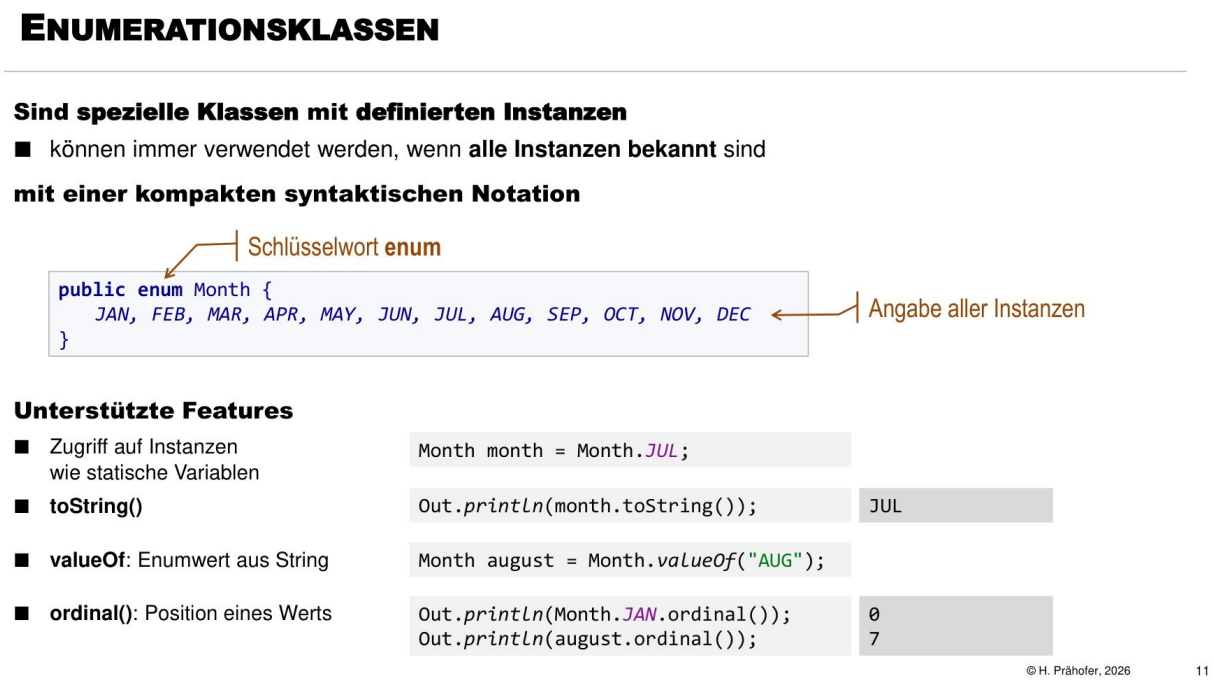
Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
enum Status {
 NEW, ACTIVE, CLOSED
}
Status s = Status.ACTIVE;
System.out.println(s == Status.ACTIVE); // true
System.out.println(s.ordinal()); // 1
```

Originalfolien und ausführliche Einordnung

S · Folie 11: ENUMERATIONSKLASSEN Sind spezielle Klassen mit definierten Instanzen

`Month.JUL` bezeichnet eine der festen Enum-Instanzen. valueOf akzeptiert einen exakt passenden Namen wie AUG; Kleinbuchstaben oder unbekannte Werte lösen eine `Exception` aus. ordinal gibt die nullbasierte Deklarationsposition zurück, daher JAN = 0 und AUG = 7. Für dauerhafte Codes besser eigene stabile Werte verwenden.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 11.

S · Folie 12: ENUMERATIONSKLASSEN Unterstützte Features (Fortsetzung)

Das Diagramm und die Beispiele zeigen das vollständige Wertearray, Identitätsvergleiche mit ==, den klassischen `switch` mit break und die natürliche Reihenfolge durch `Comparable`. Weil jede Konstante nur einmal existiert, ist == bei zwei Month-Referenzen semantisch sinnvoll. In einem modernen `switch`-Ausdruck können stattdessen Pfeilfälle verwendet werden.

ENUMERATIONSKLASSEN

Unterstützte Features (Fortsetzung)

- **values()**: Array mit allen Werten

```
for (Month m : Month.values()) {
  Out.println(m.toString());
}
```

```
JAN
FEB
...
```

- Vergleiche == und !=

```
if (month == Month.JAN) {
  Out.println(" Exams are coming :-( ");
} else if (month == Month.JUL) {
  Out.println("I am on holiday :-) ");
}
```

```
I am on holiday :-)
```

- Verwendung von **switch**

```
switch (month) {
  case JAN:
    Out.println("Exams are coming :-(");
    break;
  case JUL:
    Out.println("I am on holiday :-)");
    break;
  ...
}
```

```
I am on holiday :-)
```

- implementiert **Comparable** mit Reihenfolge wie Angabe der Werte

© H. Prähofer, 2026 12

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 12.

S · Folie 13: ENUMERATIONSKLASSEN Enumerationsklassen können haben

Den Monatskonstanten werden beim Erzeugen konkrete Konstruktorargumente übergeben. Das Attribut days ist instanzbezogen, obwohl alle Instanzen einmalig definiert sind. Der Konstruktor ist privat, sodass nur die Deklaration der Konstanten Instanzen erzeugen darf. Der Februarwert 28 ist für Schaltjahre unvollständig; methodisch wäre etwa `getDays(int year)` nötig.

ENUMERATIONSKLASSEN

Enumerationsklassen können haben

- Felder
- **privaten** Konstruktor
- Methoden

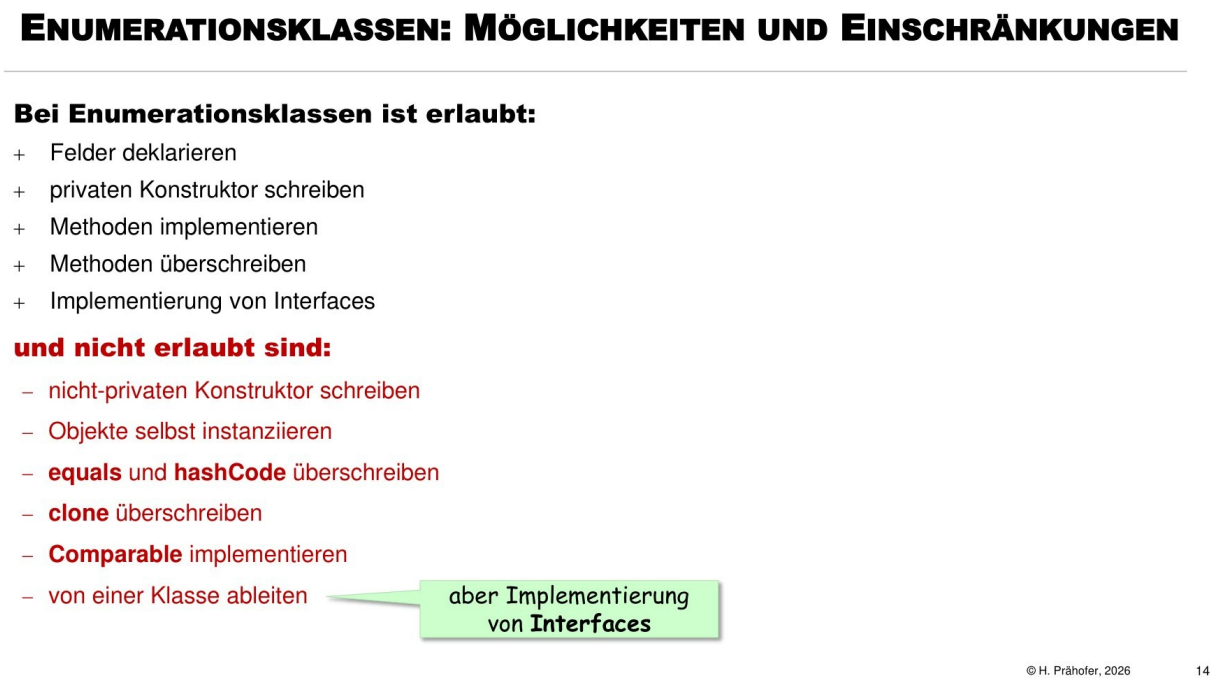
Beispiel: Month mit Anzahl der Tage

© H. Prähofer, 2026 13

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 13.

S · Folie 14: ENUMERATIONSKLASSEN: MÖGLICHKEITEN UND EINSCHRÄNKUNGEN Bei Enumerationsklassen i

Die Liste nennt zusätzliche Methoden und implementierte Interfaces als Möglichkeiten, verbietet aber eigene Instanziierung und Vererbung von einer anderen Klasse. Enum überschreibt equals und hashCode nicht: Identität der Konstanten ist festgelegt. Präzisierung: Ein Enumkonstruktor darf `private` heißen oder ohne Modifier deklariert werden, wodurch er ebenfalls privat ist; das pauschale Verbot „nicht-privater“ Konstruktoren ist im Ergebnis richtig, aber die Syntax ohne `private` ist erlaubt.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 14.

Verhalten von Enum-Konstanten überprüfen

```
Month m = Month.JUL;
System.out.println(m == Month.JUL); // true
System.out.println(Month.AUG.ordinal()); // 7
for (Month each : Month.values()) {
 System.out.println(each.name());
}
// Month.valueOf("jul") -> IllegalArgumentException
```

4.3 Geschachtelte, lokale und anonyme Klassen

In Java gibt es neben den direkt in einem Package deklarierten Top-Level-Typen auch Typen, deren Deklaration in einer anderen Klasse, einem Interface oder einem Block steht. Als Oberbegriff ist „geschachtelte Klassen“ (nested classes) präziser als „innere Klassen“: Im strengen Sprachgebrauch bezeichnet inner `class` nur eine nicht-statische Member-Klasse, eine nicht-statische lokale Klasse oder eine anonyme Klasse. Eine `static` geschachtelte Klasse besitzt gerade keine automatisch gebundene äußere Objektinstanz. Die Folien verwenden „innere Klassen“ gelegentlich für alle Formen, was hier kenntlich gemacht wird.

Bei einer statischen Member-Klasse `Outer.Node` ist der Typ zwar dem Namensraum von `Outer` zugeordnet, ein `Node`-Objekt kann aber unabhängig von einem Outer-Objekt existieren. `Node` kann auf `private` statische Mitglieder von `Outer` zugreifen; für den Zugriff auf einzelne Objektfelder benötigt es wie jede andere Klasse

eine explizite Referenz auf ein Outer-Objekt. Diese Form eignet sich besonders für interne Hilfstypen ohne Bedarf an einem äußeren Objekt, zum Beispiel Knotendaten einer verketteten Liste. Die Zugriffsmodifizierer einer geschachtelten Member-Klasse können `private`, `package-private`, `protected` oder `public` sein; für Top-Level-Klassen stehen dagegen nur `public` und `package-private` zur Verfügung.

Eine nicht-statische Member-Klasse ist an ein konkretes Objekt der äußeren Klasse gebunden. `Outer.Inner inner = outer.new Inner()` erzeugt ein neues inneres Objekt mit der äußeren Instanz `outer`. Innerhalb einer Instanzmethode von `Outer` reicht häufig `new Inner()`: das aktuelle `Outer.this` ist implizit gegeben. Ein Objekt von Inner enthält konzeptionell einen Verweis auf seine äußere Instanz und kann dadurch sogar `private` Instanzfelder des äußeren Objekts lesen oder verändern. Zwei unterschiedliche Inner-Objekte können denselben äußeren Zustand teilen. Man sollte diese enge Kopplung gezielt einsetzen, denn die äußere Instanz kann dadurch länger im Speicher gehalten werden.

Eine lokale Klasse steht innerhalb eines Blocks, beispielsweise einer Methode, und besitzt dort einen begrenzten `Geltungsbereich (Scope)`. Sie kann auf geeignete lokale Variablen und Parameter der umschließenden Methode zugreifen, jedoch nur, soweit diese `final` oder effectively `final` sind: Es findet nach der ersten Zuweisung keine weitere Zuweisung an die lokale Variable statt. Eine anonyme Klasse hat überhaupt keinen selbst gewählten Klassennamen und wird zusammen mit der Erzeugung einer einzelnen Instanz deklariert: `new Iterator<Object>() { ... }`. So lässt sich ein kleiner `Iterator` oder Adapter direkt an der Stelle erstellen, an der er gebraucht wird. Eine anonyme Klasse kann eine Klasse erweitern oder ein Interface implementieren; ausdrücklich benannte Konstruktoren sind bei ihr nicht möglich, aber Initialisierungsblöcke und Instanzfelder sind erlaubt.

Bei verschachtelten `this`-Bezügen meint `this` immer die aktuelle, innerste Objektinstanz. Die Schreibweise `Outer.this` bezeichnet die zugehörige äußere Objektinstanz. In einer statischen Member-Klasse existiert keine solche implizite Instanz. Ist ein Name sowohl im inneren als auch im äußeren Objekt deklariert, muss man entsprechend qualifizieren, um Verwechslungen zu vermeiden. Für `private`, `package-private` und geschützte Member von geschachtelten Klassen gelten außerdem die normalen Java-Zugriffsregeln.

Für `static`-Deklarationen ist das verwendete Java-Sprachniveau entscheidend: Seit Java 16 dürfen auch innere Klassen statische Member und statische Initialisierungsblöcke enthalten. Eine lokale oder anonyme Klassendeklaration selbst kann weiterhin nicht als `static` deklariert werden; die pauschale Einschränkung der Folie, diese Klassen dürften keine statischen Felder oder Methoden haben, ist für moderne Java-Versionen nicht mehr richtig. Lokale Records und lokale Enumklassen sind implizit statisch und bilden eine eigene Sonderform.

Der Oberbegriff nested `class` umfasst statische geschachtelte Klassen und echte innere Klassen. Eine `static class Node` besitzt keine implizite Referenz auf ein Objekt der äußeren Klasse. Eine nicht statische Member-Klasse besitzt dagegen eine Verbindung zu einer konkreten äußeren Instanz. Wird sie von außen erzeugt, zeigt die Syntax `outer.new Inner()` ausdrücklich diese Zuordnung. Im Inneren steht `Outer.this` für die umgebende Instanz; `this` allein für das innere Objekt.

Eine lokale Klasse wird innerhalb eines Blocks deklariert und ist nur dort mit ihrem Namen sichtbar. Eine anonyme Klasse wird direkt als Ausdruck `new Interface() { ... }` definiert; sie hat keinen eigenen einfachen Klassennamen. Beide können auf lokale Variablen der umgebenden Methode zugreifen, wenn

diese `final` oder effectively `final` sind, also nach ihrer Initialisierung nicht mehr neu zugewiesen werden. Über eine finale Arrayreferenz kann dagegen der veränderbare Arrayinhalt weiterhin geändert werden.

Statische Member geschachtelter Klassen unterliegen aktuellen Java-Regeln. Ältere Lehrdarstellungen, die statische Mitglieder jeder inneren Klasse grundsätzlich verbieten, sind seit Java 16 zu pauschal. Die Kategorien und Anwendungsgründe bleiben wichtig: Eine Hilfsklasse ohne Bezug zu einem äußeren Objekt ist oft `static`; ein `Iterator` mit Zugriff auf eine konkrete `LinkedList` kann eine nicht statische Member-Klasse oder eine anonyme Klasse sein.

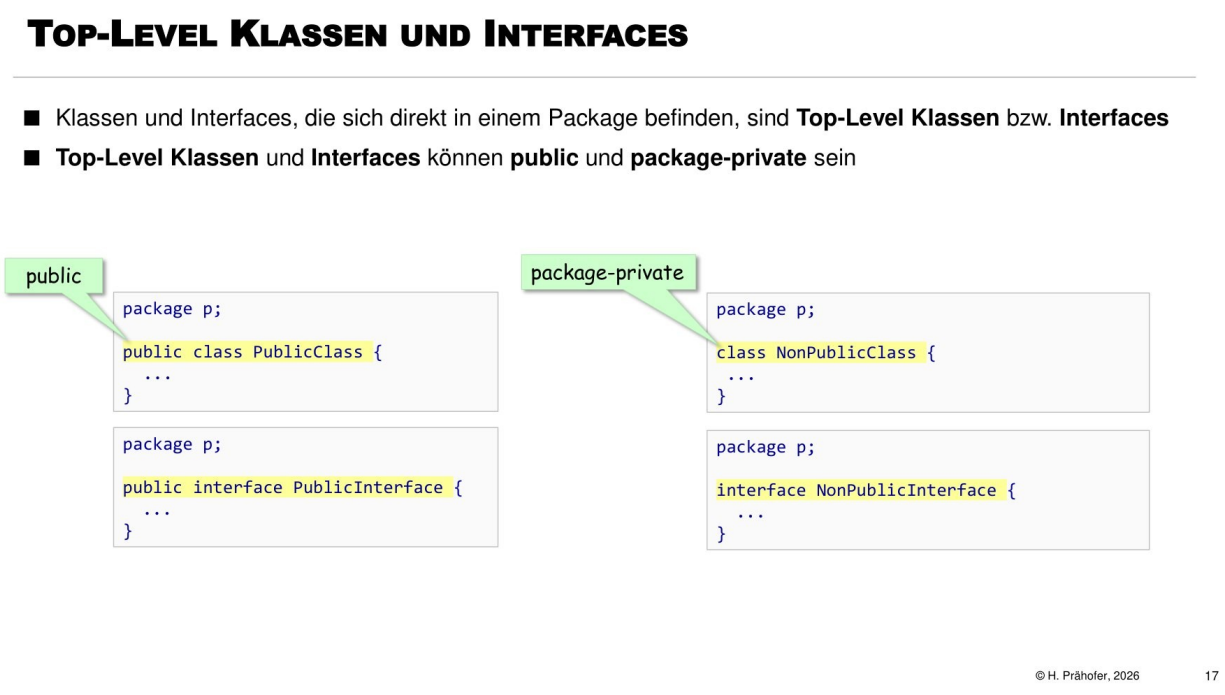
Originalfolien und ausführliche Einordnung

4.3.1 Top-Level-Typen und Arten der Verschachtelung

Eine Top-Level-Klasse steht unmittelbar innerhalb einer Compilation Unit und ihres Packages. Sie darf nicht `private` oder `protected` sein; diese Modifizierer setzen eine einschließende Klasse voraus. Eine Member-Klasse befindet sich hingegen im Klassenkörper und gehört dort zur strukturellen API oder Implementierung der äußeren Klasse.

S · Folie 17: TOP-LEVEL KLASSEN UND INTERFACES ✳ Klassen und Interfaces, die sich direkt in ein

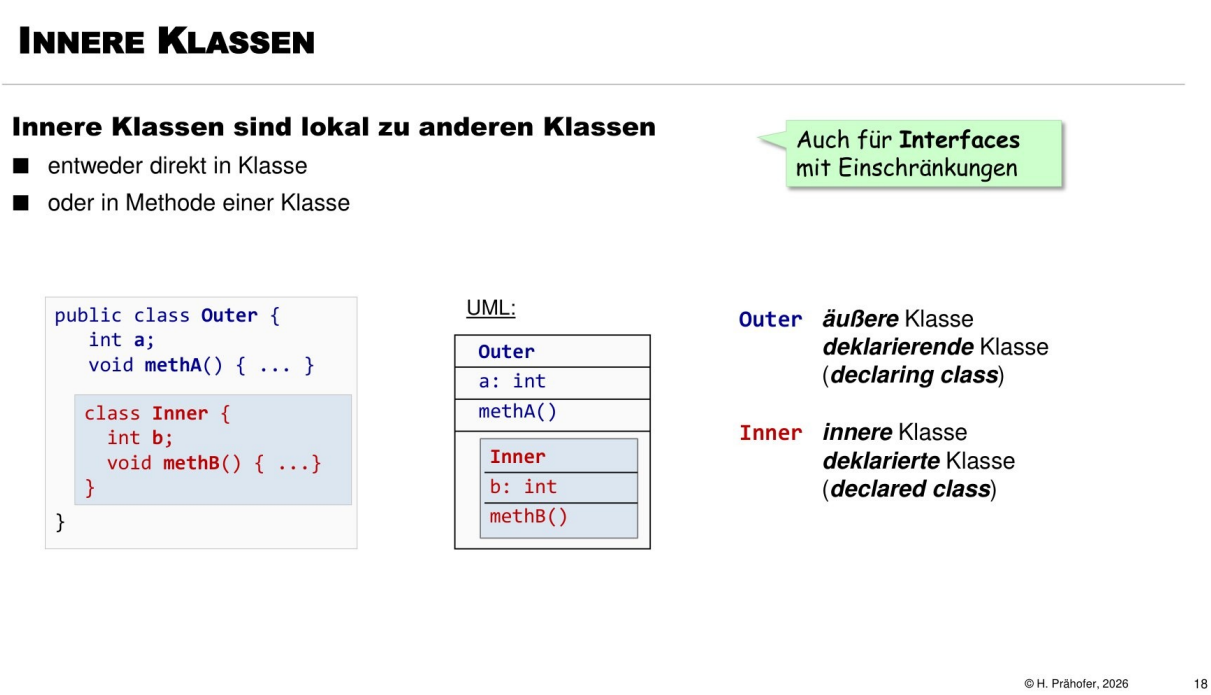
Das linke Beispiel zeigt einen öffentlich nutzbaren Top-Level-Typ, das rechte einen nur im Package sichtbaren Typ. Top-Level-Klassen und Interfaces sind nicht innerhalb einer anderen Typdefinition verschachtelt. Die Dateianordnung folgt der Package-Deklaration; mehrere Top-Level-Typen können in einer Datei stehen, aber pro Datei höchstens einer mit `public` und dann passend zum Dateinamen.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 17.

S · Folie 18: INNERE KLASSEN Innere Klassen sind lokal zu anderen Klassen

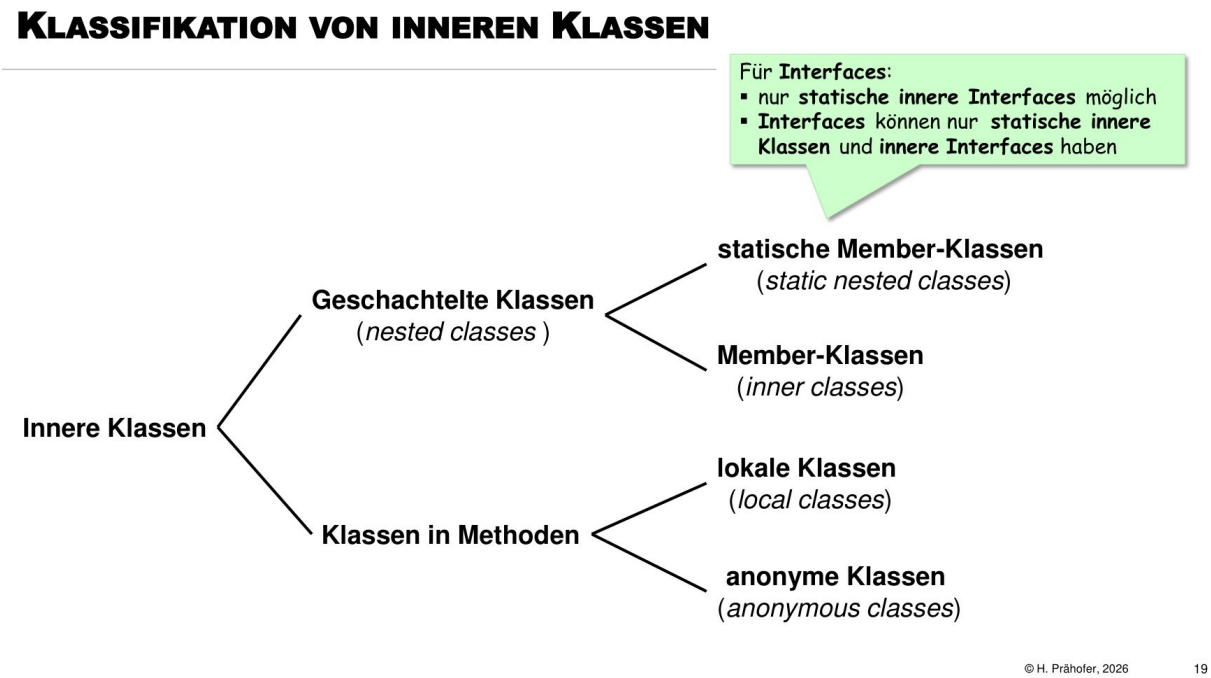
Die UML-Skizze ordnet Inner lexikalisch der äußeren Klasse Outer zu. Die lokale Klasse steht im Methodenrumpf methA und wird nur dort namentlich angesprochen. Der Bereich der Namenssichtbarkeit ist daher kleiner als bei einer Member-Klasse; eine solche lokale Klasse kann trotzdem zu einem langlebigeren Objekt führen, wenn ihre Instanz nach außen zurückgegeben wird.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 18.

S · Folie 19: KLASSIFIKATION VON INNEREN KLASSEN Innere Klassen

Die Baumgrafik unterscheidet geschachtelte Member-Typen und Typen, die in einem Block oder einem `new`-Ausdruck entstehen. Links wird „innere Klassen“ weit verwendet; strenggenommen sind `static` nested classes keine inner classes im Sinne der Java Language Specification. Die statischen Mitgliedsklassen erhalten keine implizite `Outer.this`-Referenz.

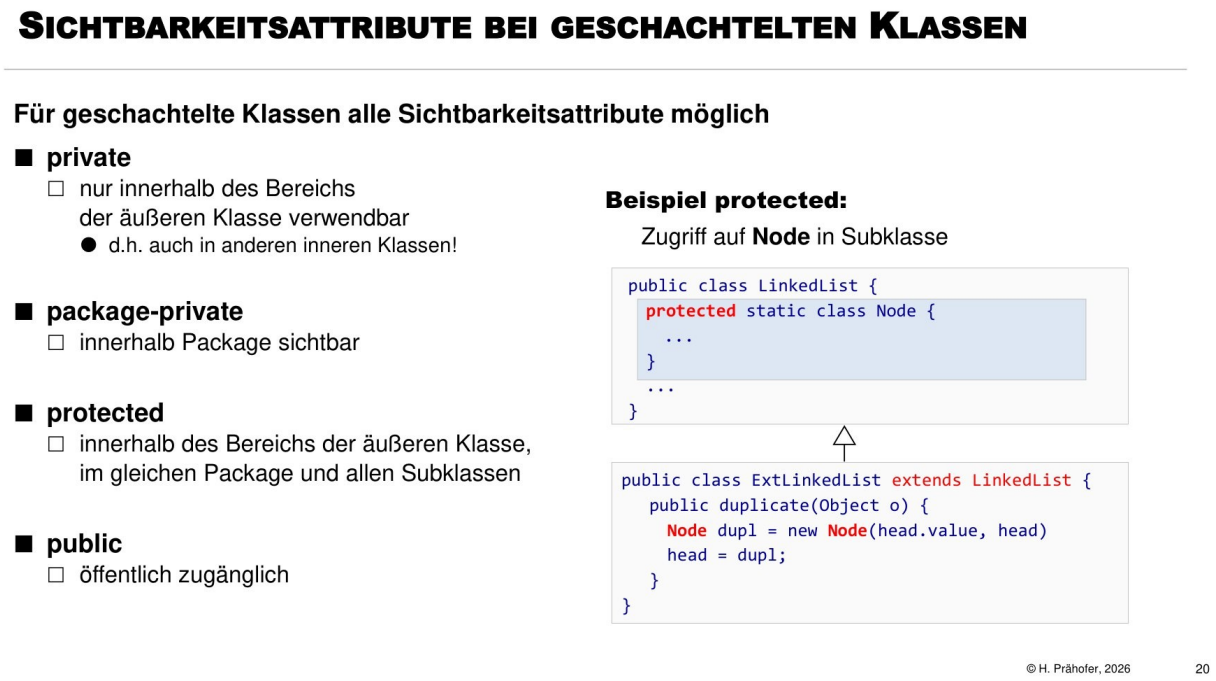


Originalabbildung: 04-Spezielle Klassen.pdf, Folie 19.

S · Folie 20: Für geschachtelte Klassen alle Sichtbarkeitsattribute möglich ⇝ private

Die vier Zugriffsstufen gelten für Member-Klassen genau wie für viele andere Member. Eine `private Node`-Klasse schützt die interne Verkettung vollständig vor direkter Benutzung von außen. Soll eine Unterklasse die `Node`-Klasse

sehen, kann `protected` nötig sein; bei Zugriffen aus fremden Packages gelten zusätzliche Zugriffsregeln für `protected`-Member.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 20.

4.3.2 Statische Member-Klassen

Die statische Knotenklasse ist ein Beispiel für Abstraktion über Sichtbarkeit. Die äußere `LinkedList` kennt `Node` intern und muss den Namen nicht als öffentliche Schnittstelle anbieten. Die `Node`-Instanzen sind dennoch ganz normale Heap-Objekte und werden unabhängig vom GC verwaltet. `static` bedeutet hier fehlende automatische äußere Instanzbindung, nicht dass nur ein einziges `Node`-Objekt existiere.

S · Folie 22: STATISCHE MEMBER-KLASSEN static deklariert

Die Syntax `static class` StaticMemberClass bindet die Typdeklaration an den Namensraum OuterClass, aber nicht an ein bestimmtes OuterClass-Objekt. Der vollqualifizierte Quellname ist etwa `p.OuterClass.StaticMemberClass`. Ohne explizite Outer-Referenz kann die nested `class` nicht direkt auf dessen einzelne Instanzfelder zugreifen.

Statische Member-Klassen

static deklariert

- damit analoge Eigenschaften wie andere **static** Members einer Klasse

```
class OuterClass {
   static class StaticMemberClass { ... }
}
```

selbstständige Klasse innerhalb des „Namensraums“ der deklarierenden Klasse

- ➔ vollqualifizierter Name der Klasse ist ***package.OuterClass.InnerClass***

```
package.OuterClass.StaticMemberClass
```

- haben Zugriff auf alle statischen Members der deklarierenden Klasse (auch private)
- auch statische innere Interfaces möglich

Anwendungen:

- um eine Klasse innerhalb der deklarierenden Klasse zu verbergen
- um eine Klasse oder ein Interface eng an eine äußere Klasse zu koppeln

© H. Prähofer, 2026 22

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 22.

S · Folie 23: BEISPIEL STATISCHE MEMBER-KLASSE Klasse Node als statische Member-Klasse, die zu

Die `Node`-Klasse ist in `LinkedList` privat, statisch und `final`. Ihre Objekte halten elem, next und prev, nicht aber automatisch einen Zeiger auf das jeweilige `LinkedList`-Objekt. Das erspart unnötige Verknüpfung mit der äußeren Instanz und ist für einfache Datenknoten ein passendes Design.

Beispiel statische Member-Klasse

Klasse **Node** als statische Member-Klasse, die zu **LinkedList** gehört

äußere Klasse

```
public final class LinkedList extends AbstractList {

    private static final class Node {
        private Object elem;
        private Node next;
        private Node prev;

        private Node(Object elem) {
            this.elem = elem;
        }
    }

    private Node head, tail;

    ...

}
```

statische Member-Klasse

© H. Prähofer, 2026 23

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 23.

S · Folie 24: INNERE KLASSEN IN INTERFACES: ANWENDUNGSBEISPIEL Konkrete Klassen die Interfaces

Die beiden Records `Rectangle` und `Circle` sind Member-Typen des Interfaces `Shape`. Sie sind implizit `public static`; sie implementieren jeweils `draw()` und können über den `Shape`-Typ polymorph verarbeitet werden. Bei

einem `Record` sind die Koordinaten und Maße die Komponenten, und die konkreten Zeichenoperationen liefern das Verhalten.

Innere Klassen in Interfaces: Anwendungsbeispiel

Konkrete Klassen die Interfaces implementieren

Beispiel: Shape mit Klassen Rectangle und Circle

```
public interface Shape {
  void draw();

  record Rectangle(int x, int y, int w, int h) implements Shape {
    public void draw() {
      Window.drawRectangle(x, y, w, h);
    }
  }

  record Circle(int x, int y, int r) implements Shape {
    public void draw() {
      Window.drawCircle(x, y, r);
    }
  }
}
```

interface **Shape**

innere Record-Klassen **Rectangle**

und **Circle**

Alle inneren Klassen in Interfaces sind implizit **public static**

© H. Prähofer, 2026 24

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 24.

S · Folie 25: INNERE KLASSEN IN INTERFACES: ANWENDUNGSBEISPIEL ⁂ Anwendung

Die erste Variante verwendet die verschachtelten Namen `Shape.Rectangle` und `Shape.Circle`. Die zweite Variante importiert die statischen Mitgliedstypen mit `import static shapes.Shape.*` und verkürzt die Schreibweise. Beide Varianten erzeugen dieselben Typen und Objekte; ein statischer Import ist keine neue Vererbung.

Innere Klassen in Interfaces: Anwendungsbeispiel

■ Anwendung

```
public class ShapeMain {
  public static void main(String[] args) {
    Shape.Rectangle r1 = new Shape.Rectangle(10, 10, 100, 200);
    r1.draw();

    Shape.Circle c1 = new Shape.Circle(10, 10, 30);
    c1.draw();
  }
}
```

■ mit **import static**

```
import static shapes.Shape.*;

public class ShapeMain {

  public static void main(String[] args) {
    Rectangle r1 = new Rectangle(10, 10, 100, 200);
    r1.draw();

    Circle c1 = new Circle(10, 10, 30);
    c1.draw();
  }
}
```

static import von **Shape.***

© H. Prähofer, 2026 25

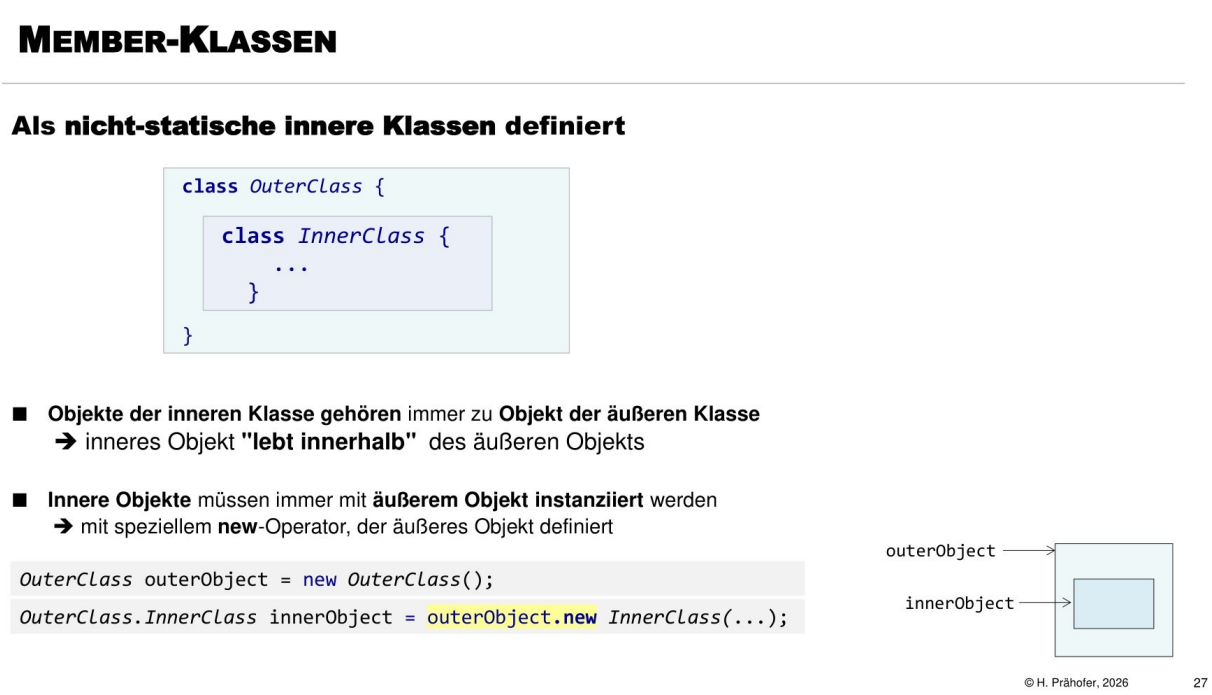
Originalabbildung: 04-Spezielle Klassen.pdf, Folie 25.

4.3.3 Nicht-statische Member-Klassen

Man darf den Ausdruck `Outer.Inner` nicht mit einer automatisch statischen Klassendeklaration verwechseln. Der Typ ist durch diese Schreibweise referenzierbar, aber seine Erzeugung benötigt eine äußere Instanz. Ein Beispiel mit zwei Outer-Objekten erzeugt dementsprechend zwei getrennte Zustandsräume; die zugeordneten Inner-Objekte greifen jeweils nur auf ihr eigenes Outer zu.

S · Folie 27: MEMBER-KLASSEN Als nicht-statische innere Klassen definiert

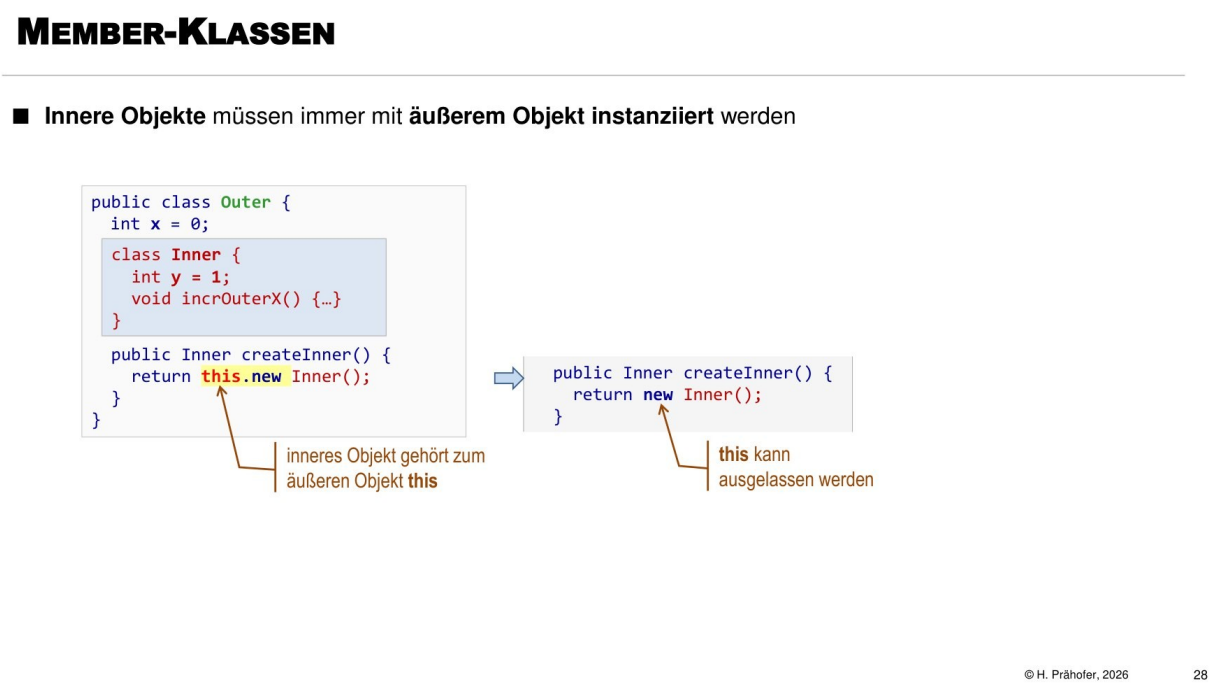
Die Deklaration `class` Inner im Rumpf von OuterClass ist nicht statisch. Ein inneres Objekt benötigt deshalb eine äußere Instanz, bei der Erzeugung außerhalb von Outer mit `outerObject.new Inner(...)`. Der Typname `OuterClass.InnerClass` sagt, zu welcher äußeren Klasse der innere Typ gehört, ist aber nicht selbst die Instanzbindung.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 27.

S · Folie 28: MEMBER-KLASSEN – Erzeugung im äußeren Objekt

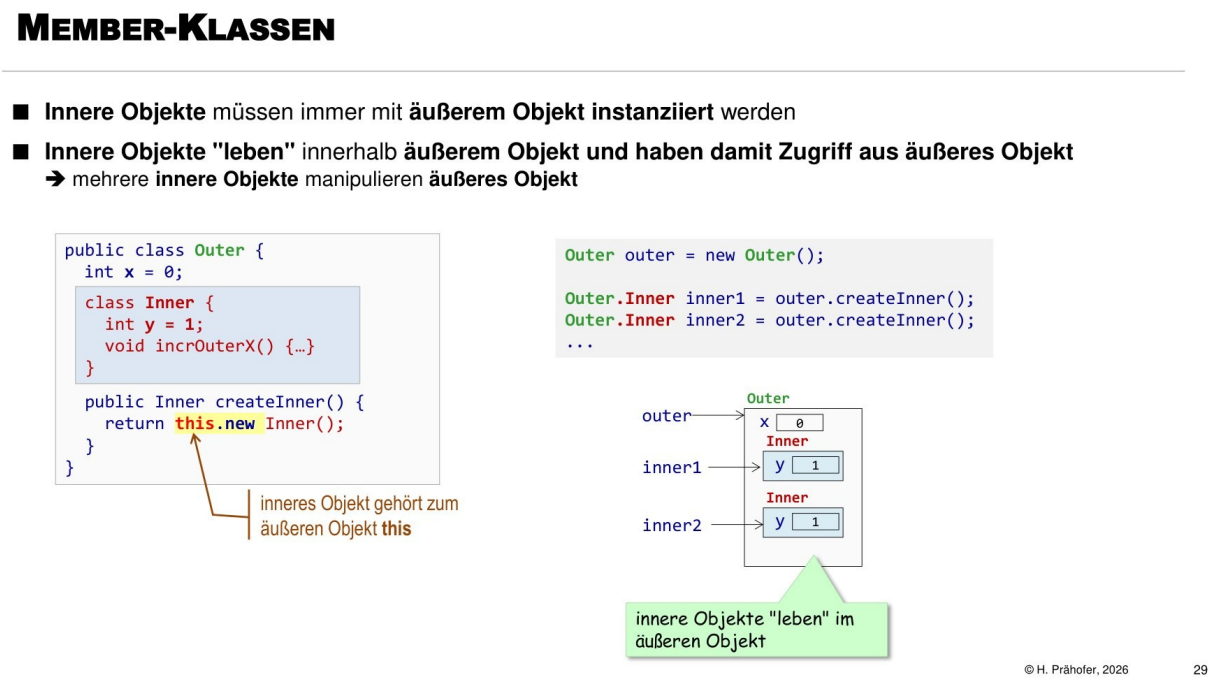
Im Rumpf einer Instanzmethode von Outer ist die umschließende Instanz bereits `this`. Deshalb sind `this.new Inner()` und `new Inner()` äquivalent, solange kein anderer gleichnamiger Typ die Auflösung verändert. Ein Aufruf aus einem statischen Kontext benötigt dagegen eine konkrete Outer-Referenz.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 28.

S · Folie 29: MEMBER-KLASSEN – mehrere innere Instanzen

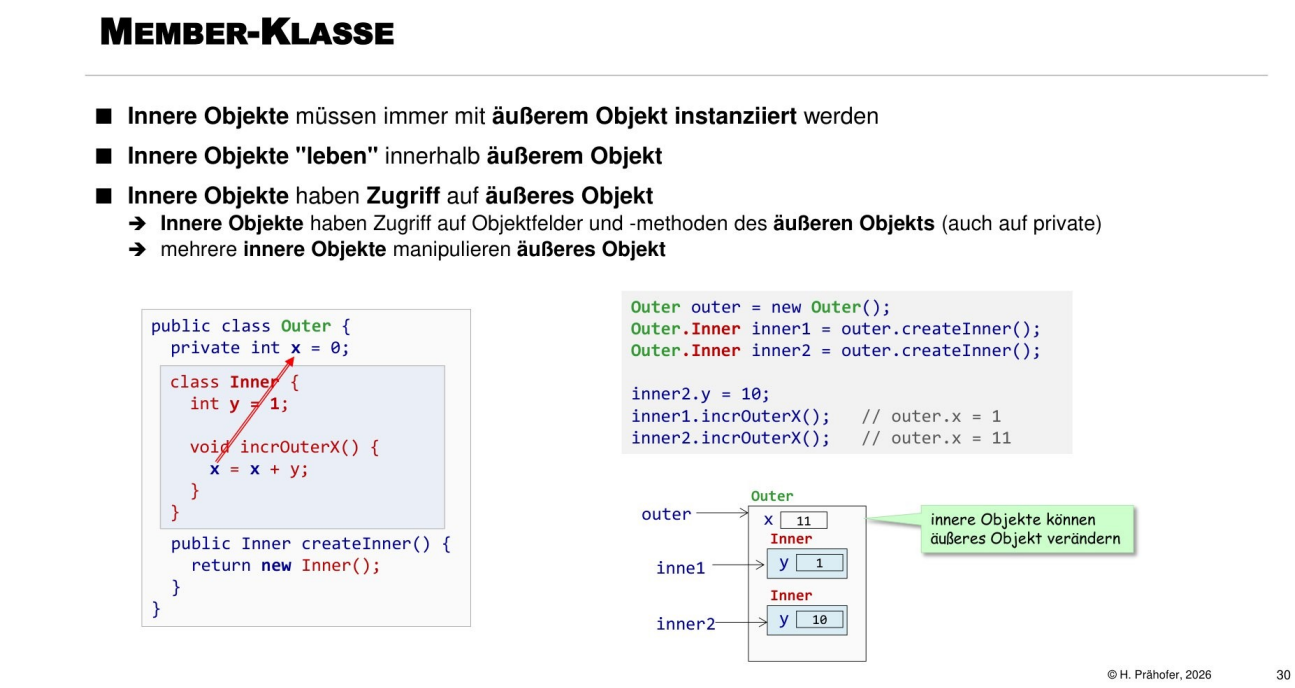
Das Objektdiagramm trennt äußeres Objekt outer von den beiden Inner-Instanzen inner1 und inner2. Die inneren Objekte besitzen jeweils ihr eigenes y-Feld, zeigen aber auf dieselbe äußere x-Variable. Die Abbildung beschreibt damit gemeinsame Zustandsänderung über verschiedene Objektwege.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 29.

S · Folie 30: MEMBER-KLASSE ⁂ Innere Objekte müssen immer mit äußerem Objekt instanziiert werde

Beim Aufruf `inner1.incrOuterX()` addiert die erste innere Instanz y = 1 auf das äußere x; inner2 addiert nach y = 10 weitere zehn. Ergebnis ist x = 11, obwohl die Klasse Outer von außen keinen öffentlichen Setter für x zeigt. Der Zugriff ist erlaubt, weil die innere Member-Klasse auf `private` Felder ihrer äußeren Klasse zugreifen darf.



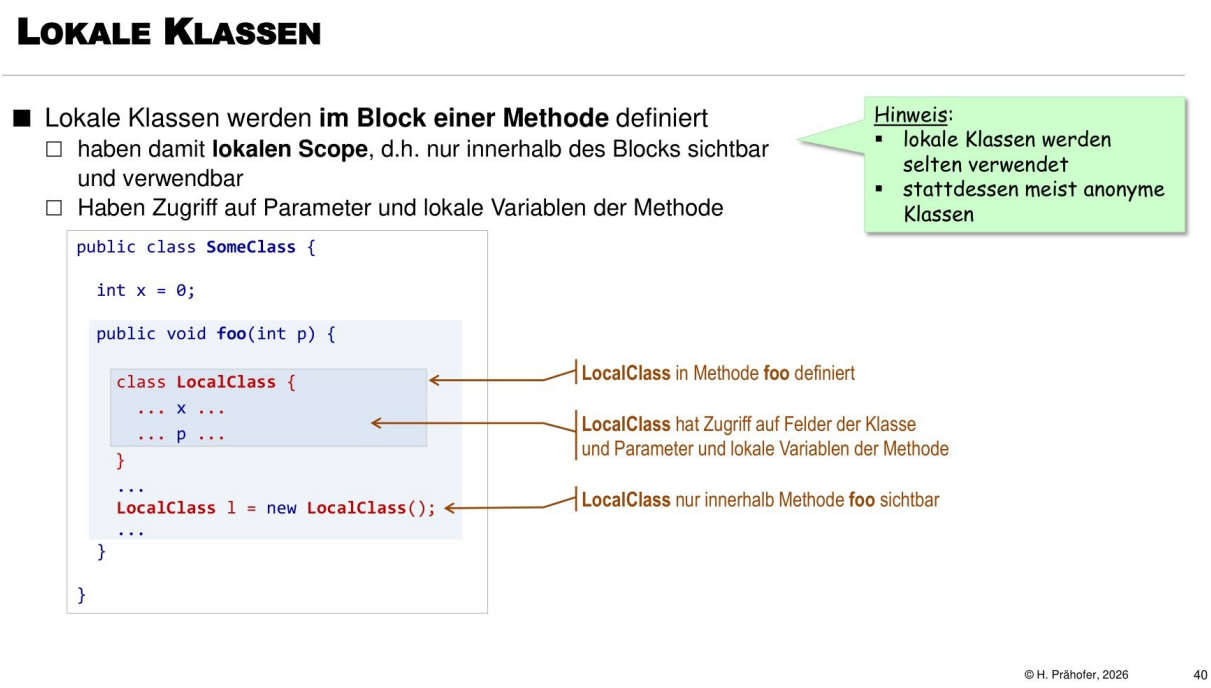
Originalabbildung: 04-Spezielle Klassen.pdf, Folie 30.

4.3.4 Lokale Klassen

Eine effektiv finale Variable muss nicht das Schlüsselwort `final` tragen. `int start = 0;` darf von einer lokalen Klasse gelesen werden, wenn start danach nicht neu zugewiesen wird. Dagegen ist schon `start++` eine erneute Zuweisung und zerstört die Bedingung. Die Erfassung einer Referenz kann den Lebenszyklus ihres Objekts verlängern, und ein zugreifendes Feld kann trotzdem veränderlich sein.

S · Folie 38: LOKALE KLASSEN ⇗ Lokale Klassen werden im Block einer Methode definiert

Eine lokale Klasse wird im Scope eines Blocks deklariert. Ihr Name ist außerhalb des Blocks unbekannt, aber Instanzen können nach außen gelangen. Die lokale Klasse darf auf das Feld x und auf den Parameter p zugreifen; p muss hierfür `final` oder effectively `final` bleiben. Ein bloßes Umkopieren einer lokalen Variablen ist keine Umgehung dieser Sprachregel.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 38.

S • Folie 39: BEISPIEL LOKALE KLASSE: LISTITERATOR public class LinkedList extends AbstractLis

Der ListIterator steht nun direkt im Rumpf der Methode `iterator()` und nicht als Member der ganzen Klasse. Dadurch wird seine Sichtbarkeit auf die Methode eingeschränkt. Der Algorithmus ist derselbe wie bei der Member-Klasse: head als Startreferenz, hasNext als Nulltest und next als schrittweises Folgen der Verkettung.

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 39.

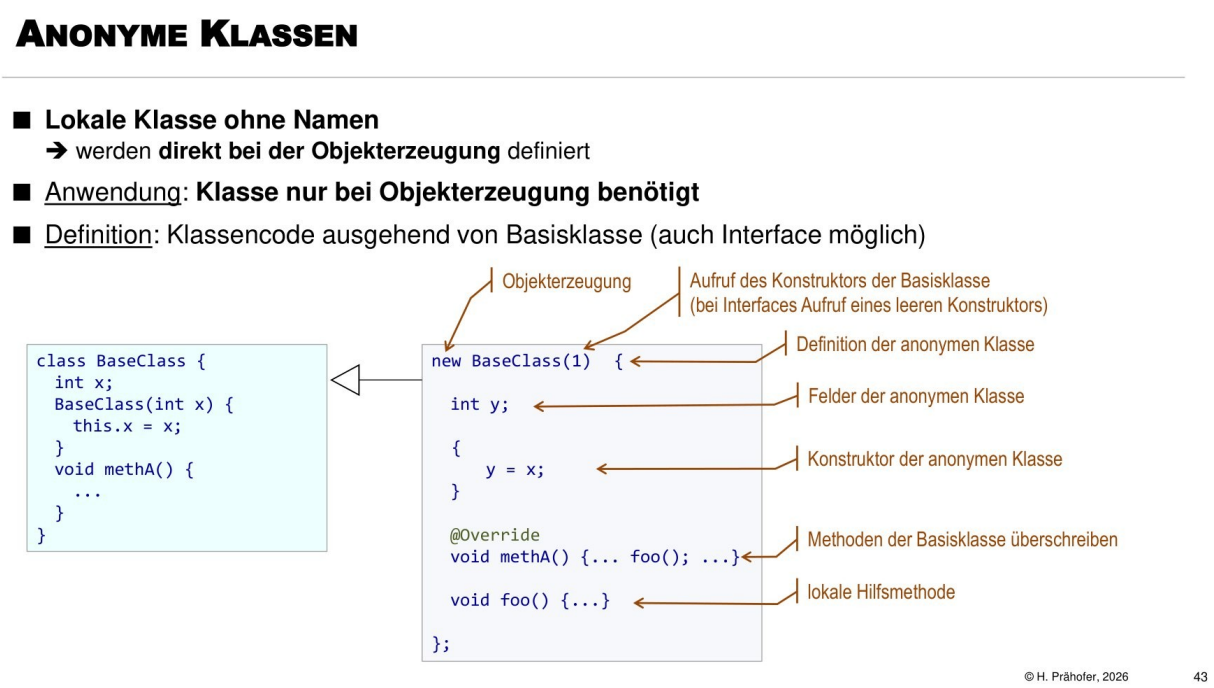
4.3.5 Anonyme Klassen und äußeres this

Eine anonyme Klasse ist kein Lambda-Ausdruck. Sie besitzt eine eigene Klassenidentität, kann Felder sowie mehrere Methoden implementieren und ist bei `this` auf ihr eigenes Objekt bezogen. Ein Lambda, das später

in der Lehrveranstaltung behandelt wird, weist bei `this` hingegen auf die umschließende Instanz und ist auf funktionale Interfaces beschränkt. Für komplexe Implementierungen ist eine benannte Klasse leichter lesbar.

S · Folie 41: ANONYME KLASSEN ⇗ Lokale Klasse ohne Namen

Die anonyme Klasse wird durch `new BaseClass(1) { … }` gleichzeitig deklariert und instanziiert. Der vorhandene Basisklassenkonstruktor wird mit 1 aufgerufen; ein eigener benannter Konstruktor für die anonyme Klasse ist nicht möglich. Der unbenannte Instanzinitialisierer mit y = x läuft bei der Erzeugung; neue Instanzfelder oder Hilfsmethoden dürfen im Klassenrumpf stehen.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 41.

S · Folie 42: BEISPIEL: ANONYME KLASSE FÜR ITERATOR IN LINKEDLIST public class LinkedList exte

Die anonyme `Iterator`-Implementierung spart den Namen ListIterator, behält aber das Feld next und die beiden Methoden bei. Weil die Deklaration innerhalb der Methode eines `LinkedList`-Objekts steht, kann der `Iterator` den äußeren Listenanfang head direkt verwenden. Der zurückgegebene Typ ist `Iterator<Object>`, nicht ein öffentlich benannter Spezialtyp.

Beispiel: Anonyme Klasse für Iterator in LinkedList

```
public class LinkedList extends AbstractList implements Iterable<Object> {
    protected Node head, tail;

    @Override
    public Iterator<Object> iterator() {

      return new Iterator<Object>() {
         Node next = head;

         @Override
         public boolean hasNext() { return next != null; }

         @Override
         public Object next() {
           if (! hasNext()) throw new NoSuchElementException("no next");
           Object ret = next.elem;
           next = next.next;
           return ret;
         }

       };

    }
    ...
}
```

Erzeugt anonyme Implementierung von **Iterator<Object>**

© H. Prähofer, 2026 44

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 42.

S · Folie 43: BEISPIEL: ANONYMER ITERATOR AUS PARAMETER ⇝ Statische Methode iteratorOf

Die statische Hilfsmethode iteratorOf nimmt ein Array und eine logische Größe entgegen; das zurückgegebene Objekt schließt diese Parameter als effectively-`final`-Werte ein. Das Feld nextIndex des anonymen Iterators ist hingegen veränderlich und dient als Cursor. Vor produktivem Einsatz müssen 0 <= size <= `array.length` und der Umgang mit späteren Arrayveränderungen geklärt werden.

Beispiel: Anonymer Iterator aus Parameter

- Statische Methode **iteratorOf** erzeugt Iterator für Parameter **array** und **size**

```
static Iterator<Object> iteratorOf(Object[] array, int size) {
  return new Iterator<>() {
    private int nextIndex;

    @Override
    public boolean hasNext() {
      return nextIndex < size;
    }

    @Override
    public Object next() {
      if (!hasNext()) throw new NoSuchElementException();
      return array[nextIndex++];
    }
  };
}
```

- **Array-Adapter** verwendet Methode **iteratorOf** mit Element-Array

```
public interface List implements Iterable<Object> {
 static IndexedList of(Object[] array) {
    return new IndexedList() {
      …
      @Override
      public Iterator<Object> iterator() {
        return iteratorOf(elems, elems.length);
      }
    …
```

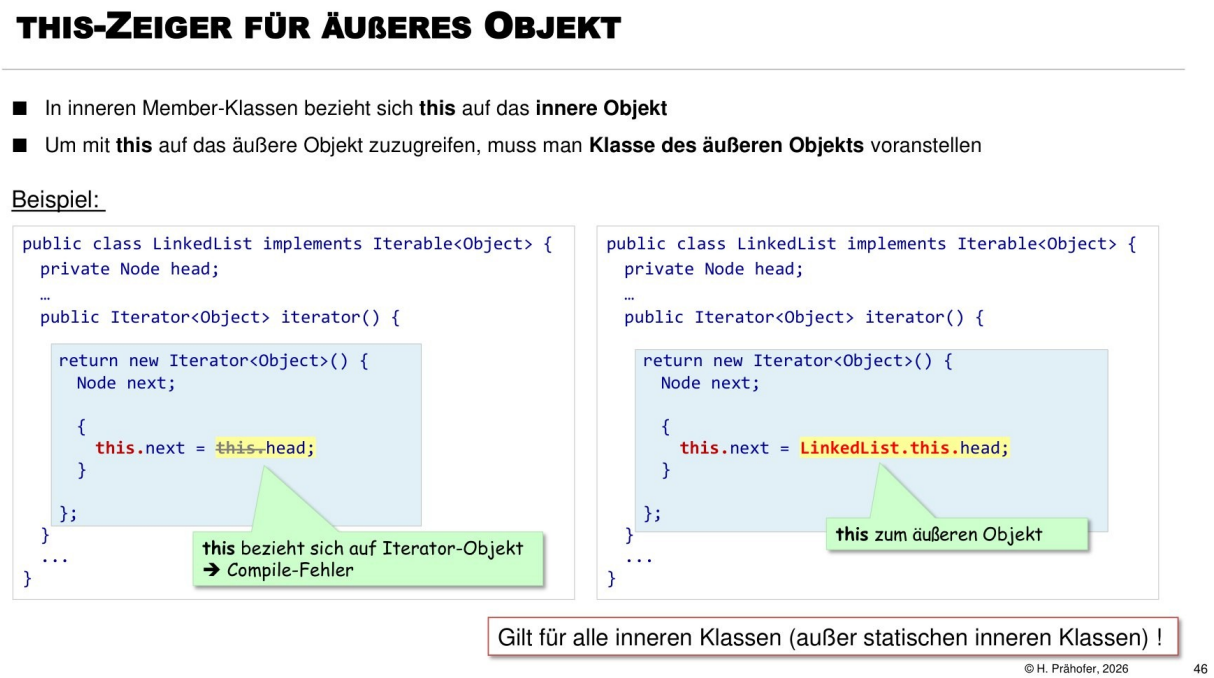
© H. Prähofer, 2026 45

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 43.

S · Folie 44: THIS-ZEIGER FÜR ÄUßERES OBJEKT ⇝ In inneren Member-Klassen bezieht sich this auf

Die linke Darstellung zeigt einen Namenskonflikt: `this.next` gehört zum inneren `Iterator`, während `this.head` ebenfalls im inneren Objekt gesucht würde und dort nicht existiert. Rechts adressiert `LinkedList.this.head`

ausdrücklich die äußere Instanz. Diese Qualifizierung hilft auch dann, wenn die innere Klasse zufällig ein gleichnamiges Feld hat.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 44.

4.3.6 Eingefangene Variablen und Sprachregeln

Der Grund für die effectively-`final`-Regel liegt darin, dass lokale Variablen bei Rückkehr aus einer Methode normalerweise nicht mehr existieren, während ein erfasstes Objekt weiterhin leben kann. Java behandelt die erfassten Werte deshalb nicht wie von außen jederzeit veränderbare lokale Stack-Plätze. Eine veränderliche Array- oder Objektkomponente umgeht nicht die Regel, sondern verlagert die Zustandsänderung in ein Objekt.

S · Folie 45: LOKALE VARIABLEN IN ANONYMEN KLASSEN ⁂ Anonyme Klassen können lokale Variablen nu

Die lokale Variable count darf im Methodenrumpf nicht nach ihrer Erfassung durch eine anonyme Klasse verändert werden. Insbesondere ist count = count + 1 innerhalb `run()` ein Compilerfehler, weil eine erfasste lokale Variable effektiv `final` bleiben muss. Das unterscheidet lokale Variablen vom veränderbaren Zustand eines Objekts.

Lokale Variablen in anonymen Klassen

- Anonyme Klassen können lokale Variablen nur verwenden, wenn diese **final** oder **effectively final** sind

effectively final =
ohne Modifier **final**
aber niemals verändert!

Beispiel: **Counter**

```
static void main() {

  int count = 0;

  Runnable counter = new Runnable() {
    @Override
    public void run() {
      count = count + 1;
    }
  };

  counter.run();
  counter.run();
  …

  System.out.println(count);
}
```

✗ **Runnable**-Objekt versucht lokale Variable zu setzen ➔ **Compilerfehler**

© H. Prähofer, 2026 47

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 45.

S · Folie 46: LOKALE VARIABLEN IN ANONYMEN KLASSEN ⁂ Anonyme Klassen können lokale Variablen nu

Das finale `int[]` refCount ist eine unveränderliche Referenz auf ein veränderbares Array. Der anonyme `Runnable` darf `refCount[0]++` ausführen, weil dabei nicht die lokale Variable refCount neu zugewiesen wird, sondern der Inhalt des referenzierten Objekts geändert wird. Zwei run-Aufrufe ergeben hier zwei Zählerinkremente.

Lokale Variablen in anonymen Klassen

- Anonyme Klassen können lokale Variablen nur verwenden, wenn diese **final** oder **effectively final** sind
- Lösung:
  - ☐ Heap-Objekt mit veränderlichem Feld
  - ☐ Referenz auf Objekt ist **final**
  - ☐ Geändert wird das veränderlichen Feldes des Objects

effectively final =
ohne Modifier **final**
aber niemals verändert!

```
static void main() {

  final int[] refCount = new int[1];

  Runnable counter = new Runnable() {
    @Override
    public void run() {
      refCount[0] = refCount[0] + 1;
    }
  };

 counter.run();
 counter.run();
 …
  System.out.println(refCount[0]);
}
```

Referenz auf Array-Objekt ist **final**

Setzen des Feldes des Array-Objekts

© H. Prähofer, 2026 48

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 46.

S · Folie 47: EINSCHRÄNKUNGEN BEI INNEREN KLASSEN UND INTERFACES Innere Interfaces

Die Folie sammelt Einschränkungen: verschachtelte Interfaces sind implizit statisch, und zusätzliche Methoden anonymer Klassen sind über eine bloße Obertyp-Referenz normalerweise nicht aufrufbar. Die Aussage, lokale und anonyme Klassen

könnten keine statischen Felder oder Methoden haben, gilt in modernen Java-Versionen ab 16 nicht mehr. Entscheidend ist, dass die lokale oder anonyme Klassendeklaration selbst nicht mit `static` modifiziert werden kann.

Einschränkungen bei Inneren Klassen und Interfaces

Innere Interfaces

- nur **statische innere Interfaces** möglich

Innere Klassen in Interfaces

- Interfaces können nur **statische innere Klassen** und **Interfaces** haben

Lokale und anonyme Klassen

- können keine **statischen Felder und Methoden** haben

Anonyme Klassen

- zusätzliche Methoden (nicht überschriebene) können nur intern verwendet werden

© H. Prähofer, 2026 49

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 47.

S · Folie 48: WANN VERWENDE ICH WELCHE INNERE KLASSE statische Member-Klasse

Die Entscheidungshilfe vergleicht Sichtbarkeit und Bindung: statische Member-Klassen sind ideal für reine Hilfstypen, nicht-statische Member-Klassen für eng gebundene Objekte, lokale Klassen für begrenzte Implementierungen im Methodenblock und anonyme Klassen für kurze einmalige Implementierungen. Zusätzliche Parameter, Wartbarkeit und Lebensdauer des äußeren Objekts spielen bei der Auswahl ebenfalls eine Rolle.

Wann verwende ich welche innere Klasse

statische Member-Klasse

- wenn ich eine Hilfsklasse für die innere Implementierung benötige
  ➔ dann **private**, **package-private** oder **protected**
- wenn man eine Klasse eng an eine äußere Klasse oder Interface koppeln will
  ➔ dann auch meist **public**

Member-Klasse

- wenn Objekte der inneren Klasse auf einem Objekt der äußeren Klasse arbeiten sollen
  ➔ wie z.B Iterators auf eine Liste
- und wenn man Objekte an mehreren stellen instanziieren muss
  oder die Klasse sehr umfangreich ist
  ➔ sonst anonyme Klasse

Anonyme Klasse

- wie Member-Klasse aber nur eine Stelle, wo man Objekt erzeugt
  und Klasse nicht zu umfangreich ist

Oft bevorzugt gegenüber Member-Klassen !

© H. Prähofer, 2026 50

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 48.

Statische und gebundene geschachtelte Klasse im Vergleich

```
class Outer {
 private int count;
 static class Node { int value; } // keine implizite Outer-Instanz
 class Inner { void inc() { Outer.this.count++; } }
 Inner create() { return new Inner(); }
}
Outer outer = new Outer();
Outer.Node node = new Outer.Node();
Outer.Inner inner = outer.new Inner();
```

Der Unterschied liegt nicht im Typnamen, sondern in der Objektbindung: Die statische `Node`-Klasse braucht kein Outer-Objekt, während Inner ohne äußere Instanz nicht erzeugt werden kann.

4.4 Fallstudie List: Iteratoren, Adapter und differenzierte Interfaces

Die Fallstudie greift die zuvor vorgestellten `List`-Entwürfe wieder auf. Ziel ist eine API, bei der die verwendenden Programme möglichst wenige Details der konkreten Speicherung kennen müssen. Eine verkettete Liste hält ihre Elemente in `Node`-Objekten, die über `next` und gegebenenfalls `prev` aufeinander verweisen. Eine Array-Liste speichert sie in einem Array mit einer logischen Größe `size`, die kleiner als die Array-Kapazität sein kann. Beide können durch das Interface `Iterable<Object>` einheitlich in einer for-each-Schleife verwendet werden, obwohl ihre Iteratoren intern völlig unterschiedlich arbeiten.

Ein `Iterator` bietet zwei zentrale Operationen: `hasNext()` sagt, ob noch ein Element verfügbar ist; `next()` liefert es und bewegt den internen Cursor genau einmal weiter. Der Cursor zeigt vor jedem Aufruf von `next()` auf das nächste auszugebende Element. Beim Ende muss `next()` eine `NoSuchElementException` werfen; es darf nicht stillschweigend `null` liefern, da `null` ein erlaubter Listeninhalt sein könnte. Wiederholte Aufrufe von `hasNext()` sollten das Ergebnis nicht konsumieren. Ein `LinkedList-Iterator` kann den nächsten `Node`-Verweis speichern; ein `ArrayList-Iterator` dagegen einen Index. Eine for-each-Schleife über ein `Iterable` ist im Grundprinzip nur eine kürzere Schreibweise für `Iterator<T> it = iterable.iterator(); while (it.hasNext()) { T x = it.next(); ... }`.

Die konkrete Wahl der Iteratorimplementierung ist für die Laufzeit entscheidend. Bei einer einfach verketteten Liste erfordert das Lesen des Elements an Position i vom Anfang aus im Allgemeinen `O(i)` Schritte. Eine Schleife, die für alle i von 0 bis n-1 wiederholt `get(i)` aufruft, benötigt daher insgesamt `O(1+2+…+n)` = $O(n^2)$ Zeit. Ein `Iterator`, der jeden `Node` genau einmal durchläuft, benötigt dagegen `O(n)`. Bei einer `ArrayList` kann der direkte Indexzugriff `get(i)` im Regelfall `O(1)` dauern, sodass auch die einfache Indexschleife insgesamt `O(n)` benötigt. Diese Abwägung motiviert die Trennung von lesendem sequenziellem Zugriff (`List`) und zusätzlichem wahlfreiem Zugriff (`IndexedList`).

Die Lehr-API trennt außerdem Lese- und Veränderungsbefugnisse. `List` stellt lesende Methoden und Iteration bereit; `MutableList` ergänzt etwa `clear()`, `add(...)` und `remove(...)`. `IndexedList` bietet den lesenden Indexzugriff `get(int)`, `MutableIndexedList` baut auf beiden Oberinterfaces auf und ergänzt indexbezogene Änderungen. Eine `LinkedList` kann sinnvollerweise nur `MutableList` implementieren und eine `ArrayList` zusätzlich `MutableIndexedList`. Damit können Methoden über ihre Parametertypen ausdrücken, welche Zugriffe sie tatsächlich benötigen. Diese Lehrschnittstellen unterscheiden sich bewusst von `java.util.List`, das sowohl get als auch mutierende Methoden kennt.

Adapter kapseln eine andere Datenstruktur und stellen sie unter einer neuen Schnittstelle bereit: Eine anonyme `List`-Implementierung kann beispielsweise lesend über ein bestehendes `Object[]` arbeiten und in `iterator()` einen weiteren anonymen `Iterator` erzeugen. Ein solcher Adapter kopiert das Array nicht automatisch. Änderungen über andere Referenzen können daher in der Sicht auftauchen. Ein Wrapper, der lediglich das lesende Interface implementiert und intern eine veränderbare Liste hält, verhindert den unmittelbaren Cast des Wrappers auf `MutableList`, macht die dahinterliegende Datenstruktur aber ebenfalls nicht ohne Weiteres tief unveränderlich. Wenn Änderungen durch andere Besitzer ausgeschlossen werden müssen, braucht man eine defensive Kopie oder eine tatsächlich unveränderliche Datensammlung.

Default-Methoden der Interfaces erlauben, häufige Operationen einheitlich auf Basismethoden aufzubauen, etwa `contains(o)` über `indexOf(o)` oder ein `addAll` über wiederholte `add`-Aufrufe. Dabei sind Randfälle explizit zu prüfen: `first()` einer leeren Liste, `remove(Object)` bei nicht gefundenem Element, Nullwerte in `indexOf`, Arraygrenzen oder ein `Iterator` während gleichzeitiger struktureller Veränderung. Die Vorlesungsfolien zeigen Schemata; für produktiven Code gehören eindeutige Verträge und Tests für solche Randfälle dazu.

Ein `Iterator` besitzt einen Cursor, der die Position des nächsten zurückzugebenden Elements beschreibt. Bei einer `LinkedList` ist das eine Knotenreferenz, bei einer `ArrayList` häufig ein Index. `hasNext()` prüft nur die Verfügbarkeit; `next()` liefert den aktuellen Wert und verschiebt den Cursor anschließend. Ein korrekter `Iterator` muss dabei auch für eine leere Liste funktionieren. Für eine Arrayliste mit internem Reserveplatz zählt die logische Anzahl `size`, nicht die physische Länge `elems.length`.

Eine Iteratorimplementierung kann ein Array oder eine Liste nur als Live-Sicht beobachten, sofern keine Kopie erzeugt wird. Wird die zugrundeliegende Struktur während des Durchlaufens geändert, hängt die Reaktion von ihrem Vertrag ab; die typische fail-fast-Prüfung der Standardbibliothek ist keine universelle `Thread`-Sicherheitsgarantie. Zur Vermeidung von `ConcurrentModificationException` darf ein `Iterator` nicht ohne dokumentierte Unterstützung durch fremde Strukturänderungen überrascht werden.

Die Unterscheidung `List`/`IndexedList` und `MutableList`/`MutableIndexedList` ist eine bewusste Interface-Aufteilung des Lehrentwurfs. Sie kodiert Fähigkeiten: Iteratorzugriff, indexbasierte Abfrage, Veränderbarkeit und indexbasierte Veränderung. Eine verkettete Liste kann dadurch auf kostspieligen Indexzugriff verzichten; eine Arrayliste stellt ihn bereit. Das lässt sich als konkrete Anwendung von Information Hiding und Interface-Segregation verstehen.

Originalfolien und ausführliche Einordnung

4.4.1 Das Iterator-Entwurfsmuster

Die Schnittstelle `java.util.Iterator<E>` ist absichtlich klein. `hasNext()` darf auch mehrfach hintereinander aufgerufen werden, ohne ein Element zu verbrauchen. `next()` hat einen Zustandsübergang. Diese Trennung ermöglicht lazy Traversal, bei der Elemente erst beim Abruf verarbeitet werden. Ein `Iterator` ist normalerweise nicht automatisch thread-sicher; Änderungen an der ursprünglichen Sammlung während des Durchlaufs können je nach Implementierung besondere Semantik haben.

S · Folie 31: ENTWURFSMUSTER: ITERATOR Zweck:

Die UML-Grafik des `Iterator`-Musters zeigt Client, abstrakten `Iterator`-Typ und konkrete Iteratoren für unterschiedliche Strukturen. Die Kante creates bedeutet, dass eine Struktur den zu ihrem Speicherlayout passenden

`Iterator` liefert. Der Client braucht nur hasNext und next; direkte Kenntnisse von Knotenzeigern oder Array-Indizes entfallen.

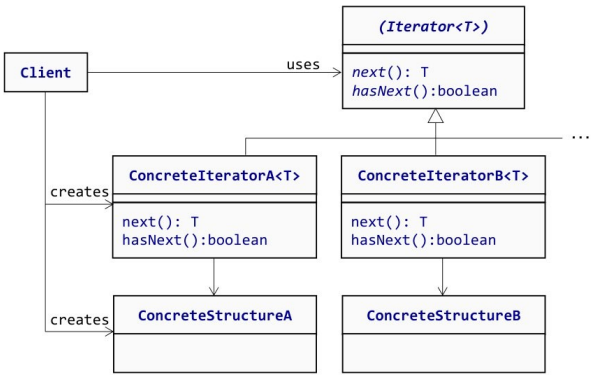
Entwurfsmuster: Iterator

Zweck:

- Auf Elemente einer Struktur sequentiell zugreifen, ohne das Innere der Struktur kennen zu müssen

Lösung:

- Abstrahiere vom konkreten Zugriff und stelle ihn über eine abstrakte Iterator-Schnittstelle bereit mit
  - ☐ Zugriff auf das nächste Element
  - ☐ Abfrage, ob nächstes Element existiert
- Implementiere Iteratoren für konkrete Strukturen



© H. Prähofer, 2026 31

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 31.

S • Folie 32: ENTWURFSMUSTER ITERATOR: INTERFACES ITERATOR<T> UND ITERABLE<T>

Iterator<T>: zu

`Iterator<T>` stellt hasNext und next als zentrale Operationen bereit. Die `default`-Methoden remove und forEachRemaining liefern zusätzliche Funktionalität, ohne jede Implementierung zu zwingen, sie selbst zu implementieren. `Iterable<T>` stellt `iterator()` bereit; daher kann der Compiler eine for-each-Schleife in passende Iteratoroperationen übersetzen.

Entwurfsmuster Iterator: Interfaces Iterator<T> und Iterable<T>

- **Iterator<T>**: zur Iteration über Collections

```
public interface Iterator<T> {

    boolean hasNext();

    T next();

    default void remove() {
        throw new UnsupportedOperationException("remove");
    }
    ...
}
```

Schleifenkonstruktion mit Iterator:

```
Iterator<Object> it = iterable.iterator();

while (it.hasNext()) {
  Object obj = it.next();
  ... o ...
}
```

- **Iterable<T>**: Interface für alle Typen die Iterator haben

```
public interface Iterable<T> {
    Iterator<T> iterator();
    ...
}
```

forEach-Schleife für alle Iterables:

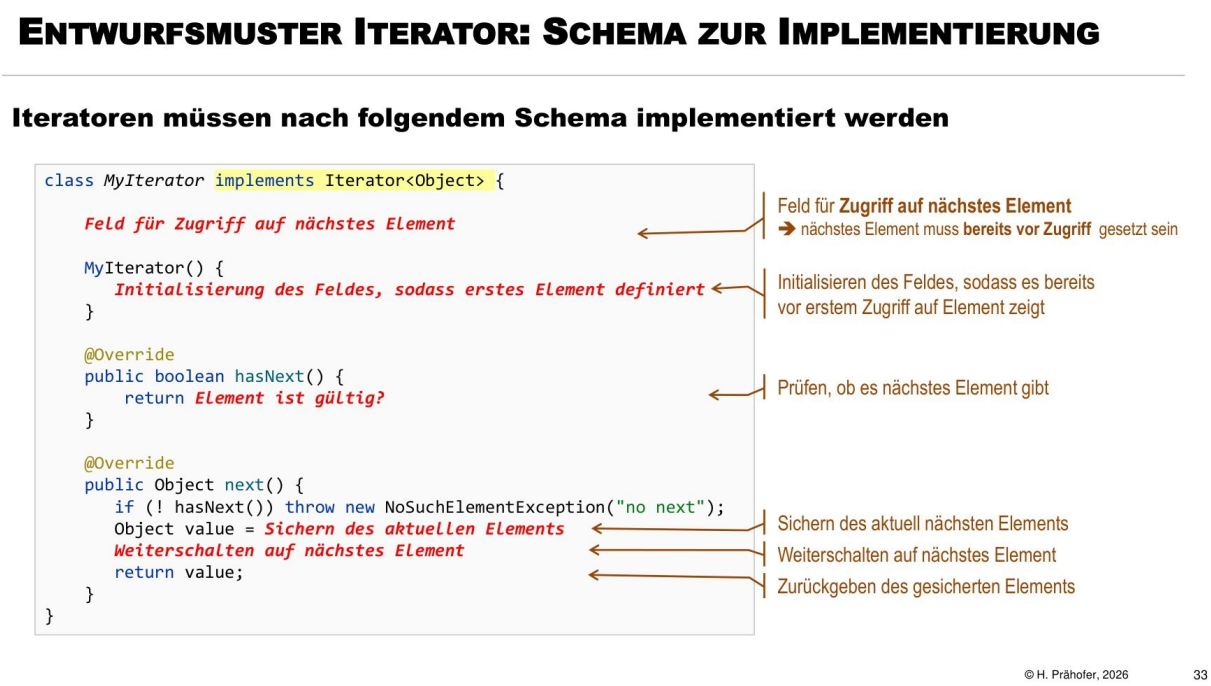
```
for (Object obj: iterable) {
   ... obj ...
}
```

© H. Prähofer, 2026 32

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 32.

S · Folie 33: ENTWURFSMUSTER ITERATOR: SCHEMA ZUR IMPLEMENTIERUNG Iteratoren müssen nach folge

Das Implementierungsschema arbeitet mit einem Cursor, der bereits vor dem ersten Aufruf auf das erste Element zeigt. hasNext prüft dessen Gültigkeit, next sichert das aktuelle Element, schaltet weiter und gibt das gesicherte Element zurück. Bei leerem oder erschöpftem `Iterator` ist `NoSuchElementException` die vertragliche Fehlerreaktion.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 33.

4.4.2 Iteratoren für verkettete Listen und Arrays

Bei der verketteten Liste liegt die gesamte benötigte Strukturinformation im Cursor `Node next`. Beim ersten Abruf wird der Kopf gespeichert und anschließend auf dessen Nachfolger geschaltet. Bei der `ArrayList` genügt ein numerischer Index; ihre logische Größe muss geprüft werden. Die Verwendung derselben Schnittstelle erlaubt eine Datenstruktur zu wechseln, ohne die Iterationsschleifen des Clientcodes zu ändern.

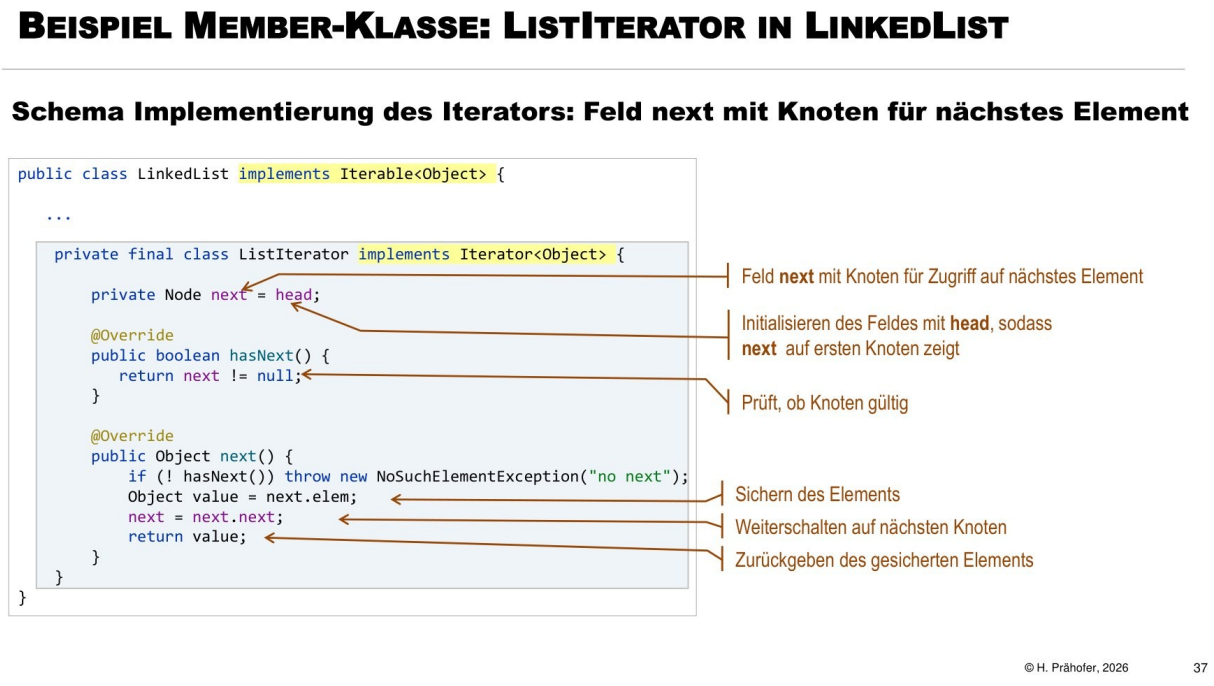
S · Folie 34: BEISPIEL MEMBER-KLASSE: LISTITERATOR IN LINKEDLIST Interface Iterable<Object> im

In `LinkedList` wird die `private` Member-Klasse ListIterator instanziiert. Ihr Feld next beginnt bei head des äußeren Listenobjekts; dadurch lässt sich jede Knotenreferenz in konstantem Aufwand weiterreichen. Besonders wichtig ist der Unterschied zwischen next als Iteratorfeld und next als Knotenfeld: die Zeile next = `next.next` folgt der Verkettung.

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 34.

S · Folie 35: BEISPIEL MEMBER-KLASSE: LISTITERATOR IN LINKEDLIST Schema Implementierung des It

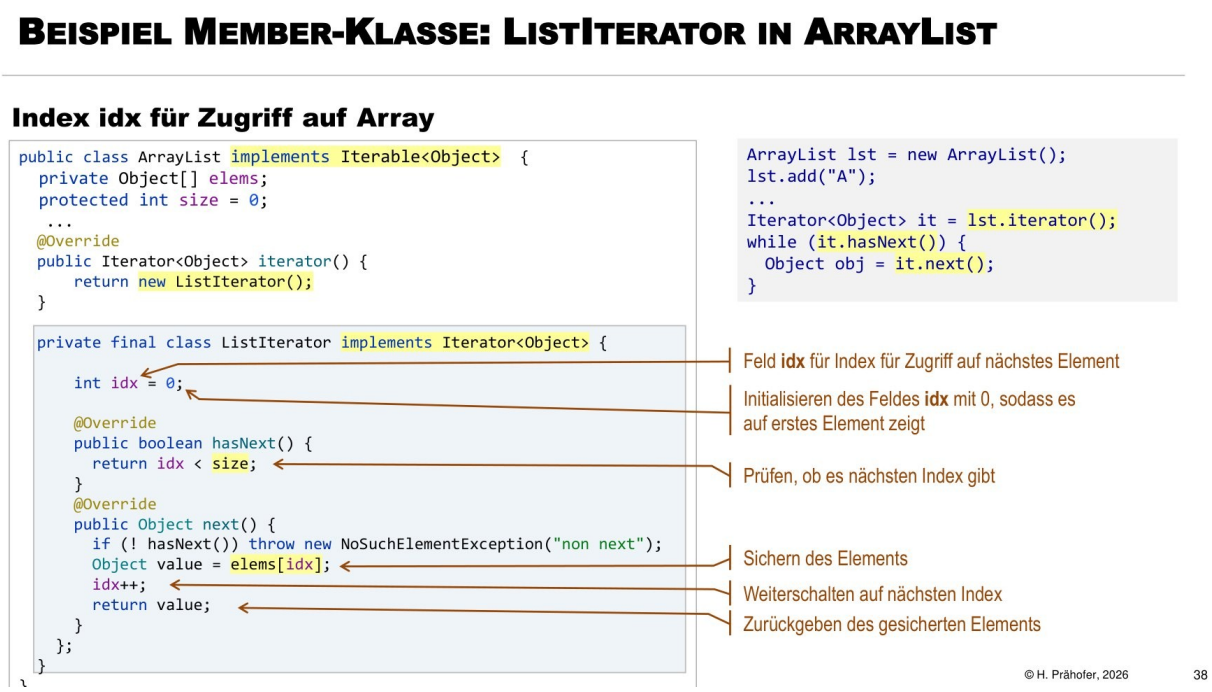
Die linke Implementierung verdeutlicht die Zustandsinvariante: vor `next()` bezeichnet das Feld next den noch nicht ausgegebenen Knoten. Danach verweist es auf dessen Nachfolger. Wenn die Referenz `null` wird, ist die Liste zu Ende; ein weiterer Abruf wirft `NoSuchElementException`.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 35.

S · Folie 36: BEISPIEL MEMBER-KLASSE: LISTITERATOR IN ARRAYLIST Index idx für Zugriff auf Arra

Beim `ArrayList-Iterator` repräsentiert idx die Position des nächsten Elements und beginnt bei `null`. hasNext prüft idx < size, nicht idx < `elems.length`, weil das Array zusätzlich unbenutzte Kapazität besitzen kann. Die Reihenfolge `value = elems[idx]; idx++; return value;` liefert genau das aktuelle Element und rückt anschließend vor.



Originalabbildung: 04-Spezielle Klassen.pdf, Folie 36.

4.4.3 List-Implementierungen und Adapter

Die zentrale Designregel lautet, interne Datenstrukturen nicht ungewollt nach außen preiszugeben. Das Feld `Node.next` ist ein Implementierungsdetail und muss beim normalen `List`-Zugriff nicht sichtbar sein. Die Lehrimplementierungen sind absichtlich vereinfacht und veranschaulichen Strukturierung, nicht sämtliche Optimierungen und Korrektheitsgarantien von `java.util.LinkedList` oder `java.util.ArrayList`.

S · Folie 51: STATISCHE INNERE KLASSE NODE VON LINKEDLIST Klasse Node von LinkedList als stati

Die Fallstudie ordnet `Node` als `private static class` der `LinkedList` zu. Weil die Klasse privat ist, ist `Node` direkt nur innerhalb der umschließenden Klasse zugänglich, nicht automatisch aus beliebigen Unterklassen. Die Annotation `final` auf `LinkedList` verhindert ohnehin Unterklassen; die Formulierung „in `LinkedList` und Unterklassen sichtbar“ passt daher nicht zum dargestellten Code.

Statische Innere Klasse Node von LinkedList

Klasse Node von LinkedList als statische innere Klasse

- damit nur in LinkedList und Unterklassen sichtbar

```
public final class LinkedList extends AbstractList {

  private static class Node {
    private Object value;
    private Node next, prev;
    private Node(Object value) {
      this.value = value;
    }
  }

  private Node head, tail;

  public LinkedList(Object... initial) {...}

  ...
  @Override
  public void add(Object elem) {
    Node n = new Node(elem);
    link(n, tail, null);
    incrSize();
  }
  ...
}
```

© H. Prähofer, 2026

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 51.

S · Folie 52: IMPLEMENTIERUNG VON ITERABLE UND ITERATOR ⇗ Basisinterface List abgeleitet von It

Die Folie verknüpft `Iterable<Object>` mit einer konkreten `LinkedList`- und `ArrayList`-Iteratorimplementierung. Beide erfüllen denselben Vertrag, verwenden aber verschiedenartige `Cursor (Node versus Index)`. Bei einem Array mit zusätzlicher Kapazität müsste iteratorOf die logische size erhalten statt unbesehen `array.length` zu verwenden.

Implementierung von Iterable und Iterator

- Basisinterface **List** abgeleitet von **Iterable<Object>**
  ➔ alle List-Implementierungen sind damit iterierbar

```
public interface List extends Iterable<Object> {
  ...
}
```

- Methode **iterator** in **LinkedList**

```
public class LinkedList extends AbstractList<T> {
  ...
  @Override
  public Iterator<Object> iterator() {

    return new Iterator<Object>() {
      Node next = head;

      @Override
      public boolean hasNext() {
        return next != null;
      }
      @Override
      public Object next() {
        if (!hasNext()) throw new NoSuchElementException("…");
        Object val = next.value;
        next = next.next;
        return val;
      }
    };
  }
}
```

- Methode **iterator** in **ArrayList<T>**

```
public class ArrayList extends AbstractList {
  ...
  @Override
  public Iterator<Object> iterator() {

    return iteratorOf(elems);
  }
}

static Iterator<Object> iteratorOf(Object[] array) {
  return new Iterator<>() {
    private int nextIndex;

    @Override
    public boolean hasNext() {
      return nextIndex < array.length;
    }
    @Override
    public Object next() {
      if (!hasNext()) throw new NoSuchElementException();
      return array[nextIndex++];
    }
  };
}
```

© H. Prähofer, 2026

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 52.

S · Folie 53: BEISPIEL: ADAPTERS MIT ANONYMEN KLASSEN List-Adapters für Array als

Ein Array-Adapter implementiert das lesende Interface `List` mit einer anonymen Klasse; deren `iterator()` gibt eine weitere anonyme `Iterator`-Instanz zurück. Die indexOf-Schleife vergleicht `array[i].equals(o)`. Steht im Array

`null`, wirft dies eine `NullPointerException`; `Objects.equals(array[i], o)` wäre für eine Liste mit Nullwerten robuster.

Beispiel: Adapters mit anonymen Klassen

List-Adapters für Array als anonyme Klassen

List-Implementierung als anonyme Klasse

anonyme Iterator-Klasse von anonymer List-Klasse

```
public interface List extends Iterable<Object> {
    ...
    static List of(Object[] array) {
        return new List() {
            @Override public int size() { return array.length; }

            @Override public int indexOf(Object o) {
                for (int i = 0; i < array.length; i++) {
                    if (array[i].equals(o)) return i;
                }
                return -1;
            }
            ...
            @Override
            public Iterator<Object> iterator() {
                return iteratorOf(array);
            }
        };
    }
}
```

© H. Prähofer, 2026 55

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 53.

4.4.4 Getrennte Interfaces für Zugriff und Mutation

Interface-Segregation bedeutet hier, dass eine Methode mit rein lesendem Bedarf ein lesendes Interface entgegennimmt, nicht vorsorglich ein Interface mit add, remove und clear. Gleichzeitig wird die Unterscheidung zwischen nur lesender Oberfläche und tatsächlich unveränderbarem Objekt ausdrücklich festgehalten: Eine lesende Sicht allein ist noch kein unänderbarer Snapshot.

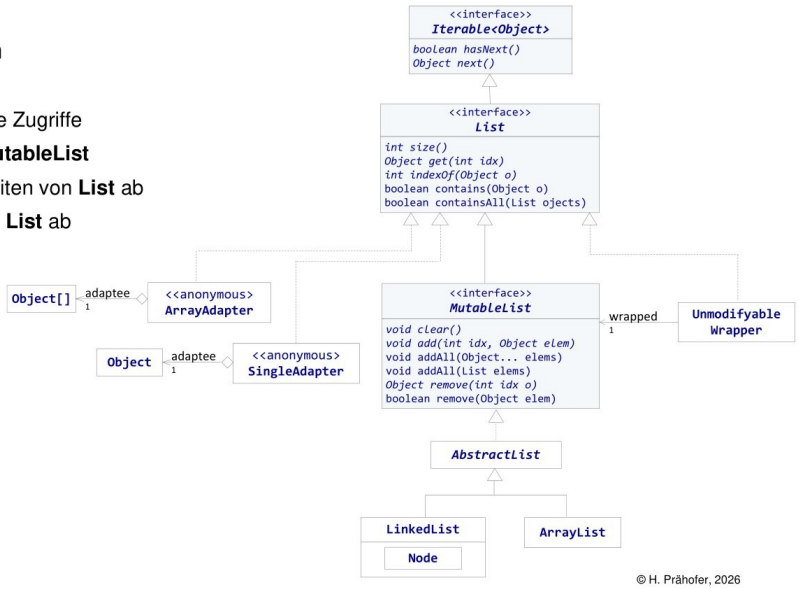
S · Folie 54: INTERFACES LIST UND MUTABLELIST Interfaces für nur-lesende

Die UML-Übersicht stellt lesende `List`-Zugriffe gegenüber zusätzlichen Änderungen in `MutableList`. Es gibt unabhängige Adapter für vorhandene `Arrays` und Einzelobjekte sowie einen read-only-Wrapper. Die abstrakte Basisklasse übernimmt gemeinsame Implementierungsanteile; die konkreten Speicherformen bleiben in `LinkedList` und `ArrayList` gekapselt.

Interfaces List und MutableList

Interfaces für nur-lesende und verändernde Operationen

- **List** – nur lesende Zugriffe
- **MutableList** – zusätzlich schreibende Zugriffe
- Konkrete Klassen implementieren **MutableList**
- **ArrayAdapter** und **SingleAdapter** leiten von **List** ab
- **UnmodifiableListWrapper** leitet von **List** ab und kapselt **MutableList**



© H. Prähofer, 2026 57

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 54.

4.4.5 Laufzeitkomplexität und indexbasierter Zugriff

Die Komplexität lässt sich direkt aus der Schleife herleiten. Für das Element mit Index i sind i Knotenübergänge notwendig. Beim Lesen aller Elemente addieren sich diese Kosten: `Summe(i, i=0..n-1)` = `n(n-1)`/2, also eine quadratische Größenordnung. Ein Cursor, der sein nächstes Element schon kennt, besucht höchstens n Knoten und benötigt bei vollständiger Traversierung lineare Zeit.

S · Folie 55: PROBLEM: ZUGRIFF ÜBER INDEX ⁂ Methode get greift über Index auf Element zu

Die linke Implementierung sucht bei jedem `get(idx)` erneut vom head über idx Nachfolger. Eine einzelne Operation kostet deshalb `O(idx)`, im schlechtesten Fall `O(n)`. Werden alle n Elemente über die rechte Indexschleife gelesen, ergibt das 0 + 1 + … + n-1 und damit `O(n²)`. Ein einmaliger Durchlauf mit einem `Iterator` ist asymptotisch günstiger.

Problem: Zugriff über Index

- Methode **get** greift über Index auf Element zu
- LinkedList implementiert get mit sequentieller Suche
  - ☐ ➔ damit hat Methode lineare Komplexität
  - ☐ ➔ und Schleife über Index hat quadratische Komplexität

```java
public final class LinkedList extends AbstractList {
  ...
  @Override
  public Object get(int idx) {
    Node n = head;
    for (int i = 0; i < idx; i++) {
      n = n.next;
    }
    return n.elem;
  }
  ...
}
```

Schleife für get !?

```java
for (int i = 0; i < l.size(); i++) {
  Object elem = l.get(i);
}
```

Einfache Schleife mit quadratischer Komplexität!?

➔ **LinkedList** sollte keine Zugriffe über Index haben!

© H. Prähofer, 2026 58

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 55.

S · Folie 56: INTERFACES INDEXEDLIST UND MUTABLEINDEXEDLIST ⁂ List – sequentieller Zugriff mit

Die neue Typstruktur trennt Listenoperationen nach ihren Kosten und Fähigkeiten. `List` steht für sequentielle Iteration, `IndexedList` zusätzlich für `get(index)`, `MutableList` für grundlegende Änderungen und `MutableIndexedList` für indexbezogene Änderungen. Gerade die Mehrfachvererbung bei Interfaces gestattet die Kombination lesender und schreibender Aspekte ohne Mehrfachvererbung von Implementierungsklassen.

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 56.

S · Folie 57: INTERFACES LIST UND INDEXEDLIST public interface List extends Iterable<Object> {

Das Interface `List` kann indexOf als Default-Methode mit einem einzigen for-each-Durchlauf implementieren. `IndexedList` ergänzt get und definiert first als `get(0)`. Hier ist eine Entscheidung über leere Listen nötig: `get(0)` wirft typischerweise IndexOutOfBoundsException. Zusätzlich sollte indexOf für mögliche Nullwerte `Objects.equals(e, o)` verwenden.

Interfaces List und IndexedList

List

```
public interface List extends Iterable<Object> {

  int size();

  Object first();

  default int indexOf(Object o) {
    int i = 0;
    for (Object e: this) {
      if (e.equals(o)) return i;
      i++;
    }
    return -1;
  }

  default boolean contains(Object o) {
    return indexOf(o) >= 0;
  }

  default boolean containsAll(List objs) {
    for (Object o : objs) {
      if (! contains(o)) return false;
    }
    return true;
  }

}
```

IndexedList

```
public interface IndexedList extends List {

Object get(int idx);

  @Override
  default Object first() {
    return get(0);
  }
}
```

© H. Prähofer, 2026

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 57.

S · Folie 58: INTERFACES MUTABLELIST UND MUTABLEINDEXEDLIST ⇗ MutableList

`MutableList` implementiert addAll, indem es add einzeln aufruft. `MutableIndexedList` ist gleichzeitig ein `MutableList` und `IndexedList` und kann `add(elem)` auf `add(size()`, elem) abbilden. Die Default-Implementierung von `remove(Object)` prüft vor `remove(idx)` ausdrücklich, ob indexOf einen gültigen Index liefert; bei -1 wird `false` zurückgegeben. Bei `addAll(this)` wäre die Mutationssicherheit zusätzlich zu klären.

Interfaces MutableList und MutableIndexedList

MutableList

```
public interface MutableList extends List {

  void clear();

  void add(Object elem);

  default void addAll(Object... elems) {
    for (Object e: elems) add(e);
  }


  default void addAll(List elems) {
    for (Object e: elems) add(e);
  }

  boolean remove(Object elem);

}
```

MutableIndexedList

```
public interface MutableIndexedList extends MutableList, IndexedList {

  void add(int idx, Object elem);

  default void add(Object elem) {
    add(size(), elem);
  }


  Object remove(int idx);

  @Override
  default boolean remove(Object elem) {
    int idx = indexOf(elem);
    if (idx >= 0) {
      remove(idx);
      return true;
    } else {
      return false;
    }
  }

}
```

© H. Prähofer, 2026

Originalabbildung: 04-Spezielle Klassen.pdf, Folie 58.

Korrigiertes Iterator-Beispiel und Aufwand

```
static <T> Iterator<T> iteratorOf(T[] values, int size) {
 if (size < 0 || size > values.length)
 throw new IllegalArgumentException("size");
 return new Iterator<>() {
 private int index;
 public boolean hasNext() { return index < size; }
 public T next() {
 if (!hasNext()) throw new NoSuchElementException();
 return values[index++];
 }
 };
}
```

Die Logik ist für eine Array-backed Collection mit logischer Größe size geeignet. Wenn Änderungen an der Sammlung parallel zur Iteration zugelassen sind, müssen Sichtbarkeit und eventuell eine fail-fast-Strategie zusätzlich definiert werden. Für eine verkettete Liste bleibt der `Node`-Cursor zweckmäßiger.

5 Generics

Die vorangegangenen Themen haben gezeigt, dass sich verschiedene Objekte mithilfe von Supertypen und Interfaces einheitlich ansprechen lassen. Eine Liste kann beispielsweise `Student`-, `Person`- oder `String`-Objekte enthalten. Soll jedoch eine bestimmte Liste ausschließlich `Student`-Objekte aufnehmen, reicht die allgemeine Typangabe `Object` nicht aus: Einfügefehler fallen erst beim Auslesen und Casten auf. `Generics (generische Programmierung)` beseitigen diese Lücke weitgehend. Ein Typparameter ist ein symbolischer Platzhalter für einen Referenztyp. Bei `List<E>` heißt dieser Platzhalter E, und bei einer Verwendung als `List<Student>` ist das zugehörige Typargument `Student`. Der Compiler prüft die daraus folgenden Typregeln, sodass viele sonst mögliche ClassCastExceptions schon vor Programmausführung verhindert werden.

Dieses Kapitel führt zunächst die Schreibweise und das Typsystem ein. Anschließend werden `Bounds (Einschränkungen erlaubter Typargumente)`, `Varianz (Beziehungen zwischen Typen mit verschiedenen Typargumenten)` und `Wildcards (unbekannte, gegebenenfalls eingeschränkte Typargumente)` hergeleitet. Danach wird geklärt, weshalb Generics in Java im Wesentlichen durch `Type Erasure (Entfernung konkreter Typargumente aus der Laufzeitrepräsentation)` realisiert werden und welche Folgen dies für Casts, `Arrays` und Objekterzeugung hat. Die abschließende Listen-Fallstudie erweitert das aus den Kapiteln 2–4 bekannte, eigens für die Vorlesung entwickelte `List`-Modell um Typsicherheit. Namen wie `List`, `LinkedList` und `ArrayList` bezeichnen dabei jeweils die Lehrbeispiele, sofern nicht ausdrücklich `java.util.List` oder die JDK-Bibliothek genannt ist.

5.1 Motivation: Typsicherheit statt Object und Casts

Eine nicht generische Liste enthält Werte des Typs `Object`. `Object` ist die Wurzel aller Java-Klassen; deshalb kann fast jedes gewöhnliche Objekt ohne Cast in eine solche Liste eingefügt werden. Genau diese Flexibilität ist problematisch, wenn die programmatische Absicht enger ist: Eine als Studentenliste verwendete Liste soll keine Professoren enthalten. Der Aufruf `add(new Professor(...))` ist dennoch aus Sicht der Signatur `add(Object)` legal. Erst beim späteren `Ausdruck (Student) list.get(2)` wird der tatsächlich gespeicherte Objekttyp geprüft und möglicherweise eine `ClassCastException` geworfen.

Generische Klassen und Methoden erlauben, die gewünschte Typbeziehung an der Deklaration auszudrücken, statt ausschließlich in Kommentaren oder unsicheren Casts. `List<Student>` bedeutet, dass ein get-Ergebnis beim normalen typisierten Zugriff als `Student` behandelt werden kann und ein add-Aufruf nur kompatible Objekte annehmen darf. Die Prüfung erfolgt zur `Übersetzungszeit (Compilezeit)`. Wiederverwendbarkeit bleibt erhalten, weil dieselbe Implementierung auch `List<Person>` und `List<String>` unterstützen kann. Dies garantiert allerdings nicht, dass grundsätzlich niemals eine `ClassCastException` vorkommt: Rohtypen, unchecked casts, Reflection oder unsicherer fremder Code können die statische Garantie unterlaufen.

Eine nicht generische Sammlung mit Elementtyp `Object` akzeptiert jedes Referenzobjekt. Der Compiler kann dann nicht sicherstellen, dass in einer fachlich als `Student`-Liste verwendeten Sammlung ausschließlich Students liegen. Bei `Student s = (Student) list.get(0);` erfolgt deshalb eine Laufzeitprüfung, die eine

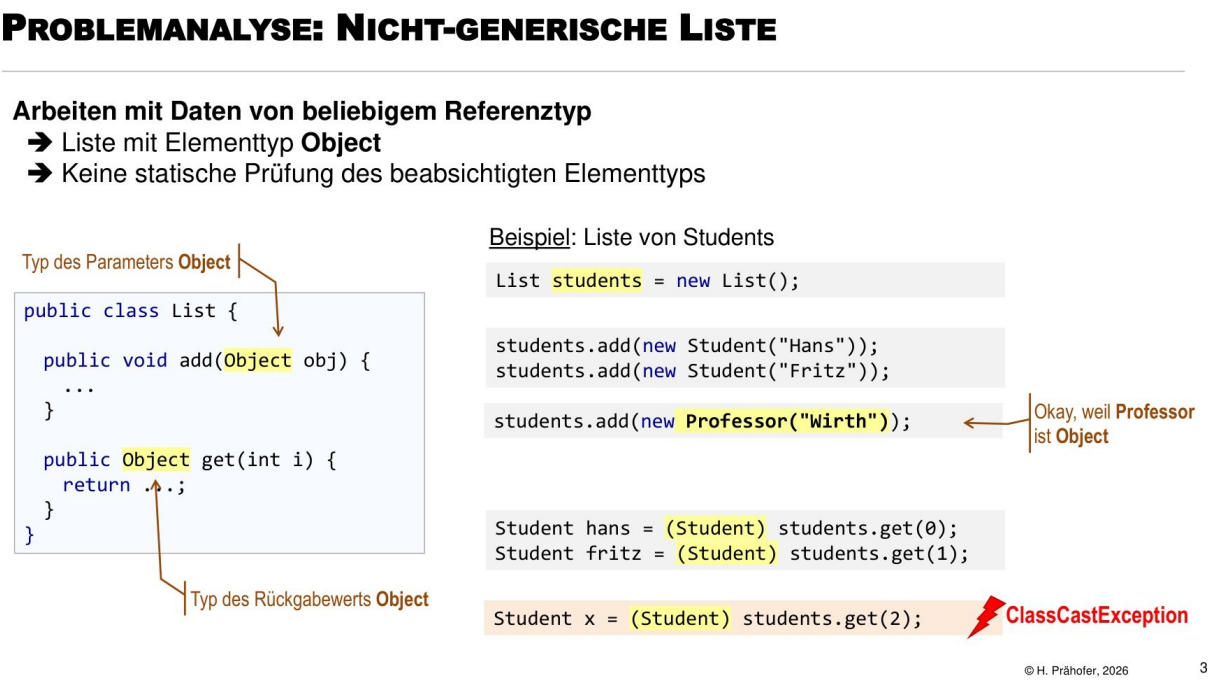
`ClassCastException` auslösen kann. Ein parametrisierter Typ `List<Student>` macht die Elementtypabsicht für den Compiler sichtbar und verhindert viele dieser Fehler bereits beim Übersetzen.

Die Typprüfung von Generics ersetzt nicht jede Laufzeitkontrolle. Sie verhindert vor allem unzulässige Quelltextoperationen in korrekt typisiertem Code. Werden Rohtypen, ungeprüfte Casts oder nicht kontrollierte Reflection verwendet, kann dennoch ein unerwarteter Elementtyp in die Sammlung gelangen. Ein generischer Rückgabewert erlaubt normalerweise eine Zuweisung ohne ausdrücklichen Cast; die JVM muss dabei unter Umständen vom Compiler eingefügte Casts ausführen.

Originalfolien, Code und fachliche Einordnung

G · Folie 3: PROBLEMANALYSE: NICHT-GENERISCHE LISTE

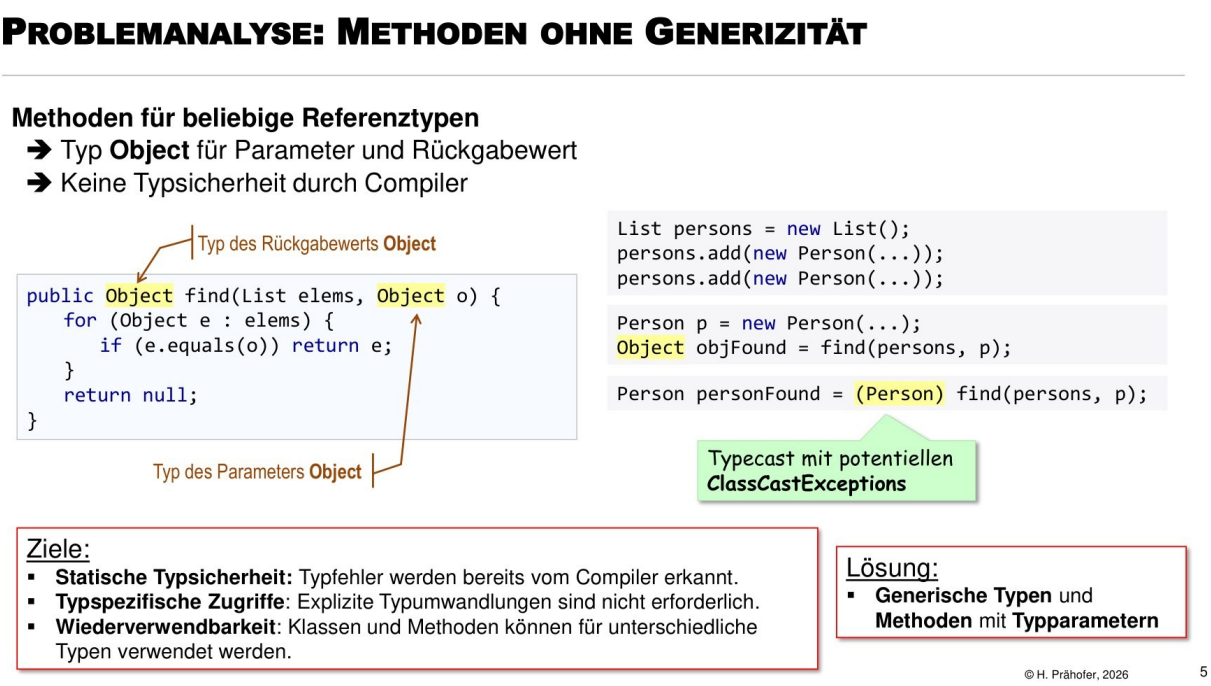
Die erste Problemanalyse zeigt, weshalb eine Liste mit Elementtyp `Object` die Entwurfsabsicht nicht absichert. `Student` und `Professor` sind zwar beides Objekte, aber nicht untereinander austauschbar. Das unzulässige Element wird beim Einfügen akzeptiert; die spätere `Umwandlung (Student)` wirft eine `ClassCastException`. Der Compiler kennt über get nur `Object` und kann die Absicht des Entwicklers nicht rekonstruieren. Generics verlagern diese Fehlererkennung in die Übersetzung.



Originalabbildung: 05.Generics.pdf, Folie 3.

G · Folie 4: PROBLEMANALYSE: METHODEN OHNE GENERIZITÄT

Die generische Motivation betrifft auch freistehende Methoden: Eine Methode find mit `Object`-Argumenten kann zwar Objekte unterschiedlicher Klassen vergleichen, garantiert aber bei der Rückgabe nur `Object`. Ein Aufrufer muss erst auf `Person` casten. Ein Typparameter für den Listenelementtyp, Suchwert und Rückgabewert formuliert die Beziehung zwischen den drei Positionen und erspart den expliziten Cast. Die Lehrziele sind frühzeitige Typprüfung, sicherer Zugang zum Ergebnis und Wiederverwendbarkeit.



Originalabbildung: 05.Generics.pdf, Folie 4.

5.2 Grundlagen: Typparameter, Parametrisierung und generische Methoden

Ein generischer Typ wird durch formale Typparameter in spitzen Klammern deklariert. `interface List<E>` ist zunächst eine Typdefinition mit einer frei benennbaren Typvariablen E. Die Methoden `void add(E element)` und `E get(int index)` stehen deshalb in Beziehung: Sie verwenden für eine bestimmte Parametrisierung denselben Elementtyp. Mit `List<String>` wird E durch das konkrete Typargument `String` ersetzt; `List<Person>` repräsentiert eine andere Parametrisierung. Anders als bei primitiven Variablen wird im Quelltext kein neuer Typ erzeugt, wenn man ein weiteres Typargument schreibt: Eine einzige generische Klassendeklaration beschreibt all diese Verwendungen.

Ein Typparameter ist nur innerhalb seines deklarierten Gültigkeitsbereichs sichtbar. Eine generische Klasse kann etwa `Map<K,V>` mit zwei unabhängigen Typvariablen K für Schlüssel und V für Werte deklarieren. `Map<String,Person>` prüft dann sowohl Schlüssel als auch Werte. Der Diamond-Operator `<>` bei `new Map<>()` lässt den Compiler die Typargumente aus dem erwarteten Kontext ableiten; in `Map<String,Person> m = new Map<>();` sind daher K und V eindeutig. Ein leerer Diamond bedeutet nicht, dass der Typ unbestimmt oder roh wird.

Vererbung und Interfaces können Typparameter weiterreichen: `class MyList<E> extends BaseList<E>` verwendet E der Unterklasse auch für die Oberklasse; `class NameList extends BaseList<String>` setzt die Typvariable dagegen fest auf `String`. Ein Bezeichner E, der nicht deklariert wurde, darf nicht als Typvariable eingesetzt werden. Eine generische Methode besitzt ihre eigenen Typparameter unmittelbar vor dem Rückgabetyp, beispielsweise `static <T> T find(List<T> list,T sought)`. Das `T` vor dem Rückgabetyp ist die Deklaration, das `T` in Parametern und Rückgabe die Verwendung. Bei Aufrufen ermittelt der Compiler unter anderem aus den Argumenttypen, welches T möglich ist. Damit können generische Methoden innerhalb nicht generischer Klassen vorkommen und umgekehrt.

Bei `class Box<T>` wird `T` in der Deklaration als Typparameter eingeführt. In `Box<String>` ist `String` das konkrete Typargument. Eine Instanz einer generischen Klasse wird beispielsweise mit `new Box<String>()` oder mit dem Diamond `new Box<>()` erzeugt. Der Diamond überlässt die Ermittlung der Typargumente dem Compiler. Java erzeugt bei gewöhnlichen Generics nicht für jedes Typargument eine eigene spezialisierte Klasse; die Umsetzung durch Type Erasure wird später erläutert.

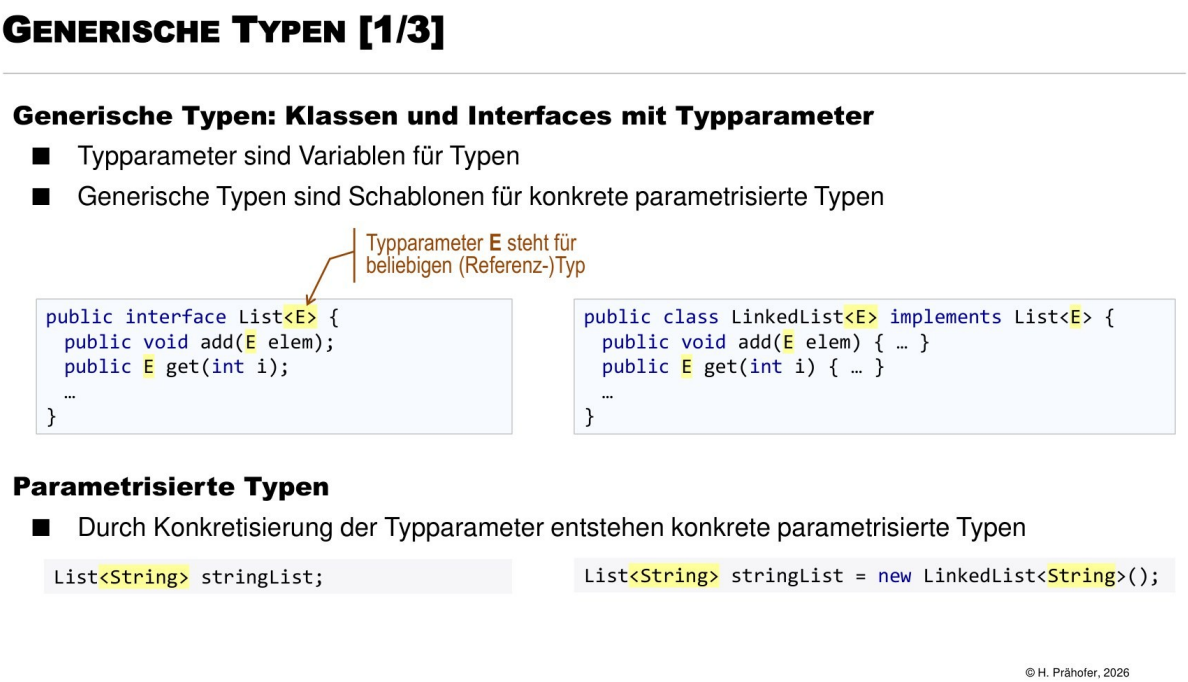
Eine generische Methode deklariert ihren Typparameter vor dem Rückgabetyp: `static <T> T first(T a, T b)`. Die beiden Parameter und das Ergebnis sind an dasselbe `T` gebunden. Bei `String s = first("A", "B");` kann der Compiler `T = String` ableiten. Ein Typparameter einer statischen Methode muss auch dann selbst eingeführt werden, wenn die Methode in einer generischen Klasse steht; eine statische Methode kann den Typplatzhalter einer konkreten Instanz nicht automatisch übernehmen.

Originalfolien, Code und fachliche Einordnung

5.2.1 Generische Typen und konkrete Typargumente

G · Folie 6: GENERISCHE TYPEN [1/3]

Die Deklaration `List<E>` führt eine freie Typvariable E ein. E erscheint als Typ des add-Parameters und des get-Ergebnisses, und `LinkedList<E>` reicht dieselbe Typvariable an das implementierte Interface weiter. Dies ist eine Quelltextschablone für statische Typbeziehungen, keine automatische mehrfache Erzeugung unterschiedlicher Klassen im ausführbaren Programm. Erst die Verwendung `List<String>` konkretisiert E für die statische Prüfung.



Originalabbildung: 05.Generics.pdf, Folie 6.

G · Folie 7: GENERISCHE TYPEN [2/3]

Die linke und rechte Darstellung zeigen die rein gedankliche Ersetzung der Typvariable durch `String`. Die tatsächliche Java-Übersetzung erzeugt keine neue Quelldeklaration `class LinkedList<String>`. Syntax wie `List<String>` bleibt eine Typverwendung. Der Compiler prüft `add(String)` und get als `String`-Rückgabe, generiert aber normalerweise eine gemeinsame Implementierung nach Type Erasure. Die grafisch ausgeschriebenen spezialisierten Klassen sind ausschließlich ein Erklärungshilfsmittel.

GENERISCHE TYPEN [2/3]

```
public interface List<E> {
  public void add(E elem);
  public E get(int i);
  …
}
```

```
public class LinkedList<E> implements List<E> {
  public void add(E elem) { … }
  public E get(int i) { … }
  …
}
```

Erzeugen von konkreten parametrisierten Typen

- durch "Zuweisung" von Typen zu Typparametern:

```
List<String> stringList;
```

E = String und damit ist folgender List-Typ verbunden

```
public interface List<String> {
  public void add(String elem);
  public String get(int i);
  …
}
```

```
List<String> stringList = new LinkedList<String>();
```

E = String und damit ist folgender LinkedList-Typ verbunden

```
public class LinkedList<String> implements List<String> {
  public void add(String elem) { … }
  public String get(int i) { … }
  …
}
```

Es werden aber keine solchen Klassen oder Interfaces erzeugt! Siehe Type Erasure

© H. Prähofer, 2026 8

Originalabbildung: 05.Generics.pdf, Folie 7.

G · Folie 8: GENERISCHE TYPEN [3/3]

Eine Liste von Strings und eine Liste von Personen entstehen aus derselben generischen Struktur. In StringList darf der Compiler get direkt als `String` behandeln und verhindert das Einfügen einer `Person`; bei PersonList ist es umgekehrt. Subtypen sind weiterhin erlaubt: Eine `Student`-Instanz darf in `List<Person>`, nicht jedoch eine beliebige `Person` in `List<Student>`. Die Folie illustriert gerade die für jede einzelne Parametrisierung gültige Typgarantie.

GENERISCHE TYPEN [3/3]

```
List<String> stringList = new LinkedList<String>();
```

```
public class LinkedList<String> implements List<String> {
  public void add(String elem) { … }
  public String get(int i) { … }
  …
}
```

```
List<Person> personList = new LinkedList<Person>();
```

```
public class LinkedList<Person> implements List<Person> {
  public void add(Person elem) { … }
  public Person get(int i) { … }
  …
}
```

Typsicheres Arbeiten

- durch konkrete Typen für Typparameter bei Variablen, Methodenparametern und Rückgabewerten

```
stringList.add("Hello");
```

add mit Parameter vom Typ String

```
String s = stringList.get(0);
```

get mit Rückgabe von Typ String

```
stringList.add(new Person(...));  ✗
```

```
personList.add(new Person(...));
```

add mit Parameter vom Typ Person

```
Person p = personList.get(0);
```

get mit Rückgabe vom Typ Person

```
personList.add("Hello");  ✗
```

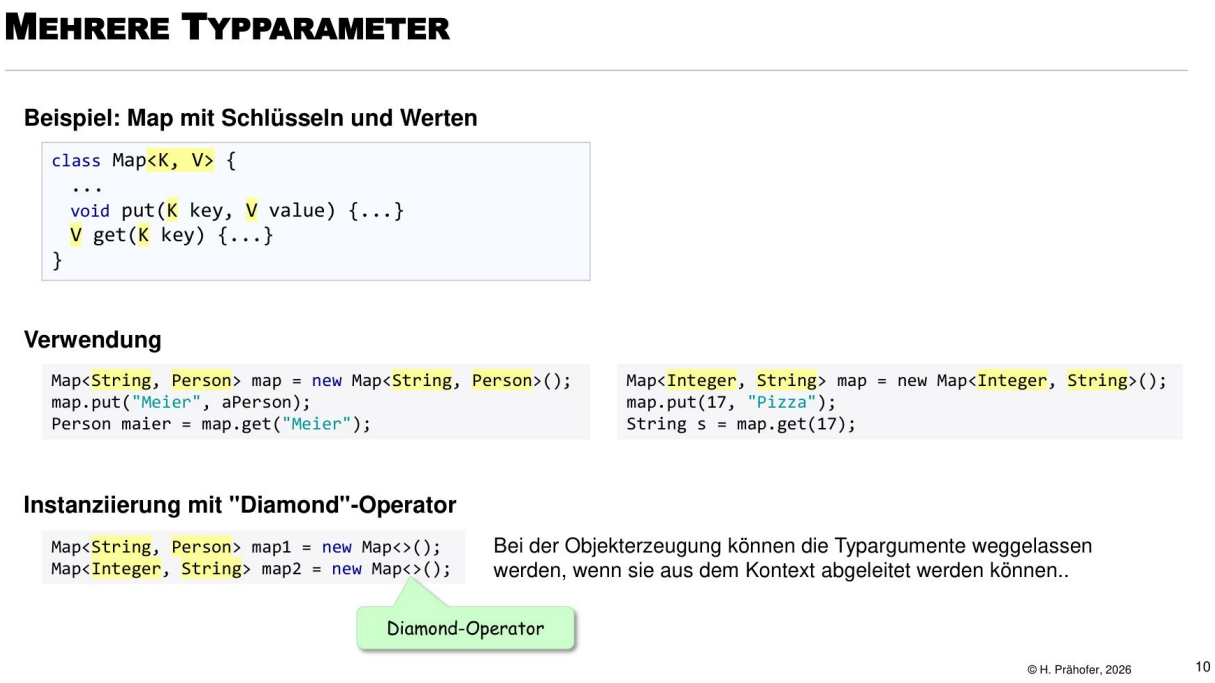
© H. Prähofer, 2026 9

Originalabbildung: 05.Generics.pdf, Folie 8.

5.2.2 Mehrere Parameter, Diamond und Vererbung

G · Folie 9: MEHRERE TYPPARAMETER

`Map<K,V>` zeigt zwei unabhängige Typparameter für Schlüssel und Wert. In `Map<String,Person>` ist der erste Parameter von put ein `String`, der zweite eine `Person`, und get liefert `Person`. Die Instanziierung `new Map<>()` setzt passende Typargumente aus dem Variablentyp ein. Die Bezeichnung `Map` ist hier eine vereinfachte, eigene generische Klasse; `java.util.Map` ist im JDK ein Interface und kann nicht direkt mit `new Map<>()` konstruiert werden.



Originalabbildung: 05.Generics.pdf, Folie 9.

G · Folie 10: GENERISCHEN TYPEN UND VERERBUNG

Eine Unterklasse einer generischen Klasse muss die Anforderungen an die Typparameter ihrer Oberklasse erfüllen. Sie kann den Typ festlegen, weiterreichen oder zusammensetzen, etwa `ListOfLists<E> extends List<List<E>>`. E muss dafür im Gültigkeitsbereich der Unterklasse deklariert sein. Analoge Regeln gelten für `extends` bei Interfaces und `implements` bei Klassen. Der konkrete Code muss außerdem beachten, ob `List` in dem Beispiel eine Klasse oder ein Interface ist; davon hängt die korrekte `extends`-/`implements`-Schreibweise ab.

Generischen Typen und Vererbung

Bei **Ableitung von einem generischen Typ** muss der **Typparameter** festgelegt werden durch

- einen konkreten Typ

Typparameter von List gleich **String**

```
class StringList extends List<String> {...}
```

- einen Typparameter der abgeleiteten Klasse

Typparameter von **List** gleich zu dem von **ExtList**

```
class ExtList<E> extends List<E> {...}
```

Gilt analog für die Implementierung und Erweiterung von Interfaces!

Typparameter von **List** ist **List<E>** mit **E** gleich zu dem von **ListOfLists**

```
class ListOfLists<E> extends List<List<E>> {...}
```

Nicht erlaubt:

- Verwendung nicht deklarierter Typparameter

```
class SubClass extends BaseClass<E>   ✗
class SubClass<E> extends BaseClass<U>   ✗
```

© H. Prähofer, 2026 11

Originalabbildung: 05.Generics.pdf, Folie 10.

5.2.3 Generische Methoden und Typinferenz

G · Folie 11: GENERISCHE METHODEN

Bei `static <T> T find(List<T> list, T item)` steht die Deklaration `<T>` unmittelbar vor dem Rückgabetyp. Das T verbindet den Listenelementtyp, den Typ des Suchwerts und das Rückgabeergebnis. Bei `List<String>` inferiert der Compiler T als `String`, bei `List<Person>` als `Person`. Das `return null` signalisiert hier „nicht gefunden“, ist aber für Referenztypen möglicherweise mehrdeutig und macht eine Nullprüfung nötig; eine spätere API kann stattdessen `Optional<T>` verwenden.

Generische Methoden

Methodendeklaration mit Typparameter

- **Typparameter** zum Bestimmen der Typen von **Parameter**, **Variablen** und **Rückgabewerten**

Typparameter **T**

T ident in **List<T> list**, **T item** und Rückgabewert

```
public static <T> T find(List<T> list, T item) {
  for (T elem: list) {
    if (elem.equals(item)) {
      return elem;
    }
  }
  return null;
}
```

- Ableitung des **konkreten Typs** aus den **aktuellen Methodenparametern**

```
List<String> keywords = …;
String found = find(keywords, "lambda");
```

Typ **T = String**

```
List<Person> persons = …;
Person found = find(persons, new Person(…));
```

Typ **T = Person**

© H. Prähofer, 2026 12

Originalabbildung: 05.Generics.pdf, Folie 11.

Zusammenhängendes Beispiel mit generischer Klasse und Methode

Das folgende eigenständige Java-Beispiel ist eine ergänzende Implementation. Das Feld enthält nur ein T; die statische Methode hat einen von der Klasse unabhängigen eigenen Typparameter U. Damit wird die Trennung von generischen Klassen und generischen Methoden sichtbar.

```
final class Box<T> {
 private T value;
 Box(T value) { this.value = value; }
 T get() { return value; }
 void set(T value) { this.value = value; }

 static <U> Box<U> of(U value) {
 return new Box<>(value);
 }
}
Box<String> greeting = Box.of("Hallo");
String text = greeting.get();
// greeting.set(42); // Compilerfehler: int -> Integer ist kein String
```

5.3 Typconstraints und vergleichbare Elemente

Ein uneingeschränkter Typparameter T erlaubt im Regelfall jeden Referenztyp. Dann darf Code, der lediglich T kennt, nur diejenigen Methoden voraussetzen, die für `Object` verfügbar sind. Eine sortierende Liste braucht mehr: Um zwei Elemente geordnet einzufügen, muss sie sie vergleichen können. Dies wird durch eine obere `Typschranke (Upper Bound)` ausgedrückt: `T extends Comparable<T>` verlangt, dass jedes zulässige T das Interface `Comparable<T>` erfüllt. Das Schlüsselwort `extends` steht hier sowohl bei Klassen als auch bei Interfaces. Dank der Schranke darf man `t1.compareTo(t2)` verwenden; das `int`-Ergebnis ist negativ bei kleiner, `null` bei gleichrangig und positiv bei größer. Es geht bei der Null nicht notwendigerweise um `equals`-Gleichheit: Konsistenz von compareTo und equals ist erwünscht, aber nicht zwangsläufig immer gegeben.

Auch generische Methoden dürfen Schranken besitzen: `static <T extends Comparable<T>> T max(T a,T b)` ist für vergleichbare Typen definiert. Es können mehrere Bounds mit dem Operator `&` kombiniert werden. Eine etwaige konkrete Oberklasse muss dabei zuerst stehen; anschließend folgen Interfaces. Jedes übergebene Typargument muss sämtliche Bedingungen erfüllen. Bei Ableitung von einer generischen Basisklasse muss der Typparameter der Unterklasse mindestens die Anforderungen der Basisklasse erfüllen. Da ein `Student` eine `Person` ist, erfüllt `S extends Student` auch die obere Schranke `S extends Person`.

Die strenge Form `T extends Comparable<T>` kann zu eng sein. Angenommen, `Person` implementiert `Comparable<Person>` und `Student` erweitert `Person`. Dann können `Student`-Objekte von der geerbten `compareTo(Person)`-Methode verglichen werden; `Student` implementiert dadurch aber nicht notwendig `Comparable<Student>`. Die später eingeführte Schranke `T extends Comparable<? super T>` erfasst genau diesen Fall. Man sollte die Form mit `<? super T>` bevorzugen, wenn natürliche Ordnung von Oberklassen geerbt werden darf.

Eine `Typschranke (Type Bound)` beschränkt erlaubte Typargumente: Bei `<T extends Number>` müssen sie `Number` selbst oder dessen Untertypen sein. Das Schlüsselwort `extends` gilt in dieser Syntax auch dann, wenn die Schranke ein Interface ist. Für Sortierung ist häufig `<T extends Comparable<? super T>>` sinnvoll: Die Werte müssen mit Objekten ihres eigenen Typs oder eines Supertyps vergleichbar sein.

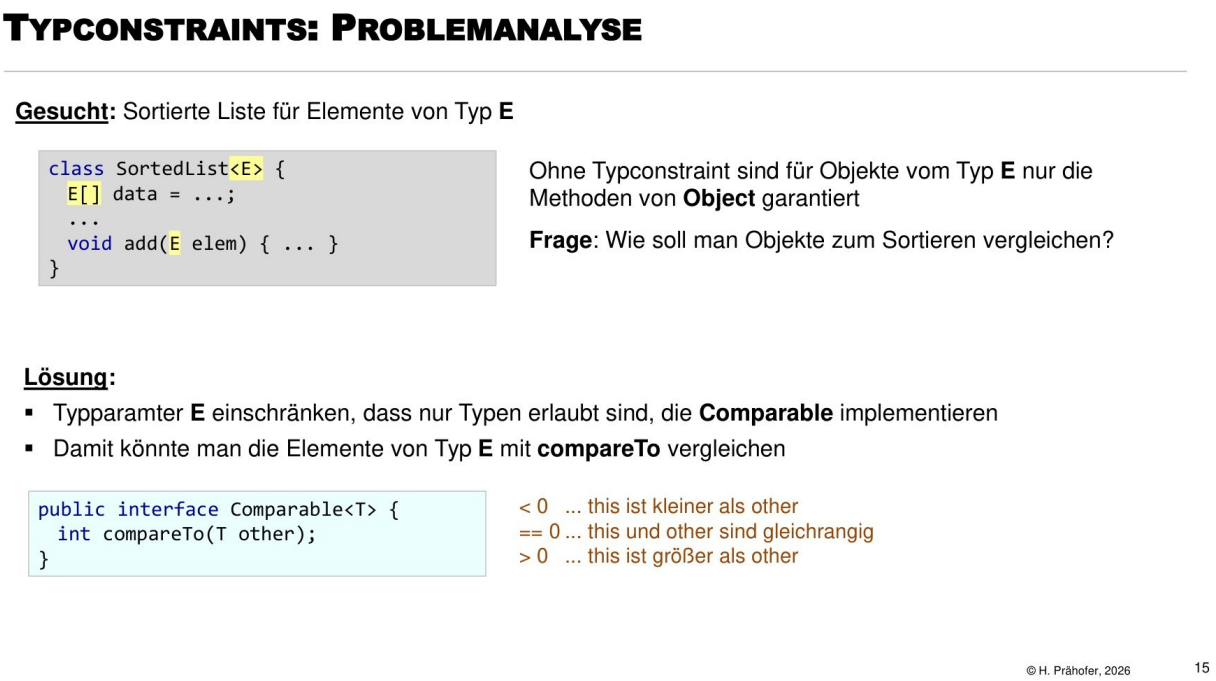
Bei mehreren oberen Schranken steht, falls vorhanden, zuerst eine Klasse und danach folgen mit `&` verbundene Interfaces, beispielsweise `<T extends Number & Comparable<T>>`. Für jede erlaubte Konkretisierung müssen sämtliche Schranken erfüllt sein. Der Compiler darf innerhalb des generischen Körpers die durch diese Schranken garantierten Operationen verwenden. Nicht garantierte Methoden eines zufälligen konkreten Typarguments stehen im generischen Code nicht zur Verfügung.

Originalfolien, Code und fachliche Einordnung

5.3.1 Warum Vergleichsmethoden Bounds erfordern

G · Folie 14: TYPCONSTRAINTS: PROBLEMANALYSE

Eine sortierte Liste muss bestimmen, an welcher Position ein Element steht. Ohne Typschranke kennt der Compiler für E nur allgemeine `Object`-Methoden; compareTo gehört nicht dazu. `Comparable<T>` stellt `int compareTo(T other)` bereit und garantiert damit eine Ordnungsrelation, deren Vorzeichen die Reihenfolge steuert. Die in der Folie gesuchte Lösung ist eine Einschränkung der zulässigen Typargumente auf vergleichbare Typen.



Originalabbildung: 05.Generics.pdf, Folie 14.

G · Folie 15: TYPCONSTRAINTS : BOUNDED TYPE PARAMETERS

`E extends Comparable<E>` ist ein bounded type parameter. Er garantiert bei jedem zulässigen Element E die Methode `compareTo(E)`. Beim Einfügen in eine aufsteigend sortierte Arrayliste wird von hinten nach vorne geschoben, solange das vorige Element größer ist als das neue. Ein vollständiges Beispiel muss vorher Kapazität und size prüfen und nach dem Verschieben size erhöhen. Die Folie verweist bereits auf die flexiblere Schranke `Comparable<? super E>`, die auch geerbte Vergleichsmethoden unterstützt.

Typconstraints : Bounded Type Parameters

`T extends TypeBound`

TypeBound kann Interface oder Klasse sein

- Das Typargument für **T** muss ein Subtyp von ***TypeBound*** sein oder ***TypeBound*** selbst.
- Mit der Typgarantie sind Operationen von ***TypeBound*** garantiert

```
public class SortedList<E extends Comparable<E>> {
  E[] es = ...;
  ...
  public void add(E e) {
    int i = size - 1;
    while (i >= 0 && es[i].compareTo(e) > 0)  {
      es[i+1] = es[i];
      i--;
    }
    es[i+1] = e;
    size++;
  }
}
```

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

Achtung:
Verbesserte Version siehe Wildcards!

© H. Prähofer, 2026 16

Originalabbildung: 05.Generics.pdf, Folie 15.

G · Folie 16: BEISPIEL COMPARABLE<T>: VERWENDUNG

Die Klasse `Person` ordnet zuerst nach Familiennamen, dann nach Vornamen und schließlich nach Alter. Jeder compareTo-Schritt entscheidet nur dann endgültig, wenn sein Vergleichsergebnis nicht `null` ist. Durch `Comparable<Person>` erfüllt `Person` den Bound der einfachen `SortedList<Person>`. Der Code benutzt für das Alter `Integer.compare` statt Subtraktion, weil die Subtraktion bei extremen Werten überlaufen könnte.

Beispiel Comparable<T>: Verwendung

- Interface für Ordnungsrelation

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

- Implementierung durch Klasse **Person**

```
public class Person implements Comparable<Person> {
  private final String firstName;
  private final String lastName;
  private final int age;
  …
  public int compareTo(Person other) {
    int c = this.lastName.compareTo(other.lastName);
    if (c != 0) return c;
    c =  this.firstName.compareTo(other.firstName);
    if (c != 0) return c;
    return Integer.compare(this.age, other.age);
  }
}
```

- **SortedList** für **Person** verwenden

```
SortedList<Person> sortedPersons = new SortedList<>();
```

Person erfüllt Constraint **Person extends Comparable<Person>**

© H. Prähofer, 2026 17

Originalabbildung: 05.Generics.pdf, Folie 16.

G · Folie 17: TYPCONSTRAINTS BEI METHODEN

Auch eine einzelne generische Methode kann eine obere Schranke haben. Bei max werden x und y durch `x.compareTo(y)` geordnet und das größere Objekt zurückgegeben. Das `>= 0` entscheidet bei gleichem

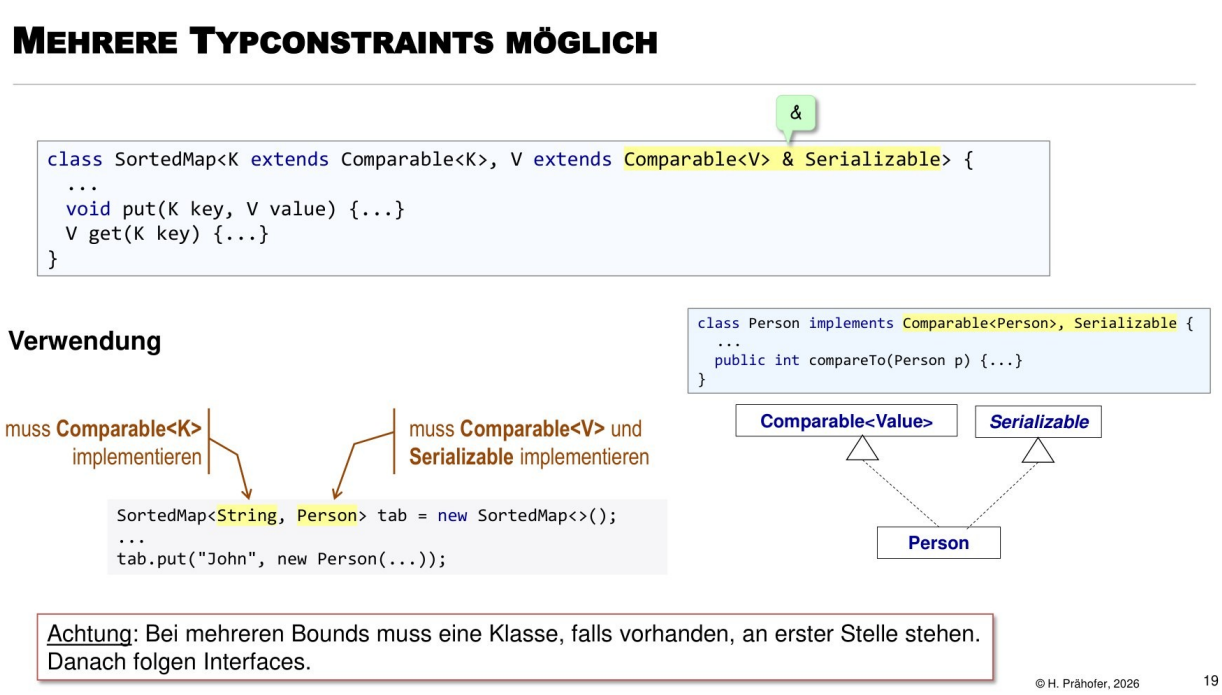
Vergleichsergebnis zugunsten des ersten Arguments. Die Variante `<T extends Comparable<T>>` ist für exakt passende Implementierungen geeignet; sie kann jedoch bei Unterklassen, die `Comparable` einer Oberklasse erben, zu eng sein. Die verbesserte Signatur mit `Comparable<? super T>` folgt später.

Originalabbildung: 05.Generics.pdf, Folie 17.

5.3.2 Mehrere Bounds und Typbeziehungen

G · Folie 18: MEHRERE TYPCONSTRAINTS MÖGLICH

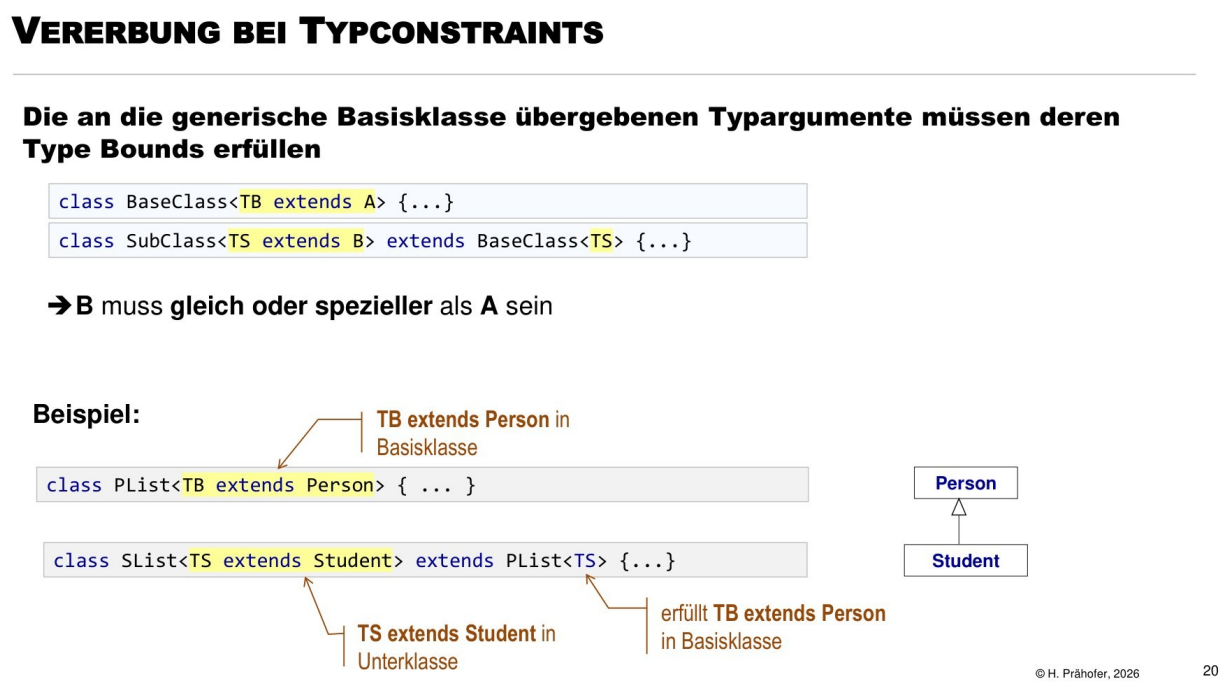
Mehrere Schranken werden mit dem Ampersand verknüpft: K muss `Comparable<K>` erfüllen, V gleichzeitig `Comparable<V>` und `Serializable`. Der Compiler erlaubt Methoden aus allen genannten Bounds. Steht eine konkrete Klasse unter den Bounds, muss sie zuerst erscheinen; danach folgen Interfaces. Hier ist `Serializable` ein Markerinterface. Ein Typargument, das nur `Comparable`, aber nicht `Serializable` implementiert, würde die zweite Anforderung nicht erfüllen.



Originalabbildung: 05.Generics.pdf, Folie 18.

G · Folie 19: VERERBUNG BEI TYPCONSTRAINTS

Die Vererbung von generischen Typen prüft die ursprünglichen Bounds erneut. Falls `PList<TB extends Person>` verlangt, darf `SList<TS extends Student>` den Typparameter TS weitergeben, weil `Student` `Person` erweitert und die Bedingung daher immer erfüllt bleibt. Dagegen wäre ein völlig uneingeschränktes TS nicht hinreichend. Diese Eigenschaft folgt aus der transitiven Java-Subtyprelation, nicht aus einer besonderen Laufzeitumwandlung.



Originalabbildung: 05.Generics.pdf, Folie 19.

Robusterer Bound für Comparable

```
public static <T extends Comparable<? super T>> T max(T a, T b) {
 return a.compareTo(b) >= 0 ? a : b;
}
```

Die Schlüsselidee ist nicht die Größe der Rückgabe, sondern die erlaubte Signatur der Vergleichsmethode: compareTo darf `Student` oder einen Obertyp wie `Person` annehmen. Ein naiver Vergleich durch Subtraktion ist bei Ganzzahlen wegen möglicher Überläufe zu vermeiden.

5.4 Invarianz, Ko- und Kontravarianz und Wildcards

Bei gewöhnlicher Vererbung ist `Student` ein Subtyp von `Person`. Daraus folgt jedoch gerade nicht, dass `List<Student>` ein Subtyp von `List<Person>` wäre. Generische Klassen wie eine veränderbare Liste sind bezüglich ihres Typarguments invariant. Würde eine `Student`-Liste als `Person`-Liste behandelt, könnte man über die vermeintliche `Person`-Liste einen `Professor` einfügen; beim anschließenden Lesen als `Student` scheiterte die Garantie. Die umgekehrte Zuweisung wäre ebenfalls gefährlich: Eine `Person`-Liste darf Professoren enthalten, die nicht als `Student` zurückgegeben werden dürfen.

Eine Wildcard ist das Fragezeichen `?` als Platzhalter für einen unbekannten Typargumenttyp. Bei `List<? extends Person>` ist das tatsächliche Typargument entweder `Person` oder ein Subtyp davon. Deshalb darf ein Element als `Person` gelesen werden, denn jeder zulässige Elementtyp ist mit `Person` kompatibel. Allerdings ist nicht bekannt, ob die Liste tatsächlich Students, Professors oder Persons enthält. Man darf über diese Referenz daher im Allgemeinen weder einen `Student` noch einen `Professor` noch eine beliebige `Person` einfügen. Die Ausnahme `null` ist erlaubt; außerdem sind Aufrufe möglich, die den fraglichen Typparameter nicht als neuen Wert benötigen.

Bei `List<? super Student>` ist der unbekannte Elementtyp dagegen `Student` selbst oder ein Supertyp wie `Person` oder `Object`. `Student`-Werte dürfen sicher eingefügt werden, weil jede dieser Listen sie aufnehmen kann. Ein beliebiges aus der Liste gelesenes Element ist jedoch nur sicher `Object`, nicht `Student`. Eine Zuweisung `Object o = list.get(0)` ist gültig; `Student s = list.get(0)` ohne Cast nicht. Diese Verwendung wird Kontravarianz genannt. Bei Java handelt es sich um Varianz der einzelnen `Verwendung (Use-site Variance)` durch Wildcards, nicht um eine uneingeschränkte Varianz der generischen Deklaration.

Daraus ergibt sich die praktische PECS-Regel: Producer Extends, `Consumer` Super. Eine Liste, aus der T-Werte produziert werden, kann häufig `<? extends T>` verwenden; eine Liste, die T-Werte konsumiert, `<? super T>`. Für eine Kopiermethode ist die allgemeine, typsichere Form `static <T> void copy(List<? extends T> from, List<? super T> to)`. In dieser Methode liefert die Quelle Werte, die als T gelesen werden können, und die Zielliste kann T-Werte aufnehmen. Sie akzeptiert daher das Kopieren von `List<Student>` in `List<Person>` und auch in `List<Object>`. Die Typvariable T verbindet beide Argumente.

PECS ist die zusammenfassende Regel „Producer Extends, `Consumer` Super“. Ein Quellparameter `List<? extends T>` erlaubt typisiertes Lesen als `T`; ein Zielparameter `List<? super T>` erlaubt sicheres Einfügen von `T`. Die flexible Kopiermethode kann daher so deklariert werden: `static <T> void copy(List<? extends T> source, List<? super T> target)`. Ihre Verwendung setzt voraus, dass die Methode das `List`-Interface mit passenden `get`- und `add`-Methoden verwendet; das Beispiel ist eine schematische Signatur für die Lehrbibliothek.

Die Schreibweise `List<?>` bedeutet einen unbekannten Elementtyp, nicht den festen Typ `Object`. Aus einer solchen Liste lässt sich ein Wert nur garantiert als `Object` entnehmen; in eine `List<?>` kann man im

Allgemeinen keinen beliebigen von `null` verschiedenen Wert einfügen. Eine `List<Object>` erlaubt dagegen das Einfügen aller Referenzobjekte. Beide Typen sind deshalb für APIs nicht austauschbar.

Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
static <T> void copy(java.util.List<? extends T> from,
 java.util.List<? super T> to) {
 to.addAll(from);
}
java.util.List<Integer> ints = java.util.List.of(1, 2);
java.util.List<Number> nums = new java.util.ArrayList<>();
copy(ints, nums);
```

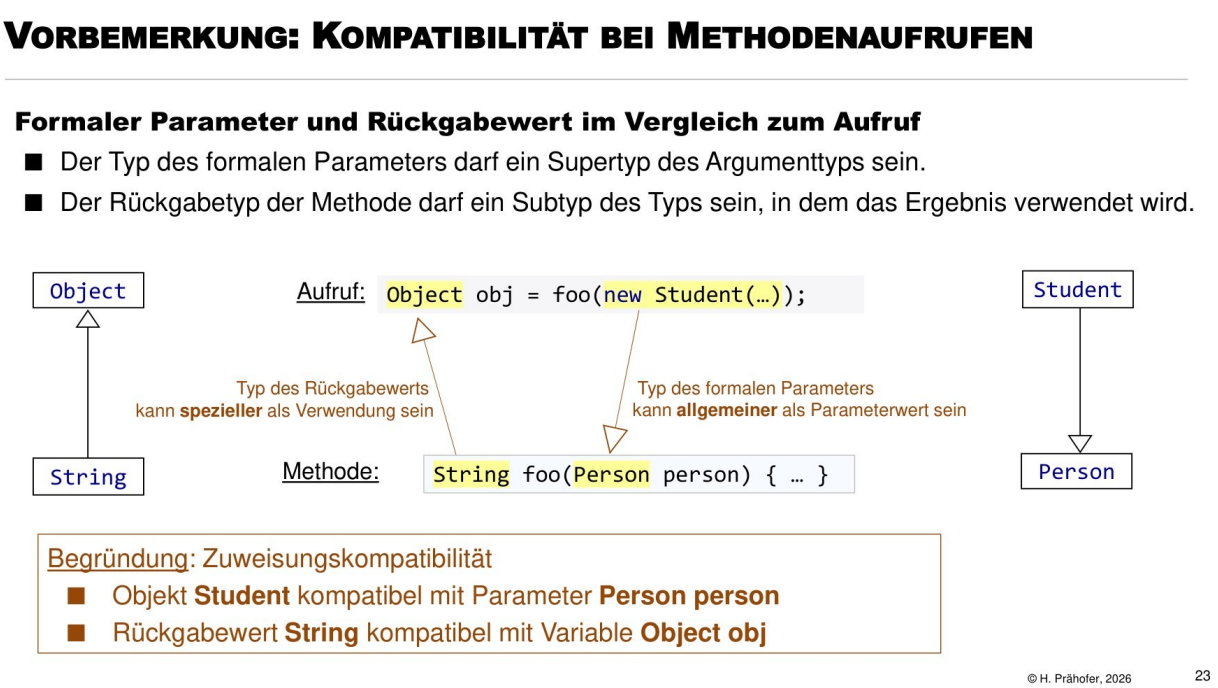
Originalfolien, Code und fachliche Einordnung

5.4.1 Grundregel der Methodenkompatibilität

Invariance beschreibt, dass der Inhaltstyp einer veränderbaren Liste nicht automatisch mit der Subtypbeziehung seines Arguments variiert. Dies lässt sich über Methodensignaturen unmittelbar herleiten: Die Eingabeposition von add verlangt mindestens eine geeignete Gegenrichtung zur Ausgabe von get. Wenn beide Funktionen im selben Interface vorhanden sind, kann eine unbeschränkte Zuweisung weder kovariant noch kontravariant sein.

G · Folie 22: VORBEMERKUNG: KOMPATIBILITÄT BEI METHODENAUFRUFEN

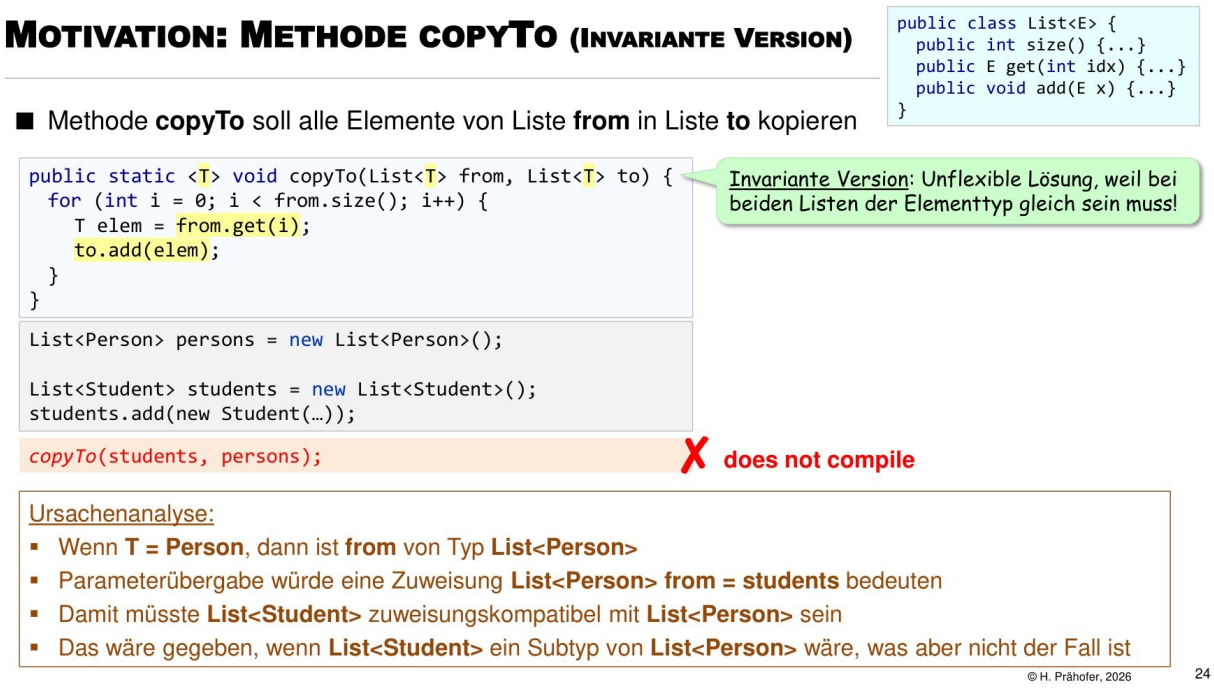
Die vorbereitende Typregel ist dieselbe wie bei gewöhnlichen Methoden: Als tatsächliches Argument darf ein Subtyp des formalen Parametertyps übergeben werden. Außerdem darf ein speziellerer Rückgabewert in einer allgemeineren Ergebnisvariable gespeichert werden. `Student` passt in `Person`, `String` passt in `Object`. Diese Zuweisungskompatibilität macht Wildcards verständlich, denn ihre Bound-Typen dienen als sichere Schranken für Ein- und Ausgaben.



Originalabbildung: 05.Generics.pdf, Folie 22.

G · Folie 23: MOTIVATION: METHODE COPYTO (INVARIANTE VERSION)

Die erste copyTo-Signatur verlangt zweimal exakt `List<T>`. Obwohl `Student` eine `Person` ist, kann der Compiler aus `List<Student>` und `List<Person>` im Allgemeinen kein einziges T gewinnen, das beide unveränderbaren Parametrisierungen gleichzeitig erfüllt. Das liegt an der Invarianz veränderbarer generischer Listen. Für den gewünschten Kopierfall muss die Quellposition den Subtyp zulassen oder die Zielposition einen Supertyp akzeptieren.

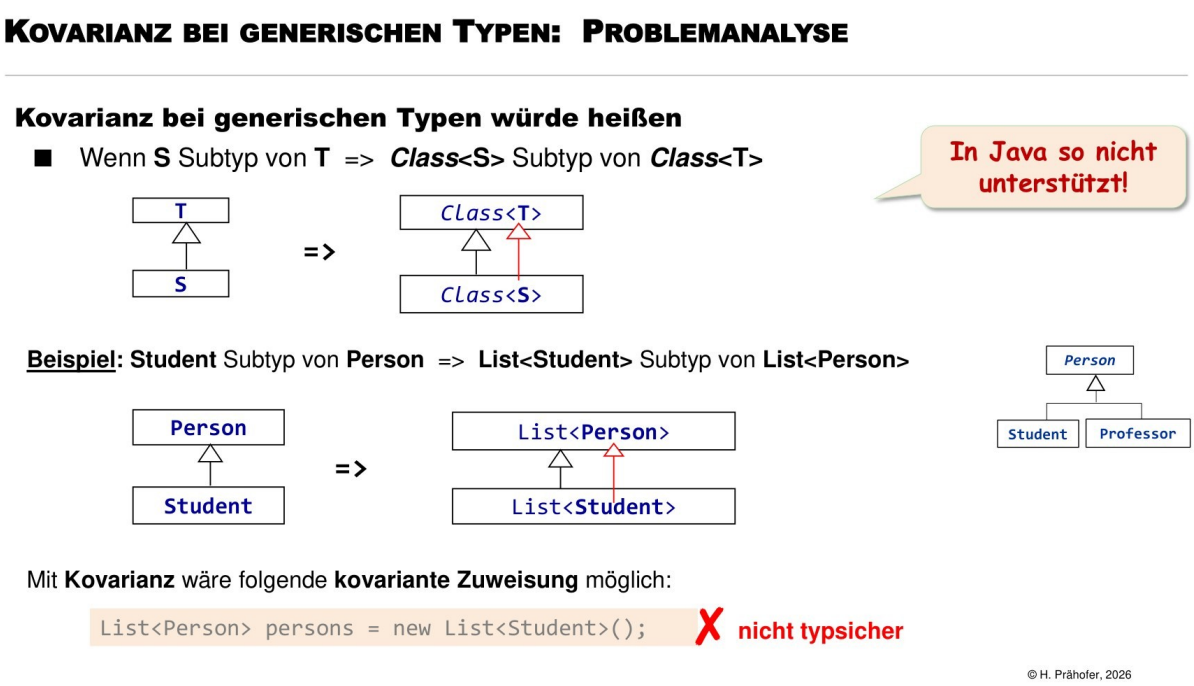


Originalabbildung: 05.Generics.pdf, Folie 23.

5.4.2 Warum veränderbare Generics invariant sind

G · Folie 24: KOVARIANZ BEI GENERISCHEN TYPEN: PROBLEMANALYSE

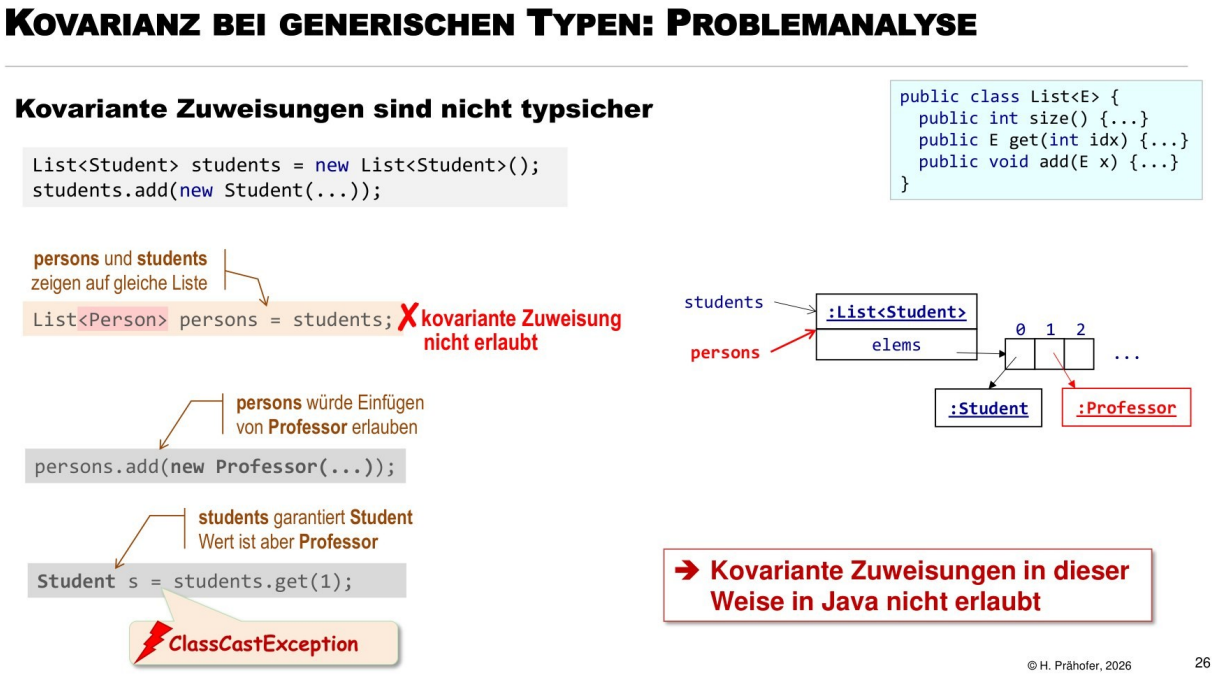
Kovarianz würde bedeuten, dass aus `Student` <: `Person` auch `List<Student>` <: `List<Person>` folgt. Dies gilt für eine normale veränderbare `List<E>` gerade nicht. Als reine Leseschnittstelle wäre eine kovariante Nutzung zwar sinnvoll, aber dieselbe Liste bietet hier auch `add(E)` an. Die Folie zeichnet die hypothetischen Hierarchiepfeile und verdeutlicht, dass die Java-Sprachregel eine unsichere allgemeine Zuweisung verhindert.



Originalabbildung: 05.Generics.pdf, Folie 24.

G · Folie 25: KOVARIANZ BEI GENERISCHEN TYPEN: PROBLEMANALYSE

Warum die hypothetische Kovarianz unsicher wäre: Zwei Variablen würden dieselbe `Student`-Liste referenzieren. Durch eine vermeintliche `List<Person>`-Sicht könnte man einen `Professor` einfügen. Anschließend würde der `Student`-lesende Code eine `Student`-Referenz erwarten, obwohl ein `Professor` gespeichert ist. Damit wäre die statische Zusicherung verletzt. Die gefährliche Anweisung `List<Person> persons = students;` wird deshalb vom Compiler zurückgewiesen.



Originalabbildung: 05.Generics.pdf, Folie 25.

5.4.3 Upper-bounded Wildcards: lesender Zugriff

Ein upper-bounded Wildcard-Typ ist nicht identisch mit `List<Person>`. Beispielsweise kann die Liste tatsächlich `List<Student>` sein. Für die Rückgabe von get lässt sich die unbekannte Subtypinformation gefahrlos zu `Person` verallgemeinern. Bei add genügt diese Information nicht; nur `null` ist als Referenzwert universell einsetzbar, falls die konkrete API Nullwerte gestattet.

G · Folie 26: UPPER-BOUNDED WILDCARD

`List<? extends Person>` bezeichnet eine Liste mit unbekanntem Elementtyp, der `Person` oder ein Untertyp ist. Dadurch darf eine `List<Student>` der Wildcard-Variablen zugewiesen werden. `<?>` ist die unbeschränkte Form und bei ungebundenen generischen Typen sinngemäß `<? extends Object>`. Das Fragezeichen wird nicht als neuer Typname mit eigener Klassendeklaration erzeugt; es steht für einen unbekannten, aber durch die Schranke begrenzten Typargumenttyp.

Originalabbildung: 05.Generics.pdf, Folie 26.

G · Folie 27: UPPER-BOUNDED WILDCARD

Aus einer `List<? extends Person>` darf get einen Wert liefern, der sicher als `Person` verwendet werden kann. Beim Einfügen über diese Sicht kann der Compiler aber nicht entscheiden, welcher konkrete Untertyp tatsächlich gespeichert wird; selbst `new Person()` ist daher im Allgemeinen nicht zulässig. `add(null)` bleibt eine Sondermöglichkeit, sofern die API `null` erlaubt. Die Aussage „nur lesend“ ist eine hilfreiche Kurzregel für typabhängige Elemente, aber nicht das generelle Verbot aller Methoden mit Parametern.

Upper-bounded Wildcard

Upper-bounded Wildcard erlaubt nur lesende Zugriffe

```
public class List<E> {
  public int size() {...}
  public E get(int idx) {...}
  public void add(E x) {...}
}
```

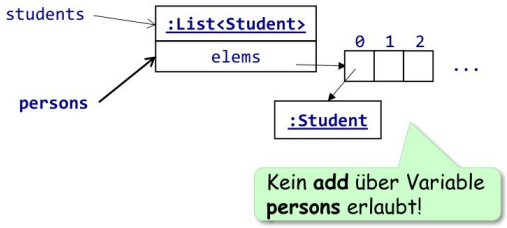
```
List<Student> students = new List<Student>();
students.add(new Student(..));
```

```
List<? extends Person> persons = students;
```

- Methoden mit **generischem Rückgabewert** sind **typsicher** und erlaubt

```
Person p = persons.get(0);
```

persons.get garantiert **Person**, **Student** kompatibel mit **Person**



- aber Methoden mit **generischem Parameter** werden vom **Compiler verboten**

```
persons.add(new Professor(...));
```

✗

extendsPerson.add würde auch **Professor** erlauben, ist aber nicht kompatibel mit **Student**

© H. Prähofer, 2026 28

Originalabbildung: 05.Generics.pdf, Folie 27.

G · Folie 28: COPYTO: KOVARIANTE VERSION

Die kovariante copyTo-Version verwendet `<T> void copyTo(List<? extends T> from,List<T> to)`. Bei students nach persons kann T als `Person` gewählt werden; die Quelle hat dann den zulässigen Typ `List<? extends Person>`. Das von get gelesene `Student`-Objekt ist eine `Person` und kann daher in das Ziel eingefügt werden. Noch allgemeiner ist die Kombination mit einem kontravarianten Ziel: `List<? super T>`. Das Ergebnis ist ein typsicheres Kopieren über Typgrenzen.

copyTo: Kovariante Version

copyTo: Lösung mit **Upper-Bounded Wildcard**

```
public static <T> void copyTo(List<? extends T> from, List<T> to) {
  for (int i = 0; i < from.size(); i++) {
    T elem = from.get(i);
    to.add(elem);
  }
}
```

from *kovariant*

Lesen von *kovarianter Liste* **from**

- Anwendung beim Kopieren von **students** zu **persons**

```
List<Student> students = new List<Student>();
```

```
List<Person> persons = new List<Person>();
```

```
copyTo(students, persons);
```

Typinferenz:

1) Ableitung von Typparameter **T** aus aktuellem Argument **persons** für zweiten Parameter **to**:
   - ➔ **persons** ist von Typ **List<Person>**
   - ➔ **T** = **Person**
   - ➔ **from** ist von Typ **List<? extends Person>**

2) Überprüfen des aktuellen Parameters **students** für ersten Argument **from**:
   - ➔ **students** ist von Typ **List<Student>**
   - ➔ Damit wird Wildcard **?** zu **Student**
   - ➔ und Wildcard-Constraint **Student extends Person** ist erfüllt

© H. Prähofer, 2026 29

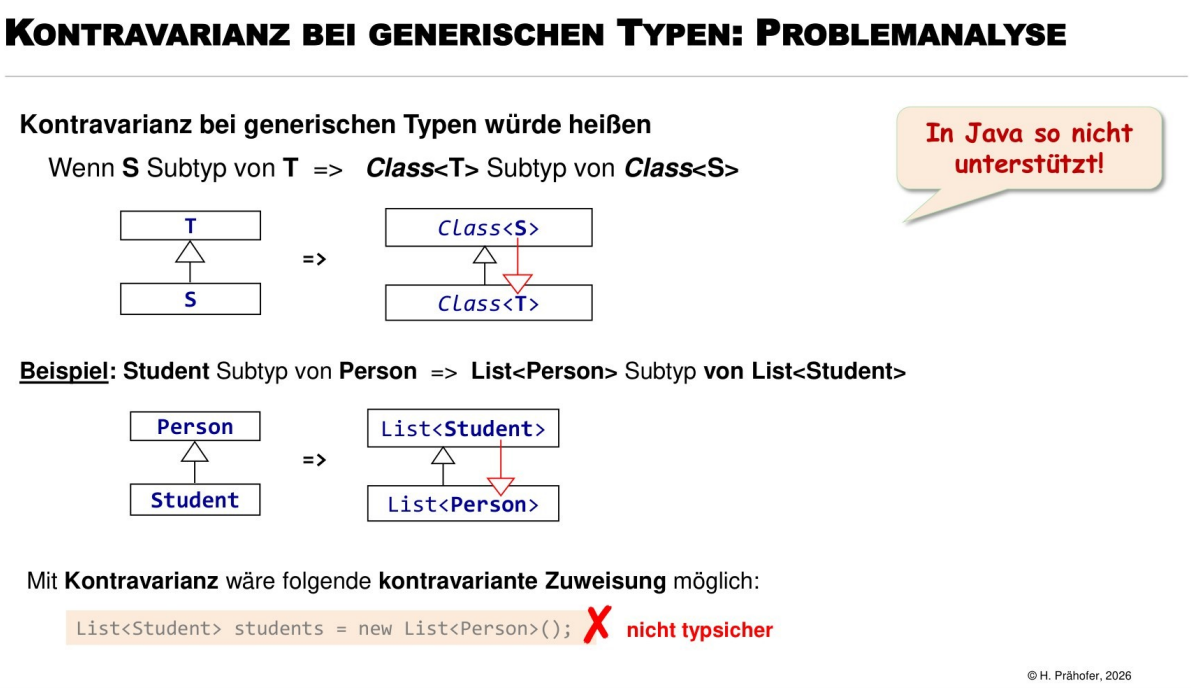
Originalabbildung: 05.Generics.pdf, Folie 28.

5.4.4 Kontravarianz und Lower-bounded Wildcards

Eine kontravariante Sicht ist kein nachträglicher Cast und ändert kein Objekt. Sie sagt vielmehr: Der unbekannte Elementtyp ist eine Oberklasse oder das angegebene T selbst. Darum darf man T einfügen, aber bei get den Typ nur als `Object` sicher annehmen. Eine vollständige Variable `List<? super Student>` kann also sowohl auf `List<Student>` als auch auf `List<Person>` zeigen.

G · Folie 29: KONTRAVARIANZ BEI GENERISCHEN TYPEN: PROBLEMANALYSE

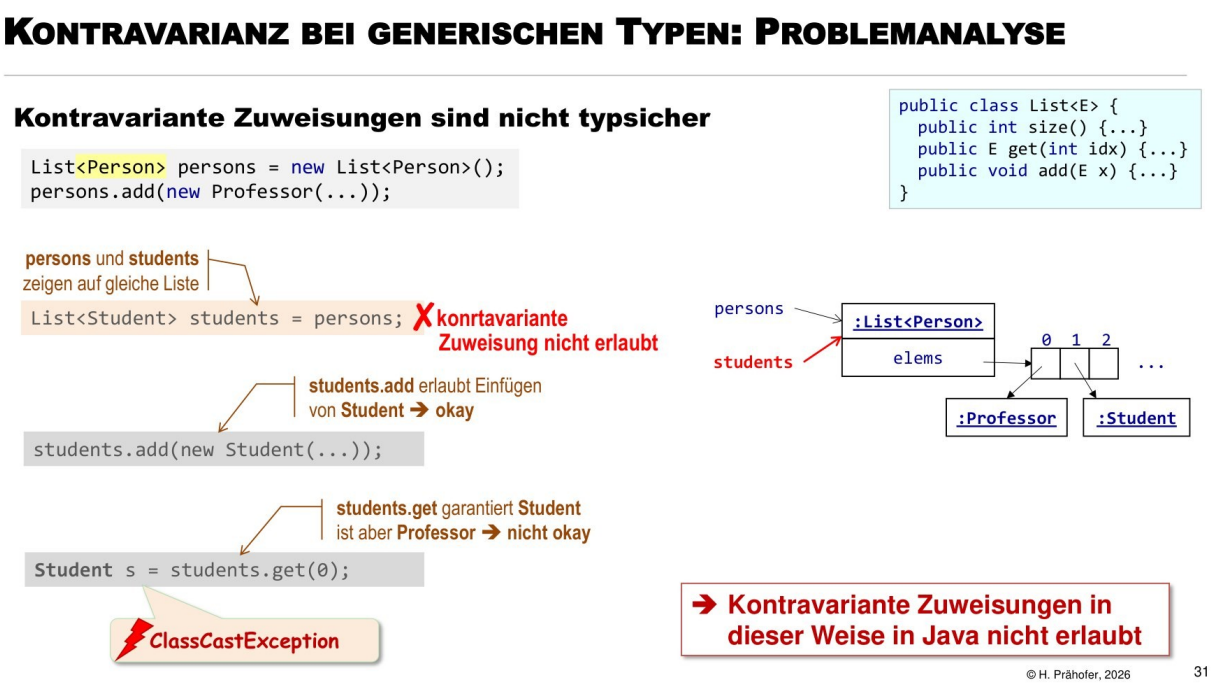
Die umgekehrte hypothetische Beziehung wäre Kontravarianz: Aus `Student` <: `Person` sollte `List<Person>` <: `List<Student>` folgen. Die Java-Sprache lässt diese Zuweisung für eine gewöhnliche generische Liste ebenso wenig zu wie die kovariante Richtung. Das Problem verlagert sich beim Lesen: Eine `Person`-Liste kann Objekte enthalten, die keine `Student`-Objekte sind, obwohl eine `List<Student>`-Variable beim get genau diesen Typ verspricht.



Originalabbildung: 05.Generics.pdf, Folie 29.

G · Folie 30: KONTRAVARIANZ BEI GENERISCHEN TYPEN: PROBLEMANALYSE

Die Folie demonstriert die gescheiterte kontravariante Zuweisung mit `Professor` und `Student` in derselben `Person`-Liste. Zwar könnte man einen `Student` gefahrlos hinzufügen, aber beim Auslesen der ersten Position würde der Code fälschlich einen `Student` erwarten, tatsächlich aber einen `Professor` erhalten. Ohne Wildcard schützt der Compiler den Vertrag von get und weist `List<Student> students = persons` zurück.



Originalabbildung: 05.Generics.pdf, Folie 30.

G · Folie 31: LOWER-BOUNDED WILDCARD

`List<? super Student>` erlaubt eine Referenz auf eine `List<Student>`, `List<Person>` oder `List<Object>`. Die tatsächliche Liste bleibt dieselbe; die Wildcard ändert weder die gespeicherten Werte noch den Laufzeittyp. Der Compiler stellt jedoch nur Operationen zur Verfügung, die für jeden möglichen oberen Elementtyp sicher sind. Ein `Student` darf eingefügt werden, weil alle denkbaren Elementtypen `Student` aufnehmen.

Originalabbildung: 05.Generics.pdf, Folie 31.

G · Folie 32: LOWER-BOUNDED WILDCARD

Beim Einfügen über `<? super Student>` ist `add(new Student(...))` typsicher. Beim Lesen darf get sehr wohl aufgerufen werden, liefert aus Sicht des Compilers aber nur `Object`. Die Formulierung auf der Folie „Kein get über

Variable erlaubt“ ist daher zu stark: Verboten ist nur das direkte Zuweisen des Ergebnisses an `Student` ohne weitere Prüfung oder Cast. Die Unterscheidung ist wichtig für Verständnis und Prüfungsaufgaben.

LOWER-BOUNDED WILDCARD

Lower-bounded Wildcards erlauben das typsichere Einfügen

Beim Lesen ist lediglich Object als Rückgabetyp garantiert

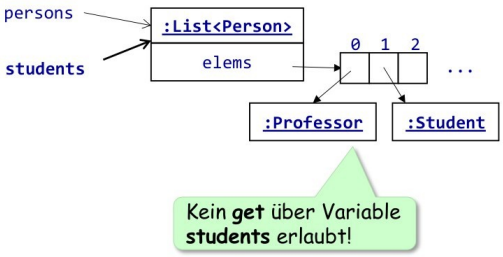
```
public class List<E> {
  public int size() {...}
  public E get(int idx) {...}
  public void add(E x) {...}
}
```

```
List<Person> persons = new List<Person>();
persons.add(new Professor(...));
List<? super Student> students = persons;
```

- Methoden mit generischen Parameter sind typsicher und erlaubt

```
students.add(new Student(...));
```

students.add erlaubt **Student**, ist kompatibel mit **Person**



- Lesen mit spezifischem Typ nicht möglich

```
Student s = students.get(0);
```

students.get verboten, weil **Student** nicht garantiert werden kann!

© H. Prähofer, 2026 33

Originalabbildung: 05.Generics.pdf, Folie 32.

G · Folie 33: COPYTO: KONTRAVARIANTE VERSION

Die kontravariante copyTo-Version nutzt `<T> void copyTo(List<T> from,List<? super T> to)`. Für `Student`-Quelle und `Person`-Ziel ist T=`Student`, und `Person` ist ein Supertyp von `Student`. Die Prüfklausel im Folienkasten enthält einen Schreibfehler: Richtig ist `Person super Student`, nicht `Person super Person`. Die Flexibilität gilt für die schreibende Zielposition, während die Quelle in dieser Variante invariant bleibt.

COPYTO: KONTRAVARIANTE VERSION

copyTo: Lösung mit **Lower-Bounded Wildcard**

```
public static <T> void copyTo(List<T> from, List<? super T> to) {
  for (int i = 0; i < from.size(); i++) {
    T elem = from.get(i);
    to.add(elem);
  }
}
```

to *kontravariant*

Schreiben auf *kontravariante Liste* **to**

- Anwendung beim Kopieren von **students** zu **persons**

```
List<Student> students = new List<Student>();
List<Person> persons = new List<Person>();
copyTo(students, persons);
```

Typinferenz:

1) Ableitung von Typparameter **T** aus aktuellem Argument **students** für ersten Parameter **from**:
   - ➔ **students** ist von Typ **List<Student>**
   - ➔ **T** = **Student**
   - ➔ **to** ist von Typ **List<? super Student>**

2) Überprüfen des aktuellen Argument **persons** für zweiten Argument **to**:
   - ➔ **persons** ist von Typ **List<Person>**
   - ➔ Damit wird Wildcard **?** zu **Person**
   - ➔ und Wildcard-Constraint **Person super Person** ist erfüllt

© H. Prähofer, 2026 34

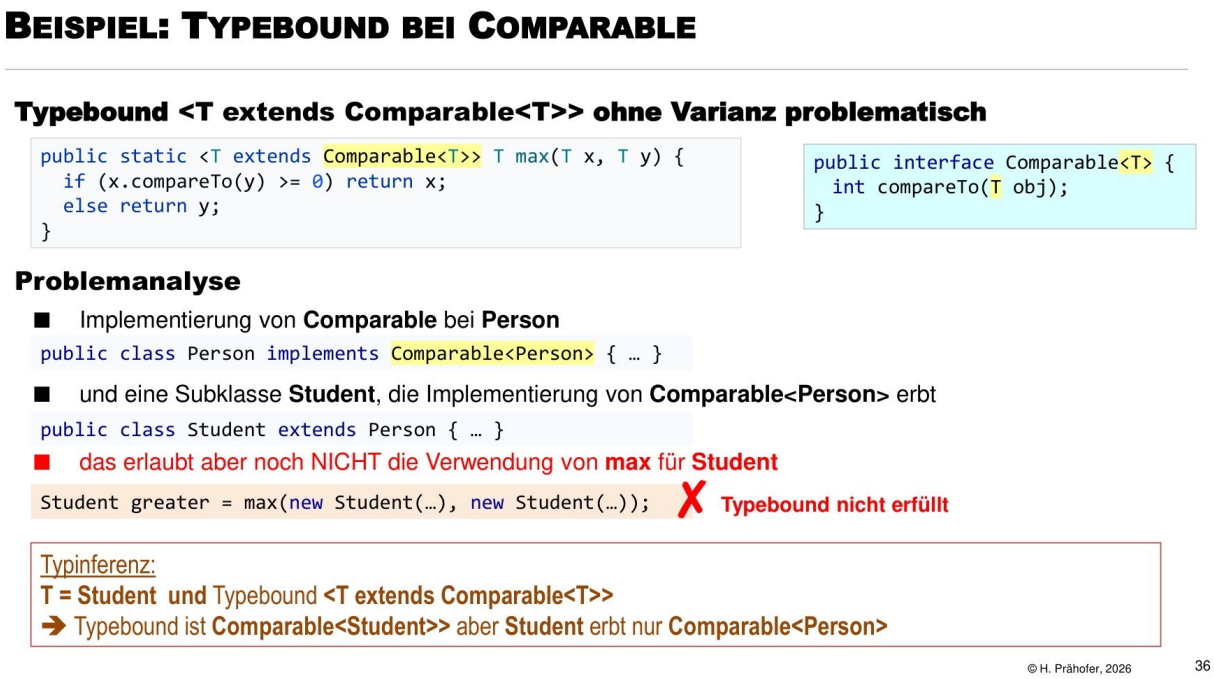
Originalabbildung: 05.Generics.pdf, Folie 33.

5.4.5 Comparable<? super T> und flexible Ordnungen

Hier sieht man, weshalb die beiden Wildcard-Richtungen zusammengehören. Wenn `Student` das Interface `Comparable<Person>` geerbt hat, ist die Vergleichsmethode über `Person` typisiert. Ein `Student` ist als Argument einer solchen Methode zulässig. Der generische Bound muss deshalb ausdrücklich eine Vergleichsoperation mit `Student` oder einem Supertyp davon zulassen: `Comparable<? super Student>`.

G · Folie 34: BEISPIEL: TYPEBOUND BEI COMPARABLE

`Comparable<T>` eines Oberklassentyps ist nicht automatisch `Comparable<Student>`. `Person` implementiert `Comparable<Person>`; `Student` erbt diese Methode und kann damit andere Studenten vergleichen, weil sie Personen sind. Die strenge Schranke `<T extends Comparable<T>>` passt jedoch nicht, wenn das erwartete Ergebnis `Student` ist. Dass Java-Typinferenz zusätzlich vom Rückgabekontext abhängen kann, ändert nichts an der fehlenden Schrankenbeziehung für T=`Student`.



Originalabbildung: 05.Generics.pdf, Folie 34.

G · Folie 35: BEISPIEL: TYPEBOUND BEI COMPARABLE

Die verbesserte Schranke `<T extends Comparable<? super T>>` erlaubt eine compareTo-Methode, die T selbst oder einen Obertyp als Vergleichsargument annimmt. Für T=`Student` erfüllt `Comparable<Person>` diese Bedingung, da `Person` ein Obertyp von `Student` ist. Daher funktioniert die generische max-Methode auch für zwei Studenten, ohne eine neue `Comparable<Student>`-Implementierung zu fordern. Diese Variante ist eine wichtige Verbindung von Bounds und Lower-bounded Wildcards.

Beispiel: Typebound bei Comparable

Lösung: Lower-Bounded Wildcard Comparable<? super T>

```
public static <T extends Comparable<? super T>> T max(T x, T y) {
  if (x.compareTo(y) >= 0) return x;
  else return y;
}
```

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

Begründung

- Implementierung von **Comparable** bei **Person**

```
public class Person implements Comparable<Person> { … }
```

- und eine Subklasse **Student**, die Implementierung von **Comparable<Person>** erbt

```
public class Student extends Person { … }
```

- mit Lower-Bounded Wildcard ist **SortedList<Student>** typ-korrekt

```
Student greater = max(new Student(…), new Student(…));
```

Typinferenz:
T = Student und Typebound **<T extends Comparable<? super T>>**
➔ **Student** erbt **Comparable<Person>** ➔ Typebound **Comparable<Person super Student>>** ist erfüllt

© H. Prähofer, 2026 37

Originalabbildung: 05.Generics.pdf, Folie 35.

G · Folie 36: BEISPIEL: TYPEBOUND BEI COMPARABLE BEI SORTEDSET

Die gleiche Korrektur gilt für eine sortierte Sammlung: `SortedList<T extends Comparable<? super T>>` akzeptiert `SortedList<Student>`, wenn `Student` `Comparable<Person>` erbt. Die Folie überschreibt im Titel versehentlich SortedSet, während im Code SortedList gemeint ist. Beim eigentlichen Einfügen vergleichen die Elemente weiterhin ihre natürliche Reihenfolge; die Wildcard ist ausschließlich Teil der statischen Zulässigkeitsbedingung.

Beispiel: Typebound bei Comparable bei SortedSet

Lower-Bounded Wildcard bei Comparable

```
public class SortedList<T extends Comparable<? super T>> {
  …
}
```

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

- Mit **Person** implementiert **Comparable<Person>** erbt

```
public class Person implements Comparable<Person> { … }
```

- und **Student** erbt **Comparable<Person>**

```
public class Student extends Person { … }
```

- Erlaubt SortedSet<Student>

```
SortedList<Student> sortedStudents = new SortedList<>();
```

Typinferenz:
Typebound **<T extends Comparable<? super T>>**
➔ **T = Student** ➔ Typebound ist **<Student extends Comparable<? super Student>>**
➔ **Student** erbt **Comparable<Person>** ➔ Typebound **Student extends Comparable<Person super Student>** ist erfüllt

© H. Prähofer, 2026 38

Originalabbildung: 05.Generics.pdf, Folie 36.

Die flexible Copy-Signatur als zentrale Herleitung

```
static <T> void copyTo(
 java.util.List<? extends T> from,
 java.util.List<? super T> to) {
 for (T elem : from) {
 to.add(elem);
 }
}
java.util.List<Integer> ints = java.util.List.of(1, 2, 3);
java.util.List<Number> nums = new java.util.ArrayList<>();
copyTo(ints, nums); // erlaubt: Integer ist ein Number
```

Die JDK-Listen in diesem Zusatzbeispiel werden ausdrücklich vollqualifiziert bezeichnet. `from` produziert Elemente eines unbekannten Untertyps von T, `to` konsumiert T beziehungsweise dessen Untertypen. Die konkrete Ermittlung von T hängt von beiden Methodenargumenten ab.

5.5 Type Erasure, Rohtypen und Laufzeitgrenzen

Java verwendet beim Übersetzen der meisten generischen Typinformationen Type `Erasure (Typlöschung)`. Ein unbeschränkter Typparameter E wird in der Laufzeitimplementierung grundsätzlich auf `Object` zurückgeführt; bei einem Bound wie `Comparable` wird dessen erste Schranke zum Erasure-Typ. Der Compiler überprüft vorab die Typregeln, fügt nötige Casts an geeigneten Zugriffen ein und kann synthetische Bridge-Methoden zur Aufrechterhaltung polymorpher Überschreibung erzeugen. Generics führt damit nicht pro Typargument automatisch eine eigene ausführbare Liste-Klasse ein. `List<String>` und `List<Person>` haben normalerweise dieselbe Laufzeitklasse.

Ein `Rohtyp (raw type)` schreibt den Namen einer generischen Klasse ohne Typargument, beispielsweise `List` statt `List<String>`. Er ist vor allem für Quellkompatibilität mit älterem Java-Code vorgesehen. Über Rohtypen wird die normale generische Sicherheit eingeschränkt; der Compiler signalisiert gefährliche Operationen als unchecked warnings. Rohtypen sind nicht mit `List<?>` zu verwechseln: Die Wildcard respektiert die unbekannte Elementtypinformation und verhindert unsichere Einfügungen, während der Rohtyp viele Prüfungen umgeht.

Konkrete Typargumente wie `String` lassen sich zur Laufzeit bei einem normalen parametrisierten `List`-Objekt nicht durch `instanceof List<String>` prüfen. Dagegen ist `instanceof List<?>` möglich, weil es nur die reifizierbare Laufzeitklasse prüft. Ein Cast auf `List<String>` kann prinzipiell geschrieben werden, ist aber nicht vollständig zur Laufzeit prüfbar; dabei entsteht gewöhnlich eine unchecked Warnung. Zur Erläuterung der Folie gilt: Nicht alle Typinformationen verschwinden restlos – generische Deklarationen können etwa als Metadaten für Reflection verfügbar bleiben. Aus dem gewöhnlichen Objekt allein lässt sich der konkrete Inhaltstyp damit trotzdem nicht zuverlässig ableiten.

Da Java-Typparameter Referenztypen verlangen, ist `List<int>` ungültig; stattdessen steht `List<Integer>` mit Auto-Boxing zur Verfügung. Ebenfalls darf eine Klasse, die direkt oder indirekt Throwable erweitert, nicht selbst einen Typparameter deklarieren, denn `Exception`-Typprüfungen in `catch`-Klauseln können zwischen erased Parametrisierungen nicht unterscheiden.

Bei Type Erasure ersetzt der Compiler Typparameter typischerweise durch deren erste obere Schranke beziehungsweise bei fehlender Schranke durch `Object`. Zum Beispiel wird ein Parameter `T` aus `class Box<T>` im erasure-basierten Laufzeitmodell grundsätzlich als `Object` behandelt. Deshalb können zwei

Objekte derselben generischen Klasse mit verschiedenen Typargumenten nicht einfach durch `instanceof Box<String>` und `instanceof Box<Integer>` unterschieden werden. Ein Test `obj instanceof Box<?>` ist dagegen erlaubt.

Zur Aufrechterhaltung polymorpher Methodenüberschreibung können vom Compiler Bridge-Methoden erzeugt werden. Sie gleichen Signaturen an, die durch Erasure voneinander abweichen würden. Das zeigt, dass die semantische Typsicherheit im Quelltext nicht allein durch die Laufzeitklassennamen sichtbar ist. Rohtypen wie `List` ohne `<...>` sind für die Kompatibilität mit älterem Code vorhanden, unterlaufen aber Teile der generischen Prüfung und verursachen bei gefährlichen Operationen Compilerwarnungen.

Erasure bedeutet nicht, dass aus jeder Klassendatei sämtliche Hinweise auf generische Deklarationen verschwinden: Bestimmte Signaturmetadaten können mit Reflection gelesen werden. Ein konkretes Objekt einer gewöhnlichen `ArrayList<String>` trägt jedoch keinen verlässlich unterscheidbaren Laufzeittyp `ArrayList<String>` gegenüber `ArrayList<Integer>`.

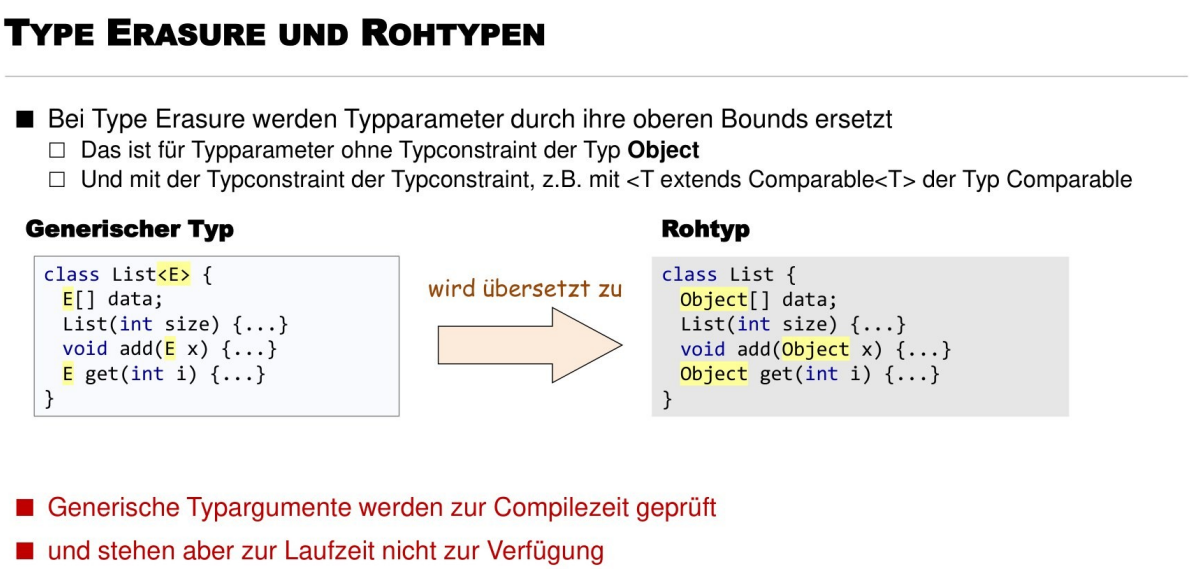
Originalfolien, Code und fachliche Einordnung

5.5.1 Erasure, Laufzeitklassen und rohe Typen

Erasure bedeutet nicht, dass beim Ausführen alle Typüberprüfungen fehlen. Der Compiler erzeugt bei Nutzung eines parametrisierten Rückgabetyps gegebenenfalls einen dynamischen Cast auf den erwarteten konkreten Referenztyp. Deshalb kann inkorrekt über einen Rohtyp eingefügter Inhalt auch bei einem scheinbar castfreien `Student s = list.get(0)` eine `ClassCastException` auslösen.

G · Folie 39: TYPE ERASURE UND ROHTYPEN

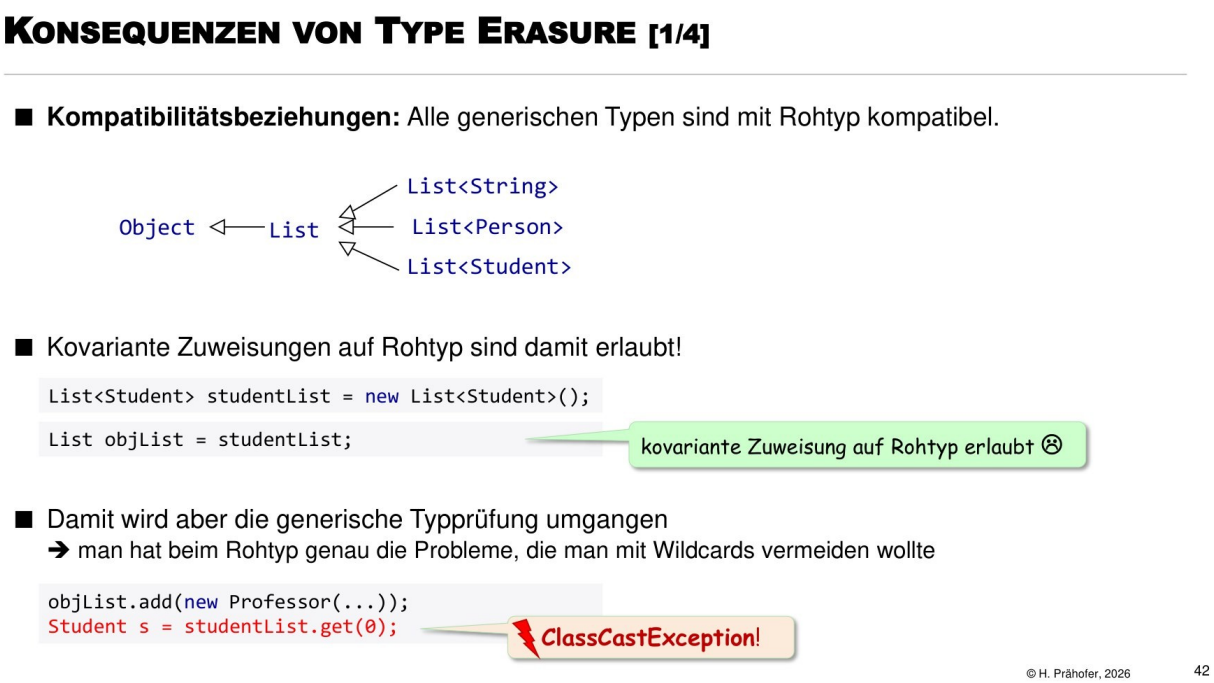
Nach Type Erasure wird ein unbeschränktes E zur Laufzeit grundlegend als `Object` behandelt; bei einem Bound wie `Comparable` ist der erste Bound ausschlaggebend. Die auf der Folie gezeigte Ersetzung generischer Signaturen durch `Object`-basierte ist eine Vereinfachung der Übersetzung. Der Compiler kann außerdem Casts und Bridge-Methoden einfügen. Wichtig bleibt, dass nicht für `List<String>` und `List<Person>` getrennte VM-Klassen entstehen.



Originalabbildung: 05.Generics.pdf, Folie 39.

G · Folie 40: KONSEQUENZEN VON TYPE ERASURE [1/4]

Ein Rohtyp `List` verzichtet auf den Typargumentvertrag. Ein mit `List<Student>` erzeugtes Objekt darf einer Rohtyp-Variablen zugewiesen werden; über diese kann aber eine unerwünschte `Professor`-Instanz eingefügt werden. Der Compiler gibt hierzu bei gefährlichen Methoden gewöhnlich eine unchecked warning aus. Beim späteren get in `Student`-Kontext droht dann eine `ClassCastException`. Deshalb sollen Rohtypen normalerweise vermieden und `List<?>` bevorzugt werden.



Originalabbildung: 05.Generics.pdf, Folie 40.

5.5.2 Typprüfung und Casting zur Laufzeit

G · Folie 41: KONSEQUENZEN VON TYPE ERASURE [2/4]

`obj instanceof List<String>` ist bei gewöhnlichen parametrisierten Typen nicht erlaubt, weil `String` als Typargument zur Laufzeit nicht überprüfbar ist. `instanceof List<?>` ist zulässig, denn überprüft wird nur die Laufzeitklasse der Liste. Auch `List<String> list = (List<String>) obj` ist syntaktisch möglich, aber nicht voll überprüfbar und führt zu einer unchecked warning. Reflection kann generische Signaturen aus Deklarationsmetadaten lesen; das beweist nicht den Inhalt eines konkreten Listenobjekts.

KONSEQUENZEN VON TYPE ERASURE [2/4]

Laufzeitprüfungen funktionieren bei Generics nicht

```
Object obj = new List<String>();
```

- Prüfen mit generischem Typ nicht möglich

```
if (obj instanceof List<String>) ...
```

✗ **Compilefehler**: VM kann nicht prüfen, ob **obj** vom Typ **List<String>** ist

- Prüfen mit **<?>** erlaubt

```
if (obj instanceof List<?>) ...
```

Bevorzugt gegenüber Rohtyp

- Prüfen auf Rohtyp erlaubt

```
if (obj instanceof List) ...
```

Rohtyp

- Typecast erlaubt, aber mit Warning

```
List<String> list = (List<String>) obj;
```

Warnung: VM garantiert, dass **obj** vom Typ **List** ist, kann aber nicht garantieren, dass **obj** vom Typ **List<String>** ist

© H. Prähofer, 2026 43

Originalabbildung: 05.Generics.pdf, Folie 41.

G · Folie 42: KONSEQUENZEN VON TYPE ERASURE [3/4]

Primitive Typen wie `int` sind keine zulässigen Java-Typargumente. Für Ganzzahlen wird `Integer` benutzt, für `double` `Double` usw. Beim Einfügen `ints.add(43)` wandelt Auto-Boxing den `int`-Wert in einen `Integer` um. Auch beim späteren Lesen in eine primitive Variable kann automatisch Unboxing erfolgen. Dabei sind die Kosten für Objektrepräsentation, mögliche NullPointerExceptions beim Unboxing und weitere Laufzeiteffekte zu beachten.

KONSEQUENZEN VON TYPE ERASURE [3/4]

Basisdatentypen können nicht für Typparameter verwendet werden

- Basisdatentypen sind nicht kompatibel mit **Object** des Rohtyps.

```
List<int> ints = new List<int>();
```

✗ **Compilefehler**: **int** kann nicht für Typparameter verwendet werden

- Man muss geboxten Referenztypen verwenden, z.B. mit **Integer**

```
List<Integer> ints = new List<Integer>();
ints.add(43);
```

geboxte Werte weniger effizient!

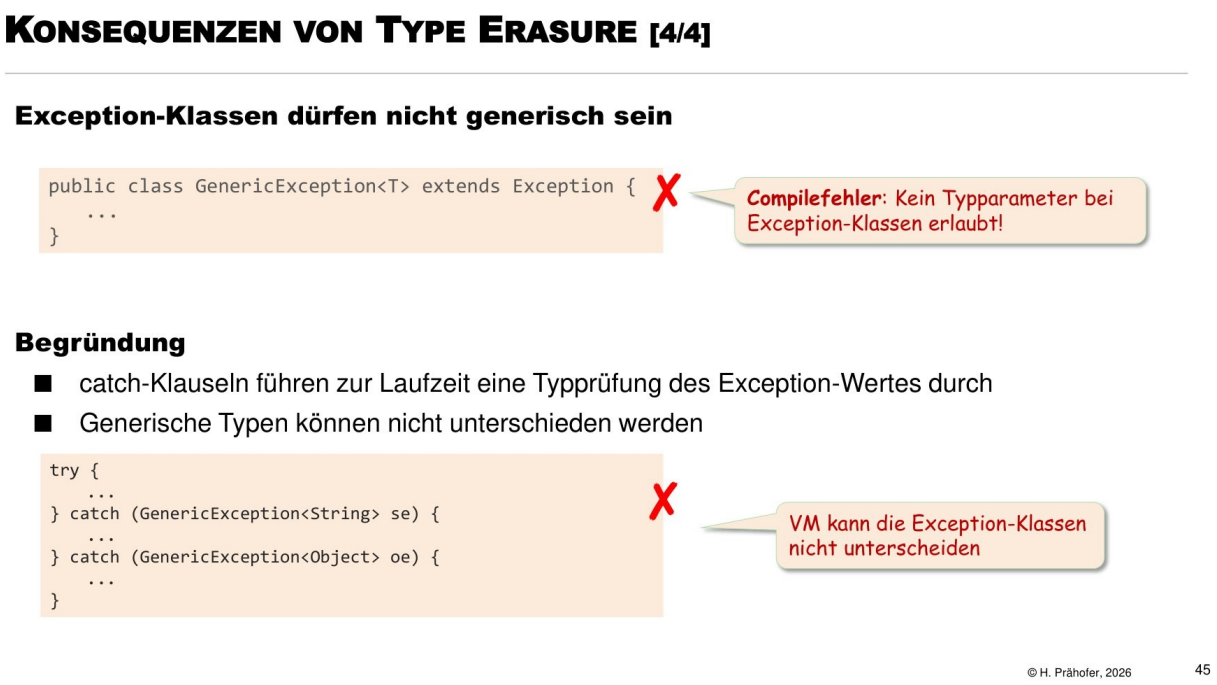
© H. Prähofer, 2026 44

Originalabbildung: 05.Generics.pdf, Folie 42.

G · Folie 43: KONSEQUENZEN VON TYPE ERASURE [4/4]

Eine Klasse, die Throwable als Oberklasse hat, darf selbst nicht generisch sein: `GenericException<T> extends Exception` ist ein Compilerfehler. Der Grund ist die Laufzeitunterscheidung von Exceptionklassen in `catch`-Klauseln

und die gleichzeitige Erasure konkreter Typargumente. Man kann allerdings in einer generischen Klasse eine nicht generische `Exception`-Klasse verwenden oder für verschiedene Fehlerarten getrennte nicht generische `Exception`-Typen definieren.



Originalabbildung: 05.Generics.pdf, Folie 43.

Kompilier- und Laufzeitprüfung auseinanderhalten

```
Object obj = java.util.List.of("A");
if (obj instanceof java.util.List<?>) {
 System.out.println("Liste vorhanden");
}
// obj instanceof java.util.List<String> // nicht zulässig
// java.util.List raw = ... // Rohtyp: vermeiden
```

Die Prüfung kann feststellen, dass obj ein `List`-Objekt ist, nicht aber allein aus dessen Laufzeitklasse folgern, dass nur Strings gespeichert sind. Die Methode `List.of` der Standardbibliothek erzeugt eine unveränderliche Liste; sie ist nicht die gleichnamige Lehr-Factory aus der eigenen `List`-Fallstudie.

5.6 Arrays, Kovarianz und generische Arraytypen

Ein Array besitzt einen Laufzeitkomponententyp. Die Typprüfung für das Einfügen eines neuen Elements erfolgt auch zur Laufzeit: Ein tatsächliches `Student[]` darf keinen `Professor` aufnehmen, selbst wenn es durch die erlaubte kovariante Zuweisung als `Person[]` referenziert wird. Diese Prüfung löst bei unpassendem Inhalt eine `ArrayStoreException` aus. Generische Listen verhalten sich anders: Sie sind invariant, um solche Fehler grundsätzlich zur Compilezeit auszuschließen. Der Unterschied beruht auf der Reifikation des Array-Komponententyps gegenüber der gewöhnlichen Typlöschung von Generics.

Eine problematische Kombination wäre `new T[10]`, weil der reale Laufzeitkomponententyp T nach Erasure nicht feststeht. Ebenso ist `new List<String>[10]` nicht erlaubt, weil der Komponentenparametertyp `List<String>` nicht reifizierbar ist. Innerhalb einer generischen Implementierung lässt sich stattdessen ein `Object[]` als internes Lager anlegen und die Werte beim Auslesen typisiert casten. Der illustrative Ausdruck `(E[]) new Object[capacity]` ist ein unchecked cast, der bei unbeschränktem E innerhalb der Klasse

durch Erasure oft verwendbar ist, aber nicht beweist, dass das Array zur Laufzeit wirklich `E[]` ist. Wer das Array als `E[]` außerhalb der Klasse veröffentlicht oder in einen `String[]`-Kontext überführt, kann eine `ClassCastException` verursachen.

Die sicherere Entwurfsvariante für viele Container ist, das interne Feld tatsächlich als `Object[]` zu führen und ausschließlich einzelne Elemente beim Lesen zu E zu casten. Für parametrisierten Arraytyp `List<String>[]` gibt es zwar ein unchecked Cast von `new List[10]`; die mangelnde Laufzeitgarantie und mögliche Heap `Pollution (Vermischung inkompatibler Parametrisierungen im selben Speicher)` müssen dann bewusst kontrolliert werden. Das Beispiel mit einem `Comparable[]` und einem bounded type E hat ähnliche Voraussetzungen: Der zur Laufzeit vorhandene Arraytyp ist `Comparable[]`, nicht ein separat erzeugtes `E[]`.

Java-`Arrays` sind zur Laufzeit bezüglich ihres Elementtyps überprüfbar und bei Referenztypen kovariant. Deshalb darf `Person[] persons = new Student[2];` geschrieben werden. Der dynamische Arraykomponententyp bleibt jedoch `Student`; ein anschließendes `persons[0] = new Professor(...);` scheitert mit `ArrayStoreException`. Die Zuweisung ist also statisch zulässig, ein unpassendes Speichern wird dynamisch verhindert.

`Arrays` von nicht konkret zur Laufzeit darstellbaren generischen Elementtypen dürfen nicht direkt mit `new T[n]` oder `new List<String>[n]` erzeugt werden. Ein interner Cast `(E[]) new Object[capacity]` ist ein unchecked cast und muss gekapselt bleiben: Das tatsächliche Array ist weiterhin ein `Object[]`. Wer die scheinbar typisierte Arrayreferenz als konkreten externen Arraytyp herausgibt, kann einen `ClassCastException`-Fehler verursachen. Viele Implementierungen verwenden daher intern bewusst `Object[]` und casten nur kontrolliert bei der einzelnen Elementrückgabe.

Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
Student[] students = new Student[2];
Person[] persons = students; // erlaubt
persons[0] = new Professor(); // ArrayStoreException
```

Originalfolien, Code und fachliche Einordnung

5.6.1 Reifizierbare und kovariante Arrays

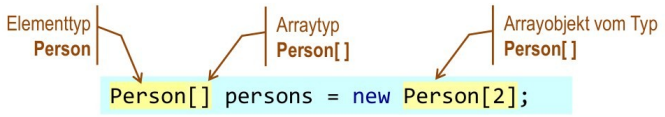
Um den Sicherheitsunterschied zu sehen, müssen der statische Variablentyp und der dynamische Arraykomponententyp getrennt werden. Bei `Person[] p = new Student[2]` ist p als `Person[]` deklariert; das Objekt bleibt aber ein `Student[]`. Jede Speicherung wird gegen den dynamischen Komponententyp geprüft. Bei typisierten Listen erlaubt der Compiler diesen Fall bereits bei der Zuweisung nicht.

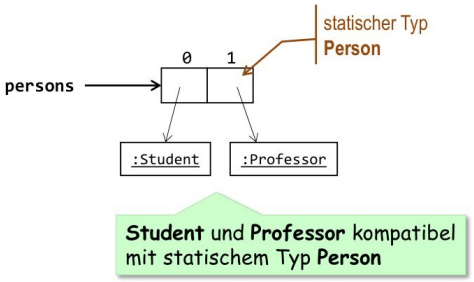
G · Folie 46: ARRAYS ALS "GENERISCHES" KONZEPT

Ein Arraytyp `E[]` beschreibt Positionen, die Elemente vom Typ E oder dessen Subtypen aufnehmen. `Person[]` kann also `Student` und `Professor` enthalten. Anders als eine generische Liste kennt das Array den konkreten Laufzeitkomponententyp. Die scheinbare „Generizität“ entsteht durch die Möglichkeit, zu jedem reifizierbaren Komponententyp einen passenden Arraytyp zu bilden, nicht durch eine Java-Typparameterdeklaration.

Arrays als "generisches" Konzept

- Array-Operator **[ ]** erzeugt aus beliebigem Elementtyp **E** einen typisierten Arraytyp **E[ ]**





- Positionen im Array **E[ ]** haben statischen Typ **E**
  ➔ erlauben Elemente von Subtypen von **E**

```
persons[0] = new Student(...);
persons[1] = new Professor(...);
```

© H. Prähofer, 2026 48

Originalabbildung: 05.Generics.pdf, Folie 46.

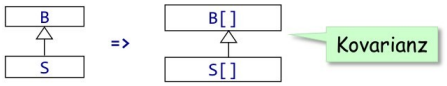
G · Folie 47: ARRAYS ALS "GENERISCHES" KONZEPT: KOVARIANZ [1/2]

`Arrays` sind kovariant: Ein `Student[]` ist zu `Person[]` zuweisungskompatibel, weil `Student` `Person` erweitert. Beide Variablen referenzieren dasselbe tatsächliche Arrayobjekt, dessen Laufzeittyp `Student[]` bleibt. Diese historisch zugelassene Kovarianz unterscheidet `Arrays` von invarianten generischen Listen; der Compiler muss sich für Schreibzugriffe auf eine spätere Laufzeitprüfung verlassen.

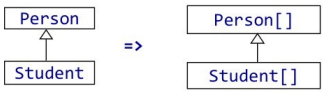
Arrays als "generisches" Konzept: Kovarianz [1/2]

Arrays sind kovariant

- Array **S[ ]** mit speziellerem Elementtyp **S** kompatibel mit Array **B[ ]** mit allgemeinerem Elementtyp **B**



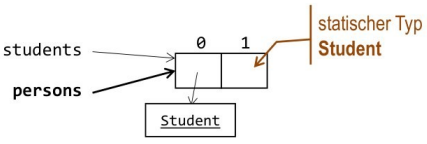
Beispiel:



- **Kovariante Zuweisung bei Arrays erlaubt**

```
Student[] students = new Student[2];
students[0] = new Student(...);
Person[] persons = students;
```

Kovariante Zuweisung erlaubt !



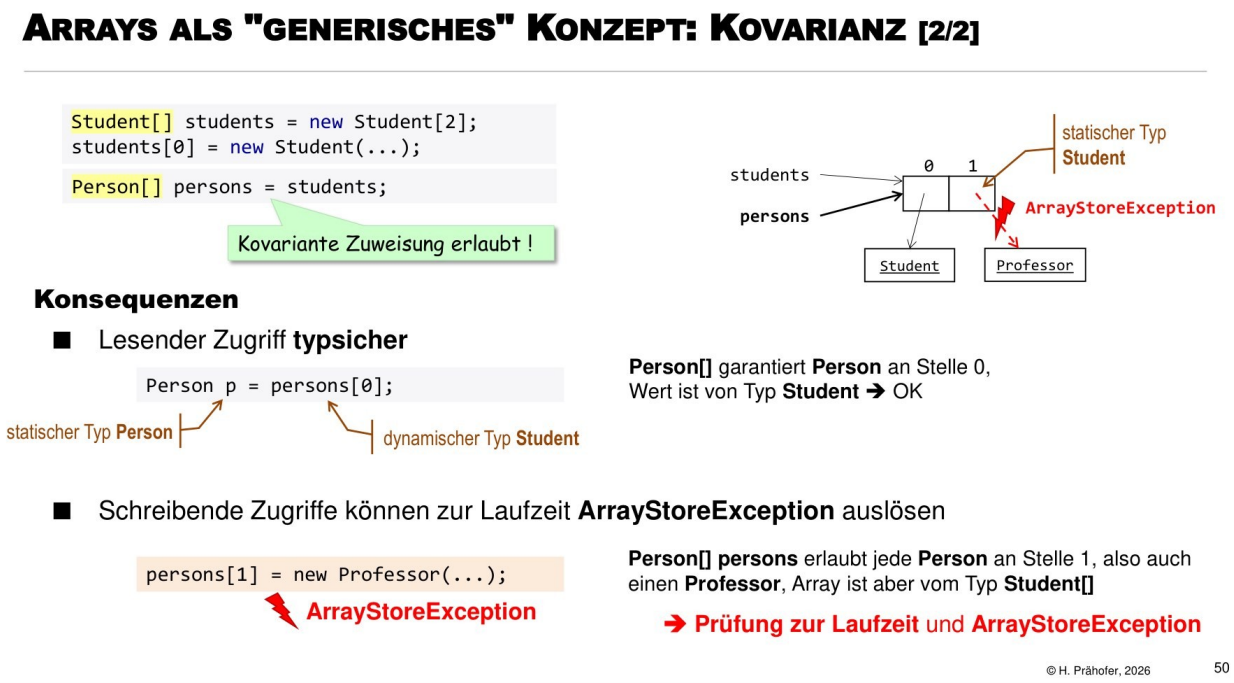
© H. Prähofer, 2026 49

Originalabbildung: 05.Generics.pdf, Folie 47.

G · Folie 48: ARRAYS ALS "GENERISCHES" KONZEPT: KOVARIANZ [2/2]

Ein durch die `Person[]`-Referenz gelesenes `Student`-Objekt ist unproblematisch und kann als `Person` behandelt werden. Ein Versuch, einen `Professor` in das tatsächlich als `Student[]` angelegte Array zu schreiben, verursacht

dagegen eine `ArrayStoreException`. Die Laufzeit kann den Fehler erkennen, weil `Student` als Komponententyp reifiziert ist. Generics verhindern vergleichbare Fehler in veränderbaren Listen weitgehend vorab durch Invarianz und Wildcards.



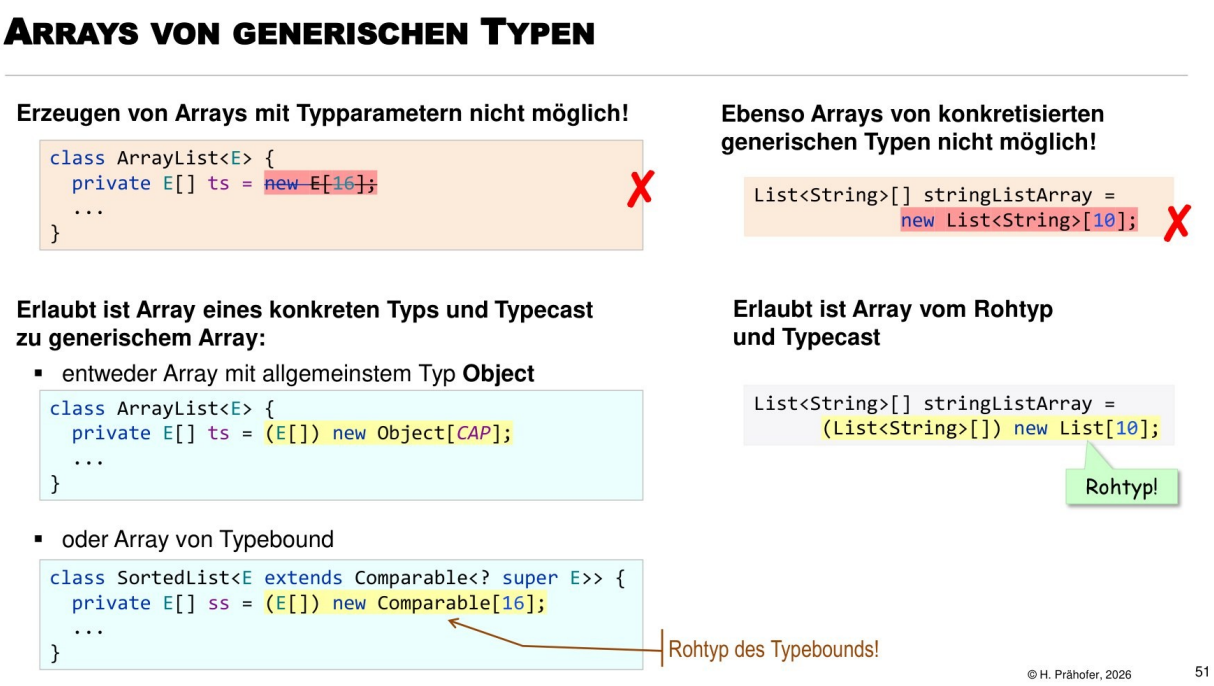
Originalabbildung: 05.Generics.pdf, Folie 48.

5.6.2 Warum new T[] nicht erlaubt ist

Ein unchecked cast lässt sich in besonderen Containerimplementierungen begründen, wenn nur Werte des Typparameters hinein gelangen und das Array vollständig privat bleibt. Nicht legitim wäre die Annahme, dass dadurch ein neues `String[]` oder `Person[]` mit passendem tatsächlichen Laufzeitkomponententyp entstünde. In sorgfältigem Anwendungscode wird häufig das interne `Object[]` direkt beibehalten.

G · Folie 49: ARRAYS VON GENERISCHEN TYPEN

`new E[16]` ist in einer generischen Klasse nicht zulässig, weil E als laufzeitprüfbarer Komponententyp fehlt. Ebenso darf `new List<String>[10]` nicht konstruiert werden. Die gezeigten Alternativen casten ein `Object[]` oder `Comparable[]` unchecked in ein `E[]` beziehungsweise ein rohes `List[]` in `List<String>[]`. Solche Casts brauchen strenge Kapselung: Wird ein `Object[]` tatsächlich als `String[]` herausgereicht, kann die Laufzeitumwandlung scheitern. Eine `Object[]`-interne Repräsentation ist oft robuster.



Originalabbildung: 05.Generics.pdf, Folie 49.

Sicherer interner Speicher für generische Container

```
final class SimpleStore<E> {
 private final Object[] elems;
 SimpleStore(int capacity) { elems = new Object[capacity]; }
 void put(int i, E value) { elems[i] = value; }
 @SuppressWarnings("unchecked")
 E get(int i) { return (E) elems[i]; }
}
```

Die unchecked Operation wird auf genau eine gekapselte Stelle begrenzt. Weil put nur E annimmt und das `Object[]` nicht nach außen gelangt, lässt sich der Cast durch eine überprüfte Klasseninvariante begründen. Das Beispiel hat bewusst keine vollständige Index-/Nullvalidierung.

5.7 Typinferenz und Erzeugung generischer Werte

Typinferenz ist die Fähigkeit des Compilers, Typinformationen aus Initialisierern, Aufrufargumenten und erwarteten Ergebnistypen herzuleiten. `var x = new ArrayList<String>();` legt einen statischen Typ `ArrayList<String>` für die lokale Variable x fest; `var` macht Java nicht dynamisch typisiert. Ein späteres Zuweisen einer nicht kompatiblen Liste bleibt unzulässig. In `static <T> List<T> of(T... elements)` kann der Compiler T aus den Elementargumenten erschließen; bei mehreren unterschiedlichen Argumenttypen bestimmt er einen geeigneten gemeinsamen Typ, möglicherweise einen Schnittstellentyp oder eine Schnittmenge mehrerer Bounds, nicht zwangsläufig nur `Object`.

Bei Methoden ohne Argumente ist der Kontext entscheidend. `List<Student> students = Lists.emptyList();` kann T als `Student` ableiten; `var students = Lists.emptyList();` besitzt ohne weitere Information gewöhnlich den Elementtyp `Object`. Mit einer expliziten Typfestlegung `Lists.<Student>emptyList()` kann man den Compiler steuern. Die konkrete Platzierung von `<Student>` unmittelbar nach dem Punkt gehört zur Syntax expliziter Typargumente beim Methodenaufruf.

`new T()` ist wegen Type Erasure nicht zulässig: Der Compiler kennt keinen konkreten Konstruktor, und die Laufzeit kann nicht allgemein ein T-Objekt erzeugen. Stattdessen lässt man die Erzeugung durch einen übergebenen Factory-Callback vornehmen. `Supplier<T>` ist ein funktionales Interface mit `T get()`: Ein übergebenes `Supplier`-Objekt entscheidet, wie ein T-Wert konstruiert wird. Die Folien zeigen hierfür zunächst eine anonyme Klasse; später lassen sich dieselben Aufrufe mit Lambda-Ausdrücken kürzer notieren. Ein `Supplier` muss nicht tatsächlich `new` verwenden, sondern kann auch gespeicherte oder berechnete T-Werte liefern.

Typinferenz bedeutet, dass der Compiler ein Typargument aus Aufrufargumenten und dem erwarteten Zieltyp ermittelt. `var x = new ArrayList<String>();` hat damit den statisch ermittelten Typ `ArrayList<String>`; `var` macht die Variable nicht dynamisch typisiert. Die lokale Typinferenz ist nicht gleichbedeutend mit einer Typprüfung zur Laufzeit. Bei einer parameterlosen generischen Methode kann der Zuweisungskontext einen Typ bestimmen, etwa `List<Student> empty = Lists.emptyList();`.

`new T()` ist im gewöhnlichen generischen Java nicht zulässig, weil der konkrete Klassenkonstruktor nach Type Erasure nicht allein aus dem Typparameter verfügbar ist. Ein Erzeugerobjekt `Supplier<T>` definiert stattdessen die Operation `T get()`. Die generische Klasse empfängt eine solche Factory als Parameter und ruft `supplier.get()` auf. Damit ist die Wahl der konkreten Erzeugung vom generischen Container getrennt und trotzdem statisch typisiert.

Ausgeführtes beziehungsweise schematisch in die genannten Typen einzuordnendes Java-Beispiel:

```
class Box<T> {
 T value;
 void init(java.util.function.Supplier<T> factory) {
 value = factory.get();
 }
}
Box<String> b = new Box<>();
b.init(() -> "fertig");
```

Originalfolien, Code und fachliche Einordnung

5.7.1 Typinferenz durch Aufruf und Zielkontext

Typinferenz führt Bedingungen aus mehreren Quellen zusammen. Beispiel: `List.of("x", "y")` liefert bei einer generischen Fabrik typischerweise eine Liste von Strings, während eine Mischung verschiedener Objektarten einen gemeinsamen Obertyp erfordert. Die häufig im Unterricht verwendete Bezeichnung „Typableitung“ ist somit ein statisches Verfahren, keine Veränderung der Variablentypen zur Laufzeit.

G · Folie 52: TYPINFERENZ [1/2]

Lokale Typinferenz mit `var` übernimmt einen statischen Typ aus dem Initialisierungsausdruck. Bei `var x = new List<String>()` wird x als `List<String>` behandelt. Bei einer generischen Factory-Methode mit mehreren Argumenten leitet der Compiler einen gemeinsamen passenden Typparameter ab. Er darf dabei nicht blind einen gewünschten Spezialtyp annehmen, sondern muss alle übergebenen Werte typkompatibel einordnen.

TYPINFERENZ [1/2]

Teil des Compilers

Typinferenz leitet die Typen aus dem Kontext ab

- Keyword **var:** Typ einer Variablen wird vom Initialwert abgeleitet

```
var x = new List<String>();
```

Typ von **x** ist **List<String>**

- Methoden mit Parameter: es erfolgt die Ableitung der Typen aus aktuellen Argumenten

```
public static <T> List<T> of(T...ts) {
    List<T> list = new List<T>();
    for (T t: ts) list.add(t);
    return list;
}
```

```
var strings = List.of("Hallo", "World");
```

Ableitung von **List<String>**

```
var objects = List.of("Hallo", new Point(1, 2), new Student( … ));
```

Ableitung eines gemeinsamen Obertyps der Argumenttypen

© H. Prähofer, 2026 54

Originalabbildung: 05.Generics.pdf, Folie 52.

G · Folie 53: TYPINFERENZ [2/2]

Eine generische Methode kann ihren Rückgabetyp sogar aus dem Zieltyp einer Zuweisung ableiten, wenn sie keinerlei Argumente erhält. `List<Student> result = List.emptyList()` bietet den Kontext T=`Student`; mit `var result = List.emptyList()` fehlt dieser Kontext und T wird typischerweise `Object`. Mit der expliziten Syntax `List.<Student>emptyList()` wird T unabhängig vom Ziel festgelegt. Das illustriert den Unterschied zwischen Typinferenz und Laufzeittypprüfung.

TYPINFERENZ [2/2]

Typinferenz leitet die Typen aus dem Kontext ab

- Generische Methoden ohne Argumente:
  Ableitung derTypargumente aus dem Aufrufkontext
  - ☐ z.B. aus Typ der Variablen bei Zuweisung

```
List<Student> emptyStudentList = List.emptyList();
```

List<Student>

```
public class List<E> implements Iterable<E> {
  public static <T> List<T> emptyList() {
    return new List<T>();
  }
  …
}
```

  - ☐ Wenn Kontext nicht ausreichend, dann **List<Object>**

```
var list = List.emptyList();
```

List<Object>

  - ☐ oder durch explizite Angabe eines Typs für den Typparameter

```
var emptyStudentList = List.<Student>emptyList();
```

List<Student>

© H. Prähofer, 2026 55

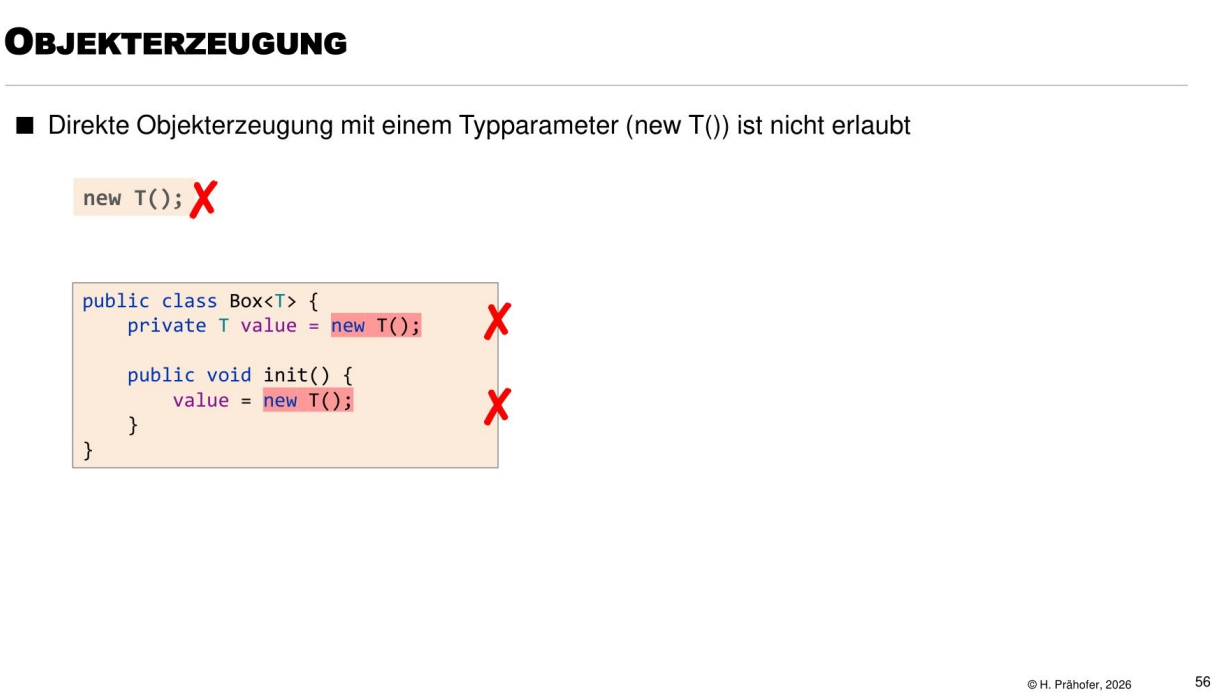
Originalabbildung: 05.Generics.pdf, Folie 53.

5.7.2 Supplier statt new T()

Der Standardweg zur kontrollierten Erzeugung generischer Objekte ist Dependency Injection eines Erzeugers: eine Factory, ein `Supplier<T>` oder in geeigneten Situationen ein Konstruktorreferenzobjekt. Der Container fordert lediglich einen T-Wert an und muss nicht selbst wissen, wie er konstruiert wird. Das hält den Code flexibler und mit Type Erasure vereinbar.

G · Folie 54: OBJEKTERZEUGUNG

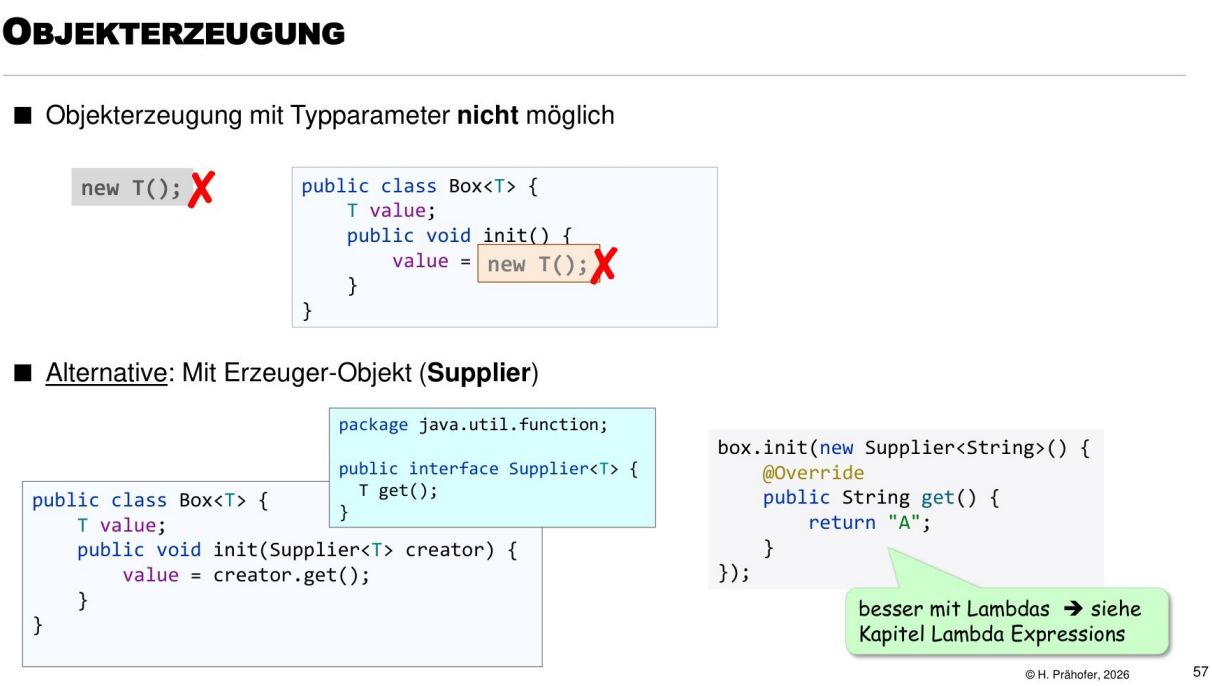
Eine Typparameterdeklaration T beschreibt keinen konkreten Konstruktor. `new T()` ist daher verboten, auch wenn manche später verwendeten Typen einen parameterlosen Konstruktor besitzen. Der Compiler muss die generische Klasse für alle zulässigen T übersetzen können, und nicht jeder mögliche Typ ist instanziierbar. Die Abbildung zeigt das Verbot sowohl bei einem Feldinitialisierer als auch innerhalb einer Methode.



Originalabbildung: 05.Generics.pdf, Folie 54.

G · Folie 55: OBJEKTERZEUGUNG

Ein `Supplier<T>` kapselt die Erzeugung eines T-Werts in der Methode `get()`. Der generische Box-Code erhält ein solches Erzeugerobjekt als Parameter und weist dessen Rückgabewert an das Feld zu. Die Folie verwendet dazu eine anonyme Klasse, die `String` zurückgibt. Mit später behandelten Lambdas ist dieselbe Übergabe kürzer, etwa `box.init(() -> "A")`. Ein `Supplier` kann auch `null` liefern; falls Box das verbieten will, ist eine explizite Prüfung nötig.



Originalabbildung: 05.Generics.pdf, Folie 55.

Vollständiges Supplier-Beispiel

```
import java.util.function.Supplier;
class Holder<T> {
 T value;
 void init(Supplier<? extends T> factory) {
 value = factory.get();
 }
}
Holder<String> h = new Holder<>();
h.init(() -> new String("Text"));
```

Der `Supplier` stellt den Wert bereit, ohne dass der generische Container selbst `new T()` benötigt. Die Signatur mit `? extends T` lässt auch einen `Supplier` zu, der einen Untertyp von T produziert.

5.8 Fallstudie List: die generische Listenarchitektur

Die bisherige Listen-Fallstudie wurde entlang zweier Fragen aufgebaut: Ist der Zugriff sequentiell über Iteratoren oder über einen Index möglich, und darf die Datenstruktur verändert werden? Daraus entstehen die Interfaces `List<E>`, `IndexedList<E>`, `MutableList<E>` und `MutableIndexedList<E>`. Mit dem Typparameter E beschreibt jede Schnittstelle jetzt, welchen Elementtyp ihre Iteratoren, get-Methoden und add-Operationen liefern oder erwarten. Die Lehrmodell-Interfaces sind ausdrücklich nicht mit `java.util.List` gleichzusetzen; ihr Architekturziel ist die demonstrative Trennung einzelner Fähigkeiten.

`List<E> extends Iterable<E>` bedeutet, dass ein `List<E>`-Objekt durch eine for-each-Schleife traversiert werden kann und dass `iterator()` ein `Iterator<E>` liefert. `IndexedList<E>` stellt darüber hinaus `E get(int index)` bereit. Ein `ArrayList<E>` speichert Werte in einem internen Array und kann diese typisiert zurückgeben; `LinkedList<E>` verwendet Knoten. Eine statische Member-Klasse `Node<V>` in `LinkedList<E>` braucht einen eigenen Typparameter V, weil der statische verschachtelte Typ nicht automatisch E der äußeren Klasse übernimmt. Eine nicht statische Member-Klasse `LinkedListIterator` kann dagegen E verwenden, weil sie an eine `LinkedList<E>`-Instanz gebunden ist.

Die Adapter `of(E[] array)` und `of(E single)` können als anonyme Klassen implementiert werden. Eine Factory-`Methode (Fabrikmethode)` ist eine Methode, die ein geeignetes Objekt erzeugt und über einen abstrakten Typ zurückgibt. Der Array-Adapter liefert eine Sicht auf ein vorhandenes Array: Solange nicht ausdrücklich kopiert wird, können Änderungen des Originalarrays durch den Adapter sichtbar bleiben. Bei `indexOf` ist `Objects.equals(element, gesucht)` gegenüber `element.equals(gesucht)` auch dann sicher, wenn ein Arrayeintrag `null` sein darf. Bei selbstgeschriebenen Listen muss dokumentiert werden, ob Nullwerte überhaupt erlaubt sind.

Die schreibende Schnittstelle `MutableList<E>` bietet `add(E)` sowie `addAll(List<? extends E>)`. Eine Liste mit `Person`-Elementen darf dadurch Elemente aus einer Liste mit `Student`-Elementen übernehmen. Dagegen erwarten `contains(Object)`, `indexOf(Object)` und `remove(Object)` absichtlich `Object` statt E: Bei einer rein lesenden oder entfernden Operation kann ein Objekt eines anderen Typs schlicht „nicht vorhanden“ sein, ohne die Sammlung zu beschädigen. Zudem kann eine so deklarierte Methode auch bei `MutableList<? extends T>` aufgerufen werden, während `remove(E)` aufgrund des unbekannten konkreten E nicht ohne Weiteres aufrufbar wäre.

Die Folien führen eine Methode transfer ein. Ihre zentrale Typidee ist korrekt, aber ihr konkreter Ablauf benötigt eine Zusatzprüfung: Wenn die durch `asReadonlyList()` gelieferte Sicht dieselbe veränderbare Quellliste verwendet und während der Iteration Elemente entfernt werden, kann ein `Iterator` inkonsistent werden oder eine ConcurrentModificationException auslösen. Auch Duplikate und Gleichheitsvergleiche verdienen Beachtung. Für eine robuste Übertragung ist häufig eine echte Snapshot-Kopie der Quellwerte oder eine Entfernung direkt über einen dafür vorgesehenen `Iterator` sinnvoll. Eine nur lesende Sicht ist noch kein Snapshot und nicht automatisch unveränderlich.

Die Typbeziehungen in der finalen Lehrarchitektur haben zwei unabhängige Aufgaben: `Iterable<E>` ermöglicht die Iteration mit Elementtyp `E`, und `MutableList<E>` erlaubt das typsichere Einfügen. Ein `LinkedList<E>` kann mittels eines `Node<E>`-Feldes und eines `Iterator<E>` arbeiten; ein `ArrayList<E>` bietet direkten Indexzugriff. Die Fallstudie demonstriert, wie die zuvor getrennten Interfaces und die Speicherrepräsentationen durch einheitliche Typargumente zusammengeführt werden.

Bei `addAll(List<? extends E> elems)` ist die Quelle bewusst kovariant: Eine `MutableList<Person>` kann daher alle Werte einer `List<Student>` übernehmen. Bei `remove(Object)` und `contains(Object)` ist dagegen kein `E`-Parameter erforderlich: Diese Operationen können lediglich festzustellen versuchen, ob ein gegebenes Objekt vorhanden ist, ohne dadurch einen nicht erlaubten Wert in die Sammlung einzufügen. Ein korrekt implementiertes `remove(Object)` sollte bei Nichtvorhandensein wohldefiniert reagieren, statt unnötig einen Cast auf `E` zu erzwingen.

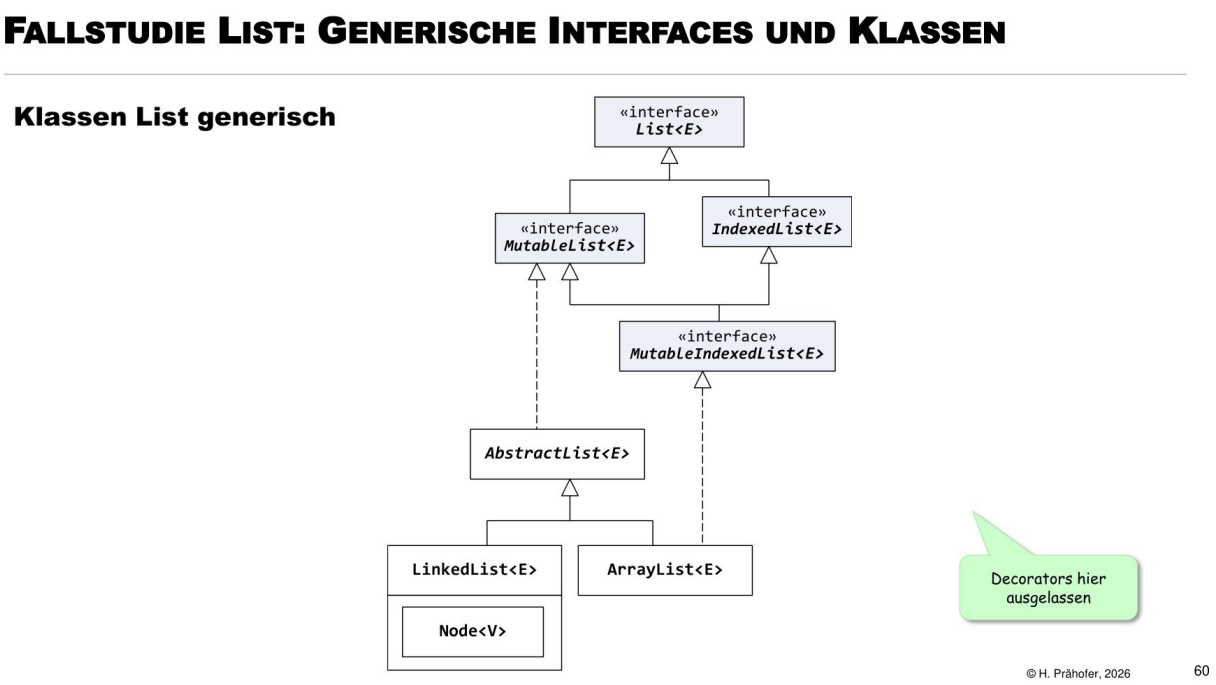
Originalfolien, Code und fachliche Einordnung

5.8.1 Parametrisierte Listeninterfaces

Diese Fallstudie führt nun alle Sprachmittel zusammen: Interfaces aus Kapitel 3, geschachtelte Klassen und anonyme Adapter aus Kapitel 4 sowie generische Typparameter, Bounds und Wildcards aus diesem Kapitel. Entscheidend ist, dass alle durch E verbundenen Methoden einen einheitlichen Elementtyp verwenden, auch wenn mehrere konkrete `List`-Unterklassen unterschiedliche interne Datenstrukturen haben.

G · Folie 58: FALLSTUDIE LIST: GENERISCHE INTERFACES UND KLASSEN

Das UML-Diagramm verknüpft die zuvor eingeführten Listenfähigkeiten mit Typparametern. `List<E>`, `MutableList<E>`, `IndexedList<E>` und `MutableIndexedList<E>` bilden Schnittstellen für Traversierung, Änderung und Indexzugriff. `AbstractList<E>` führt gemeinsamen Zustand, `LinkedList<E>` verwaltet Knoten und `ArrayList<E>` nutzt ein Array. Alle Pfeile sind als Beziehungen der generischen Typen zu lesen, nicht als automatische Kovarianz verschiedener E-Argumente.



Originalabbildung: 05.Generics.pdf, Folie 58.

G · Folie 59: FALLSTUDIE LIST: GENERISCHE INTERFACES

Die generischen Interfaces reichen E an `Iterable<E>` weiter, sodass eine foreach-Schleife den korrekten Elementtyp liefert. `IndexedList<E>` ergänzt `get(E-Rückgabe)`; `MutableList<E>` erlaubt `add(E)` und addAll mit `List<? extends E>`; `MutableIndexedList<E>` ergänzt positionsbezogene Änderungen. Das Beispiel ist schematisch und enthält Ellipsen. Die kombinierten Vererbungsbeziehungen aus dem Diagramm müssen bei einer vollständigen Implementierung konsistent deklariert werden.

Fallstudie List: Generische Interfaces

■ Interfaces **List<E>**, **IndexedList<E>**, **MutableList<E>** und **MutableIndexedList<E>**

```
public interface List<E> extends Iterable<E> {
  …
  public int size();
  default boolean contains(Object o) { … };
}
```

```
public interface Iterable<T> {
  Iterator<T> iterator();
  …
}
```

```
public interface IndexedList<E> extends List<E> {
  …
  E get(int idx);
  int indexOf(Object o);
  default boolean contains(Object o) { return indexOf(o) >= 0; }
}
```

```
public interface MutableList<E> extends List<E> {
  …
  void clear();
  void add(E elem);
  default void addAll(E... elems) {…}
  default void addAll(List<? extends E> list) {…}
  boolean remove(Object elem);
}
```

```
public interface MutableIndexedList<E> extends IndexedList<E> {
  …
  void add(int idx, E elem);
  E remove(int idx);
}
```

© H. Prähofer, 2026 61

Originalabbildung: 05.Generics.pdf, Folie 59.

G · Folie 60: FALLSTUDIE LIST: GENERISCHE KLASSEN

`AbstractList<E>` stellt eine gemeinsame size-Implementierung bereit. `LinkedList<E>` enthält generische `Node<E>`-Referenzen, während `ArrayList<E>` Werte in einem mit einem unchecked cast initialisierten internen Array hält. Das Cast-Ergebnis garantiert nicht, dass sich das Feld gefahrlos als konkretes `E[]` nach außen geben lässt. Das Beispiel zeigt einen häufigen internen Implementierungsansatz, bei dem der Entwickler die Einhaltung der Elementtypinvariante selbst kontrollieren muss.

Fallstudie List: Generische Klassen

■ Klassen **AbstractList<T>** , **ArrayList<E>** und **LinkedList<T>**

```
public abstract class AbstractList<E> implements MutableList<E> {
  private int size = 0;

  protected AbstractList() { }

  @Override
  public int size() { return size; }
  …
}
```

```
public final class LinkedList<E> extends AbstractList<E> {

  private static class Node<V> {…}

  private Node<E> head;
  private Node<E> tail;
  …
}
```

```
public final class ArrayList<E> extends AbstractList<E> implements MutableIndexedList<E> {
  private E[] elems;
  public ArrayList(E...initial) {
    elems = (E[])new Object[INITIAL_CAPACITY];
    addAll(initial);
  }
  …
}
```

Erzeugen von **Object[]** aber Typecast zu **E[]**!
➔ siehe **Generische Typen und Arrays**

© H. Prähofer, 2026 62

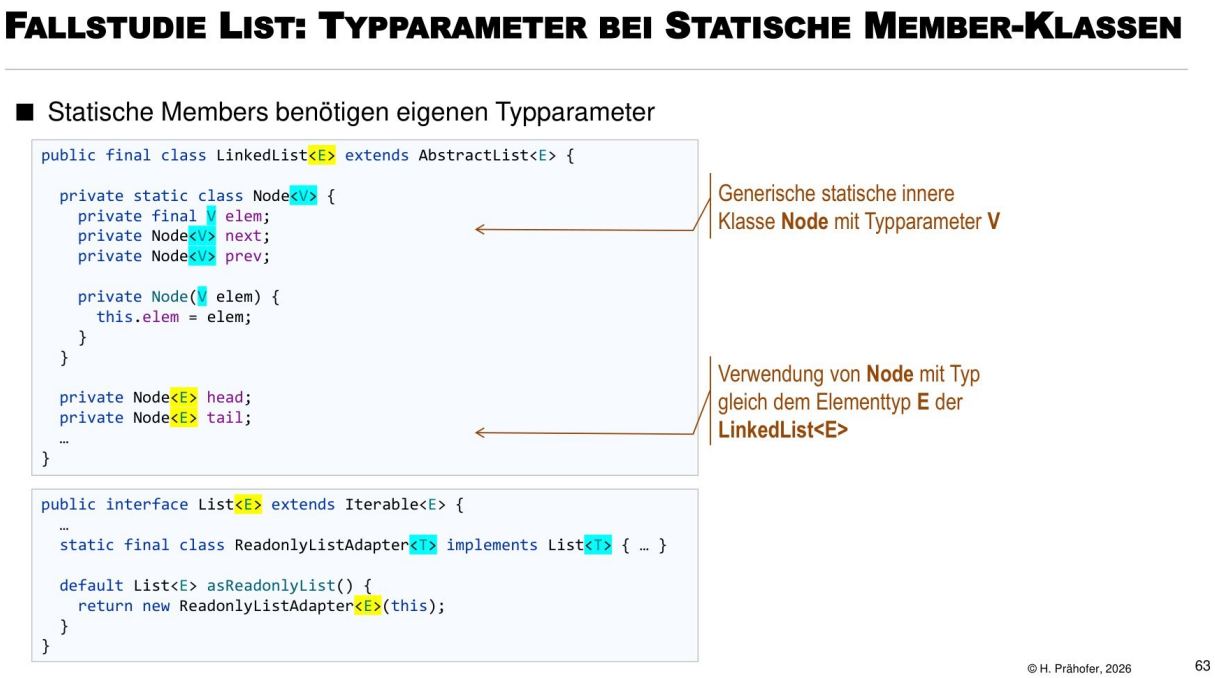
Originalabbildung: 05.Generics.pdf, Folie 60.

5.8.2 Statische und nicht statische generische Hilfsklassen

Statische Verschachtelung trennt den eigenen Typparameter der Hilfsklasse von dem Typparameter der äußeren Klasse. Ein `LinkedList<E>` verwaltet im Feld head eine `Node<E>`, aber die Knotendeklaration selbst heißt `Node<V>`. Bei einer echten, nicht statischen Member-Klasse dagegen gibt es eine implizite äußere Instanz, weshalb dort der Typparameter E sichtbar ist.

G · Folie 61: FALLSTUDIE LIST: TYPPARAMETER BEI STATISCHE MEMBER-KLASSEN

Eine statische geschachtelte `Node<V>`-Klasse kann den Typparameter E ihrer äußeren `LinkedList<E>` nicht unmittelbar übernehmen, denn sie ist nicht an ein bestimmtes äußeres Listenobjekt gebunden. Stattdessen führt `Node<V>` ihren eigenen Typparameter ein. Die Liste verwendet dann `Node<E>` für head und tail. Vergleichbar ist eine statisch geschachtelte `ReadonlyListAdapter<T>`-Klasse innerhalb eines generischen Interfaces: Auch sie benötigt ihren eigenen Parameter T.



Originalabbildung: 05.Generics.pdf, Folie 61.

G · Folie 62: FALLSTUDIE LIST: TYPPARAMETER BEI MEMBER-KLASSEN

Die nicht statische Member-Klasse LinkedListIterator ist an eine `LinkedList<E>`-Instanz gebunden. Daher darf sie den Typparameter E der äußeren Klasse verwenden, ohne einen eigenen generischen Klassennamen zu deklarieren. Ihr Feld `Node<E> next = head` referenziert den nächsten Knoten; `next()` liefert E und verschiebt den Cursor. Bei Erreichen des Listenendes wird, gemäß `Iterator`-Vertrag, `NoSuchElementException` ausgelöst.

Fallstudie List: Typparameter bei Member-Klassen

■ Member-Klassen verwenden Typparameter der äußeren Klasse

```
public final class LinkedList<E> extends AbstractList<E> {

  private class LinkedListIterator implements Iterator<E> {

    private Node<E> next = head;

    @Override
    public boolean hasNext() {
      return next != null;
    }

    @Override
    public E next() {
      if (!hasNext()) { throw new NoSuchElementException(); }
      E element = next.elem;
      next = next.next;
      return element;
    }
  }
  …
}
```

Innere Klasse **LinkedListIterator** ohne eigenen Typparameter

Innere Klasse **LinkedListIterator** verwendet Typparameter **E** der äußeren Klasse

© H. Prähofer, 2026

Originalabbildung: 05.Generics.pdf, Folie 62.

G · Folie 63: FALLSTUDIE LIST: TYPPARAMETER BEI ANONYMEN KLASSEN

Eine anonyme Klasse innerhalb einer generischen Methode kann den Methodentypparameter E verwenden, solange er im lexikalischen Gültigkeitsbereich steht. `iteratorOf(E[] array,int size)` liefert daher `Iterator<E>`, dessen `next()` die Arrayelemente typisiert zurückgibt. Die Folie setzt eine gültige Größe voraus; für robusten Code müsste geprüft werden, dass size zwischen `null` und `array.length` liegt. Änderungen am Array während des Durchlaufs wirken weiterhin auf die gelesenen Elemente.

Fallstudie List: Typparameter bei Anonymen Klassen

■ Anonyme Klassen können Typparameter von äußeren Klasse oder Methode verwenden

```
public class Lists {

  static <E> Iterator<E> iteratorOf(E[] array, int size) {

    return new Iterator<E>() {
      private int nextIndex;

      @Override
      public boolean hasNext() {
        return nextIndex < size;
      }

      @Override
      public E next() {
        if (!hasNext()) throw new NoSuchElementException();
        return array[nextIndex++];
      }
    };

  }
  …
}
```

Anonyme Klasse verwendet Typparamter **E** von Methode **iteratorOf**

© H. Prähofer, 2026

Originalabbildung: 05.Generics.pdf, Folie 63.

5.8.3 Generische Adapter und Zugriffsmethoden

Die statischen Fabrikmethoden sind mit `<E>` eigenständig generisch, selbst wenn sie in einem generischen Interface deklariert sind. Das Ergebnis ist ein Interface-Typ und die tatsächliche Implementierung eine anonyme Klasse. Dies ist dieselbe Trennung von Schnittstelle und Implementierung, die bereits im Kapitel über Interfaces motiviert wurde.

G · Folie 64: FALLSTUDIE LIST: ADAPTERS GENERISCH

Zwei generische Factory-Methoden erzeugen Adapter: `of(E[] array)` bietet eine indexbasierte Sicht auf ein bestehendes Array und `of(E single)` eine Ein-Element-Liste. Die anonymen Klassen implementieren `IndexedList<E>`. get prüft die zulässigen Indizes und wirft andernfalls IndexOutOfBoundsException. `indexOf` vergleicht sicher mit `Objects.equals` und berücksichtigt damit auch `null`. Die Methode ist ein Lehrbeispiel, nicht das Verhalten der unveränderlichen JDK-Fabrik `java.util.List.of`.

Originalabbildung: 05.Generics.pdf, Folie 64.

G · Folie 65: FALLSTUDIE LIST: GENERISCHE ZUGRIFFE

Für `List<Student>` liefert `iterator()` einen `Iterator<Student>`, dessen `next()` ohne Cast einen `Student` ergibt. Die foreach-Schleife `for (Student p : students)` basiert auf dieser `Iterable<E>`-Struktur. `IndexedList<Student>` garantiert bei `get(0)` ebenfalls `Student`. Die Folie belegt somit, dass Generics nicht nur add-Parameter einschränkt, sondern auch alle lesenden Schnittstellen sicher verbindet.

Fallstudie List: Generische Zugriffe

Interface List<E>

- Zugriffe auf Elemente mit generischem Rückgabewert **T** sind typsicher

```
public interface Iterable<E> {
  public Iterator<E> iterator();
  …
}
```

```
public interface List<E> extends Iterable<E> {
  …
  public int size();
  default boolean contains(Object o) { … };
}
```

```
public interface IndexedList<E> extends List<E> {
  …
  E get(int idx);
  int indexOf(Object o);
  default boolean contains(Object o) {
    return indexOf(o) >= 0;
  }
}
```

```
List<Student> students = …;
```

```
Iterator< Student > iterator = students.iterator();
while (iterator.hasNext()) {
  Student p = iterator.next();
  …
}
```

```
for (Student p: students) { … }
```

```
IndexedList<Student> students = …;
```

```
Student first = students.get(0);
```

© H. Prähofer, 2026 67

Originalabbildung: 05.Generics.pdf, Folie 65.

G · Folie 66: FALLSTUDIE LIST: GENERISCHE ZUGRIFFE

`MutableList<Student>`.add akzeptiert `Student` und Unterklassen, aber keinen `Professor`. Die Default-Methode `addAll(List<? extends E>)` ist flexibler: Eine `MutableList<Person>` darf alle `Student`-Werte aus einer `List<Student>` aufnehmen. Die Schleife liest jedes Quellobjekt als `Person` und fügt es typsicher ein. Zu beachten sind mögliche Seiteneffekte, falls Quell- und Zielliste identisch sind, denn darüber macht die gezeigte Signatur allein keine Garantie.

Fallstudie List: Generische Zugriffe

Interface MutableList<E>

- **add** typsicher

```
public interface MutableList<E> extends List<E> {
  void add(E elem);
  …
}
```

```
MutableList<Student> mutStudents = new ArrayList<>();

mutStudents.add(new Student("Anna"));
```

- **addAll** mit kovariantem Wildcard **List<? extends E>**
  - ☐ erlaubt auch Listen mit Subtypen

```
public interface MutableList<E> extends List<E> {
  …
  default void addAll(List<? extends E> elems) {
    for (E t: elems) add(t);
  }
}
```

Z.B. Anfügen von **List<Student>** zu **MutableList<Person>**

```
MutableList<Person> mutPersons = new ArrayList<>();

List<Student> students = List.of( … );

mutPersons.addAll(students);
```

List<Student> kompatibel mit **List<? extends Person>**

© H. Prähofer, 2026 68

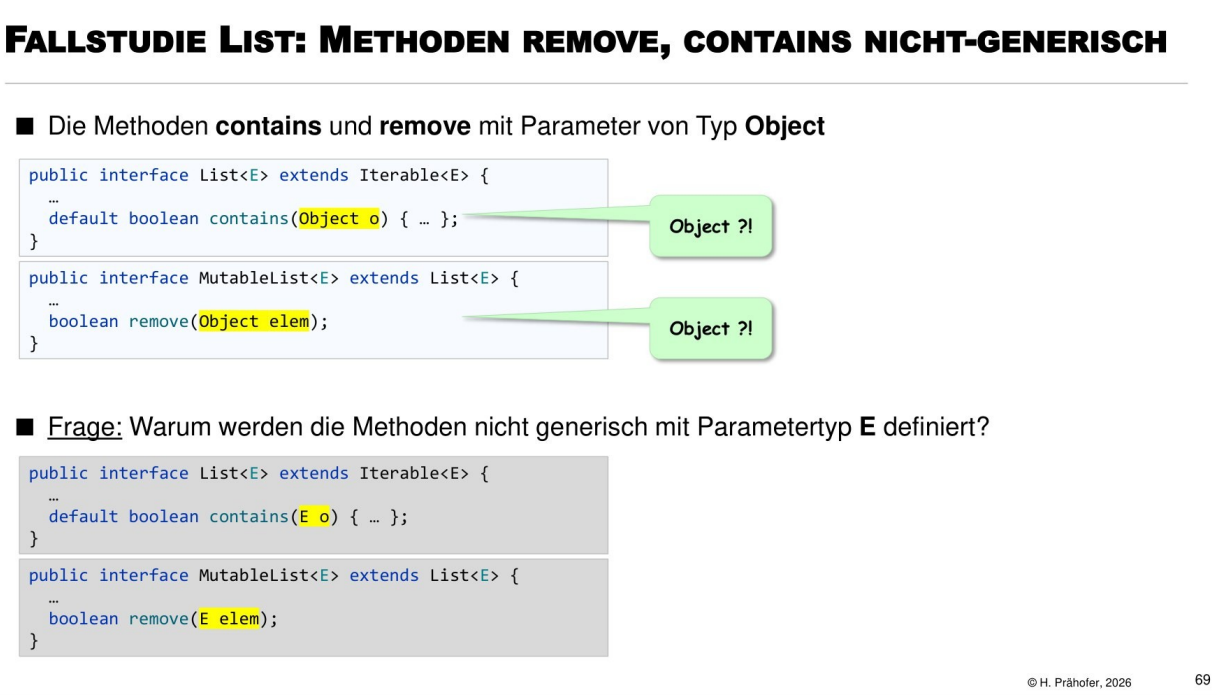
Originalabbildung: 05.Generics.pdf, Folie 66.

5.8.4 Warum remove(Object) sinnvoll ist

Die Wahl `Object` als Such- beziehungsweise Löschparameter ist bewusst asymmetrisch zu `add(E)`. Einfügen könnte den Elementtypvertrag verletzen und benötigt daher die starke Schranke E. Suchen oder Entfernen eines nicht passenden Objekts schadet diesem Vertrag nicht. Bei richtig implementierter Gleichheit wird die Operation dann schlicht kein passendes Element finden.

G · Folie 67: FALLSTUDIE LIST: METHODEN REMOVE, CONTAINS NICHT-GENERISCH

Die Folien fragen, warum `contains(Object)` und `remove(Object)` nicht den Typparameter E benutzen. Ein Vergleich oder eine Entfernung fügt keinen unzulässigen neuen Wert hinzu; es ist deshalb sicher, eine Suchanfrage für ein Objekt anderen Typs zu stellen. Außerdem sind solche Aufrufe bei Wildcard-Sichten möglich. Ein allgemeineres `Object`-Argument bedeutet nicht, dass jede Anfrage ein vorhandenes Element finden oder entfernen muss.



Originalabbildung: 05.Generics.pdf, Folie 67.

G · Folie 68: FALLSTUDIE LIST: PROBLEMANALYSE REMOVE

Die hypothetische Signatur `remove(E elem)` lässt sich auf einer Quelle `MutableList<? extends T>` kaum sinnvoll aufrufen, weil der unbekannte konkrete Quell-Elementtyp durch T selbst nicht feststeht. Die Methode transfer versucht von einer read-only Sicht zu lesen und ein Element anschließend aus der veränderbaren Quelle zu entfernen. Zusätzlich zur statischen Typfrage muss man die Iterationssicherheit beachten, wenn Lesen und Entfernen dieselbe Struktur verändern.

FALLSTUDIE LIST: PROBLEMANALYSE REMOVE

Interface MutableList<T>

- **Annahme: remove** mit Parameter von generischem Typ **E**

```
public interface MutableList<E> extends List<E> {
  …
  void remove(E elem);
  …
}
```

Beispiel Methode **transfer**: Lesen von **to**, kopieren in **to**, Löschen in **from**

```
public static <T> void transfer(MutableList<? extends T> from, MutableList<T> to) {
  List<? extends T> fromCopy = from.asReadonlyList();
  for (T elem: fromCopy) {
    to.add(elem);
    from.remove(elem);
  }
}
```

✗ does not compile

Methode **remove** kann für Liste mit kovariantenTyp **List<? extends T>** nicht verwendet werden!

Ursachenanalyse:
- Kovarianter Parameter **List<? extends T> from**
- Methode **remove** mit generischem Parameter kann bei kovariantem Parameter nicht verwendet werden

© H. Prähofer, 2026 70

Originalabbildung: 05.Generics.pdf, Folie 68.

G · Folie 69: FALLSTUDIE LIST: REMOVE NICHT GENERISCH

Die Umstellung auf `remove(Object)` behebt den generischen Signaturkonflikt, weil der Aufruf kein Objekt in die Sammlung schreibt. Das Beispiel transfer ist jedoch nur mit einer geeigneten Snapshot-/Iteratorstrategie allgemein korrekt: Wird über eine live view iteriert und die Ursprungsliste mutiert, können Ausnahmen oder übersprungene Elemente auftreten. Die Begriffe read-only und immutable unterscheiden sich, wie schon in Kapitel 3 und 4 erklärt.

FALLSTUDIE LIST: REMOVE NICHT GENERISCH

Lösung

- **remove** nicht generisch, sondern mit Parameter von Typ **Object**

```
public interface MutableList<T> extends List<T> {
  …
  void remove(Object elem);
  …
}
```

- **remove** nun bei kovariantem **MutableList<? extends Person> from** erlaubt

```
public static <T> void transfer(MutableList<? extends T> from, MutableList<T> to) {
  List<? extends T> fromCopy = from.asReadonlyList();
  for (T elem: fromCopy) {
    to.add(elem);
    from.remove(elem);
  }
}
```

- **remove** ist typsicher, weil es keine neuen (potentiell inkompatiblen) Elemente einfügt

```
MutableList<Person> mutPersons = new ArrayList<>( … );
MutableList<Student> mutStudents = new ArrayList<>( … );

transfer(mutStudents, mutPersons);
```

Gilt auch für **contains**, **indexOf** etc.

© H. Prähofer, 2026 71

Originalabbildung: 05.Generics.pdf, Folie 69.

G · Folie 70: FALLSTUDIE LIST: REMOVEALL MIT KOVARIANTEM WILDCARD LIST<?>

`removeAll(List<?> elems)` nimmt Listen beliebiger Elementtypen entgegen. Aus ihnen werden die Elemente nur als `Object` gelesen und über `remove(Object)` entfernt; dazu ist keine besondere Beziehung zwischen dem

Typparameter der Ziel- und Eingabeliste nötig. Ein ungebundenes Fragezeichen wird in diesem Kontext als unbekannter Referenztyp mit `Object` als implizitem oberem Bound behandelt. Die gezeigte Schleife illustriert den Typvertrag; bei identischer Eingabe-/Zielliste ist ihre Iterationssemantik gesondert zu klären.

Originalabbildung: 05.Generics.pdf, Folie 70.

Korrigiertes Transfer-Beispiel: erst kopieren, dann entfernen

Das Beispiel der Folie kann bei einer live view die Datenstruktur verändern, während ihr `Iterator` läuft. Die folgende Fassung hält die Elementliste zunächst als Snapshot fest. Sie verwendet JDK-Listen, um eine ausführbare, eindeutig spezifizierte Demonstration zu ermöglichen.

```
static <T> void transferSafe(
 java.util.List<? extends T> from,
 java.util.List<? super T> to) {
 if (from == to) return;
 java.util.List<T> snapshot = new java.util.ArrayList<>(from);
 for (T value : snapshot) {
 to.add(value);
 }
 from.clear(); // Quelle ist eine veränderbare JDK-List
}
```

Diese Version trennt die Iteration von der späteren Quelländerung. Beachte: Wenn das Ziellisten-add mitten im Vorgang scheitert, ist die Operation nicht atomar; auch eine geänderte Quellliste durch einen anderen `Thread` wird nicht automatisch verhindert. Für starke Transaktions- oder `Thread`-Sicherheitsgarantien ist eine zusätzliche Architektur nötig.

5.9 Zusammenfassung und systematische Einordnung

Generische Klassen und Methoden machen aus einer einzigen Quelltextdefinition unterschiedliche, statisch typisierte Verwendungen. Die Verbindung von Typparametern und Java-Vererbung ist nicht einfach eine Vererbung der Parametrisierungen: Invarianz bleibt die sichere Grundeinstellung für veränderbare Container. Bounds begrenzen die erlaubten Typargumente, Wildcards flexibilisieren Lese- und Schreibpositionen. Dabei

gilt `? extends T` für das Erzeugen beziehungsweise Lesen von T-Werten und `? super T` für das Aufnehmen beziehungsweise Schreiben von T-Werten.

Type Erasure erklärt, weshalb die Laufzeit normalerweise keinen `List<String>`- und `List<Person>`-Unterschied prüfen kann. Daraus folgen die Beschränkungen von `instanceof`, parametrisierten `Arrays` und `new T()`. Die Listen-Fallstudie zeigt, wie sich diese Regeln gemeinsam anwenden lassen: typsichere Iteratoren, statisch typisierte Knoten, generische Fabrikmethoden und flexible addAll-Schnittstellen. Zur Prüfung des Verständnisses sollte man stets dieselben drei Fragen stellen: Was ist der statische Typ? Welche Laufzeitrepräsentation besitzt das Objekt? Und ist die Operation ein Lesen, Schreiben oder lediglich ein Vergleich/Entfernen?

Die fünf Themen bilden eine zusammenhängende Folge: Klassen definieren Zustand und Operationen; Vererbung und Interfaces definieren Subtypbeziehungen und dynamische Verarbeitung; Records, Enums und geschachtelte Klassen spezialisieren die Struktur von Typen; Generics ergänzen die statische Typsicherheit bei wiederverwendbaren Abstraktionen. Alle Ebenen benötigen konsistente Sichtbarkeit, gültige Objektzustände und präzise Methodensignaturen.

Beim Lesen eines Java-Ausdrucks ist daher systematisch zu prüfen: Welcher statische Typ ist bekannt? Welche Methoden sind damit zugänglich? Ist eine ausgewählte Instanzmethode dynamisch gebunden? Welche Parametertypen und checked Exceptions gelten? Bei generischen Typen kommt hinzu: Welche Bounds und Wildcards begrenzen das Lesen und Schreiben? Welche Information geht durch Erasure zur Laufzeit verloren? Diese Reihenfolge verhindert häufige Verwechslungen zwischen statischer Zugriffsprüfung und tatsächlichem Laufzeitverhalten.

Originalfolien, Code und fachliche Einordnung

G · Folie 72: ZUSAMMENFASSUNG

Die Schlussfolie verbindet Motivation, Constraints, Wildcards und Erasure. Generische Typparameter werden zur Compilezeit zur statischen Typprüfung eingesetzt, können aber durch Erasure nicht als beliebig unterscheidbare Laufzeittypen behandelt werden. Bounds erzwingen Fähigkeiten und Wildcards ermöglichen flexible, sichere Benutzung; obere Bounds passen zu lesender, untere zu schreibender Verwendung. Die Beziehungen sind unabhängig von der grundlegenden Klassenvererbung, müssen aber im gemeinsamen Typensystem korrekt kombiniert werden.

Zusammenfassung

- Generische Typen und Methoden verwenden Typparameter für ein Arbeiten mit unterschiedlichen Typen
  - Typparameter sind Variablen/Platzhalter für Typen
  - Werden durch konkrete Typen belegt
- Generizität dient der statischen Typprüfung zur Compilezeit
  - Generics ermöglichen typsicheres Arbeiten ohne explizite Typumwandlungen
- Generizität ist zu Vererbung und Typhierarchien orthogonal; dies erfordert:
  - Typconstraints schränken die zulässigen Typargumente ein.
  - Wildcards ermöglichen flexible Beziehungen zwischen parametrisierten Typen.
- Wildcards
  - Wildcards mit Upper Bounds ermöglichen kovariante Verwendung generischer Typen.
  - Wildcards mit Lower Bounds ermöglichen kontravariante Verwendung generischer Typen.
- Type Erasure in Java:
  - Generische Typen und Methoden werden auf eine gemeinsame Laufzeitimplementierung abgebildet
  - Typparameter werden durch ihren oberen Bound ersetzt (bei unbeschränkten Typparametern durch **Object**)
  - Konkrete Typargumente stehen zur Laufzeit für Typprüfungen nicht zur Verfügung

© H. Prähofer, 2026 74

Originalabbildung: 05.Generics.pdf, Folie 72.

Merkschema mit drei Entscheidungen

Bei einer Java-Generics-Aufgabe ist zuerst zu klären, welche Typvariable oder welcher Bound deklariert wurde. Zweitens ist zu fragen, ob die betrachtete Position einen Wert produziert (`extends`) oder konsumiert (`super`). Drittens muss man die Laufzeitseite prüfen: Objekte besitzen eine tatsächliche Klasse, aber gewöhnliche generische Typargumente stehen nach Erasure nicht für jede dynamische Prüfung bereit. Dieses Verfahren erklärt die meisten Compilerfehler der Folien, ohne auf auswendig gelernte Einzelfälle angewiesen zu sein.

Anhang A Fachliche Korrekturen und besonders wichtige Codefallen

01.Klassen · Folie 19

Die Schreibweise `private abstract` ist illegal: Eine abstrakte Methode muss für eine Implementierung durch Unterklassen überschreibbar sein, eine `private` Methode ist es nicht.

01.Klassen · Folie 22

Eine Klasse darf mehrere `static { ... }`-Blöcke enthalten, nicht höchstens einen. Java arbeitet ihre Ausführung bei der Klasseninitialisierung in der Reihenfolge im Quelltext ab.

01.Klassen · Folie 36

Im illustrierten `Timer.add(int sec)` werden für Übertrag und Rest teils der Parameter `sec` und das Feld `this.sec` verwechselt. Beispiel: Ausgangszustand 00:00:50, `add(20)` muss 00:01:10 statt 00:00:20 ergeben. Die Berechnung muss zuerst 50 + 20 = 70 und danach Übertrag und Rest verwenden.

01.Klassen · Folien 45–52

`final` verbietet das erneute Zuweisen einer Variablen, nicht automatisch jede Mutation des verwiesenen Objekts. Tiefe Unveränderlichkeit verlangt Kontrolle aller veränderbaren referenzierten Objekte.

02.Vererbung · Folie 13

Der Beispielcode mit der Positionsrückgabe `new Pos(x,y)` ist in der extrahierten Folie unvollständig gesetzt. Das Prinzip der abstrakten Deklaration bleibt gültig.

02.Vererbung · Folie 25

Die Schleife über `new Shape[10]` mit nur zwei befüllten Positionen führt ohne Nullprüfung nach den ersten zwei Elementen zu einer `NullPointerException`.

02.Vererbung · Folie 29

Ein Switch-Ausdruck über einen beliebigen `int month` muss vollständig sein. Das dargestellte Beispiel ohne `default` ist als Java-Switch-Ausdruck nicht vollständig.

02.Vererbung · Folie 32

Im `readInt(new FileReader(fileName);`-Ausschnitt fehlt eine schließende Klammer.

02.Vererbung · Folie 34

Eine `abstract`-Methode muss mit Semikolon enden und darf keinen Methodenrumpf besitzen.

02.Vererbung · Folie 36

Die Formulierung „Methoden sind dynamisch gebunden“ ist als Regel für überschreibbare Instanzmethoden zu lesen. Statische und `private` Methoden sind nicht virtuell überschreibbar; finale Methoden werden nicht überschrieben.

02.Vererbung · Folie 75

Bei den Array-Adapter-Klassennamen werden unterschiedliche Varianten verwendet. Es muss konsequent der tatsächlich deklarierte Typ erzeugt werden.

02.Vererbung · Folie 81

`this` ist in einem statischen Methodenrumpf nicht verfügbar. Für `setStrategy` muss die Strategie entweder auf einem Objekt gespeichert werden oder als Klassenfeld deklariert und ohne `this` gesetzt werden.

03-Interfaces · Folie 16

Das Vergleichsergebnis in `Fraction.compareTo` hat das umgekehrte Vorzeichen. Richtig für positive Nenner ist `num other.denom - other.num denom`; Overflow berücksichtigen.

03-Interfaces · Folie 17

`java.util.Collection` ist ein Interface, keine Basisklasse.

03-Interfaces · Folie 38

Eine Methode `remove(Object)` muss berücksichtigen, wenn `indexOf(o)` -1 liefert; unmittelbar `remove(-1)` wäre fehlerhaft.

03-Interfaces · Folie 39

Ein Wrapper, der nur `List` implementiert, ist eine schreibgeschützte Sicht. Andere Referenzen können seine intern referenzierte `MutableList` weiterhin verändern; er garantiert ohne Kopie keine tiefe Unveränderlichkeit.

Mehrere Folien zur List-Fallstudie

Die als Beispiel entworfenen Klassen `List`, `AbstractList`, `LinkedList`, `ArrayList` sind didaktische Entwürfe. Sie sind nicht identisch mit allen Signaturen und Regeln der gleichnamigen Java-Standardbibliotheksklassen.

04-Spezielle Klassen · Folie 5

Die Folie gibt nur eine vereinfachte Expansion des Records wieder. Die tatsächlich generierte toString-Ausgabe enthält in der üblichen Form die Komponentennamen, `z.B.` `Fraction`[num=1, denom=2]; außerdem sind Records keine frei erweiterbaren Unterklassen von `Record`.

04-Spezielle Klassen · Folie 8

Im Beispiel `Fraction.add` wird im Ergebnisnenner `n * f.denom` geschrieben, obwohl n nicht definiert ist. Korrekt ist `denom * f.denom`. Auch eine Nennerprüfung, mögliche Überläufe und optionale Normalisierung sind bei realer Brucharithmetik zu berücksichtigen.

04-Spezielle Klassen · Folie 13

Das Enum-Beispiel ordnet FEB konstant 28 Tage zu. Dies trifft nicht auf Schaltjahre zu; die Klasse zeigt einen vereinfachten Ausgangswert und keinen vollständigen Kalender.

04-Spezielle Klassen · Folien 19 und 47

Der Oberbegriff „innere Klassen“ ist an diesen Stellen weit gefasst. Formal sind `static` nested classes keine inner classes. Seit Java 16 sind statische Felder, Methoden und Initialisierer auch in echten inneren Klassen möglich; die pauschale Verbotsformulierung auf Folie 47 ist veraltet.

04-Spezielle Klassen · Folie 43

Ein `Iterator`, dem Array und size übergeben werden, muss eine gültige Größe zwischen `null` und `array.length` voraussetzen oder validieren. Änderungen am übergebenen Array bleiben möglich und beeinflussen die Iteration.

04-Spezielle Klassen · Folie 51

Die abgebildete `LinkedList` ist `final`; zusätzlich ist `Node private`. Die Begleitformulierung, `Node` sei „in `LinkedList` und Unterklassen sichtbar“, widerspricht beiden Festlegungen. Unterklassen gibt es von einer `final`-Klasse nicht, und `private Node` ist ohnehin nicht direkt außerhalb der einschließenden Klasse sichtbar.

04-Spezielle Klassen · Folien 52 und 53

`iteratorOf(elems)` auf einem Array mit möglicher Überkapazität darf nicht automatisch alle Arrayplätze als logische Listenelemente liefern. Außerdem sollte indexOf für potenzielle Nullwerte `Objects.equals` statt array[i].equals verwenden.

04-Spezielle Klassen · Folien 55–58

Eine indexbasierte Traversierung einer `LinkedList` kann `O(n²)` benötigen, während ein `Iterator O(n)` benötigt. Die von der Vorlesung entworfenen Interfaces sollen diese Kostenunterschiede ausdrücken und sind nicht deckungsgleich mit `java.util.List`.

Klassen · statischer Initialisierungsblock (Originalfolie 22)

Die Originalfolie behauptet, eine Klasse habe höchstens einen statischen Initialisierungsblock. Java erlaubt mehrere solcher Blöcke; sie werden zusammen mit statischen Feldinitialisierern in textueller Reihenfolge ausgeführt, wenn die Klasse initialisiert wird. Die Aussage ist daher fachlich falsch und wird hier ausdrücklich korrigiert.

Klassen · technische Methodensignatur (Originalfolie 19)

Die Folie verwendet „Methodensignatur“ für eine umfassende Methodendeklaration mit Rückgabetyp und `throws`-Klausel. In der Java Language Specification besteht die eigentliche Methodensignatur im Wesentlichen aus Methodenname und `Parametertypen (einschließlich relevanter Typparameterregeln)`, nicht aus Rückgabetyp, Sichtbarkeit oder `throws`-Klausel. Deshalb können Methoden nicht allein durch den Rückgabetyp überladen werden.

Klassen und Vererbung · Beispiele mit Platzhaltern

Einige Folien zeigen Teile von Methoden als `...` oder lassen Deklarationen benachbarter Hilfsklassen weg. Solche Fragmente illustrieren Sprachregeln, sind aber keine vollständigen, selbständig übersetzbaren Java-

Dateien. Für eine lauffähige Fassung sind fehlende Imports, Konstruktoren, Exceptionbehandlung und geeignete Implementierungen zu ergänzen.

Spezielle Klassen · verschachtelte Typen und static

Die pauschale Aussage, lokale und innere Klassen könnten keine statischen Mitglieder enthalten, ist für modernes Java so nicht mehr richtig. Seit Java 16 können innere Klassen statische Member deklarieren. Der rechtlich präzise Unterschied zwischen `static` nested `class` und `(nicht statischer)` inner `class` bleibt erhalten; nur letztere ist an eine äußere Instanz gebunden.

Spezielle Klassen · Einschränkungen von Read-only-Sichten

Die zuvor vorgestellten lesenden Interfaces sind nicht automatisch gleichbedeutend mit einem echten unveränderlichen Snapshot. Ein Cast kann Zugriff auf eine darunterliegende `MutableList` erlauben, wenn tatsächlich dieselbe konkrete Instanz zurückgegeben wird. Ein Wrapper verbietet bestimmte Casts, ersetzt aber keine Kopie, wenn die dahinterliegende Datenstruktur andernorts mutiert werden darf.

05.Generics · Folien 7–8

Das dort ausgeschriebene Ersetzen von `List<E>` durch separate `List<String>`- oder `List<Person>`-Klassen ist eine begriffliche Darstellung. Bei Java Type Erasure entstehen dadurch normalerweise keine separaten Laufzeitklassen für die verschiedenen Typargumente.

05.Generics · Folie 9

Das gezeigte `new Map<>()` bezeichnet ein eigenes Lehrbeispiel mit einer als Klasse deklarierten `Map`. Das JDK verwendet `java.util.Map` als Interface; dieses kann man nicht direkt instanziieren.

05.Generics · Folie 27

Die Kurzform „nur lesende Zugriffe“ bei `? extends T` ist nicht absolut: Nullwerte können bei `add(E)` über eine Wildcard-Referenz grundsätzlich übergeben werden, und nicht typabhängige Methoden bleiben verwendbar.

05.Generics · Folie 32

Bei `List<? super Student>` ist `get()` nicht verboten. Der Rückgabewert hat aus Sicht des Compilers den sicheren Typ `Object`; eine unmittelbare Zuweisung an `Student` ist dagegen unzulässig.

05.Generics · Folie 33

Im Ablauf zur Typinferenz steht am Ende fälschlich `Person super Person`. Korrekt muss die Forderung `Person super Student` lauten: Der tatsächliche Zielelementtyp `Person` ist ein Supertyp des Quelltyps `Student`.

05.Generics · Folie 36

Die Überschrift spricht von SortedSet, während das Beispiel tatsächlich `SortedList<T extends Comparable<? super T>>` verwendet. Entscheidend für den Bound ist dieselbe `Comparable`-Supertypregel.

05.Generics · Folien 39–43

Type Erasure entfernt viele konkrete Laufzeit-Typargumente, aber nicht alle generischen Informationen aus jeder Klassendatei: Deklarationsmetadaten können per Reflection vorhanden sein. Unchecked casts und Rohtypen schränken die statische Sicherheit ein.

05.Generics · Folie 49

Der Cast von `new Object`[CAP] zu `E[]` ist unchecked und nur bei kontrolliert gekapselter Verwendung erklärbar; er erzeugt nicht tatsächlich ein Array mit dem Laufzeitkomponententyp E. Das Herausgeben als konkretes `String[]` kann fehlschlagen.

05.Generics · Folie 59

Das vereinfachte Interface-Diagramm und die schematischen Deklarationen müssen beim Zusammenbau konsistent sein. Eine vollständige `MutableIndexedList<E>` kombiniert, wie das Diagramm zeigt, die erforderlichen verändernden und indexbasierten Fähigkeiten.

05.Generics · Folien 68–69

Die gezeigte transfer-Methode kann beim Entfernen aus einer Quelle, über die gleichzeitig ein live `Iterator` läuft, fehlerhaft sein. Eine read-only Sicht ist nicht automatisch eine Snapshot-Kopie. Eine korrekte Implementierung muss das Verhalten bei Änderung, doppelten Elementen und Fehlern festlegen.

05.Generics · Folie 70

Bei `removeAll(List<?> elems)` ist ein Elementtyp nicht notwendig; über `remove(Object)` werden keine unerlaubten Inhalte eingefügt. Als Ergänzung ist bei identischer Quell- und Zielsammlung das Iterationsverhalten gesondert zu behandeln.

Anhang B Vollständiger Folienabgleich

Jeder Eintrag entspricht einer Originalfolienseite. Die Angabe nennt die Stelle im Skript, an der die fachlichen Inhalte erläutert werden. Die inhaltlichen Originalfolien sind in der jeweiligen Sektion zusätzlich als Faksimile abgedruckt. Gliederungs-, Titel- und reine Lernziel-Folien sind durch (G) gekennzeichnet und werden aus Redundanzgründen nicht gesondert als Bild wiederholt.

01.Klassen.pdf · 60 Folien

Folie	Kapitel	Titel / Prüfstatus
1	1.1	(G) SOFTWAREENTWICKLUNG MIT JAVA 2
2	1.1	(G) 1. KLASSEN
3	1.1	KLASSEN
4	1.1	VARIABLENDEKLARATION UND OBJEKTERZEUGUNG
5	1.1	REFERENZEN UND REFERENZZUWEISUNGEN
6	1.1	WERTESEMANTIK VS. REFERENZSEMANTIK
7	1.1	WRAPPER-KLASSEN UND BOXING / UNBOXING
8	1.1	BOXING UND UNBOXING
9	1.1	NULL-REFERENZ
10	1.1	NULL HAT MEHRERE BEDEUTUNGEN/VERWENDUNGEN
11	1.1	UMGANG MIT NULL-WERTEN: PRÜFEN UND ERSETZEN
12	1.1	UMGANG MIT NULL-WERTEN: NULL-SICHERE OPERATIONEN
13	1.1	(G) LERNZIELE ÜBERPRÜFEN
14	1.2	(G) 1. KLASSEN
15	1.2	BESTANDTEILE VON KLASSEN
16	1.2	FELDER: STATISCHE VS. OBJEKTVARIABLEN
17	1.2	METHODEN: STATISCHE VS. OBJEKTMETHODEN
18	1.2	OBJEKTMETHODEN: AUFRUF
19	1.2	METHODEN: METHODENSIGNATUR
20	1.2	METHODEN: VARARG-PARAMETER
21	1.2	METHODEN: ÜBERLADEN VON METHODEN
22	1.2	STATISCHER INITIALISIERUNGSBLOCK
23	1.2	KONSTRUKTOREN
24	1.2	STANDARDKONSTRUKTOR
25	1.2	(G) LERNZIELE ÜBERPRÜFEN
26	1.3	(G) 1. KLASSEN
27	1.3	VERZEICHNISSTRUKTUR UND PACKAGES
28	1.3	VERZEICHNISSTRUKTUR UND PACKAGES
29	1.3	VOLLQUALIFIZIERTER KLASSENNAME
30	1.3	IMPORTS
31	1.3	STATISCHE IMPORTS
32	1.3	WICHTIGE PACKAGES AUS JDK
33	1.3	(G) LERNZIELE ÜBERPRÜFEN
34	1.4	(G) 1. KLASSEN
35	1.4	SICHTBARKEITSATTRIBUTE
36	1.4	TIMER MIT INFORMATION HIDING
37	1.4	GETTER UND SETTER
38	1.4	GETTER UND SETTER
39	1.4	GETTER UND SETTER: VORTEILE
40	1.4	GETTER: ALTERNATIVE NAMENSGEBUNG
41	1.4	RICHTLINIEN ZUR VERWENDUNG DER SICHTBARKEITSATTRIBUTE
42	1.4	(G) LERNZIELE ÜBERPRÜFEN
43	1.5	(G) 1. KLASSEN
44	1.5	FINAL: STATISCHE FELDER
45	1.5	FINAL: OBJEKTFELDER
46	1.5	UNVERÄNDERLICHE DATENKLASSEN
47	1.5	IMPLEMENTIERUNG EQUALS UND HASHCODE
48	1.5	IMPLEMENTIERUNG EQUALS UND HASHCODE
49	1.5	IMPLEMENTIERUNG EQUALS UND HASHCODE
50	1.5	UNVERÄNDERLICHE DATENOBJEKTE: OPERATIONEN

51	1.5	VERÄNDERLICHE VS. UNVERÄNDERLICHE KLASSEN: BEISPIEL TIMER UND TIME
52	1.5	BEST PRACTICE: MÖGLICHST UNVERÄNDERLICHE KLASSEN
53	1.5	(G) LERNZIELE ÜBERPRÜFEN
54	1.6	(G) 1. KLASSEN
55	1.6	RICHTLINIEN ZUR GESTALTUNG VON KLASSENSYSTEMEN
56	1.6	RICHTLINIEN ZUR GESTALTUNG VON KLASSENSYSTEMEN
57	1.6	FALLBEISPIEL TELEFONBUCH
58	1.6	FALLBEISPIEL TELEFONBUCH: KLASSEN
59	1.6	FALLBEISPIEL TELEFONBUCH: KLASSEN
60	1.6	FALLBEISPIEL TELEFONBUCH: AUFGABEN

Kontrolle: 60 von 60 Folien zugeordnet; 48 inhaltliche Originalfolien als Abbildungen eingebettet; 12 reine Titel-/Gliederungs-/Lernziel-Folien ohne separate Reproduktion.

02.Vererbung.pdf · 86 Folien

Folie	Kapitel	Titel / Prüfstatus
1	2.0	(G) SOFTWAREENTWICKLUNG MIT JAVA 2
2	2.0	VORBEMERKUNG: STATISCH VS. DYNAMISCH
3	2.0	UNIFIED MODELING LANGUAGE (UML)
4	2.1	(G) 2. VERERBUNG UND DYNAMISCHE BINDUNG
5	2.1	VERERBUNG (ABLEITUNG, SPEZIALISIERUNG)
6	2.1	VERERBUNG IN JAVA
7	2.1	DIE KLASSE OBJECT
8	2.1	ÜBERSCHREIBEN VON METHODEN UND SUPER-AUFRUFE
9	2.1	ANNOTATION @OVERRIDE
10	2.1	SICHTBARKEITSATTRIBUT PROTECTED
11	2.1	SICHTBARKEITSATTRIBUT PROTECTED
12	2.1	ERWEITERUNG DER SICHTBARKEIT
13	2.1	ABSTRAKTE KLASSEN UND METHODEN
14	2.1	ABLEITUNG VON ABSTRAKTER KLASSE
15	2.1	ABSTRAKTE KLASSEN ALS TYPEN
16	2.1	FINAL KLASSEN
17	2.1	FINAL METHODEN
18	2.1	ÜBERDECKEN VON FELDERN
19	2.1	(G) LERNZIELE ÜBERPRÜFEN
20	2.2	(G) 2. VERERBUNG UND DYNAMISCHE BINDUNG
21	2.2	TYPEN, SUBTYPEN UND TYPHIERARCHIEN
22	2.2	TYPKOMPATIBILITÄT UND POLYMORPHE ZUWEISUNG
23	2.2	STATISCHER UND DYNAMISCHER TYP
24	2.2	DYNAMISCHE BINDUNG VON METHODEN
25	2.2	POLYMORPHE VERARBEITUNG
26	2.2	LAUFZEIT-TYPTEST
27	2.2	TYPUMWANDLUNG (TYPECAST)
28	2.2	KOMBINATION TYPTEST UND TYPECAST
29	2.2	EXKURS: SWITCH-AUSDRUCK
30	2.2	EXKURS: SWITCH-AUSDRUCK
31	2.2	SWITCH MIT TYPE PATTERNS
32	2.2	AUSNAHMEBEHANDLUNG: FANGEN VON EXCEPTIONS
33	2.2	HIERARCHIE DER EXCEPTION-KLASSEN
34	2.2	THROWS-KLAUSEL BEI METHODEN
35	2.2	THROWS-KLAUSEL BEI METHODEN
36	2.2	MERKSÄTZE
37	2.2	(G) LERNZIELE ÜBERPRÜFEN
38	2.3	(G) 2. VERERBUNG UND DYNAMISCHE BINDUNG
39	2.3	KOVARIANTE RÜCKGABETYPEN
40	2.3	KOVARIANTE RÜCKGABEWERTE UND POLYMORPHE ZUWEISUNGEN
41	2.3	KOVARIANZ BEI GEWORFENEN EXCEPTIONS
42	2.3	KOVARIANZ BEI GEWORFENEN EXCEPTIONS
43	2.3	KOVARIANZ BEI GEWORFENEN EXCEPTIONS
44	2.3	(G) LERNZIELE ÜBERPRÜFEN
45	2.4	(G) 2. VERERBUNG UND DYNAMISCHE BINDUNG
46	2.4	FALLSTUDIE LIST

47	2.4	FALLSTUDIE LIST
48	2.4	INTERFACE LIST UND IMPLEMENTIERENDE KLASSEN
49	2.4	ListBASISKLASSE LIST
50	2.4	ListKONKRETE KLASSE: LINKEDLIST
51	2.4	ListKONKRETE KLASSE: ARRAYLIST
52	2.4	PROBLEMANALYSE
53	2.4	PROBLEMANALYSE
54	2.4	PROBLEMANALYSE: IMPLEMENTIERUNG VON ZUSÄTZEN
55	2.4	PROBLEMANALYSE: IMPLEMENTIERUNG VON ZUSÄTZEN
56	2.5	(G) 2. VERERBUNG UND DYNAMISCHE BINDUNG
57	2.5	ENTWURFSMUSTER (DESIGN PATTERNS)
58	2.5	DESIGN PATTERNS: ELEMENTS OF REUSABLE OBJECT-ORIENTED SOFTWARE
59	2.5	BESTANDTEILE EINES MUSTERS
60	2.5	EXKURS: UML NOTATION
61	2.5	ENTWURFSMUSTER WRAPPER
62	2.5	ENTWURFSMUSTER WRAPPER: LOGGINGLISTWRAPPER
63	2.5	ENTWURFSMUSTER WRAPPER: LOGGINGLISTWRAPPER
64	2.5	(G) 2. VERERBUNG
65	2.5	ENTWURFSMUSTER DECORATOR
66	2.5	ENTWURFSMUSTER DECORATOR: LISTDECORATOR
67	2.5	ENTWURFSMUSTER DECORATOR: UNMODIFIABLELIST
68	2.5	ENTWURFSMUSTER ADAPTER
69	2.5	ENTWURFSMUSTER ADAPTER
70	2.5	ENTWURFSMUSTER: ADAPTER [3/3]
71	2.5	ENTWURFSMUSTER FACTORY-METHODE [1
72	2.5	ENTWURFSMUSTER FACTORY-METHODE [1
73	2.5	FACTORY-METHODE
74	2.5	ENTWURFSMUSTER ADAPTER: ARRAYTOLISTADAPTER
75	2.5	ENTWURFSMUSTER ADAPTER: SINGLE2LISTADAPTER
76	2.5	TEMPLATE METHOD (SCHABLONENMETHODE)
77	2.5	TEMPLATE METHOD (SCHABLONENMETHODE)
78	2.5	TEMPLATE METHOD (SCHABLONENMETHODE)
79	2.5	TEMPLATE METHODE COPYTO BEI LIST
80	2.5	STRATEGY (STRATEGIE)
81	2.5	STRATEGY (STRATEGIE)
82	2.5	STRATEGY (STRATEGIE)
83	2.5	LIST: METHODE SORT MIT STRATEGIE FÜR SORTIERUNG
84	2.5	STRATEGY (STRATEGIE)
85	2.5	ZUSAMMENFASSUNG FALLSTUDIE LIST
86	2.5	(G) LERNZIELE ÜBERPRÜFEN

Kontrolle: 86 von 86 Folien zugeordnet; 75 inhaltliche Originalfolien als Abbildungen eingebettet; 11 reine Titel-/Gliederungs-/Lernziel-Folien ohne separate Reproduktion.

03-Interfaces.pdf · 42 Folien

Folie	Kapitel	Titel / Prüfstatus
1	3.0	(G) SOFTWAREENTWICKLUNG MIT JAVA 2
2	3.0	VORSCHAU: GENERICS TypparameterNäheres zu G
3	3.0	VORSCHAU: GENERICS
4	3.1	(G) 3. INTERFACES
5	3.1	EINFACHVERERBUNG BEI KLASSEN (SINGLE INHERITANCE)
6	3.1	MEHRFACHVERERBUNG BEI KLASSEN (MULTIPLE INHERITANCE)
7	3.2	(G) 3. INTERFACES
8	3.2	INTERFACES IN JAVA
9	3.2	<<interface>>INTERFACES IN JAVA
10	3.2	MEHRFACHVERERBUNG BEI INTERFACES IN JAVA
11	3.2	UNIFIED MODELING LANGUAGE (UML)
12	3.2	INTERFACES ALS TYPEN
13	3.3	(G) 3. INTERFACES
14	3.3	DEFAULT-METHODEN
15	3.3	ERBEN VON DEFAULT-METHODEN
16	3.3	ANWENDUNG DEFAULT-METHODEN: "RICH INTERFACES"

17	3.3	ANWENDUNG DEFAULT-METHODEN: SCHNITTSTELLENERWEITERUNGEN
18	3.3	EINSCHRÄNKUNGEN BEI DEFAULT-METHODEN
19	3.4	(G) 3. INTERFACES
20	3.4	STATISCHE KONSTANTEN UND METHODEN
21	3.4	PRIVATE METHODEN
22	3.5	(G) 3. INTERFACES
23	3.5	INTERFACE JAVA.LANG.RUNNABLE
24	3.5	ITERATOR<T> UND ITERABLE<T>
25	3.5	COMPARABLE<T>
26	3.5	COMPARATOR<T>
27	3.5	SONDERFALL: TAGGING INTERFACES Achtung: Statt Tagging Interfaces werden
28	3.5	FALLBEISPIEL: INTERFACE CLONEABLE UND METHODE CLONE
29	3.5	FALLBEISPIEL: INTERFACE CLONEABLE UND METHODE CLONE
30	3.5	ALTERNATIVE ZU CLONE: COPY-KONSTRUKTOREN
31	3.5	(G) LERNZIELE ÜBERPRÜFEN
32	3.6	(G) 3. INTERFACES
33	3.6	INTERFACE LIST
34	3.6	INTERFACE LIST
35	3.6	INTERFACE LIST
36	3.6	(G) 3. INTERFACES
37	3.6	UNMODIFIABLELISTDECORATOR: PROBLEMANALYSE
38	3.6	Approach von Kotlin!INTERFACES LIST UND MUTABLELIST
39	3.6	WIRKLICH UNVERÄNDERLICHE LISTE
40	3.6	KLASSENDIAGRAMM LIST UND MUTABLELIST
41	3.6	(G) 3. INTERFACES
42	3.6	WEITERE VERBESSERUNGEN

Kontrolle: 42 von 42 Folien zugeordnet; 32 inhaltliche Originalfolien als Abbildungen eingebettet; 10 reine Titel-/Gliederungs-/Lernziel-Folien ohne separate Reproduktion.

04-Spezielle Klassen.pdf · 59 Folien

Folie	Kapitel	Titel / Prüfstatus
1	4.1 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
2	4.1 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
3	4.1	RECORD-KLASSEN Record-Klassen sind spezielle Klassen für unveränderliche Objek
4	4.1	RECORD-KLASSEN Für Record-Klassen ist definiert
5	4.1	Record-Klassen werden expandiert
6	4.1	RECORD-KLASSEN: MÖGLICHKEITEN UND EINSCHRÄNKUNGEN Bei Record-Klassen ist erlau
7	4.1	"COMPACT"-KONSTRUKTOR "Compact"-Konstruktor = Variante zu kanonischem Konstruk
8	4.1	public record Fraction(int num, int denom) { public Fraction(int num, int deno
9	4.1 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
10	4.2 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
11	4.2	ENUMERATIONSKLASSEN Sind spezielle Klassen mit definierten Instanzen
12	4.2	ENUMERATIONSKLASSEN Unterstützte Features (Fortsetzung)
13	4.2	ENUMERATIONSKLASSEN Enumerationsklassen können haben
14	4.2	ENUMERATIONSKLASSEN: MÖGLICHKEITEN UND EINSCHRÄNKUNGEN Bei Enumerationsklassen
15	4.2 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
16	4.3 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
17	4.3	TOP-LEVEL KLASSEN UND INTERFACES Klassen und Interfaces, die

		sich direkt in e
18	4.3	INNERE KLASSEN Innere Klassen sind lokal zu anderen Klassen
19	4.3	KLASSIFIKATION VON INNEREN KLASSEN Innere Klassen
20	4.3	Für geschachtelte Klassen alle Sichtbarkeitsattribute möglich ↳private
21	4.3 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
22	4.3	STATISCHE MEMBER-KLASSEN static deklariert
23	4.3	BEISPIEL STATISCHE MEMBER-KLASSE Klasse Node als statische Member-Klasse, die
24	4.3	INNERE KLASSEN IN INTERFACES: ANWENDUNGSBEISPIEL Konkrete Klassen die Interfac
25	4.3	INNERE KLASSEN IN INTERFACES: ANWENDUNGSBEISPIEL ↳Anwendung
26	4.3 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
27	4.3	MEMBER-KLASSEN Als nicht-statische innere Klassen definiert
28	4.3	public class Outer { int x = 0;
29	4.3	public class Outer { int x = 0;
30	4.3	MEMBER-KLASSE ↳Innere Objekte müssen immer mit äußerem Objekt instanziiert wer
31	4.4	ENTWURFSMUSTER: ITERATOR Zweck:
32	4.4	ENTWURFSMUSTER ITERATOR: INTERFACES ITERATOR<T> UND ITERABLE<T> ↳Iterator<T>:
33	4.4	ENTWURFSMUSTER ITERATOR: SCHEMA ZUR IMPLEMENTIERUNG Iteratoren müssen nach fol
34	4.4	BEISPIEL MEMBER-KLASSE: LISTITERATOR IN LINKEDLIST Interface Iterable<Object>
35	4.4	BEISPIEL MEMBER-KLASSE: LISTITERATOR IN LINKEDLIST Schema Implementierung des
36	4.4	BEISPIEL MEMBER-KLASSE: LISTITERATOR IN ARRAYLIST Index idx für Zugriff auf Ar
37	4.3 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
38	4.3	LOKALE KLASSEN ↳Lokale Klassen werden im Block einer Methode definiert
39	4.3	BEISPIEL LOKALE KLASSE: LISTITERATOR public class LinkedList extends AbstractL
40	4.3 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
41	4.3	ANONYME KLASSEN ↳Lokale Klasse ohne Namen
42	4.3	BEISPIEL: ANONYME KLASSE FÜR ITERATOR IN LINKEDLIST public class LinkedList ex
43	4.3	BEISPIEL: ANONYMER ITERATOR AUS PARAMETER ↳Statische Methode iteratorOf
44	4.3	THIS-ZEIGER FÜR ÄUßERES OBJEKT ↳In inneren Member-Klassen bezieht sich this au
45	4.3	LOKALE VARIABLEN IN ANONYMEN KLASSEN ↳Anonyme Klassen können lokale Variablen
46	4.3	LOKALE VARIABLEN IN ANONYMEN KLASSEN ↳Anonyme Klassen können lokale Variablen
47	4.3	EINSCHRÄNKUNGEN BEI INNEREN KLASSEN UND INTERFACES Innere Interfaces
48	4.3	WANN VERWENDE ICH WELCHE INNERE KLASSE statische Member-

		Klasse
49	4.3 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
50	4.4 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
51	4.4	STATISCHE INNERE KLASSE NODE VON LINKEDLIST Klasse Node von LinkedList als sta
52	4.4	IMPLEMENTIERUNG VON ITERABLE UND ITERATOR ↳Basisinterface List abgeleitet von
53	4.4	BEISPIEL: ADAPTERS MIT ANONYMEN KLASSEN List-Adapters für Array als
54	4.4	INTERFACES LIST UND MUTABLELIST Interfaces für nur-lesende
55	4.4	PROBLEM: ZUGRIFF ÜBER INDEX ↳Methode get greift über Index auf Element zu
56	4.4	INTERFACES INDEXEDLIST UND MUTABLEINDEXEDLIST ↳List – sequentieller Zugriff mi
57	4.4	INTERFACES LIST UND INDEXEDLIST public interface List extends Iterable<Object>
58	4.4	INTERFACES MUTABLELIST UND MUTABLEINDEXEDLIST ↳MutableList
59	4.4 (G)	Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert

Kontrolle: 59 von 59 Folien zugeordnet; 46 fachliche Originalfolien als Abbildungen eingebettet; 13 Titel-, Übersichts- oder Lernzielfolien durch die Kapitelabdeckung und die beantworteten Lernziele mitberücksichtigt.

05.Generics.pdf · 72 Folien

Folie	Kapitel	Titel / Prüfstatus
1	5.0 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
2	5.0 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
3	5.1	PROBLEMANALYSE: NICHT-GENERISCHE LISTE
4	5.1	PROBLEMANALYSE: METHODEN OHNE GENERIZITÄT
5	5.1 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
6	5.2	GENERISCHE TYPEN [1/3]
7	5.2	GENERISCHE TYPEN [2/3]
8	5.2	GENERISCHE TYPEN [3/3]
9	5.2	MEHRERE TYPPARAMETER
10	5.2	GENERISCHEN TYPEN UND VERERBUNG
11	5.2	Methodendeklaration mit Typparameter
12	5.2 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
13	5.3 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
14	5.3	TYPCONSTRAINTS: PROBLEMANALYSE
15	5.3	TYPCONSTRAINTS : BOUNDED TYPE PARAMETERS
16	5.3	BEISPIEL COMPARABLE<T>: VERWENDUNG
17	5.3	TYPCONSTRAINTS BEI METHODEN
18	5.3	MEHRERE TYPCONSTRAINTS MÖGLICH
19	5.3	VERERBUNG BEI TYPCONSTRAINTS
20	5.3 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
21	5.4 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
22	5.4	VORBEMERKUNG: KOMPATIBILITÄT BEI METHODENAUFRUFEN
23	5.4	MOTIVATION: METHODE COPYTO (INVARIANTE VERSION)
24	5.4	KOVARIANZ BEI GENERISCHEN TYPEN: PROBLEMANALYSE

25	5.4	KOVARIANZ BEI GENERISCHEN TYPEN: PROBLEMANALYSE
26	5.4	UPPER-BOUNDED WILDCARD
27	5.4	UPPER-BOUNDED WILDCARD
28	5.4	COPYTO: KOVARIANTE VERSION
29	5.4	KONTRAVARIANZ BEI GENERISCHEN TYPEN: PROBLEMANALYSE
30	5.4	KONTRAVARIANZ BEI GENERISCHEN TYPEN: PROBLEMANALYSE
31	5.4	LOWER-BOUNDED WILDCARD
32	5.4	LOWER-BOUNDED WILDCARD
33	5.4	COPYTO: KONTRAVARIANTE VERSION
34	5.4	BEISPIEL: TYPEBOUND BEI COMPARABLE
35	5.4	BEISPIEL: TYPEBOUND BEI COMPARABLE
36	5.4	BEISPIEL: TYPEBOUND BEI COMPARABLE BEI SORTEDSET
37	5.4 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
38	5.5 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
39	5.5	TYPE ERASURE UND ROHTYPEN
40	5.5	KONSEQUENZEN VON TYPE ERASURE [1/4]
41	5.5	KONSEQUENZEN VON TYPE ERASURE [2/4]
42	5.5	KONSEQUENZEN VON TYPE ERASURE [3/4]
43	5.5	KONSEQUENZEN VON TYPE ERASURE [4/4]
44	5.5 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
45	5.6 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
46	5.6	ARRAYS ALS "GENERISCHES" KONZEPT
47	5.6	ARRAYS ALS "GENERISCHES" KONZEPT: KOVARIANZ [1/2]
48	5.6	ARRAYS ALS "GENERISCHES" KONZEPT: KOVARIANZ [2/2]
49	5.6	class ArrayList<E> {
50	5.6 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
51	5.7 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
52	5.7	TYPINFERENZ [1/2]
53	5.7	TYPINFERENZ [2/2]
54	5.7	OBJEKTERZEUGUNG
55	5.7	OBJEKTERZEUGUNG
56	5.7 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
57	5.8 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
58	5.8	FALLSTUDIE LIST: GENERISCHE INTERFACES UND KLASSEN
59	5.8	FALLSTUDIE LIST: GENERISCHE INTERFACES
60	5.8	FALLSTUDIE LIST: GENERISCHE KLASSEN
61	5.8	FALLSTUDIE LIST: TYPPARAMETER BEI STATISCHE MEMBER-KLASSEN
62	5.8	FALLSTUDIE LIST: TYPPARAMETER BEI MEMBER-KLASSEN
63	5.8	FALLSTUDIE LIST: TYPPARAMETER BEI ANONYMEN KLASSEN
64	5.8	FALLSTUDIE LIST: ADAPTERS GENERISCH
65	5.8	FALLSTUDIE LIST: GENERISCHE ZUGRIFFE
66	5.8	FALLSTUDIE LIST: GENERISCHE ZUGRIFFE
67	5.8	FALLSTUDIE LIST: METHODEN REMOVE, CONTAINS NICHT-GENERISCH
68	5.8	FALLSTUDIE LIST: PROBLEMANALYSE REMOVE
69	5.8	FALLSTUDIE LIST: REMOVE NICHT GENERISCH
70	5.8	FALLSTUDIE LIST: REMOVEALL MIT KOVARIANTEM WILDCARD LIST<?>
71	5.9 (G)	Titel-, Gliederungs- oder Lernzielfolie – Inhalte im Kapitel erläutert
72	5.9	ZUSAMMENFASSUNG

Kontrolle Thema 5: 72 von 72 Folien zugeordnet. Davon 56 fachliche Originalfolien als Abbildungen übernommen; die übrigen 16 Titel-, Übersichts- und Lernzielfolien durch die Kapitelabdeckung und Erklärung der Lernziele berücksichtigt.

Quellenbasis und Abgrenzung: 01.Klassen.pdf (60), 02.Vererbung.pdf (86), 03-Interfaces.pdf (42), 04-Spezielle Klassen.pdf (59) und 05.Generics.pdf (72). Zusammen 319 PDF-Seiten, vollständig im Folienabgleich aufgeführt. 00.Organisatorisches.pdf nur als Orientierung. Eigene Ergänzungen, Codevarianten und Fehlerhinweise werden getrennt von den Originalfolien erläutert. Für die genaue Sprachregel gelten die Java Language Specification und JDK-API-Dokumentation als ergänzender fachlicher Referenzrahmen.